COMP3511 Midterm Cheat Book

Fall 2026 · Midterm 29-Oct-2026 (Thu) 8:00–10:00 PM · L1 **LTD** · L2 **LSKG012** · L3 **LTB**
Coverage: Chapters 1–6 (to Synchronization Tools) · open-book · calculator allowed · bring rough sheets
中文解釋 + English keywords / model answers · 所有 fork / scheduling / sync 答案已用程式驗證

點用（How to use）

1. **先睇 E（考場一頁通）**：公式卡、本科 conventions、陷阱表 — MC 前掃一次。
2. **計數題用 P（Playbooks）**：fork / pipe / scheduling / sync 步驟 + 預計表（Amdahl、powers、RM bound）。
3. **概念題 / short question** → 對應章節（1–6）入面 📝 Model answer（英文可直接抄）。
4. **PA / lab API / coding 題** → L（Labs & PA1/PA2）+ A（API 速查）。
5. **見到似曾相識嘅題目** → Q（歷屆全解）/ M（MC key）。**搵字** → 最後嘅 Index（紅色 = 主要頁）。
6. 右邊（雙數頁左邊）有**彩色 tab**：同一顏色 = 同一部分。

Tab legend

E	Exam-Day Essentials · 考場一頁通：公式・慣例・陷阱	4	P	Playbooks · 解題流程 + 預計表	6
1	Ch1 Introduction · 引言	9	2	Ch2 OS Structures · 作業系統結構	16
3	Ch3 Processes · 進程・fork・IPC・interrupt	26	4	Ch4 Threads · 線程與並行	39
5	Ch5 CPU Scheduling · CPU 排程	47	6	Ch6 Synchronization · 同步工具	59
L	Labs & PA1/PA2 · Lab 1–6・PA 考點・lab 例題	66	Q	Past Papers Solved · 歷屆長題全解	73
M	MC Answer Key · MC 題庫 + 答案	85	A	C / Pthreads API · Lab / PA API 速查	94
G	Glossary A-Z · 術語表	98	I	Index · 索引	102

題型 Router（見到 → 翻去）

Formula card / course conventions / top traps	4	Gantt chart procedure + tie rules	7
Interrupt / DMA / trap / dual mode / privileged	26	Metrics: TAT / WT / response (course def)	47
Cache, locality, average access time	11	MLFQ rules + example	50
von Neumann, storage hierarchy, main memory	10	RM / EDF / real-time	53
SMP / multicore / NUMA / multiprocessor	12	PCS / SCS, affinity, load balancing, CMT	51
Multiprogramming vs timesharing; virtualization; cloud	13	Past scheduling answers (all papers)	56
API vs system call, printf→write, parameters	17	Race condition possible values	63
Linker vs loader, DLL	19	Semaphores, spinlock with TAS / CAS / swap	60
Monolithic / layered / microkernel / LKM / hybrid	20	Critical-section requirements	59
Process states, PCB, schedulers, context switch	28	Lab 3 system calls, exec family, dup / dup2	67
fork() counting / tree / outputs / exec cases	6	PA1 simplified shell (what the exam may ask)	68
pipe / dup / open / read / write fill-in	7	Named pipe / FIFO, popen, Pthreads (Lab 4)	69
Zombie / orphan / termination	32	Priority + RR with arrivals; PA2 MLFQ spec	70
IPC, message passing, pipes, sockets	35	Peterson + Lab 6 sync sample questions	71
Threads: shared vs private, benefits, models	39	C / Pthreads API	94
Amdahl's Law (pre-computed table)	8	MC answer key	85
Signals, cancellation, fork/exec semantics	43	Glossary A–Z / Index	98

Contents 目錄

E · Exam-Day Essentials 4
E1. 必背公式卡 Formula Card 4
E2. 本科 Conventions（考試照用，唔確定就寫 assumption） 4
E3. 最易中伏 Top Traps（MC 前睇一次） 4
E4. 答題模板 Answer Templates 5

P · Playbooks 6
P1. fork() Playbook 6
P2. Pipe / dup / exec 填空 Playbook 7
P3. CPU Scheduling Playbook 7
P4. Synchronization Playbook 7
P5. 預先計好嘅表 Pre-computed Tables 8

1 · Ch1 Introduction 9
1. What is an Operating System?（OS 係乜？）[1.20–1.24] 9
2. Computer System Organization（電腦系統組織）[1.25–1... 9
3. Storage（儲存）[1.28–1.35] 10
4. Caching ⭐ ⭐（必考 short question）[1.32–1.33] 11
5. Processor Systems（處理器系統）[1.36–1.41] 12
6. Multiprogramming & Timesharing ⭐ [1.42] 13
7. Virtualization ⭐ [1.43–1.46] 13
8. Cloud Computing [1.47–1.48] 14
9. I/O Subsystem（Chapter 1 summary 提及） 14
10. 常見陷阱 & MC 速查（Traps） 14
11. Past-paper 題目（Ch1 相關） 15
12. 自我測驗（Self-test） 15

2 · Ch2 OS Structures 16
1. Operating System Services（OS 服務）[2.4–2.7] 16
2. User Interfaces [2.8–2.12] 17
3. System Calls ⭐ ⭐ [2.13–2.26] 17
4. System Programs（系統程式）[2.27–2.30] 19
5. Linkers and Loaders ⭐ ⭐（F19 Q1.1、F20 Q2.2 都考過... 19
6. OS Design and Implementation [2.33–2.37] 20
7. Operating System Structure ⭐ ⭐ ⭐（必考比較題）[2.... 20
8. History of OS [2.60] 24
9. 補充：Protection & Security（Chapter 2 summary） 24
10. 常見陷阱 & MC 速查 24
11. Past-paper 題目（Ch2） 24
12. 自我測驗 24

3 · Ch3 Processes 26
0. Computer-System Operation：I/O、Interrupt、DMA、D... 26
· 0.1 I/O 基本運作 [3.21] 26
· 0.2 Interrupts ⭐ ⭐ [3.22–3.24] 26
· 0.3 DMA (Direct Memory Access) ⭐ ⭐ [3.25–3.26] 26
· 0.4 Dual-mode Operation ⭐ ⭐ [3.27–3.31] 27
· 0.5 Three Types of Mode Transfer（user → kernel） ⭐ [3.... 27
1. Process Concept（進程概念）[3.4–3.8] 27
· 1.1 Four Fundamental OS Concepts [3.4] 27
· 1.2 Process 嘅組成部分 [3.5] ⭐ 28
· 1.3 Loading a program into memory [3.7] 28
· 1.4 Memory Layout of a C Program [3.8] 28
2. Process State（進程狀態） ⭐ ⭐ [3.9] 28
3. Process Control Block (PCB) ⭐ [3.10, 3.13] 29
4. Threads（簡介，詳見 Ch4）[3.11–3.12] 29
5. Process Scheduling（進程排程） ⭐ [3.14–3.18] 29
· 5.1 基本 29
· 5.2 Scheduling Queues [3.15–3.16] 30
· 5.3 Schedulers ⭐ ⭐ [3.17–3.18] 30
6. Context Switch（上下文切換） ⭐ ⭐ [3.19–3.20] 30
7. Process Creation（建立進程） ⭐ ⭐ ⭐ [3.32–3.46] 31
· 7.1 概念 31
· 7.2 fork() ⭐ ⭐ ⭐ [3.35–3.39] 31
· 7.3 Copy-on-Write (COW) & clone() [3.46] 31
· 7.4 exec / wait / exit / signal [3.41–3.43] 32
8. Process Termination（終止） ⭐ ⭐ [3.47–3.50] 32
· 8.1 Zombie ⭐ ⭐ 32
· 8.2 Orphan ⭐ 32
9. fork() 解題大法 ⭐ ⭐ ⭐（考試必用） 32
· 9.1 基本規則 32
· 9.2 常見 pattern 公式（全部已用真 C 程式驗證 ✅） 33
· 9.3 有 break / return 嘅 loop（追蹤「仲 active」嘅 proces... 33
· 9.4 Short-circuit 題 33
· 9.5 Nested / combined 33
· 9.6 Possible output orderings（輸出次序題） ⭐ 34
· 9.7 fork + exec 成功/失敗情況（S26 Q3.2） ⭐ 35
· 9.8 更多歷屆 fork 題（Lab 3 review 提供：F22 / F24 / S25... 35
· 9.9 Buffering 陷阱 35
10. Interprocess Communication (IPC) ⭐ ⭐ [3.51–3.64] 35
· 10.1 Independent vs Cooperating 35
· 10.2 Producer–Consumer Problem（Bounded Buffer, Shar... 35
· 10.3 Message Passing [3.57–3.64] 35
11. Pipes ⭐ ⭐ ⭐ [3.65–3.69] 36
· 11.1 設計問題（4 個） 36
· 11.2 Ordinary Pipes vs Named Pipes（必背表） 36
· 11.3 Pipe example（Parent → Child）[3.68] 37
· 11.4 dup / dup2 & I/O redirection（Lab 3 / PA1） ⭐ ⭐ 37
12. Client–Server Communication：Sockets & RPC ⭐ [3.7... 37
13. 常見陷阱 & MC 速查 37
14. Past-paper 題目（Ch3） 38
15. 自我測驗 38

4 · Ch4 Threads 39
1. Motivation & Overview（動機）[4.4–4.6] 39
2. Benefits（四大好處） ⭐ ⭐ [4.7] 40
3. Multicore Programming（多核心編程） ⭐ [4.8–4.11] 40
4. Amdahl's Law ⭐ ⭐ ⭐（計數題必考）[4.12–4.13] 41
5. Multithreaded Process & TCB ⭐ [4.14–4.18] 41
6. User Threads vs Kernel Threads ⭐ ⭐ [4.19–4.20] 42
7. Multithreading Models ⭐ ⭐ ⭐ [4.21–4.26] 42
8. Threading Issues ⭐ ⭐ [4.27–4.33] 43
9. OS Examples [4.34–4.38] 44
10. Pthreads API 速查（Lab 4 / Ch6 都用） 44
11. 常見陷阱 & MC 速查 45
12. Past-paper 題目（Ch4） 45
13. 自我測驗 45

5 · Ch5 CPU Scheduling 47
1. Basic Concepts（基本概念）[5.4–5.7] 47
· 1.1 CPU–I/O Burst Cycle 47
· 1.2 When does scheduling happen? ⭐ ⭐ [5.6] 47
· 1.3 Dispatcher [5.7] 47
2. Scheduling Metrics（排程指標）⭐ ⭐ ⭐ [5.8, 5.12] 47
· 2.1 公式（本科 convention，必背）⭐ 48
3. FCFS — First-Come, First-Served ⭐ [5.9–5.10] 48
4. RR — Round Robin ⭐ ⭐ ⭐ [5.11–5.16] 48
· 4.1 Time quantum 大細 [5.14, 5.16] ⭐ 48
· 4.2 FCFS vs RR ⭐（F19 Q3.2 考過）[5.15] 48
· 4.3 RR 做 Gantt chart 規則（考試用） 49
5. SJF / SRTF — Shortest Job First ⭐ ⭐ ⭐ [5.17–5.24] 49
· 5.1 SJF（non-preemptive） 49
· 5.2 SRTF — Shortest-Remaining-Time-First（preemptive ... 49
· 5.3 比較 [5.24] ⭐ 49
· 5.4 Exponential Averaging（預測下一個 CPU burst）⭐... 49
6. Priority Scheduling ⭐ ⭐ [5.25–5.27] 49
7. Multilevel Queue（多級佇列）⭐ [5.28–5.29] 50
8. MLFQ — Multilevel Feedback Queue ⭐ ⭐ ⭐（每年必... 50
· 8.1 Slide example 規則（考試照用）[5.31] ⭐ 51
· 8.2 Lecture example [5.32–5.34]（Q0 q=4, Q1 q=8, Q2 FC... 51
· 8.3 MLFQ advantages ⭐ ⭐（F19 Q3.3、F20 Q2.5 必背... 51
· 8.4 MLFQ 做題 checklist ✍️ 51
9. Thread Scheduling ⭐ ⭐ [5.36–5.37] 51
10. Multiple-Processor Scheduling ⭐ ⭐ [5.38–5.46] 52
· 10.1 AMP vs SMP [5.38–5.39] 52
· 10.2 Multicore & Multithreaded cores (CMT) ⭐ [5.40–5.44] 52
· 10.3 Processor Affinity ⭐ ⭐ [5.45] 52
· 10.4 Load Balancing ⭐ [5.46] 53
11. Real-Time CPU Scheduling ⭐ ⭐ ⭐ [5.47–5.52] 53
· 11.1 Hard vs Soft ⭐ 53
· 11.2 Periodic tasks [5.48] 53
· 11.3 Rate-Monotonic (RM) ⭐ ⭐ [5.49–5.50] 53
· 11.4 EDF — Earliest Deadline First ⭐ ⭐ [5.51–5.52] 54
· 11.5 RM / EDF 做題方法 ✍️ 54
12. Algorithm Evaluation [5.53–5.60] 55
13. Gantt Chart 萬用流程（考試步驟）✍️ ⭐ ⭐ ⭐ 55
· 13.1 Tie-breaking（寫 assumption！） 56
14. Past-paper 計算題全集（全部用程式驗證 ✅） 56
· 14.1 S26 Q4.1 MLFQ（Q0 RR4, Q1 RR8, Q2 FCFS；preem... 56
· 14.2 S26 Q4.3 Dynamic priority（larger = higher；waiting... 56
· 14.3 S26 Q2.3 自訂 policy（揀 f_i 最大） 56
· 14.4 F19 Q3.5 LUF (Least Used First)（揀用 CPU 最少嘅；t... 56
· 14.5 F19 Q3.6（P1 0/8, P2 7/13, P3 9/1, P4 11/5, P5 20/7, ... 57
· 14.6 F20 Q4.1（P1 0/9, P2 3/3, P3 8/2, P4 10/6, P5 16/8, P... 57
· 14.7 F20 Q4.2 Two-level feedback queue with aging（Q0 ... 57
· 14.8 F20 Q4.3 Priority + RR（smaller = higher；preempti... 57
· 14.9 Sp21 Q2.1（P1 0/2, P2 4/12, P3 6/6, P4 9/4, P5 13/5） 57
· 14.10 Sp21 Q2.2 MLFQ（Q0 q=2, Q1 q=4, Q2 FCFS） 57
· 14.11 HW2（P1 0/7, P2 1/3, P3 4/5, P4 10/2） 57
15. 常見陷阱 & MC 速查 57
16. 自我測驗 58

6 · Ch6 Synchronization 59
1. Background（背景）[6.4–6.8] 59
2. Race Condition & Critical-Section Problem ⭐ ⭐ ⭐ [6.9... 59
3. Synchronization Hardware ⭐ ⭐ [6.13–6.21] 60
4. Mutex Locks ⭐ ⭐ [6.22–6.23] 62
5. Semaphores ⭐ ⭐ ⭐ [6.24–6.28] 62
6. 做題方法 ✍️ ⭐ ⭐ ⭐ 63
7. 補充：Peterson（Lab 6）& classic problems（Ch7 預覽） 64
8. 常見陷阱 & MC 速查 65
9. Past-paper 題目（Ch6） 65
10. 自我測驗 65

L · Labs & PA1/PA2 66
L1. Lab 1 — Linux 環境 & Shell 指令 66
L2. Lab 2 — C in Linux、編譯流程、Ch1–3 Review 66
· L2.1 Compile pipeline（同 Ch2 linker/loader 掛鉤） 66
· L2.2 I/O Subsystem 補充（Lab 2 review slides） 66
· L2.3 Lab 2 fork 題（已驗證 ✅） 66
L3. Lab 3 — Linux Process & Pipe System Calls（⭐ ⭐ ⭐ ... 67
· L3.1 本 lab 用到嘅 system calls 67
· L3.2 Zombie / Orphan demo（lab 示範） 67
· L3.3 File operations（低階 vs stdio） 67
· L3.4 Pipes 67
· L3.5 exec family 67
· L3.6 dup / dup2（redirection 核心） 68
· L3.7 Lab 3 fork 例題 + 更多歷屆 fork 題（全部驗證 ✅） 68
· L3.8 PA1 — Simplified Linux Shell（考試可能問） 68
L4. Lab 4 — Named Pipes & Pthreads 69
· L4.1 Anonymous vs Named pipes（lab 版本） 69
· L4.2 Exam example — 兩條 ordinary pipes 雙向通訊（填... 69
· L4.3 Pthreads 69
· L4.4 Exam example — thread vs process 共享 memory（... 70
L5. Lab 5 — Scheduling Review & PA2（MLFQ） 70
· L5.1 Review 例題（P1 7, P2 4, P3 1, P4 4，全部 t=0，已驗... 70
· L5.2 ⭐ Priority + RR 有唔同 arrival time（HW / Exam 熱... 70
· L5.3 RM / EDF 例題（deadline < period） 70
· L5.4 PA2 — N-level MLFQ simulator 70
L6. Lab 6 — Synchronization Review（Sample Questions） 71
· L6.1 Peterson's solution（software-only，2 processes） 71
· L6.2 POSIX primitives 速查 71
· L6.3 Sample Questions（官方答案，已驗證 ✅） 71
L7. 常見陷阱（Labs / PA） 72

Q · Past Papers Solved 73
Spring 2026 Midterm（lecturer 提供） 73
Fall 2019 Midterm 75
Fall 2020 Midterm 77
Spring 2021 Midterm 79
Homework 1（Fall 2026） 80
Homework 2（Fall 2026） 80
Fall 2022 Midterm（lab slides 引用） 81
Fall 2024 Midterm（lab slides 引用） 82
Spring 2025 Midterm（lab slides 引用） 82
Spring 2023 Midterm（lab slides 引用） 82
Lab 2 Exercises 82
Lab 4 Exam Example 83
Lab 6 Sample Questions（官方答案） 83

M · MC Answer Key 85
歷屆 MC 85
新練習 MC 90

A · C / Pthreads API 94
1. Process APIs 94
2. File I/O（POSIX，唔係 stdio） 95
3. Pipes 95
4. Pthreads（#include <pthread.h>，compile with -pthrea... 96
5. Compile / Run（Lab 1–2） 97

G · Glossary A-Z 98

I · Index 102

E Exam-Day Essentials 考場一頁通：公式・慣例・陷阱

E1. 必背公式卡 Formula Card

主題	公式	例
Average access time [1.32]	$\text{AMAT} = h\,T_{\text{hit}} + (1-h)\,T_{\text{miss}}$（若畀 penalty：$T_{\text{miss}} = T_{\text{hit}} + \text{penalty}$）	$0.9 \times 2 + 0.1 \times 50 =$ **6.8 ns**
Amdahl's Law [4.12]	$\text{speedup} \le \dfrac{1}{S + (1-S)/N}$；$N \to \infty$：$\dfrac{1}{S}$；$S$ = **serial**	$S = 0.25,\ N = 2 \Rightarrow$ **1.6**
Amdahl 反解	$S = \dfrac{1/X - 1/N}{1 - 1/N}$	$X = 3,\ N = 4 \Rightarrow S = 1/9$
Turnaround [5.8]	$\text{TAT} = C - A$（completion − arrival）	
Waiting [5.8]	$\text{WT} = \text{TAT} - \text{burst}$（= Gantt 上嘅 gaps 總和）	
Response（本科） [5.12]	$\text{RT} = (\text{end of FIRST run}) - A$（RR/MLFQ：第一個 quantum 完；non-preemptive：= TAT）	RR4：P1=4, P2=7, P3=10
Exponential averaging [5.19]	$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n$	$\alpha = .25,\ t_n = 6,\ \tau_n = 10 \Rightarrow$ **9**
Little's formula [5.57]	$n = \lambda \times W$	$\lambda = 3/\text{s},\ n = 12 \Rightarrow W =$ **4 s**
RR bound [5.11]	冇 process 等多過 $(n-1)\,q$	
RT utilization [5.49]	$U = \sum_i \dfrac{t_i}{p_i}$；$U > 1 \Rightarrow$ **一定 miss**	$25/50 + 35/80 = 0.94$
RM bound（課本）	保證可排：$U \le N\left(2^{1/N} - 1\right)$：N=1 1.000、2 0.828、3 0.780、4 0.757、5 0.743、∞ 0.693	
EDF [5.52]	$U \le 1$（$D = T$ 時）理論上可排	
fork 數目	n 個無條件 fork → 2^n；`if(fork()) fork();` 每輪 ×3 → 3^n	見 Playbook P1
Bounded buffer (in/out) [3.54]	最多 **BUFFER_SIZE − 1** items	
Semaphore（blocking） [6.26]	$\text{value} < 0 \Rightarrow \lvert\text{value}\rvert$ = 等緊嘅 processes 數	
Storage units [1.28]	KB $= 2^{10}$ B；MB $= 2^{20}$；GB $= 2^{30}$；networks 用 **bits**	

E2. 本科 Conventions（考試照用，唔確定就寫 assumption）

1. **Response time = completion of the first run − arrival**（slide 5.12；S26 官方答案照用）。
2. **RR**：quantum 用完 → **tail**；同一時刻有新 arrival → **新 arrival 先入 queue**。
3. **MLFQ**：新 job → **Q0 tail**；用晒 quantum 仍未完 → **降一級 tail**；**被高 queue 新 arrival preempt → 返自己 queue 嘅 HEAD，保留剩餘 quantum（Q1）/ 剩餘 burst（Q2），唔會降級**；Q0 job 唔會被 preempt；Q1 job 唔會 preempt Q2 job。
4. **Priority**：**細數字 = 高 priority**（除非題目話 larger = higher，例如 S26 dynamic priority）。
5. **Ties**：題目有講就跟（S26：smaller ID；dynamic priority：larger ID）；冇講：FCFS（早到先）→ 細 ID；**SRTF / EDF 平手 → 唔 preempt running process**。
6. **Soft real-time**：miss 咗 deadline 嘅 job **繼續跑完**，但計一次 miss。
7. **fork 題**：除非另有說明，假設 **printf 即時 flush**、**fork 全部成功**。
8. **Gantt chart**：每段寫 **start / process / end**；idle 都要畫；MLFQ 標 queue（P1@Q0）。

E3. 最易中伏 Top Traps（MC 前睇一次）

✗ 錯誤講法	✓ 正確
Main memory is nonvolatile / registers slower than memory	volatile；registers 最快；**main memory byte addressable**
von Neumann：instructions 同 data 分開存	**same memory**（stored-program）
Speedup with N CPUs = N	**< N**（contention / overhead）
NUMA：所有 memory 同樣 latency	**remote memory slower**
VM 一定比 native 快	minor slowdown
printf() 係 system call	printf **calls write()** system call
API 比 system call 快 / 可以 bypass protection	優點 = **portability + hide details（convenient）**
Microkernel performance 好過 monolithic	**worse**（message passing）
Layered：任何 layer 可 call 任何 layer	嗰個係 **LKM**；layered 只 call 下層
LKM 要 recompile kernel + reboot	可以 **run time load/remove**

✗ 錯誤講法	✓ 正確
Policy = how	**Policy = what；mechanism = how**
Interrupts 全部同一 priority / 只由 I/O 產生	有 **priority levels**；**trap = software interrupt**
DMA 每個 byte interrupt 一次	**block 完先 interrupt 一次**
Mode bit 1 = kernel	**1 = user，0 = kernel**
Context switch = user→kernel mode switch	context switch = **save/restore process state**；越複雜 OS 越慢
fork() 令 parent 等 child	要 **wait()**
child 改 global variable → parent 都變	各自 **copy**
exec() 成功會 return	**唔會 return**
Zombie 仲行緊 code / 冇 PID	terminated，**PID + PCB 仍在**，等 parent wait()
Ordinary pipe 任何 process 用到	**parent–child only**、單向、用完消失
Named pipe 用完自動消失	**explicitly delete**；UNIX FIFO **half-duplex**
Threads 有自己 address space / files	**share** code、data、heap、files；**own** PC、registers、**stack**
Concurrency = 同時執行	同時 = **parallelism**；concurrency 單核都得
Many-to-many 易 implement / user thread map 去多個 kernel threads	**difficult**；冇 one-to-many
所有 signals deliver 畀所有 threads	sync → causing thread；部分 async → all
Asynchronous cancellation = 定期 check	嗰個係 **deferred**（Pthreads default）
FCFS 可以 preemptive	**FCFS must be non-preemptive**
RR 一定比 FCFS 好	同長度 jobs 時 **TAT 更差**
q 越大 TAT 越好	唔一定；q 太大 → **FCFS**
MLFQ 要知 burst / 永不 starve	**唔使知**；可能 starve（要 aging/reshuffle）
Multilevel queue 可搬 queue	只有 **MLFQ** 可以
Multicore 簡化 scheduling	**complicates**（faster + less power）
Hard RT 遲咗都得 / RT scheduler 唔使 preemption	遲 = 冇服務；要 **priority + preemption**
EDF 要 periodic	**唔需要**；只需 announce deadline
Little's formula 只適用 FCFS	**any algorithm**
"Synchronization" 係 CS 三條件之一	**ME + progress + bounded waiting**
test_and_set 返回新值 / 會 release lock	返回 **old value**；release = `lock = false`
Binary semaphore = mutex	可當 mutex，但**更通用**（init 0 做 ordering）

E4. 答題模板 Answer Templates

Short question（2–4 分）：*[Definition sentence with keywords]. [Why / how — one sentence]. [Example or advantage/disadvantage].*

Gantt + averages：

```
Gantt:  0 P1 4 P2 8 P3 12 ...          (每個 boundary 都寫)
P | Arr | Burst | Comp | TAT=C-A | WT=TAT-B | RT=end1-A
avg WT  = (w1 + w2 + ... + wn) / n = x.xx
avg TAT = (t1 + ... + tn) / n = x.xx
Assumptions: ties -> ..., new arrival enters queue before preempted process.
```

fork 題：*Draw tree → count per level → formula → final number*；output 題：*constraints (wait / same process order) → list all interleavings*。

Race values：*Case 1 (branch ...): interleaving ... → c = ...* 逐個列。

Semaphore 填空：用 init-0 semaphore：**先做者 `signal(s)`，後做者開頭 `wait(s)`**；冇需要嘅 blank 寫 `/* Nothing */`。

P Playbooks 解題流程 + 預計表

P1. fork() Playbook

Step 1 — 規則

- fork() 返回：**child 0 / parent child-PID (>0) / fail −1（只有 parent，冇 child）**。
- Child 由 **fork() 下一個 operation** 開始，複製晒 variables（包括 loop 嘅 `i`）。
- 短路：`A && B` → A 係 0（child）唔做 B；`A || B` → A 非 0（parent）唔做 B。
- `return` / `exit` / `break` 會停止該 process 嘅 loop；`exec` 成功 = 之後 code 唔執行。

Step 2 — Pattern 表（每個 iteration、所有 process 都繼續 loop）

每 iteration	每 process 變	n 次後	例
`fork();`	×2	2^n	n=3 → 8；n=10 → 1024
`fork(); fork();`	×4	4^n	n=3 → 64
`if (fork()) fork();`	×3	3^n	n=3 → 27
`if (fork() == 0) fork();`	×3	3^n	n=3 → 27（Sp21 MC8）
`fork() && fork();`	×3	3^n	parent 做兩次
`fork() || fork();`	×3	3^n	child 做第二次
`if (fork() == fork()) printf`	×4	4^n；lines $(4^n-1)/3$	F22 Q3
`if (fork()) {fork(); fork();} else fork();`	×6	6^n	Lab 3 Ex3
`if (fork()) fork(); else break;`	active ×2	$2^{n+1}-1$ total	breaks 2^n-1（S26 Q3.1）
`if (fork()) printf; else printf;`	lines 2^i per iter	$2^{n+1}-2$ lines	F19 Q2.6
`pid = fork(); if (!pid) for (j<N) fork();`（M 輪）	$\times(2^N+1)$	$(2^N+1)^M$	F24 P3.1
`if (fork() == 0) break;`（fan）	+1 child	$n+1$	只 parent 繼續
`if (fork() > 0) break;`（chain）	+1 child	$n+1$	只 child 繼續

Step 3 — 其他已驗證答案

Code	答案
`if (fork() || fork()) fork();`	**5** processes
`if (fork() && fork()) fork();`	**4**（P0: T&&T → fork → 2；C1 false → 1；C2 false → 1）
`fork(); fork() && fork(); fork();`	**12**
`pid1=fork(); pid2=fork(); if(pid1>0) if(pid2==0) fork();`	**5**
`if (fork()==fork() || fork()!=fork()) fork(); printf`	**23** lines
S26 MC10：loop 3：`if(fork()==0){fork(); printf X; return;}` 之後 printf Y	**7 processes, X = 6, Y = 1**
S26 Q3.1：`if(fork()) fork(); else break;`	$2^{n+1}-1$ processes, 2^n-1 breaks（n=2：7, 3）
F20 Q3.2：`if(!fork()){loop n: if(fork()) fork();} fork(); printf a`	$2(3^n+1)$
Sp21 Q1.2：`if(fork()==0){loop 5: if(fork()==0) fork();} fork();`	$(3^5+1)\times 2 = 488$
F19 Q2.6：loop n，parent/child 都 printf	$2^{n+1}-2$ lines
Slide 3.44：loop 10 `fork(); fork();`	$4^{10} = 1{,}048{,}576$
Slide 3.45：loop 10 `if (fork()) fork();`	$3^{10} = 59{,}049$
Lab 3 Ex3：loop n `if(fork()){fork();fork();} else fork();`	每輪 ×6 → 6^n
F22 Q3：loop n `if (fork() == fork()) printf`	processes 4^n；lines $(4^n-1)/3$（1, 5, 21）
F24 P3.1：loop M `pid=fork(); if(!pid) for(j<N) fork();`	$(2^N+1)^M$（line Y 由 **child** 執行；M=N=1 → 3）
S25 MC12：`if (fork() || fork()) { if (fork() && !fork()) fork(); }`	**9** processes
Lab 2：`pid=fork(); if(pid==0){cnt+=10; pid=fork(); if(pid){cnt+=10; pid=fork();}}` 印 pid、cnt	非 0 pid × **2**；'0' × **3**；'10' × **1**

Step 4 — Process tree：每個 node 寫 PID 名（P0, P1…）同佢喺邊個 iteration 出世；child 繼承 parent 嘅 `i` 值繼續。

Step 5 — Output ordering：

1. 同一 process 內嘅 print 次序固定。
2. `wait()`：parent 之後嘅 print > 所有 child（及其子孫，如 child 等咗佢哋）嘅 print。
3. fork 之前印嘅（已 flush）最先。

4. 列所有符合約束嘅排列（topological orders）；可以用「插空位」數：k 個自由字元插入固定序列。

Step 6 — exec 情況：(I) fork ✓ exec ✓：child exec 前嘅 print + parent 嘅 print；(II) fork ✓ exec ✗：再加 exec 後嘅 print；(III) fork ✗：只有 `ret < 0` branch。

P2. Pipe / dup / exec 填空 Playbook

要做	Code
建 pipe（**fork 之前**）	`int fd[2]; pipe(fd);` → **fd[0] read，fd[1] write**
Writer	`close(fd[0]); write(fd[1], &x, sizeof(x)); close(fd[1]);`
Reader	`close(fd[1]); read(fd[0], &x, sizeof(x)); close(fd[0]);`
stdin ← file	`int fin = open(name, O_RDONLY); close(0); dup(fin);`（或 `dup2(fin, 0)`）
stdout → file	`int fo = open(name, O_WRONLY|O_CREAT|O_TRUNC, 0644); dup2(fo, 1); close(fo);`
stdout → pipe	`dup2(fd[1], 1); close(fd[0]); close(fd[1]); execlp(...)`
stdin ← pipe	`dup2(fd[0], 0); close(fd[0]); close(fd[1]); execlp(...)`
等指定 child	`waitpid(pid, &status, 0);`
exec 一個 program	`execlp("ls", "ls", "-l", (char *)NULL);` / `execvp(argv[0], argv);`（argv 尾要 NULL）
數偶數（S26）	`if (number % 2 == 0) cnt++;`

P3. CPU Scheduling Playbook

萬用步驟：(1) 抄表；(2) 由 t=0 起逐個 event（arrival / completion / quantum expiry / check point）；(3) 每個 event 寫 ready queue 內容；(4) 照算法揀；(5) 畫 Gantt（每個 boundary）；(6) C → TAT → WT → RT；(7) 平均連 working；(8) 檢查：最後完成時間 $= \sum \text{burst} + \text{idle}$。

Algorithm	揀邊個	幾時重新揀	注意
FCFS	最早到	完成時	non-preemptive；convoy effect
SJF	**已到達**嘅最短 burst	完成時	一旦開始跑到完
SRTF	最短 **remaining**	完成 + **每個 arrival**	tie → keep running
RR (q)	queue head	完成 / quantum 用完	新 arrival 先入 queue，再放被 preempt 嘅
Priority NP	最高 priority（細數字）	完成時	tie → FCFS
Priority P	最高 priority	完成 + 高 priority arrival	
Priority + RR	最高 priority；同級 RR	完成 / quantum / 高 priority arrival	同級得一個就繼續跑
MLFQ	最高非空 queue 嘅 head	完成 / quantum / 高層 arrival	preempted → **head + 剩餘 quantum**
Dynamic priority	計 $\text{init} + \alpha \cdot \text{waited} + \beta \cdot \text{ran}$	每 check interval / 完成	tie 跟題目
LUF（F19）	用 CPU 最少	完成 + arrival	descheduled → tail
RM	period 最短	每個 release / 完成	static
EDF	absolute deadline 最早	每個 release / 完成	dynamic；tie → keep running

MLFQ 追蹤表（考試用 rough paper 畫）：

```
t   | event           | Q0       | Q1 (rem q)   | Q2      | run (queue, q left, burst left)
0   | P1 arrives      | P1       |              |         | P1@Q0 q=4 b=9
4   | P1 q used       | P2       | P1(8)        |         | P2@Q0 ...
```

RM / EDF 追蹤：列 release times：task i 喺 $0, p_i, 2p_i, \ldots$ release；deadline $=$ release $+ d_i$。逐個 time unit 揀；記低每個 job 完成時間，比較 deadline。

P4. Synchronization Playbook

Race 可能值（F19 Q4.3 / F20 Q5 type）

1. 將 T1 拆做 atomic steps：**read for condition** → **read b** → **read a** → **write c**（`c = b + a` 由左至右讀）。
2. T2 嘅 writes（次序固定）可以插入任何 gap。
3. 分 branch（condition read 前 / 後寫）→ 每個 read 見新值 / 舊值 → 計 c。
4. 列晒所有 **distinct** 值 + 每個值嘅 interleaving。

Semaphore 次序：每條 dependency「X 要喺 Y 前」→ semaphore s = 0：**X 尾 `signal(s)`、Y 頭 `wait(s)`**；多個前置 → 多個 `wait`。

Semaphore 序列可行性：逐字追蹤每個 semaphore 嘅值同每個 process 做到邊；任何 `wait` 時值必須 > 0；有 loop 嘅 process 可以重複。

Spinlock 任何 atomic instruction：`while (instr(...) 返回「攞唔到」) ;` + release 寫返 free 值：

Instruction（return）	acquire	release
`test_and_set(&lock)`（old）	`while (test_and_set(&lock)) ;`	`lock = false;`
`compare_and_swap(&lock,0,1)`（old）	`while (compare_and_swap(&lock,0,1) != 0) ;`	`lock = 0;`
`swap(locked,1)`（old）	`while (swap(locked,1) == 1) ;`	`locked = 0;`
`compare_and_reset(&v,0,1)`（1 = 成功）	`while (compare_and_reset(&v,0,1) == 0) ;`	`v = 0;`

Bounded-waiting TAS 檢查點：(1) **cyclic scan** `j=(i+1)%n … j!=i` ；(2) **冇人等就** `lock = false` ；(3) 有人等就 `waiting[j] = false`（直接交 lock）；三個性質：ME（只有第一個 TAS 得 key=false / 只放一個 waiting）、progress（exit 一定放 lock 或者叫醒一個）、bounded waiting（≤ n−1 turns）。

P5. 預先計好嘅表 Pre-computed Tables

Amdahl speedup $= \dfrac{1}{S + (1-S)/N}$（S = serial fraction；parallel $= 1 - S$）

$S \backslash N$	2	3	4	5	6	7	8	10	16	32	∞
0.05 (par 95%)	1.90	2.73	3.48	4.17	4.80	5.38	5.93	6.90	9.14	12.55	**20.00**
0.10 (par 90%)	1.82	2.50	3.08	3.57	4.00	4.38	4.71	5.26	6.40	7.80	**10.00**
0.15 (par 85%)	1.74	2.31	2.76	3.13	3.43	3.68	3.90	4.26	4.92	5.66	**6.67**
0.19 (par 81%)	1.68	2.17	2.55	2.84	3.08	3.27	3.43	3.69	4.16	4.64	**5.26**
0.20 (par 80%)	1.67	2.14	2.50	2.78	3.00	3.18	3.33	3.57	4.00	4.44	**5.00**
0.25 (par 75%)	1.60	2.00	2.29	2.50	2.67	2.80	2.91	3.08	3.37	3.66	**4.00**
0.30 (par 70%)	1.54	1.88	2.11	2.27	2.40	2.50	2.58	2.70	2.91	3.11	**3.33**
0.35 (par 65%)	1.48	1.76	1.95	2.08	2.18	2.26	2.32	2.41	2.56	2.70	**2.86**
0.39 (par 61%)	1.44	1.69	1.84	1.95	2.03	2.10	2.14	2.22	2.34	2.44	**2.56**
0.40 (par 60%)	1.43	1.67	1.82	1.92	2.00	2.06	2.11	2.17	2.29	2.39	**2.50**
0.45 (par 55%)	1.38	1.58	1.70	1.79	1.85	1.89	1.93	1.98	2.06	2.14	**2.22**
0.50 (par 50%)	1.33	1.50	1.60	1.67	1.71	1.75	1.78	1.82	1.88	1.94	**2.00**
0.60 (par 40%)	1.25	1.36	1.43	1.47	1.50	1.52	1.54	1.56	1.60	1.63	**1.67**
0.70 (par 30%)	1.18	1.25	1.29	1.32	1.33	1.35	1.36	1.37	1.39	1.41	**1.43**
0.80 (par 20%)	1.11	1.15	1.18	1.19	1.20	1.21	1.21	1.22	1.23	1.24	**1.25**

Powers（fork 用）

n	2^n	3^n	4^n	6^n	$2^{n+1}-1$	2^n-1	$2^{n+1}-2$	$2(3^n+1)$	$(4^n-1)/3$
1	2	3	4	6	3	1	2	8	1
2	4	9	16	36	7	3	6	20	5
3	8	27	64	216	15	7	14	56	21
4	16	81	256	1296	31	15	30	164	85
5	32	243	1024	7776	63	31	62	488	341
6	64	729	4096	46656	127	63	126	1460	1365
7	128	2187	16384	279936	255	127	254	4376	5461
8	256	6561	65536	1679616	511	255	510	13124	21845
9	512	19683	262144	10077696	1023	511	1022	39368	87381
10	1024	59049	1048576	60466176	2047	1023	2046	118100	349525

RM utilization bound $N\left(2^{1/N}-1\right)$

N	bound	N	bound
1	1.000	7	0.729
2	0.828	8	0.724
3	0.780	9	0.721
4	0.757	10	0.718
5	0.743	∞	0.693 ($\ln 2$)
6	0.735		

Exponential averaging（$\alpha = 1/2$）**權重**：t_n ½、t_{n-1} ¼、t_{n-2} ⅛、t_{n-3} 1/16、… τ_0 權重 $(1/2)^{n+1}$。$\alpha = 0.25$：t_n 0.25、t_{n-1} 0.1875、t_{n-2} 0.1406、… τ_0 權重 0.75^{n+1}。

1 Ch1 Introduction 引言

📌 **考試範圍提示**：Ch1 主要出 **MC（2–4 題）** 同 **short question（定義題）**。最常考：**caching / locality / average access time**、**von Neumann**、**storage hierarchy**、**SMP / multicore / NUMA**、**multiprogramming vs timesharing**、**virtualization / cloud**。 ⚠️ 留意：今個 term（Fall 2026）嘅 slides 將 **interrupt / DMA / dual-mode** 放咗喺 **Ch3 slides 3.21–3.31**（舊 paper 寫 "1.40" 係舊版 slide number）。呢部分喺 **Ch3 notes §0**，但 MC 一樣會考。

1. What is an Operating System?（OS 係乜？）[1.20–1.24]

1.1 Computer system 四個組成部分 [1.20]

Component	解釋
Users	人、機器、其他電腦 / devices
Application programs	定義點樣用 system resources 去解決問題（editors, compilers, web browsers, database, games）
Operating system	**controls and coordinates use of computing resources among various applications and users**
Hardware	basic computing resources：**CPU, memory, I/O devices**

1.2 OS 嘅定義 [1.21, 1.23]

- **OS is a system software that acts as an intermediary between users (or application software) and computer hardware.**（OS 係用戶/應用程式同硬件之間嘅中介）
- OS **hides the complicated details of the hardware** 並提供 **simple and easy-to-use interface**（例如你處理 photos、web pages、files，而唔係 bytes 同 blocks）→ 呢個叫 **abstraction**。
- **There is no universally accepted definition of OS.** "Everything a vendor ships when you order an operating system" 只係一個 approximation。
- 兩個角色（必背）：
  - **OS is a resource allocator** — manages all resources (hardware and software); **decides between conflicting requests for efficient and fair resource use**.
  - **OS is a control program** — **controls execution of programs to prevent errors and improper use of the computer**.
- 一句總結：**"OS manages and controls hardware and helps to facilitate programs to run on computers."**

1.3 OS 嘅兩大目標（Chapter summary）

1. **User convenience** — provides an environment for users to execute programs in a **convenient, safe, protected and efficient** manner.
2. **Resource allocation** — allocates resources (hardware: CPU, memory; software: signals, locks) **in a fair and efficient manner**.

📝 **HW1 MC3 例子**：「幾個 job 喺 shared computer 等運行，OS 決定邊個 job 用 CPU、memory 點分配」→ **OS acts as a resource allocator**。

1.4 唔同環境嘅 OS [1.22]

環境	重點
Shared computers（mainframe / minicomputer）	要令所有 user 滿意 → **performance vs. fairness**
Individual systems（workstations）	dedicated resources，重 **performance** 多過 fairness
Mobile devices	**resource constrained**；touch screen、voice（Siri）；optimized for **usability and battery life**
Embedded systems	好少或無 UI；OS **pre-installed in ROM**（冷氣機、汽車、太空船）
Real-time systems	**strict time constraints**

1.5 常見名詞 [1.24]

- **Kernel** — **"the one program running at all times on the computer"**（核心功能）。
- **Middleware** — a set of software frameworks providing **additional services to application developers**（databases, multimedia, graphics）；mobile OS（iOS, Android）好常見。
- **System programs** — ships with the OS but **not part of the kernel**（例如 compilers）。
- **Application programs** — 同 OS 無關嘅程式（App Store 啲 app）。
- **OS = always-running kernel + middleware frameworks + system programs.**

2. Computer System Organization（電腦系統組織）[1.25–1.27]

- **One or more CPU cores** 同 **device controllers** 經 **common bus** 連接，**share memory**。[1.25]
- **CPUs and devices execute concurrently，compete for memory cycles through the shared bus**。

- 每個 **device controller** 管一類 device，有自己嘅 **local buffer**；OS 用 **device driver** 同 controller 溝通（device driver = kernel module that controls a device）。
- [1.27] 圖：processor + cache ↔ bridge/memory controller ↔ memory；**PCI bus** 連 graphics controller、SCSI controller、IDE disk controller、expansion bus interface（keyboard, parallel/serial port）。

2.1 von Neumann Architecture [1.26] ⭐ （MC 常客）

- **Stored-program architecture**：**instructions and data are stored in the same memory**（同一個 memory space，用 address 存取）。
- **CPU** contains an **ALU (arithmetic logic unit)** and **processor registers** — **PC (Program Counter)**, **AC (Accumulator)**, **MAR (Memory Address Register)**, **MDR (Memory Data Register)**。
- **Control unit** — **IR (Instruction Register)** and **PC**。
- **Memory** stores data and instructions（along with caches）；**external mass storage** = secondary storage；**input and output mechanisms**。
- **Instruction execution steps（instruction–execution cycle）**：
  1. **Fetch instruction**（from memory/cache → 放入 IR）
  2. **Decode instruction**
  3. **Fetch data**（operands → data registers）
  4. **Execute instruction**
  5. **Write back**（if any，結果寫返 memory/cache）
- **Programs and data must be brought into memory before execution.**
- **MMU (memory management unit) sees only a stream of memory addresses** — 佢唔知 address 點產生、亦唔知係 instruction 定 data。

> ⚠️ **陷阱**：
> - "Instructions and data are stored in **separate** physical memories" ✗（嗰個係 Harvard architecture）。
> - "PC stored in external mass storage" ✗；"CPU has no ALU" ✗。
> - Sp21 MC：「processing unit contains a/an ___ and processor registers」→ **ALU**（唔係 "all of the mentioned"，因為 IR/PC 屬 control unit）。

---

3. Storage（儲存） [1.28–1.35]

3.1 單位 [1.28]

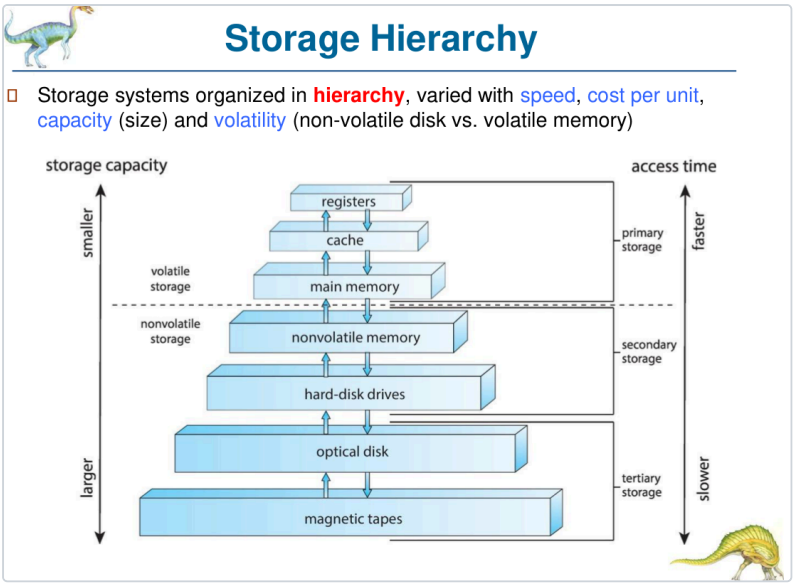
- **bit**（0/1）；**byte = 8 bits**（大部分電腦最細嘅 convenient chunk，冇 move 1 bit 嘅 instruction，但有 move 1 byte）。
- **word** = 一個 architecture 嘅 **native unit of data**（64-bit registers → 64-bit (8-byte) word）。
- **KB** = 2^{10} = **1,024 bytes**；**MB** = 1024^2 **bytes**；**GB** = 1024^3；**TB** = 1024^4；**PB** = 1024^5。
- 廠商通常 round off（MB ≈ 1 million bytes）；**networking measured in bits**（Mb/s, Gb/s）。

3.2 Storage Hierarchy [1.29] ⭐

按 **speed, cost per unit, capacity (size), volatility** 分層：

```
          registers        ┐
          cache            │ primary storage   (volatile)
          main memory      ┘
- - - - - - - - - - - - - - - - - - - - - - - - - - - - -
          nonvolatile memory (NVM) ┐ secondary storage (nonvolatile)
          hard-disk drives (HDD)   ┘
          optical disk     ┐ tertiary storage
          magnetic tapes   ┘
 上面：smaller, faster, more expensive per byte
 下面：larger, slower, cheaper per byte
```

- 越近 CPU → **越快、越細、每 byte 越貴**；越落 → **cost per byte 減、access time 增、capacity 增**。



Storage hierarchy [1.29]

3.3 Main Memory [1.30]

- **Main memory is the only large storage media that the CPU can access directly.**（CPU 係 **indirectly** 存取 secondary storage）
- **Volatile**；通常係 **DRAM (Dynamic Random-Access Memory)**。
- **load / store** instructions to specific addresses；**byte addressable — each address refers to one byte**。⭐（S26 MC3 答案）
- 開機第一個程式 **bootstrap program** 存喺 **EEPROM**（firmware）。

3.4 Secondary Storage [1.31]

- **Extension of main memory providing large non-volatile storage capacity**。
- 最常見：**HDDs** 同 **NVM devices**（例如 SSD）。
- **Mechanical**（HDD, optical disk, magnetic tape）vs **Electrical**（flash, SSD, FRAM, NRAM → 統稱 **NVM**）。
- **Mechanical: larger and less expensive per byte. Electrical: more costly, smaller, more reliable, and faster.**

3.5 Characteristics of storage [1.34]（睇表記「Managed by」）

Level	**1 registers**	**2 cache**	**3 main memory**	**4 SSD**	**5 magnetic disk**
Typical size	< 1 KB	< 16 MB	< 64 GB	< 1 TB	< 10 TB
Technology	custom memory, multiple ports CMOS	on/off-chip CMOS SRAM	CMOS SRAM（slide 原文）	flash memory	magnetic disk
Access time (ns)	0.25–0.5	0.5–25	80–250	25,000–50,000	5,000,000
Bandwidth (MB/s)	20,000–100,000	5,000–10,000	1,000–5,000	500	20–150
Managed by	**compiler**	**hardware**	**operating system**	**OS**	**OS**
Backed by	cache	main memory	disk	disk	disk or tape

- **Movement between levels of storage hierarchy can be explicit or implicit.**（例如 cache ↔ memory 由 hardware 隱式處理；memory ↔ disk 由 OS 顯式處理）

3.6 Range of Timescales [1.35]（Jeff Dean "Numbers Everyone Should Know"）

L1 cache 0.5 ns；branch mispredict 5 ns；L2 cache 7 ns；mutex lock/unlock 25 ns；**main memory reference 100 ns**；compress 1KB 3 µs；send 2KB over 1 Gbps 20 µs；read 1MB from memory 250 µs；round trip in datacenter 500 µs；**disk seek 10 ms**；read 1MB from disk 20 ms；packet CA→Netherlands→CA 150 ms。

---

4. Caching ⭐⭐（必考 short question）[1.32–1.33]

4.1 Fundamental idea

> 📝 **Model answer**：*The fundamental idea of caching is to temporarily copy a subset of information from a slower but larger storage to a faster but smaller storage, so that frequently used data can be accessed more quickly.*（make frequently used case faster, less frequent case less dominant）

- Caching **performed at many levels**：memory、address translation、file blocks、file names、file directories、network routes...

- 存取時先 check cache：
  - **Hit** — information is in the cache → used directly (fast)
  - **Miss** — not in cache → **data copied from slower storage to cache** and used there
- **Cache is usually much smaller than the storage being cached.**
- **Cache management**：**cache size** 同 **replacement policy**。
- **Major criterion: cache hit ratio**（percentage of content found in cache）。

4.2 Average access time（公式必背）

> **Average Access Time（AMAT）**
>
> $$\text{AMAT} = (\text{Hit Ratio} \times \text{Hit Time}) + (\text{Miss Ratio} \times \text{Miss Time})$$

- $\text{Miss Ratio} = 1 - \text{Hit Ratio}$。
- **HW1 例**：hit ratio 90%、hit time 2 ns、miss time 50 ns → $0.9 \times 2 + 0.1 \times 50 = 1.8 + 5 = \mathbf{6.8\ ns}$。
- 如果題目畀 **miss penalty**（額外時間）而唔係 miss time：$T_{\text{miss}} = T_{\text{hit}} + \text{penalty}$ → $\text{AMAT} = T_{\text{hit}} + (1-h) \times \text{penalty}$（兩條公式結果一樣）。

> 📝 **S26 Q2.2 Model answer（high hit ratio 夠唔夠？）**：*No. Cache performance is not judged only by cache hit ratio. Although hit ratio is a major criterion, the overall performance should be evaluated using the average access time, which depends on both the hit time and the miss time (miss penalty). A cache with a high hit ratio but a long hit time or a huge miss penalty can still perform poorly.*

4.3 Locality — why caching works ⭐

- **Temporal locality (locality in time)** — **recently accessed items are likely to be accessed again** in the near future.
  - 例：loop counter `i`、loop 入面重複執行嘅 instructions、成日 call 嘅 function。
- **Spatial locality (locality in space)** — **items near (contiguous to) the recently accessed items are likely to be accessed shortly**（data 同 program 都係）。
  - 例：順序讀 array `a[0], a[1], a[2]…`；順序執行 instructions。
- **Without access locality（e.g. all items accessed with equal probability）, cache would never work!**
- F20 MC1：**"Locality is essential to ___" → Cache**。

---

5. Processor Systems（處理器系統）[1.36–1.41]

5.1 Single-Processor Systems [1.36]

- 一個 CPU，**single processing core**；core 執行 instructions，有 registers 存 data。
- 仲有 **special-purpose processors**（device-specific：disk controller、GPU）→ 跑 **limited instruction set**，**do not execute instructions from user processes**。

5.2 Multiprocessor Systems [1.37] ⭐

- 而家由 mobile 到 server 都係 multiprocessor 主導。
- **Speed-up ratio with N processors is less than N** — 因為 **overhead, e.g., contention for shared resources (bus or memory)**。
- **三大優點（必背）**：
  1. **Increased throughput** — more computing capability
  2. **Economy of scale** — share other devices（I/O devices, power supplies, housings, peripherals）→ 平過多部單 CPU 機
  3. **Increased reliability** — **graceful degradation or fault tolerance**
- 缺點（summary）：hardware design 同 software（**parallel programming**）都複雜好多。
- **兩類**：
  - **Asymmetric multiprocessing (AMP)** — **master–slave**：**master** assigns specific tasks to slaves and **handles I/O**。（Ch5：master 可以變 **bottleneck**）
  - **Symmetric multiprocessing (SMP)** — **each processor performs all tasks, including OS functions and user processes**；I/O 可以由任何 processor 處理。

5.3 SMP [1.38]

- **Each CPU has its own set of registers and a private (local) cache; all processors share physical memory through the system bus.**

5.4 Multicore [1.39] ⭐

- **Multiple computing cores reside on a single physical chip.**
- 優點：**faster on-chip communication than between-chip communication**；**uses significantly less power**（mobile/laptop 重要）。

📝 **F19 Q2.5**：*Multicore SMP systems are faster and consume less power than systems in which each CPU has its own physical chip (on-chip communication is faster).*

5.5 NUMA (Non-Uniform Memory Access) [1.40] ⭐

- 問題：加更多 CPU 未必 scale，因為 **contention for the system bus → bottleneck**。
- 解決：**each CPU (or group of CPUs) has its own local memory accessed via a small, fast local bus**；CPUs connected by a **shared system interconnect**；**all CPUs share one physical memory address space**。
- **Drawback: increased latency when a CPU must access remote memory across the system interconnect** → 影響 **scheduling and memory management**（Ch5 processor affinity）。

⚠️ HW1 MC4：「In NUMA, every CPU accesses every part of memory with exactly the same latency」→ **False**（Non-**uniform** 就係唔同 latency）。

5.6 名詞定義 [1.41]（MC 易混）

Term	Definition
CPU	The hardware that executes instructions
Processor	A physical chip that contains one or more CPUs
Core	The basic computation unit of the CPU（executes instructions + registers）
Multicore	Multiple computing cores on a single physical processor chip
Multiprocessor	Including multiple processors

---

6. Multiprogramming & Timesharing ⭐ [1.42]

- 所有 modern OS 都有兩個特徵：
  1. **Multiprogramming (batch system)** — needed for **efficiency**。
     - 以前一次 load 一個 program；**single program cannot keep CPU or I/O devices busy at all times**。
     - **Multiprogramming organizes jobs so that the CPU always has one to execute.**（一個 job 等 I/O → CPU 轉做另一個 job）
     - Mainframe：jobs submitted remotely and queued → **job scheduling** 揀邊啲 load 入 memory。
  2. **Timesharing (multitasking)** — **logical extension of multiprogramming in which the CPU switches jobs so frequently that users can interact with each job while it is running → interactive computing**。
     - **Response time should be under a certain threshold**（通常 < 1 秒）。
- 相關概念：
  - A program in execution → **process**
  - Several processes ready at the same time → **CPU scheduling**
  - Processes don't fit in memory → **swapping** moves them in and out
  - **Virtual memory** allows execution of processes **not completely in memory**

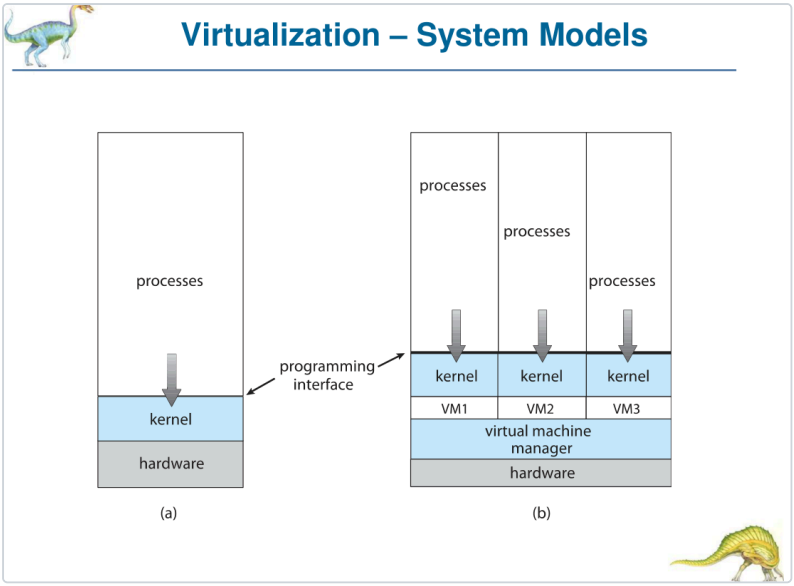
📝 **HW1 Q2.2 Model answer**：*Multiprogramming keeps several jobs in memory at the same time. When the running job has to wait (e.g., for I/O), the OS switches the CPU to another job, so the CPU always has something to execute and CPU utilization increases. Time sharing (multitasking) extends multiprogramming by switching the CPU among jobs so frequently (using CPU scheduling and timer interrupts) that each user gets a fast response time and can interact with the program while it is running — this supports interactive computing.*

---

7. Virtualization ⭐ [1.43–1.46]

- **Virtualization abstracts the hardware of a single computer into multiple different execution environments**，令每個 user/program 以為自己有部 **"private computer"**。
- 創造 **virtual machine (VM)**，OS 同 applications 可以喺上面跑。
- **It allows an operating system to run as an application within another operating system.** ⭐（F20 MC2 答案：**Virtualization**，唔係 Emulation）
- **Components** [1.44]：
  - **Host** — underlying hardware system
  - **Virtual machine manager (VMM) or hypervisor** — **creates and manages virtual machines** by providing an interface **identical to the host**
  - **Guest OS** — process provided with a **virtual copy of the host**（通常係一個 OS）
- **Motivations** [1.43]：
  - **Server consolidation** — 將多個 OS 合併到較少 hardware 上
  - Desktop 上跑一個 OS（Linux/macOS）但仍用到另一個平台（Windows）嘅 native apps
  - **Developers can run many OS types/versions on one machine** → easier testing and debugging

- [1.45] 圖：(a) processes → kernel → hardware；(b) 每個 VM 有自己 kernel，VM1/VM2/VM3 → **VMM** → hardware。



Virtualization system models [1.45]

- **History** [1.46]：IBM mainframes (1972)；VMware 喺 Intel x86 跑多個 Windows guest；**programs in a VM show only minor performance decreases**（多咗幾層 software）；**VMM is in complete control of system resources**；1990s 後期 Intel CPU 夠快；Xen、VMware。
- **Emulation vs Virtualization**（課本背景）：**Emulation** = 模擬 **different** CPU architecture（source CPU ≠ target CPU，慢）；**Virtualization** = guest OS **natively compiled for the CPU**。

> ⚠️ HW1 MC8：「An application always runs **faster** inside a VM」→ **False**（只會 minor performance decrease）。

8. Cloud Computing [1.47–1.48]

- **Delivers computing, storage, and apps as a service over a network.**
- **Logical extension of virtualization**（用 virtualization 作基礎）；例：Amazon EC2（pay based on usage）。
- **Types**：
  - **Public cloud** — available via Internet to anyone willing to pay
  - **Private cloud** — run by a company for its own use
  - **Hybrid cloud** — both public and private components
- **Service models**（MC 易考 match）：
  - **SaaS (Software as a Service)** — applications via Internet（e.g. word processor）
  - **PaaS (Platform as a Service)** — **software stack** ready for application use（e.g. **database server**）
  - **IaaS (Infrastructure as a Service)** — **servers or storage** over Internet（e.g. storage for backup）
  - **MaaS** — Machine learning as a Service

9. I/O Subsystem（Chapter 1 summary 提及）

- I/O 硬件三元素：**buses, device controllers, devices**。
- 搬 data（device ↔ main memory）：**CPU in programmed I/O** 或者 **offloaded to a DMA controller**。
- **Device driver** = kernel module that controls a device。
- **Kernel I/O subsystem services**：**I/O scheduling, buffering, caching, spooling, device reservation, error handling**。
  - **Buffering** — store data temporarily while it is being transferred；**Caching** — faster storage holding a **copy** of data（Lab 2）。⭐ **A buffer may hold the only existing copy of a data item, whereas a cache only holds a copy of an item that resides elsewhere.**
  - **Spooling** — hold output for a device that **cannot accept interleaved data**（e.g. **printer**），coordinating concurrent output。
- I/O hardware（Lab 2）：**port**（connection point）、**bus**（PCI/PCIe；expansion bus for slow devices）、**controller (host adapter)**；device registers：**data-in, data-out, status, control**。
- （Interrupt、DMA、dual-mode 詳細見 **Ch3 notes §0**（slides 3.21–3.31））

10. 常見陷阱 & MC 速查（Traps）

敘述	真/假	原因
Main memory is byte addressable	✓	each address refers to one byte [1.30]

敘述	真/假	原因
Main memory is nonvolatile	✗	DRAM 係 **volatile**
Registers are slower than main memory	✗	registers 最快
HDD is smaller than volatile memory	✗	HDD 大好多
CPU can directly access secondary storage	✗	只可 directly access main memory
Instructions and data stored in same memory (von Neumann)	✓	stored-program
Speed-up with N processors = N	✗	**< N** due to overhead/contention
Multicore uses less power than multiple single-core chips	✓	[1.39]
NUMA: same latency for all memory	✗	remote memory slower
SMP: each processor has private cache, shares memory	✓	[1.38]
AMP: master handles I/O	✓	[1.37]
Timesharing is an extension of multiprogramming	✓	[1.42]
VM always faster than native	✗	minor slowdown
Virtualization lets an OS run as an app within another OS	✓	[1.43]
Cache works even with uniform random access	✗	needs locality
High hit ratio alone guarantees performance	✗	also hit time & miss time
Cache is managed by OS	✗	**hardware**（[1.34]）；registers by **compiler**

11. Past-paper 題目（Ch1 相關）

1. **S26 MC3**：storage hierarchy 正確嘅係？→ **D. Main memory is byte addressable**。
2. **S26 MC4**：von Neumann 核心特徵？→ **B. Instructions and data stored in the same memory space, accessed by address**。
3. **S26 Q2.2**：caching 基本概念 + 高 hit ratio 夠唔夠？→ 見 §4。
4. **F19 Q1.2**：temporal vs spatial locality？→ 見 §4.3。
5. **F20 MC1**：Locality is essential to → **Cache**。
6. **F20 MC2**：___ allows a guest OS to run as an application on a native OS → **Virtualization**。
7. **Sp21 MC1**：von Neumann processing unit contains a/an ___ and processor registers → **ALU**。
8. **HW1 MC3** resource allocator；**MC4** NUMA same latency → false；**MC8** VM always faster → false；**Q2.1** AMAT = 6.8 ns；**Q2.2** multiprogramming vs timesharing。

12. 自我測驗（Self-test）

1. 寫出 average access time 公式；hit ratio 95%、hit time 1 ns、miss time 100 ns？（答：0.95 + 5 = **5.95 ns**）
2. 點解 N 個 processors speedup < N？（contention for shared resources e.g. bus/memory, overhead）
3. Multiprocessor 三個優點？（throughput, economy of scale, reliability/graceful degradation）
4. NUMA 嘅 drawback？（remote memory access latency）
5. VMM 又叫咩？做咩？（hypervisor; creates and manages VMs, interface identical to host）
6. Cache 由邊個 manage？Main memory 呢？（hardware；OS）

2 Ch2 OS Structures 作業系統結構

📌 **考試範圍提示**：Ch2 每年都有 **3–5 題 MC + 1–2 題 short question**。必考：**API vs system call（點解要分開）**、**printf() → write()**、**parameter passing 三種方法**、**linker vs loader**、**monolithic / layered / microkernel / LKM / hybrid 比較**（優缺點）、**policy vs mechanism**、**OS services**。 ⚠️ **Dual-mode / interrupt / trap** 今個 term 放咗喺 **Ch3 slides（3.x）**，見 Ch3 notes §0。

1. Operating System Services（OS 服務）[2.4–2.7]

OS provides an environment for **execution of programs** and **other services to programs and users**。分兩組：

1.1 對 user 有用嘅服務（helpful to the user）

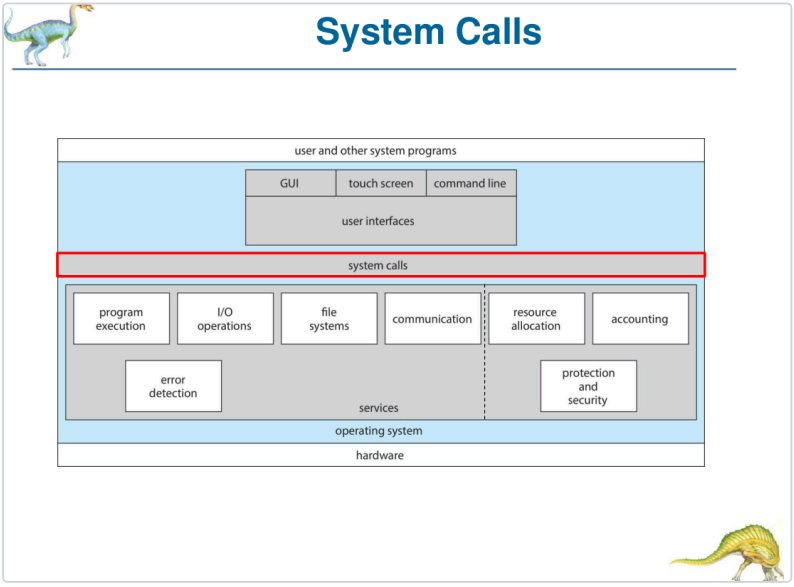
Service	解釋
User interface (UI)	**CLI (command-line)**, **GUI**, **touch-screen**
Program execution	load a program into memory and run it; end execution **normally or abnormally (indicating error)**
I/O operations	running program may require I/O（file 或 I/O device）
File-system manipulation	read/write/create/delete/search files & directories, list info, **permission management**
Communications	processes exchange information（同一部機或 network）via **shared memory** or **message passing (packets moved by the OS)**
Error detection	errors **may occur in CPU and memory, in I/O devices, in user programs**；OS takes appropriate actions；**debugging facilities**

1.2 確保系統本身有效運作（efficient operation via resource sharing）

Service	解釋
Resource allocation	多 user/job 同時跑 → 分配 **CPU cycles, memory, file storage, I/O devices**
Logging (accounting)	**keep track of which users use how much and what kinds of computer resources**
Protection and security	**Protection**: ensure all access to system resources is controlled；**Security**: defend from outsiders → **user authentication**, defend external I/O devices from invalid access

⚠️ **S26 MC5 陷阱**：「**Errors only occur in memory and I/O devices**」→ **False**（CPU、memory、I/O devices、user programs 都可以出錯）。 ✓「Logging service should keep track of computer resources that each user uses」→ True。

- Summary 補充：OS services 兩大類 —（1）**enforce protection among concurrent processes**（process 只可以存取自己 address space；唔可以直接存取 hardware）；（2）**provide functionalities not directly supported by hardware**（memory management、file systems）。
- [2.7/2.13 圖] user & system programs → user interfaces (GUI/touch/CLI) → **system calls** → services (program execution, I/O, file systems, communication, resource allocation, accounting, error detection, protection & security) → OS → hardware。



OS services and the system-call layer [2.13]

2. User Interfaces [2.8–2.12]

- **CLI / command interpreter** — allows **direct command entry**。
  - **Sometimes implemented in kernel, sometimes by system programs.**
  - 有多個 flavor → **shells**（C shell, **Bourne-Again shell (bash)**, Korn shell）。
  - **Shell 嘅主要功能：(1) fetches a command from user, (2) interprets it, (3) executes it.** ⭐（PA1 myshell 就係做呢樣）
- **GUI** — user-friendly **desktop metaphor**；icons；mouse；**invented at Xerox PARC (early 1970s)**，first widely used in **Apple Macintosh (1984)**。Windows = GUI + CLI "command" shell；Mac OS X = **"Aqua" GUI** + UNIX kernel；Unix/Linux = CLI + optional GUI（KDE, GNOME）。
- **Touchscreen** — gestures, virtual keyboard, voice command（Siri）。

---

3. System Calls ⭐ ⭐ [2.13–2.26]

3.1 定義

- **System calls provide the programming interface to the services provided by the OS.**（Sp21 MC：「___ provide an interface to the services provided by an OS」→ **System calls**）
- 通常以 **high-level language (C/C++) functions** 提供；**certain low-level tasks written in assembly** for accessing hardware。
- 例：`cp in.txt out.txt` 涉及**一連串 system calls**（acquire input file name、open input file、create output file、loop read/write、close、write completion message、terminate normally）。

3.2 API (Application Programming Interface) [2.15]

- **API specifies a set of functions available to application programmers, including the parameters passed to each function and the return values it may expect.**
- **The functions that make up an API invoke the actual system calls on behalf of the programmer.**
  - 例：Windows `CreateProcess()` invokes `NTCreateProcess()` system call。
- Programmer 經 OS 提供嘅 **library** 用 API；UNIX/Linux C → **libc**。
- **三大 API**：**Win32 API**（Windows）、**POSIX API**（UNIX, Linux, Mac OS X）、**Java API**（JVM）。
- [2.16] `ssize_t read(int fd, void *buf, size_t count)` — `fd` file descriptor；`buf` buffer；`count` max bytes；**return 0 = end of file；−1 = error**。

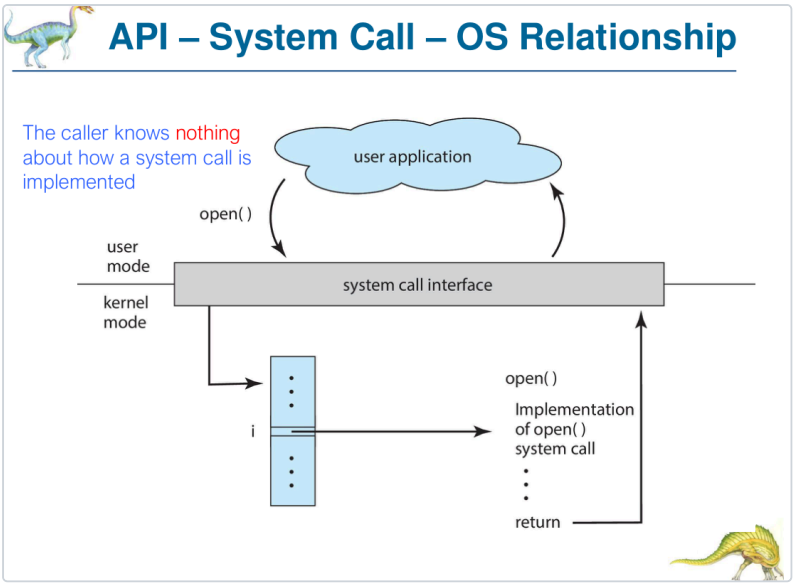
3.3 點解用 API 而唔直接 call system call？⭐（F19 Q1.3、S26 MC6）[2.17]

> 📝 **Model answer**：*(1) **Program portability** — a program using an API can compile and run on any system that supports the same API, even though the implementation of system calls varies from machine to machine. (2) **Hide the complex details of system calls** — actual system calls are often more detailed and difficult to work with; the caller only needs to obey the API and understand what the OS will do, not how the system call is implemented.*

- S26 MC6 答案：**APIs provide a more convenient programming interface**。
- ✗「APIs always execute faster than system calls」、✗「APIs can bypass kernel protection」、✗「APIs eliminate the need for linkers and loaders」。
- F20 MC4：「Programmers interface with ___ to obtain services from the OS」→ **APIs**（best answer）。

3.4 System Call Implementation [2.18–2.19]

- 大部分語言嘅 **run-time environment (RTE)**（a set of functions built into libraries）提供 **system-call interface**，作為連接 OS system calls 嘅 **link**。
- **Each system call is associated with an identity number**；system-call interface **maintains a table indexed by these numbers**。
- System-call interface 做三件事：**intercepts function calls in the API → invokes the necessary system call within the OS → returns the status of the system call and return value(s)**。
- **The caller knows nothing about how a system call is implemented.**（只需要 obey API）



API – system call – OS relationship **[2.19]**

- [2.19 圖]：user application 呼叫 `open()` → (user mode) **system call interface** → (kernel mode) 用 index *i* 查 table → implementation of `open()` → return。

> **HW1 Q2.4：user program call read() 點樣 user→kernel→user？（三步）**
>
> 1. *The program calls the `read()` API in libc (user mode); the library/system-call interface puts the system-call number and parameters in registers (or a block/stack) and executes a **trap (system-call) instruction**.*
> 2. *The trap switches the CPU to **kernel mode** (mode bit 1→0); the kernel uses the system-call number to index the **system-call table** and runs the `read()` service routine (which may block waiting for the device).*
> 3. *When finished, the kernel puts the return value/status in a register, executes a **return-from-trap** which resets the mode to **user mode**, and control returns to the instruction after the call.*
>
> - HW1 MC6 答案：**The API and run-time environment use the system-call interface to invoke the corresponding kernel service, and control later returns to the application.**

3.5 System Call Parameter Passing ⭐ [2.20–2.21]

三種方法（F20 MC5：**All of the above**）：

1. **Registers** — simplest；但可能 parameters 多過 registers。
2. **Block (table) method** — parameters stored in a **block/table in memory**, **address of the block passed in a register**（Linux & Solaris 用）。
3. **Stack method** — parameters **pushed onto the stack by the program and popped off by the OS**。

- **Block and stack methods do not limit the number or length of parameters.**

3.6 Types of System Calls（六類） ⭐ [2.22–2.25]

類別	例子	UNIX 例	Windows 例
Process control	create/terminate process, end/abort, load/execute, get/set attributes, wait for time, wait/signal event, allocate/free memory, dump memory, debugger, **locks**	`fork()`, `exit()`, `wait()`	`CreateProcess()`, `ExitProcess()`, `WaitForSingleObject()`
File management	create/delete, open/close, read/write/reposition, get/set attributes	`open()`, `read()`, `write()`, `close()`	`CreateFile()`, `ReadFile()`, `WriteFile()`, `CloseHandle()`
Device management	request/release device, read/write, get/set attributes, attach/detach	`ioctl()`, `read()`, `write()`	`SetConsoleMode()`, `ReadConsole()`, `WriteConsole()`
Information maintenance	get/set time/date, system data, attributes	`getpid()`, `alarm()`, `sleep()`	`GetCurrentProcessID()`, `SetTimer()`, `Sleep()`
Communications	create/delete connection, send/receive messages (**message passing**), **shared-memory** create/access regions	`pipe()`, `shm_open()`, `mmap()`	`CreatePipe()`, `CreateFileMapping()`, `MapViewOfFile()`
Protection	control access, get/set permissions, allow/deny user access	`chmod()`, `umask()`, `chown()`	`SetFileSecurity()`, …

3.7 Standard C Library Example ⭐（S26 MC8）[2.26]

- **C program invoking `printf()` library call, which calls the `write()` system call.**
- ✓ **"printf() invokes write() system call to get service from OS"**
- ✗「printf() and write() are both system calls」（printf 係 library function）
- ✗「program enters kernel mode immediately when printf() is called」（printf 喺 user mode 處理 formatting/buffering，到 write() 先 trap 入 kernel）
- ✗「write() invokes printf()」

---

4. System Programs（系統程式）[2.27–2.30]

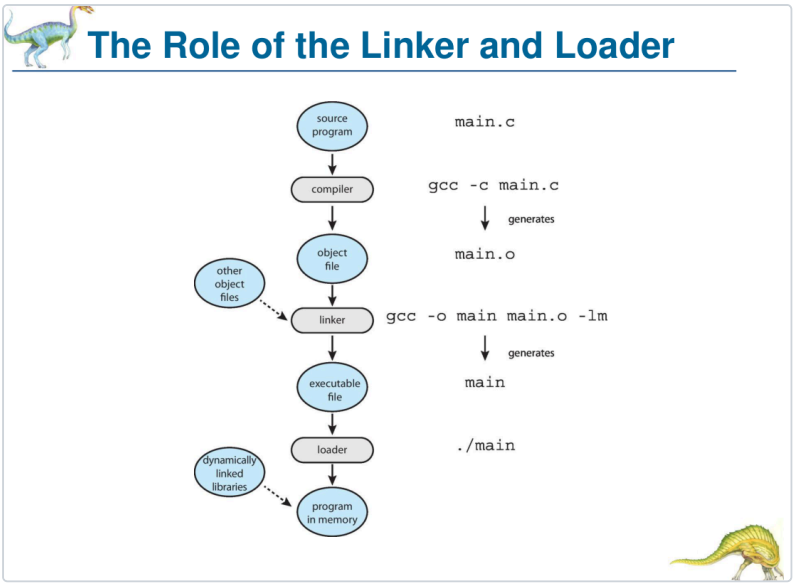
- **System services / system programs / system utilities provide a convenient environment for program development and execution.**
- 有啲只係 **user interfaces to system calls**；有啲複雜得多。
- **The view of the OS seen by most users is defined by the application and system programs, not the actual system calls.**（GUI 同 CLI 用同一套 system calls，但 user 睇落好唔同）
- **類別**：
  - **File management** — create, delete, copy, rename, print, list files/directories
  - **Status information** — date, time, available memory, disk space, number of users；performance, logging, debugging info
  - **File modification** — text editors；search/transform file contents
  - **Programming-language support** — compilers, assemblers, debuggers, interpreters
  - **Program loading and execution** — absolute loaders, relocatable loaders, linkage editors, overlay loaders, debugging systems
  - **Communications** — virtual connections among processes/users/systems（message, web, email, remote login, file transfer）
  - **Background services** — launch at boot；有啲跑到系統關機 → **services, subsystems, daemons**（disk checking, process scheduling, error logging）
- **Application programs** — **not part of the OS**（browsers, word processors, spreadsheets, databases, games）。

---

5. Linkers and Loaders ⭐⭐（F19 Q1.1、F20 Q2.2 都考過）[2.31–2.32]

```
main.c --(compiler: gcc -c main.c)--> main.o (relocatable object file)
main.o + other object files + libraries --(linker: gcc -o main main.o -lm)--> main (executable file)
main --(loader: ./main)--> program in memory  (+ dynamically linked libraries)
```

- **Source code is compiled into relocatable object files**（designed to be loaded into any physical memory location）。
- **Linker combines these relocatable object files into a single binary executable file**（along with libraries if needed, e.g. `-lm` math library）。
- **Programs reside on secondary storage as binary executables, which must be brought into memory by the loader to be executed.**
- **Relocation** assigns final addresses to program parts and adjusts code & data to match those addresses。
- Modern OS **don't link libraries statically**；而係用 **dynamically linked libraries (DLLs in Windows)** — **loaded when needed, shared by all programs using the same version (loaded only once)** → 慳 memory，避免 link 用唔到嘅 library。
- Object files / executables 有 **standard formats**（machine code + symbol table；Linux = **ELF**）→ OS 知點 load 同 start。



Role of the linker and loader [2.32]

> 📝 **Model answer**：*The **linker** combines several relocatable object files (generated by the compiler), together with any needed libraries, into a single binary executable file stored on disk. The **loader** loads the binary executable file into memory (possibly together with dynamically linked libraries) so that it can be run on a CPU.*

6. OS Design and Implementation [2.33–2.37]

6.1 Design goals [2.33]

- Design 受 **choice of hardware, type of system**（desktop, mobile, distributed, real-time）影響；先定 **goals and specifications**。
- **User goals**：convenient to use, easy to learn and use, reliable, safe, fast。
- **System goals**：easy to design, implement, maintain; flexible, reliable, error-free, efficient。

6.2 Policy vs Mechanism ⭐ [2.34]

- **Policy: What will be done?　Mechanism: How to do it?**
- 例：**A timer is a mechanism for ensuring CPU protection, but deciding how long the timer is set for a particular user is a policy decision.**
- **Separation of policy from mechanism allows flexibility if policy decisions are changed later**（同一個 mechanism 可以支持「I/O-intensive 優先」或者相反嘅 policy）。
- 總結：**goals → determine policies → implemented through mechanisms**。

6.3 Implementation [2.35]

- 早期用 **assembly**；而家多數用 **C/C++**；最底層 kernel 可能仍用 assembly。Android：system libraries C/C++，frameworks Java。
- **Advantages of high-level languages**：**easier to port to other hardware**；code written faster, more compact, easier to understand and debug；compiler 進步 → recompile 就變好。
- **Disadvantages**：reduced speed、increased storage requirements（今日唔係大問題）。

6.4 Examples [2.36–2.37]

- **MS-DOS**：**single-tasking**；開機 invoke shell；**no process created**；single memory space；load program overwriting all but the kernel；program exit → shell reloaded。
- **FreeBSD**（Unix variant）：**multitasking**；user login → shell；**shell executes `fork()` to create a new process, `exec()` to load a new program into the process**；shell **waits for the process to terminate** or continues with user commands（background）；需要 CPU scheduling、process coordination、memory management。

7. Operating System Structure ⭐ ⭐ ⭐（必考比較題）[2.38–2.53]

7.0 一覽表

Structure	核心概念	優點	缺點	例子
Simple structure	無 well-defined structure	most functionality in least space	interfaces/levels 唔分明；**app 可直接 access I/O**；no dual mode/no hardware protection	**MS-DOS**

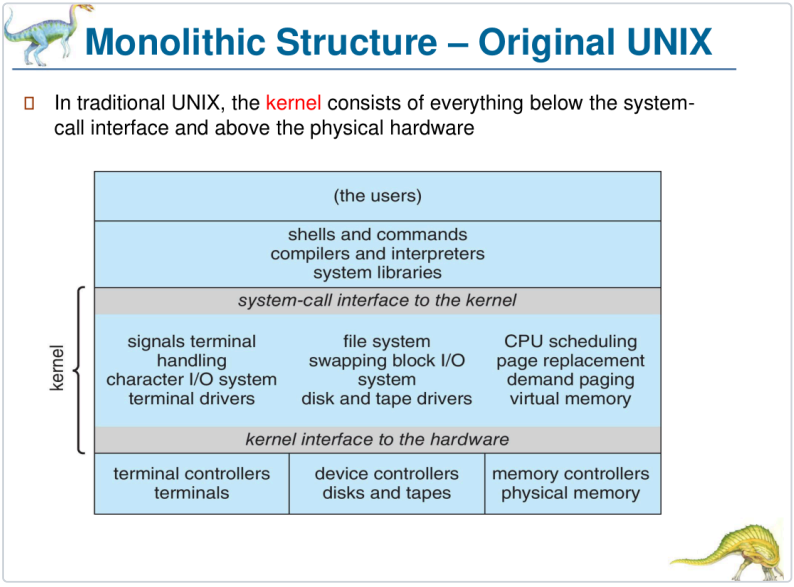
Structure	核心概念	優點	缺點	例子
Monolithic	**all kernel functionality in a single, static binary file running in a single address space**	**performance**：system-call interface overhead 好少，kernel 內部溝通快	**difficult to implement, debug, maintain/modify**；**tightly coupled**	**Original UNIX**, Linux, Windows（largely）
Layered	layer 0 = hardware ... layer N = user interface；每層只用下層服務	**simplicity of construction and debugging**；**information hiding**	**難定義 layers**；**performance overhead**（traverse 多層）	TCP/IP networks；(pure layered OS 好少)
Microkernel	**remove nonessential components from kernel, implement them as user-level programs in separate address spaces**；靠 **message passing**	**easier to extend**、**easier to port**、**more secure and reliable**（less code in kernel mode）	**performance suffers** due to **increased system-function overhead / user–kernel communication / message copying**	**Mach**, Darwin（macOS/iOS 部分），Windows NT（layered microkernel）
Loadable kernel modules (LKMs)	core kernel + **dynamically loaded modules（boot time or run time）**	唔使 recompile/reboot；**any module can call any other module**；**no message passing → more efficient than microkernel**	（仍喺 kernel mode，module 出錯可影響 kernel）	**Linux**（device drivers, file systems）, Solaris
Hybrid	混合多種結構	performance + security + usability	複雜	**Linux**（monolithic + modular）, **Windows**（monolithic + microkernel-like **personalities** + LKMs）, **macOS/iOS**（Darwin = Mach + BSD）

7.1 Simple Structure — MS-DOS [2.39]

- **Written to provide the most functionality in the least space**；not carefully divided into modules。
- **Application programs can access I/O directly**（interfaces/levels 唔分開）。
- Written for **Intel 8088 — no dual mode and no hardware protection**。

7.2 Monolithic Structure — Original UNIX ⭐ [2.40–2.42]

- **Traditional UNIX kernel = everything below the system-call interface and above the physical hardware.**



Original UNIX: monolithic structure [2.40]

- **Place all functionality of the kernel into a single, static binary file that runs in a single address space.**
- **UNIX = two separable parts: kernel + system programs**；kernel 再分成 interfaces 同 device drivers。
- **Drawback**：enormous functionalities combined into one level → **difficult to implement, debug, and maintain**。
- **Advantage**：**distinct performance advantage — very little overhead in the system-call interface, and communication within the kernel is fast.** → 所以 UNIX、Linux、Windows 仍然用。
- Sp21 MC：Original UNIX 採用 → **Monolithic structure**。
- **Linux** [2.42]：based on UNIX；**monolithic — runs entirely in kernel mode in a single address space**；apps 用 **glibc** 同 system call interface 溝通；**modular design 可以 run time 改 kernel（LKM）**。

7.3 Modular Design [2.43]

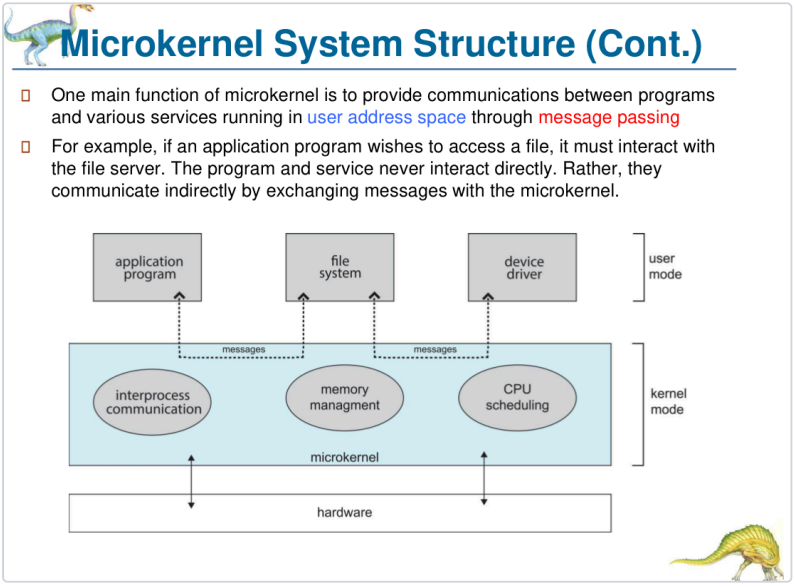
- Monolithic = **tightly coupled system**（改一部分影響其他部分）。
- **Loosely coupled system**：分成細 components，每個有 specific/limited 功能 → **changes in one component affect only that component** → 更自由靈活。
- **Layered approach 係 modular design 嘅其中一種。**

7.4 Layered Approach ⭐ [2.44–2.45]

- **OS divided into layers, each built on top of lower layers. Bottom layer (layer 0) = hardware; highest (layer N) = user interface.**
- 每層有 data structures + functions，可被上層調用、可調用下層。
- **Information hiding**：上層唔需要知下層點 implement，只需知 **what** these operations do（interface）；每層隱藏自己嘅 data structures、operations、hardware。
- **Main advantage：simplicity of construction and debugging**（由最底層開始逐層 debug）。
- 成功例子：**computer networks（TCP/IP）**、web applications。
- **Few OSes use a pure layered approach because**：
  1. **Significant challenges in appropriately defining layers and their functionalities**
  2. **Overall performance affected by the overhead of traversing multiple layers to obtain a service**
- Trend：**fewer layers with more functionality**，modularized code。
- S26 MC9 ✓：**The interfaces between adjacent layers are usually well defined**。✗ any layer may directly access any other layer；✗ eliminates all performance overhead；✗ pure layered is dominant。

7.5 Microkernel ⭐ ⭐ [2.46–2.48]

- Kernel 變大難管理 → **removing all nonessential components from the kernel and implementing them as system or user-level programs in separate address spaces → much smaller kernel**。
- **Mach**（CMU, mid-1980s）；best-known：**Darwin**（Mac OS X/iOS，包含 Mach microkernel）。
- 微核心通常提供：**minimal process and memory management + communication facility (IPC)**。
- **Main function：provide communication between client programs and services running in user space through message passing.** 例：app 要讀 file → 同 **file server** 經 microkernel **exchange messages**，兩者**唔直接 interact**。



Microkernel: services in user mode talk via message passing [2.47]

- **Advantages**：
  - **Easier to extend** — new services added to **user space without modifying the kernel**
  - Kernel 要改時改動較少（kernel 細）
  - **Easier to port to new architectures**
  - **More secure and reliable** — less code in kernel mode；**if a service fails, the rest of the OS remains untouched**
- **Drawbacks**：
  - **Performance can suffer due to increased system-function overhead (user space ↔ kernel space communication)**
  - Two user-level services communicate → **messages must be copied between services in separate address spaces**
  - Windows NT（layered microkernel）比 Windows 95 慢 → Windows XP 變返 more monolithic。

📝 **F20 Q2.1 Model answer**：*Main idea: a microkernel removes all nonessential components from the kernel and implements them as system or user-level programs in separate address spaces; the kernel keeps only minimal process/memory management and IPC (message passing). Advantage: easier to extend/port, more secure and reliable since less code runs in kernel mode. Disadvantage: performance suffers due to increased system-function overhead — message passing / copying between user space and kernel space.*

⚠️ **S26 MC7**：「microkernel typically has better performance than monolithic」→ **False**。（「runs memory management in user space」亦被接受為 false，因為 minimal memory management 喺 kernel 入面。）HW1 MC5 ✓「microkernel keeps only essential functions in the kernel and provides other services outside the kernel through message passing」。

7.6 Loadable Kernel Modules (LKMs) — Modular Approach ⭐ ⭐ [2.49–2.51]

- **Best current methodology for OS design.**
- **Kernel has a set of core components and links in additional services via modules, either at boot time or during run time.**
- 好處：**dynamic linking is preferable to adding features directly to the kernel, which would require recompiling the entire kernel every time**。例：kernel 內有 CPU scheduling + memory management，**file systems 經 loadable modules 加入**。
- **Similar to layered**：each kernel section has **well-defined, protected interfaces**；**but more flexible — any module can call any other module**（無上下層限制）。
- **Similar to microkernel**：primary module 只有 core functions + 知點 load/communicate 其他 modules；**but more efficient — modules do not need message passing to communicate**。
- **Linux uses LKMs primarily for device drivers and file systems**；可以 boot/run time **insert** 或 **remove**。例：插 USB，kernel 冇 driver → dynamic load。
- → **dynamic and modular kernel while keeping the performance benefits of a monolithic system**。
- [2.51] Solaris：core kernel + scheduling classes, file systems, loadable system calls, executable formats, STREAMS modules, misc modules, device & bus drivers。
- F20 MC6：「core components in kernel + adds services via modules during run time」→ **LKMs**。

📝 **F19 Q1.4 Model answer（LKM vs layered）**：*Similarity: like a layered design, OS functionality is divided into modules, each with a well-defined (protected) interface. Difference: LKM is much more flexible — any module can call any other module directly, with no higher/lower layer restriction (and modules can be loaded/removed at run time).*

7.7 Hybrid Systems [2.52–2.59]

- **Very few OSes adopt a single strictly defined structure** → **hybrid systems** combine approaches for **performance, security, usability**。
- **Linux**：**monolithic**（single address space → efficient）+ **modular**（LKM）。
- **Windows**：**largely monolithic**，但保留 microkernel 特色 — **separate subsystems (personalities) running as user-mode processes**；亦支援 dynamically loadable kernel modules。
- **macOS / iOS** [2.54–2.57]：kernel environment **Darwin = Mach microkernel + BSD UNIX kernel**（**XNU kernel**，hybrid）。
  - **Two system-call interfaces**：**Mach system calls (traps)** + **BSD system calls (POSIX)**。
  - **Mach** → memory management, CPU scheduling, IPC；**BSD** → POSIX API, networking, file system, security。
  - **I/O kit** for device drivers；dynamically loadable modules = **kernel extensions (kexts)**。
  - Mac：**Aqua** UI（mouse/trackpad）、**Cocoa**（Objective-C API）、Core frameworks（QuickTime, OpenGL）。
  - iOS：**Springboard** UI（touch）、**Cocoa Touch**、Media services、Core services（cloud, databases）；**ARM**（vs Intel）；不能原生跑 Mac apps。
- **Android** [2.58–2.59]：Open Handset Alliance（Google）；**open-source**（iOS closed-source）；layered；**Android API for Java** → 跑喺 **Android RunTime (ART)**（VM optimized for mobile）；**JNI (Java Native Interface)** bypass VM 存取 hardware（**not portable**）；libraries：webkit, SQLite, SSL；**HAL (hardware abstraction layer)** → consistent view；**Bionic** C library（唔係 glibc）；modified Linux kernel（power management）。
- **iOS vs Android**（summary）：相同 — 都基於現有 kernel、software stack、developer frameworks；唔同 — iOS closed / Android open；Objective-C vs Java；**Android uses a VM, iOS executes natively**。

7.8 Summary in OS Design [2.53]（原句，MC 用）

- **Monolithic**: no structure; all functionality in a single static binary in a single address space; difficult to modify; **primary benefit is efficiency**.
- **Layered**: discrete layers (bottom = hardware interface, top = user interface); **not ideal for OS due to performance problems and difficulty in defining layers**.
- **Microkernel**: minimal kernel; most services run as user-level applications; **communication via message passing**.
- **LKM**: services through modules **loaded and removed during runtime**.
- **Many contemporary OSes are hybrid: monolithic kernel + modules.**

8. History of OS [2.60]

- 硬件貴、人平：**Hand programmed (1945–55) → Batch processing (1955–65) → Multiprogramming (1965–80)**
- 硬件平、人貴：**Time sharing → Personal computer → Distributed OS → Real-time OS**

9. 補充：Protection & Security（Chapter 2 summary）

- **Protection** — controlling access of users/processes to resources。**Access right** = permission to perform operations on an object；**Domain** = a set of access rights；process 喺 domain 入面執行。
- **Security** — defend system against internal/external attacks（DoS, worms, viruses）。

10. 常見陷阱 & MC 速查

敘述	真/假
printf() is a system call	✗（library function，calls write()）
API always faster than system calls	✗
API can bypass kernel protection	✗
Parameters can be passed by registers, block/table, stack	✓（all of the above）
Block & stack methods limit number of parameters	✗（**do not** limit）
Shell may be implemented in kernel or as system program	✓
Linker loads program into memory	✗（loader 做）
DLLs are loaded once and shared	✓
Microkernel has better performance than monolithic	✗
Microkernel: services communicate by message passing	✓
Microkernel easier to port and extend	✓
Layered: any layer may call any other layer	✗（LKM 先可以 any module calls any module）
Layered: main advantage = simplicity of construction & debugging	✓
LKM requires recompiling entire kernel	✗
LKM more efficient than microkernel (no message passing)	✓
Monolithic: fast but hard to maintain	✓
Original UNIX = monolithic	✓
Linux = monolithic + modular (hybrid)	✓
Windows NT = layered microkernel	✓
Policy = how; mechanism = what	✗（**policy = what，mechanism = how**）
Darwin = Mach + BSD	✓
Android apps run on ART VM; iOS runs natively	✓

11. Past-paper 題目（Ch2）

1. **S26 MC5** false about OS services → **"Errors only occur in memory and I/O devices"**
2. **S26 MC6** advantage of API → **more convenient programming interface**
3. **S26 MC7** false about microkernel → **better performance than monolithic**（C 亦接受）
4. **S26 MC8** printf/write → **printf() invokes write() system call**
5. **S26 MC9** layered → **interfaces between adjacent layers are well defined**
6. **F19 Q1.1 / F20 Q2.2** linker vs loader（§5）
7. **F19 Q1.3** 點解分開 API 同 system call（§3.3）
8. **F19 Q1.4** LKM vs layered（§7.6）
9. **F20 MC4** APIs；**MC5** all of the above；**MC6** LKMs；**Q2.1** microkernel（§7.5）
10. **Sp21 MC**：two modes → **user mode and kernel mode**；interface to OS services → **system calls**；original UNIX → **monolithic**
11. **HW1 MC5** microkernel true statement；**MC6** read() path；**MC7** privileged instruction in user mode → **trap to OS**；**Q2.4** read() user→kernel→user

12. 自我測驗

1. 寫出 system call 六大類別。
2. 點解 block/stack parameter passing 優於 registers？（no limit on number/length）
3. LKM 同 microkernel 有咩相似同唔同？（both: core kernel small; LKM no message passing → efficient）

4. 舉例說明 policy vs mechanism。（timer = mechanism; how long = policy）
5. FreeBSD shell 點樣 run 一個 program？（fork → exec → wait）

3 Ch3 Processes 進程 • fork • IPC • interrupt

📌 **考試範圍提示**：Ch3 係**分數最多**嘅章節之一。必考：

1. **fork() 計數 / process tree / possible outputs**（每年 2–3 題，8–13 分一題）
2. **fork + exec + wait 成功/失敗情況、pipe / dup 填空**（S26 Q3.3）
3. MC：**process state、PCB、context switch、schedulers、zombie/orphan、pipes（ordinary vs named）、interrupt、DMA、dual mode** ⚠️ 今個 term **interrupt / DMA / dual-mode** 喺 Ch3 slides（3.21–3.31）。

0. Computer-System Operation：I/O、Interrupt、DMA、Dual Mode [3.21–3.31]

0.1 I/O 基本運作 [3.21]

- **I/O devices and the CPU execute concurrently and asynchronously.**
- **Each device controller is in charge of a particular device type and has a local buffer.**
- **Device controller moves data between the peripheral devices and its local buffer storage.**（I/O = device ↔ controller local buffer）
- **CPU moves data from/to main memory to/from local buffers**（typically for slow devices: keyboard, mouse）。
- **Device controller informs the CPU that it has finished an operation by causing an interrupt.**
  - Input device：data available in local buffer；Output device：I/O operation completed。

0.2 Interrupts ⭐ ⭐ [3.22–3.24]

- **An I/O device may trigger an interrupt by sending a signal to the CPU**（經 **interrupt-request line**）。CPU **handles the interrupt and then returns to the interrupted instruction**。
- **Interrupts are widely used to handle asynchronous events — device controllers and hardware faults.**
- **Interrupt transfers control to an interrupt service routine (ISR) or interrupt handler — part of kernel code.**
- **Interrupt vector**（summary）：a table/array holding the **addresses of the interrupt handlers**；每個 interrupt 有 unique **interrupt number** 作為 index。
- 運作（summary）：CPU **每執行完一條 instruction 就 sense interrupt-request line** → 有 signal → 讀 interrupt number → 用佢做 index 入 interrupt vector → jump 去 handler。
- **兩條 interrupt request lines**（summary）：
  - **Nonmaskable interrupt** — reserved for events like **unrecoverable memory errors, power failure**
  - **Maskable interrupt** — **can be turned off by the CPU** before executing critical instruction sequences；device controllers 用（可以 delay）
- **Interrupt priority levels** ⭐ — **a high-priority interrupt can preempt the execution of a low-priority interrupt**；CPU 可以 defer low-priority interrupts 而唔使 mask 全部。
  - Lab 2 補充：**a trap (software interrupt) is given a relatively LOW interrupt priority compared with device interrupts**；**multi-CPU systems can process interrupts concurrently**。
- **A trap or exception is a software-generated interrupt caused either by an error (e.g., arithmetic error / divide by zero) or a user request (a system call).** ⭐
- **All modern operating systems are interrupt-driven**；每秒幾百個 interrupts。

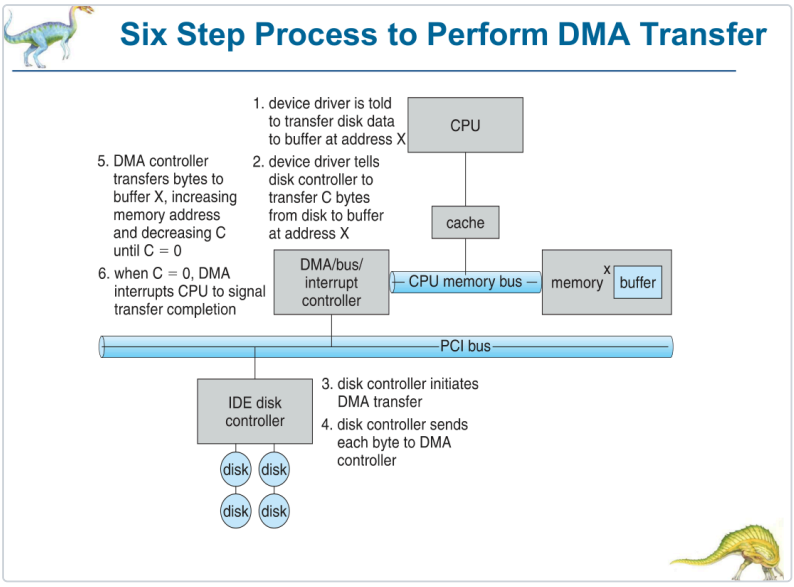
⚠️ **MC 陷阱**：

- S26 MC1 ✗「**All interrupts are serviced in exactly the same way, with no difference in priority**」（有 priority levels）
- HW1 MC2 ✗「**Interrupts can only be generated by I/O devices**」（software trap/exception 都係）；✓ hardware interrupts asynchronous；✓ trap caused by error or system call；✓ different priority levels
- F20 MC3 ✗「**Interrupt allows software to access hardware including the CPU**」（嗰個係 **system call** 嘅作用）
- Sp21 MC2：直接 trigger interrupt 嘅 event → **I/O job completion**

0.3 DMA (Direct Memory Access) ⭐ ⭐ [3.25–3.26]

- **Programmed I/O** — **CPU runs special I/O instructions to move one byte at a time between memory and slow devices**（keyboard, mouse）。
- **DMA** — for **fast devices and large amounts of data**：**DMA controller bypasses the CPU to transfer data between I/O device and memory directly**；CPU/OS 只係 **initialize** DMA controller。→ **relieves the CPU from slow data movement**。
- 步驟：OS writes **DMA command block** into memory（**source and destination addresses, read or write mode, number of bytes**）→ writes location of command block to DMA controller → **bus mastering**（DMA controller grabs bus from CPU — **momentarily prevents the CPU from accessing main memory**, Lab 2）→ **when done, sends an interrupt to the CPU to signal completion**。
- **Six-step DMA transfer** [3.26]：
  1. Device driver is told to transfer disk data to buffer at address X
  2. Device driver tells disk controller to transfer C bytes from disk to buffer at address X

3. Disk controller initiates DMA transfer
4. Disk controller sends each byte to DMA controller
5. DMA controller transfers bytes to buffer X, increasing memory address and decreasing C until C = 0
6. **When C = 0, DMA interrupts CPU to signal transfer completion**



Six-step DMA transfer **[3.26]**

> ⚠️ **MC 陷阱：**
> - HW1 MC1 ✗「**DMA controller sends an interrupt after every byte**」→ 只係 **block 完成後 interrupt 一次**
> - S26 MC2 ✓「**DMA allows data transfer between device and memory with less CPU involvement**」
> - ✗「DMA eliminates interrupts completely」；✗「CPU and DMA can't execute concurrently」；✗「DMA for slow devices, CPU for fast devices」（相反）

0.4 Dual-mode Operation ⭐ ⭐ [3.27–3.31]

- **Dual-mode operation allows the OS to protect itself and other system components.**
- **User mode** vs **kernel mode**（亦叫 supervisor / system / privileged mode）— distinguish execution of OS code and user code。
- **Provides the basic means of protecting the OS from errant users, and errant users from one another.**
- **Mode bit provided by hardware：user mode = 1，kernel mode = 0.** ⭐
- **Privileged instructions — can only be executed in kernel mode**，例：**I/O control, timer management, interrupt management**（summary：set timer value, clear memory, disable interrupts, modify device-status table, access I/O devices）。
- **If a privileged instruction is attempted in user mode, the hardware treats it as illegal and traps it to the OS.**（HW1 MC7 答案 A）
- **Beyond two modes：Intel — four protection rings（ring 0 = kernel, ring 3 = user；rings 1, 2 rarely used）；ARMv8 — seven modes；CPUs supporting virtualization have a separate VMM mode（more privileges than user, fewer than kernel）**。VMM 做 **machine switch**（save entire machine state 包括 privileged hardware state）。
- **Initial control in kernel mode → set to user mode when giving control to user app → back to kernel via interrupt, trap, or system call.**
- **Timer** — ensures a user program always returns control to the OS（防止 infinite loop 霸住 CPU）；設定 timer 係 privileged。
- **System call**：user mode → kernel mode to fulfil request；**return from system call resets it to user mode**。

0.5 Three Types of Mode Transfer（user → kernel） ⭐ [3.31]

Type	性質	例子
System call	user process **requests a service** from OS；"like a function call but outside the user process"	`exit()`, `write()`, `read()`
Interrupt	**external, asynchronous** events；**independent of user process**	timer, I/O completion
Trap / Exception	**internal, synchronous** events in the process	**protection violation (segmentation fault)**, **divide by zero**

---

1. Process Concept（進程概念） [3.4–3.8]

1.1 Four Fundamental OS Concepts [3.4]

1. **Process** — **an instance of an executing program**；consists of **an address space and one or more threads of control**。

2. **Thread — a single unique execution context**；fully describes program state — **program counter, registers, execution flags, stack**。
3. **Address space** — program 喺自己嘅 address space 執行，**distinct from other programs'**（logical/virtual address，Ch9–10）。
4. **Dual-mode operation** — basic protection（見 §0.4）。

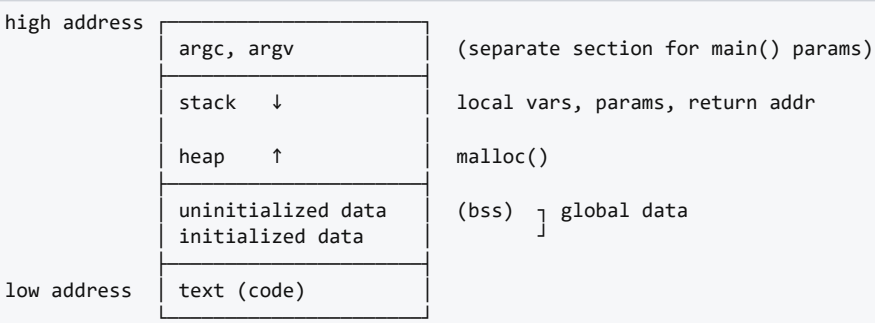
1.2 Process 嘅組成部分 [3.5] ⭐

- **Text section** — the program code
- **Current activity** — **program counter**, processor registers
- **Stack** — **temporary data when invoking functions: function parameters, return addresses, local variables** ⭐（Sp21 MC 答案：stack）
- **Data section** — **global variables**
- **Heap** — **memory dynamically allocated during run time**（`malloc()` / `free()`）
- **Program = passive entity**（executable file on disk）；**Process = active entity**（有 PC 指住下一條 instruction + associated resources）。
- **A program becomes a process when an executable file is loaded into memory**（double-click icon / shell command）。
- **One program can be executed by multiple processes.**
- A process can be an execution environment for other programs（例：**JVM** 執行 Java program）。

1.3 Loading a program into memory [3.7]

1. **Load code and static data**（initialized variables）into the address space
2. **Allocate the runtime stack**；fill in `main()` parameters **argc, argv**
3. **Allocate heap**（`malloc()` / `free()`）
4. **Other initialization (I/O)** — UNIX：每個 process 預設有 **three open file descriptors: stdin (0), stdout (1), stderr (2)** ⭐

1.4 Memory Layout of a C Program [3.8]

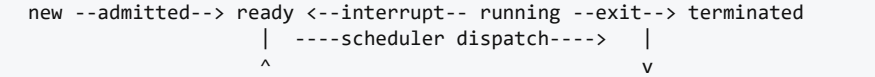


- **Global data section is divided into (1) initialized data and (2) uninitialized data.**

---

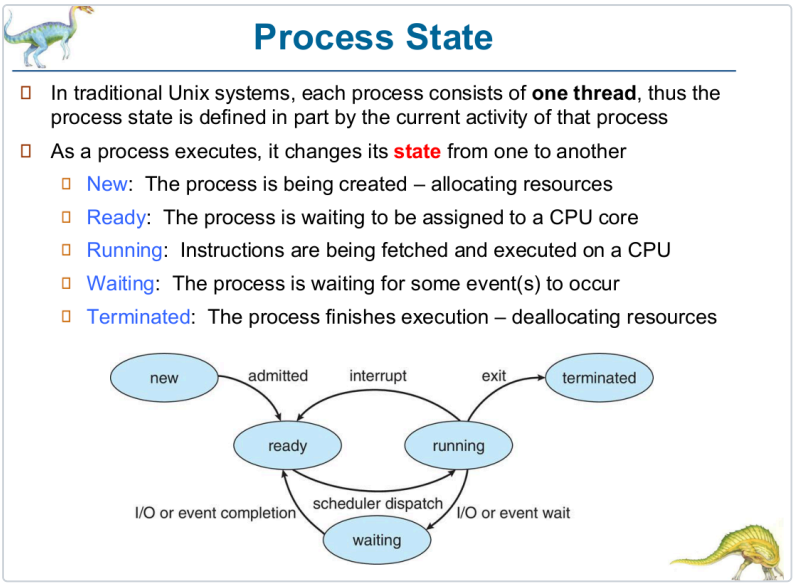
2. Process State（進程狀態） ⭐ ⭐ [3.9]

State	意思
New	process is being created — **allocating resources**
Ready	**waiting to be assigned to a CPU core**
Running	**instructions are being executed** on a CPU
Waiting	**waiting for some event(s)** to occur（I/O completion, signal）
Terminated	finished execution — **deallocating resources**



轉換（F19 Q2.1 答案）：

- New → Ready：**admitted**
- Ready → Running：**scheduler dispatch**（CPU scheduling）
- Running → Ready：**interrupt**（e.g. timer / time slice expired / preempted）
- Running → Waiting：**I/O or event wait**
- Waiting → Ready：**I/O or event completion**
- Running → Terminated：**exit**



Process state diagram [3.9]

📝 **HW1 Q2.3**：admitted → dispatched → disk I/O → I/O completes → timer interrupt → dispatched → exits **New → Ready → Running → Waiting → Ready → Running → Ready → Running → Terminated** 離開 Running 時 OS 喺 **PCB** 記錄：**process state, program counter, CPU registers**（context），以及更新 scheduling info、accounting info（CPU time used）、I/O status（e.g. pending I/O request）。

⚠️ 注意：**Waiting → Running 係冇嘅**（要先返 Ready）；**Ready → Waiting 亦冇**。S26 MC14：令 process 離開 running state 嘅 event → **It issues an I/O request**（call function / 加 local variable / read cache 都唔會）。Sp21 MC6：might force a process to leave the CPU → **all of the above**（fork system call、I/O request、interrupt）。

3. Process Control Block (PCB) ⭐ [3.10, 3.13]

PCB（又叫 task control block）= information associated with each process（kernel data structure）：

Field	內容
Process state	running, waiting, etc.
Program counter	location of the next instruction to execute
CPU registers	accumulators, index registers, **stack pointers**, general-purpose registers, condition codes
CPU-scheduling information	**priorities, pointers to scheduling queues**, scheduling parameters
Memory-management information	memory allocated（page tables / segment tables）
Accounting information	CPU used, clock time elapsed, time limits
I/O status information	I/O devices allocated, **list of open files**

- 每個 process 有 unique **PID (process identifier)**（UNIX/Linux 係 positive integer；**PID 1 = init / systemd**，root process）。
- **Process table / process list** — kernel data structure keeping track of all processes（indexed by PID, pointing to PCBs）。
- **Linux** [3.13]：**task list = circular doubly linked list** of `struct task_struct`（process descriptor = PCB）：`pid_t pid; long state; unsigned int time_slice; struct task_struct *parent; struct list_head children; struct files_struct *files; struct mm_struct *mm;`

4. Threads（簡介，詳見 Ch4） [3.11–3.12]

- **A thread is represented by a program counter, registers, execution flags, and a stack — part of a process.**
- 一個 thread 喺 CPU 執行 = 佢嘅 state 喺 CPU **hardware registers**。
- **Multiple threads of a process run within the same address space (share code, data, files); each thread has its own thread state.**（multicore 上可以 parallel）
- 例：web browser 一個 thread display、一個 thread 由 network 拎 data。
- 傳統 process（single thread）= **heavy-weight process**。

5. Process Scheduling（進程排程） ⭐ [3.14–3.18]

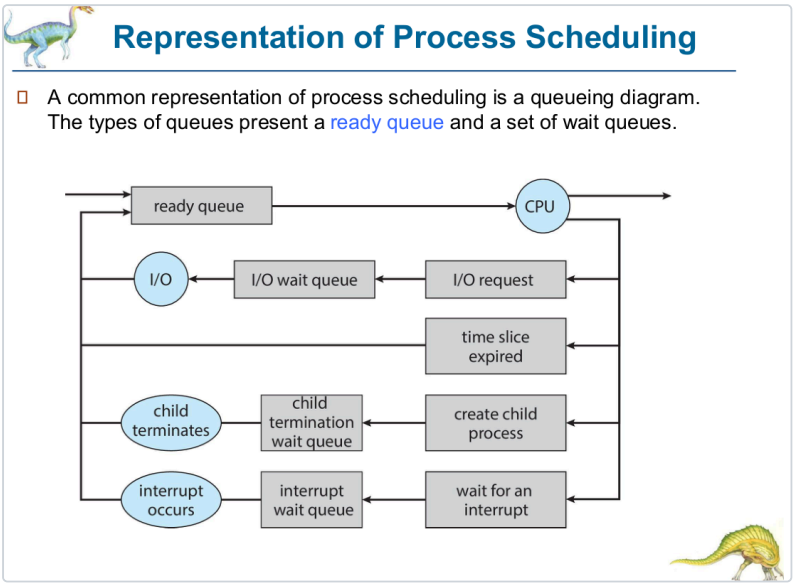
5.1 基本

- **Objective of multiprogramming：have some process running at all times to maximize CPU utilization.**
- **Process scheduler** — selects an available process (in memory) for execution on a CPU core。

- **Single CPU core → never more than one process running at a time.**
- **Degree of multiprogramming = number of processes currently residing in memory.** ⭐ （Sp21 MC）
- **I/O-bound process** — more time doing I/O；**many short CPU bursts**。
- **CPU-bound process** — infrequent I/O；**few very long CPU bursts**。

5.2 Scheduling Queues [3.15–3.16]

- **Ready queue** — processes in main memory, ready and waiting to execute
- **Wait queues** — processes waiting for an event（e.g. I/O completion）
- **Processes migrate among the various queues during their lifetime.**
- **Queueing diagram** [3.16]：ready queue → CPU → 出口有：**I/O request**（→ I/O wait queue → I/O → ready）、**time slice expired**（→ ready）、**create child process**（→ child termination wait queue）、**wait for an interrupt**（→ interrupt wait queue → interrupt occurs → ready）。



Queueing diagram [3.16]

- Summary：**the state of a process can be determined by which queue it resides on**（running 嗰個例外）。

5.3 Schedulers ⭐ ⭐ [3.17–3.18]

Scheduler	又叫	做咩	頻率
Long-term scheduler	**job scheduler**	**selects which processes should be brought into memory**（from job queue, after resource allocation）；**dictates the degree of multiprogramming**；aims for a **good process mix** of I/O-bound and CPU-bound；mainframes/minicomputers 用，**PC 冇**	**very infrequent**（seconds, minutes）→ may be slow
Short-term scheduler	**CPU scheduler**	**selects a process from the ready queue to execute next and allocates a CPU core**	**very frequent**（milliseconds）→ **must be fast**
Medium-term scheduler	—	**reduces degree of multiprogramming — removes a process from memory, stores it on disk, brings it back later = swapping**（swap space）	需要時

- F20 MC10：「___ swaps a process out of memory and temporarily stores it on disk」→ **medium-term scheduler**。
- Sp21 MC：「selects from the ready queue to execute on the CPU」→ **short-term scheduler**。

6. Context Switch（上下文切換） ⭐ ⭐ [3.19–3.20]

- **When the CPU switches to another process, the OS must save the state of the old process and load the saved state of the new process via a context switch.**
- **Context of a process is represented in the PCB**（or thread）：registers, program counter, stack pointer。
- **Context-switch time is pure overhead — the system does no useful work while switching.**
- **The more complex the OS and the PCB, the longer the context switch.**
- 速度取決於 **memory speed, number of registers to copy, special instructions (load/store all registers) → highly dependent on hardware**。
- **Some processors provide multiple sets of registers（SUN UltraSPARC）→ context switch = changing a pointer to another register set.**

⚠️ S26 MC11 ✓「**During context switch, OS saves the state of one process/thread and restores another**」。✗「Context switch means switching from user mode to kernel mode」（嗰個係 **mode switch**，唔係 context switch）✗「requires same hardware on all architectures」；✗「shorter if OS is more complex」

7. Process Creation（建立進程） ⭐⭐⭐ [3.32–3.46]

7.1 概念

- **Parent process creates child processes → tree of processes.** 每個 process 有 unique **pid**。
- Linux tree [3.33]：`systemd (pid 1)` 有三個 children：`logind (8415)`、`python (2808)`、`sshd (3028)`；`logind` → `bash (8416)` → `ps (9298)`、`vim (9204)`；`sshd (3028)` → `sshd (3610)` → `tcsh (4005)`。
- Child 需要 resources（CPU time, memory, files, I/O devices）；parent 可傳 initialization data。
- **Resource sharing 三種選擇** [3.34]：
  1. **Parent and children share (almost) all resources** — `fork()`
  2. **Children share a subset of parent's resources** — `clone()`（Ch4）
  3. **Parent and child share no resources**
- **Execution 兩種選擇**：
  1. **Parent continues to execute concurrently with its children**（例：`a.out &`）
  2. **Parent waits until some or all of its children have terminated**（例：shell 等 `a.out` 完成）
- **Address space 兩種選擇** [3.35]：
  1. **Child is a duplicate of the parent (same program and data)** — UNIX/Linux `fork()`
  2. **Child has a new program loaded into it** — `exec()`

7.2 fork() ⭐⭐⭐ [3.35–3.39]

- `fork()` **creates a new process that duplicates the entire address space of the parent. Both processes continue execution at the instruction after** `fork()`**.**
- **Return value**（each process receives exactly one return value）[3.36–3.37]：

Return value	喺邊個 process	意思
> 0	**parent**	值 = **PID of the newly created child**
= 0	**child**	新建嘅 child
< 0（−1）	**parent (original)**	**fork failed，no child created**，errno set

- **All state of the parent is duplicated in the child**：memory (program, data), **file descriptors**...；每個 process 有**自己一份** program 同 data（time consuming）。
- **Steps to implement fork()** [3.38]：
  1. **Create and initialize the PCB** in the kernel
  2. **Create a new address space**（allocate memory）
  3. **Initialize the address space with a copy of the parent's entire address space**（time consuming）
  4. **Inherit the execution context of the parent**（e.g. open files）
  5. **Inform the CPU scheduler that the child is ready to run**
- **Parent vs Child after fork()** [3.39]：
  - **Duplicated**：address space, **global & local variables**, current working directory, root directory, process resources, resource limits, **program counter (PC)**...
  - **Different**：**PID**, **return value from fork()**, running time, running state
- ⚠️ **Child 改 variable 唔會影響 parent**（各自一份 copy）— HW1 MC9 ✗「If the child changes a global variable, the parent's copy changes automatically」。
- ⚠️ **Execution order of parent and child depends on the scheduler** unless synchronization（`wait()`）is used。
- F20 MC7 ✗「**The parent process waits until the completion of the child process**」（fork 本身唔會等，要 `wait()`）。
- Sp21 MC ✗「**The child process can call wait() in order to finish earlier than the parent**」。

7.3 Copy-on-Write (COW) & clone() [3.46]

- **Process creation in UNIX is unique — separated into two system calls:** `fork()` **and** `exec()`**.**（其他 OS：create new address space + read executable + start 一次過）
- **Linux implements fork() with copy-on-write (COW) pages** — parent and child **share a single copy；copying is delayed until a page is written**。若 `fork()` 後即刻 `exec()`，pages **never need to be copied → greatly speeds up process creation**。
- **Linux also implements fork() via** `clone()`（more general；flags 指定 share 邊啲 resources）。

7.4 exec / wait / exit / signal [3.41–3.43]

- `exec()` / `execlp()` — **changes the program being run by the current process；creates a new process image from a regular executable file**（**replaces the address space**；**成功就唔會 return**，fail 先 return −1 並繼續執行下一行）。
- `wait()` — wait for a (child) process to finish；`pid = wait(&status);` 返回 terminated child 嘅 PID，`status` 存 exit status。
- `exit()` — terminate the current process and free all its resources。
- `signal()` — send a notification to another process（slide 3.41 原文）。
  - ⚠️ slides 前後唔一致：slide 4.31 話真正 **deliver** signal 畀 process 嘅 UNIX function 係 `kill(pid, signal)`；而實際 UNIX 嘅 `signal(SIGINT, handler)` 係**裝 handler**（PA1 就係咁用）。MC 照題目用字對返邊張 slide 作答。
- 經典 code [3.42]：

```
pid = fork();
if (pid < 0) { fprintf(stderr, "Fork Failed"); exit(-1); }   /* error */
else if (pid == 0) { execlp("/bin/ls", "ls", NULL); }        /* child */
else { wait(NULL); printf("Child Complete"); exit(0); }      /* parent */
```

- **Implementing a shell** [3.43]：loop `readAndParseCmdLine` → `fork()` → child `exec(prog, args)`（NOT REACHED after）→ parent `wait(child_pid)`。
- F19 Q2.4：shell 要 start new process 必須用嘅兩個 system calls → **fork()**（clones the current process）+ **exec()**（overlays a new program onto the calling process）。

---

8. Process Termination（終止） ⭐ ⭐ [3.47–3.50]

- Process 執行完最後一句 → `exit()` 要求 OS 刪除佢；可以經 `wait()` 回傳 **status value** 畀 parent。
- **Resources (memory, open files, I/O buffers) are deallocated and reclaimed by the OS.**
- **Parent can terminate children**：`abort()`（UNIX/Linux）、`TerminateProcess()`（Windows）；parent 需要知 child PID（由 fork() 返回）。理由：child **exceeded allocated resources**；**task no longer required**；**parent is exiting**（有啲 OS 唔准 child 冇 parent）。
- **Cascading termination** — if a process terminates, **all its children must also be terminated**（initiated by OS）。
- Normal termination：`exit()` 直接或間接被 call（C runtime library 預設 call `exit()`）。

8.1 Zombie ⭐ ⭐

- **A process that has terminated, but whose parent has not yet called `wait()`, is a zombie process.**
- 佢嘅 **entry in the process table, PID, and PCB still exist**（因為 process table entry 存住 **exit status**）；其他 resources 已釋放；**not runnable**（冇 address space）。
- **All processes transition into the zombie state before termination, but generally only briefly.**
- **Once the parent calls `wait()`, the zombie's PID, process-table entry and PCB are released.**
- S26 MC13 ✓「**A zombie process has terminated but its parent has not yet collected its status**」；*X* still executing user code；*X* created when exec() fails；*X* owns no PID。

8.2 Orphan ⭐

- **If a parent terminates without invoking `wait()`, its child becomes an orphan**（如果冇 cascading，可能仍 runnable，最終變 zombie）。
- `init` / `systemd` **is assigned as the new parent and periodically calls `wait()`** → 收集 exit status，釋放 PID 同 process-table entry。

---

9. fork() 解題大法 ⭐ ⭐ ⭐ （考試必用）

9.1 基本規則

1. **每 call 一次 fork()，「每個執行到嗰句嘅 process」都會變兩個**：parent（return child PID > 0 → **true**）、child（return 0 → **false**）。
2. **Child 由 fork() 下一句開始行**，有 parent 所有 variables 嘅 **copy**（包括 loop counter `i`）。
3. **n 句連續 fork()（無條件）→ 2^n processes。**
4. **Short-circuit evaluation** ⭐：
   - `A && B`：A **false (0 → child)** 就**唔會**執行 B；A **true (parent)** 先執行 B。
   - `A || B`：A **true (parent)** 就**唔會**執行 B；A **false (child)** 先執行 B。
5. **printf 次數 = 執行到該句 printf 嘅 process 數目**（每個 printf 位置分開計）。
6. `return` / `exit()` / `break` 會令 process 停止繼續 loop 或者唔再執行後面。
7. `exec()` 成功 → 之後嘅 code **唔會執行**；失敗 → return −1 繼續。

9.2 常見 pattern 公式（全部已用真 C 程式驗證 ✓）

Code（每個 iteration，所有 process 都繼續 loop）	每 iteration 每個 process 變成	n 次後 processes
`fork();`	2	2^n
`fork(); fork();` [3.44]	4	4^n（slide：10 次 → 4^{10}）
`if (fork()) fork();` [3.45]	3（parent 再 fork 一次）	3^n（slide：3^{10}）
`if (fork() == 0) fork();`（Sp21 MC8）	3（child 再 fork）	3^n（n=3 → **27**）
`if (fork() && fork()) ...` / `fork() || fork()`	3	3^n

9.3 有 break / return 嘅 loop（追蹤「仲 active」嘅 process）

S26 Q3.1：

```
for (int i = 0; i < n; i++) {
    if (fork()) fork();   // parent: fork 多一次
    else break;            // 第一個 child: break（停 loop）
}
```

- 每個 active process 每 iteration：第一個 fork 產生 child（→ else → **break**，停）；parent 再 fork 一次，**新 child（第二個）繼續 loop**（佢喺 if branch 入面，return 0 但已經喺 if 入面）。
- 所以 1 個 active → 3 個 processes，其中 **2 個繼續 active**、1 個 stopped。
- $A_k = 2^k$（iteration k 開始前 active 數）；每 iteration 新增 2^k 個 break。
- **Total processes** $= \underbrace{2^n}_{\text{active}} + \underbrace{(2^n - 1)}_{\text{stopped}} = 2^{n+1} - 1$；**break 次數** $= 2^n - 1$。
- n = 2：**7 processes，3 breaks** ✓；n = 3：15 processes，7 breaks ✓。
- Process tree（n=2）：P0 → P1 (break), P2 → {P5 (break), P6}, P3 (break), P4...（P0 第 0 輪產生 P1(break)、P2；第 1 輪產生 P3(break)、P4；P2 第 1 輪產生 P5(break)、P6）

```
P0
├── P1   (break at i=0)
├── P2   (continues)
│   ├── P5 (break at i=1)
│   └── P6
├── P3   (break at i=1)
└── P4
```

S26 MC10：

```
for (i = 0; i < 3; i++) {
    if (fork() == 0) { fork(); printf("X "); return 1; }
}
printf("Y ");
```

- 只有 original parent 繼續 loop；每 iteration 產生 1 個 child，child 再 fork → 2 個 process 各印 "X " 後 return。
- 3 iterations → 3 × 2 = 6 個 process 印 X；original 印 1 個 Y。**Total = 7 processes, X = 6, Y = 1 → 答案 C** ✓

9.4 Short-circuit 題

Sp21 MC7：`if (fork() || fork()) { fork(); }`

- P0：fork1 → P0 得 >0（true）→ **跳過第二個 fork** → 入 if → fork → 2 個
- C1（fork1 得 0）→ 執行 fork2：C1 得 >0（true）→ 入 if → fork → 2 個；C2 得 0 → false → 唔入 if → 1 個
- **Total = 2 + 2 + 1 = 5** ✓

F19 Q2.7：`if (fork() == fork() || fork() != fork()) fork(); printf("Fork!\n");`

- `fork() == fork()`：4 個 processes，返回值組合 (x,y)：P0 (c1,c2) 唔等；C2 (c1,0) 唔等；C1 (0,c3) 唔等；**C3 (0,0) 相等** → **true**（只有收到兩次 0 嘅個 grandchild）。
- C3：|| 短路 → 執行 body fork() → **2 個** process 印。
- 其餘 3 個 process 執行 `fork() != fork()` → 每個變 4 個 = 12；每組 4 個入面 3 個 true（(c,c')、(0,c)、(c,0)），1 個 false (0,0)。
- 9 個 true → fork → **18**；3 個 false → **3**。
- **Total = 2 + 18 + 3 = 23 lines** ✓

9.5 Nested / combined

Sp21 Q1.1：

```
pid1 = fork(); pid2 = fork();
if (pid1 > 0) if (pid2 == 0) fork();
```

- 兩個 fork 後 4 個 processes；只有「pid1 > 0（P0 或 P0 嘅第二個 child）且 pid2 == 0」嘅 **P0 第二個 fork 生出嘅 child** 符合 → 再 fork 1 次 → **5 processes** ✓

Sp21 Q1.2：

```
if (fork() == 0) { for (int i = 0; i < 5; i++) if (fork() == 0) fork(); }
fork();
```

- 第一個 fork：child 入 loop；loop 5 次，每次每個 process 變 3 個 → $3^5 = 243$（包括 child 自己）。
- 加 original parent = 243 + 1 = 244；最後 fork 全部 ×2 → $(3^5 + 1) \times 2 = 488$ ✓

F20 Q3.2：

```
if (!fork()) { for (int i = 0; i < n; i++) if (fork()) fork(); }
fork(); printf("a");
```

- 同上：child branch 變 3^n，加 parent → $3^n + 1$，最後 fork ×2 → **"a" 數** $= 2(3^n + 1)$ ✓（n=1 → 8；n=2 → 20）

F19 Q2.6（loop 入面 parent 同 child 都 printf）：

```
for (i = 0; i < n; ++i) { if (fork()) printf("Fork!\n"); else printf("Fork!\n"); }
```

- 第 i 次 iteration（由 1 數）有 2^i 個 process 各印一行 → $2 + 4 + \cdots + 2^n = \sum_{i=1}^{n} 2^i = 2^{n+1} - 2$ ✓

9.6 Possible output orderings（輸出次序題）⭐

方法：

1. 畫 process tree，寫低每個 process 印咩、依咩次序（**同一 process 內次序固定**）。
2. 加 **wait()** 約束：parent `wait()` 之後嘅 print 一定喺 child **全部** print 之後。
3. Fork 之前印嘅一定最先（假設 flushed）。
4. 列出所有符合約束嘅 **interleavings**（topological orders）。

F20 Q3.1：

```
if (fork() == 0) {            // child C
    if (fork() > 0) { printf("1"); wait(NULL); printf("2"); }  // C
    else printf("3");         // grandchild G
} else printf("4");           // parent P
```

- 約束：C 印 "1" 然後等 G（印 "3"）→ 再印 "2"。即 **"2" 喺 "1" 同 "3" 之後**；"4" 隨意。
- "2" 最後：1、3、4 任意排 = 3! = 6 種：**1342, 1432, 3142, 4132, 3412, 4312**
- "4" 喺 "2" 之後：**1324, 3124**
- **Total = 8**（unbuffered）。若 buffered（"1" 同 "2" 同一 buffer 一齊 flush）→ 只剩 **4312, 3412, 3124**（3 種）。

Sp21 Q1.3：`printf("A\n"); if (fork()==0) printf("B\n"); else { printf("C\n"); ___A___ } printf("D\n");`

- (a) A 空白：**ABDCD, ABCDD, ACBDD, ACDBD**（4 種；A 一定最先；child 印 B,D；parent 印 C,D）
- (b) A = `wait(NULL)`：parent 嘅 D 一定最後 → **ABDCD, ABCDD, ACBDD**
- (c) A = `printf("E\n")`：parent 印 C,E,D；child 印 B,D → 7 種：**ABDCED, ABCDED, ABCEDD, ACBDED, ACBEDD, ACEBDD, ACEDBD**

HW1 Q3.1（x 值）：

```
int x = 1; pid_t c1 = fork();
if (c1 == 0) { x += 2; printf("C1: %d\n", x); return 0; }   // 3
x *= 2; pid_t c2 = fork();
if (c2 == 0) { x += 3; printf("C2: %d\n", x); return 0; }   // 2+3 = 5
wait(NULL); wait(NULL); printf("P: %d\n", x);                // 2
```

- **C1: 3, C2: 5, P: 2** — 因為 **each process has its own copy of x**（fork 時複製）。
- Possible outputs：`C1:3 → C2:5 → P:2` 或 `C2:5 → C1:3 → P:2`（P 一定最後，因為 wait 兩次）。
- 若將第一個 `wait(NULL)` 移到 `x *= 2` 前：parent 先等 C1 完，先 fork C2 → **保證 C1: 3, C2: 5, P: 2**；**數值不變**（wait 唔改 x）。

HW1 Q3.2（exec）：

```
printf("A\n"); pid_t p = fork();
if (p == 0) { pid_t q = fork();
    if (q == 0) { execlp("./worker","worker",(char*)NULL); printf("X\n"); return 1; }
    wait(NULL); printf("C\n"); return 0; }
printf("P\n"); wait(NULL); printf("D\n");
```

- **3 processes**：parent → child → grandchild（**grandchild exec worker**，印 W）。
- **X 唔會印**：execlp 成功 → address space 被取代，**never returns**。
- 約束：A 最先；W < C < D；P < D。→ **A P W C D、A W P C D、A W C P D**（3 種）。

9.7 fork + exec 成功/失敗情況（S26 Q3.2）⭐

```
int ret = fork();
if (ret == 0) { printf("Hello1\n"); exec("some_executable"); printf("Hello2\n"); }
else if (ret > 0) { wait(NULL); printf("Hello3\n"); }
else { printf("Hello4\n"); }
```

Case	Output
fork 成功、exec 成功	`Hello1`, `Hello3`（Hello2 唔印：image replaced）
fork 成功、exec 失敗	`Hello1`, `Hello2`, `Hello3`（exec 返回 −1，繼續）
fork 失敗	`Hello4`（only parent, ret < 0）

- 因為有 `wait(NULL)`，Hello3 一定喺 child 嘅輸出之後。

9.8 更多歷屆 fork 題（Lab 3 review 提供：F22 / F24 / S25，全部驗證 ✓）

題目	Code（重點）	答案
Fall 2022 Q3	`for(i<n){ if (fork() == fork()) printf(...); }`	processes 4^n；printed lines $4^0 + 4^1 + \cdots + 4^{n-1} = \frac{4^n - 1}{3}$
Fall 2024 P3.1	`for(i<M){ pid = fork(); if(!pid) for(j<N) fork(); }`	**child** 行 line Y；M=N=1 → 3；一般 $(2^N + 1)^M$
Spring 2025 MC12	`if (fork() || fork()) { if (fork() && !fork()) fork(); }`	**9**（外層 2 入 if × 每個變 4 + 1 入 else）
Lab 3 Ex3	`for(i<n){ if(fork()){fork();fork();} else fork(); }`	6^n（parent 變 4 + child 變 2）

- 詳細解釋：**Labs notes L3.7**。Fall 2022 嘅 lines：每輪只有收到兩個 0 嘅 grandchild 印，第 k 輪有 4^{k-1} 個這樣嘅 process。

9.9 Buffering 陷阱

- `printf` 冇 `\n` 或者 output 去 file/pipe → **stdout buffer 未 flush 就 fork → buffer 被複製 → 重複印**。
- 考試通常講明「stdout is flushed after each printf」或「printf not buffered」。如果冇講，可以寫一句 assumption。

---

10. Interprocess Communication (IPC) ⭐ ⭐ [3.51–3.64]

10.1 Independent vs Cooperating

- **Cooperating processes can affect or be affected by other processes**（e.g. share data）。
- **Reasons for cooperation**：**information sharing**（copy & paste）、**computation speedup**（parallel subtasks）、**modularity**（+ convenience）。
- 需要 **IPC mechanism** 交換 data。**兩大模型**：
  1. **Shared memory** — a region of memory is shared；processes read/write the shared region（**implicit** communication；快，但要自己 synchronize）
  2. **Message passing** — **explicit** messages exchanged（**useful in distributed environment**；Mach OS 主要用；唔需要 share address space）

10.2 Producer–Consumer Problem（Bounded Buffer, Shared Memory）[3.53–3.56]

- **Unbounded buffer**：no practical limit；**bounded buffer**：fixed size。

```
#define BUFFER_SIZE 10
item buffer[BUFFER_SIZE]; int in = 0, out = 0;
/* Producer */
while (true) {
    while (((in + 1) % BUFFER_SIZE) == out) ;   /* buffer full → do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
/* Consumer */
while (true) {
    while (in == out) ;                         /* buffer empty → do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
}
```

- ⭐ **This solution only allows at most `BUFFER_SIZE − 1` items in the buffer at the same time**（因為 `in == out` 表示 empty，要留一格分辨 full）。

10.3 Message Passing [3.57–3.64]

- **Mechanism for processes to communicate and synchronize without sharing the address space.**
- 兩個 operations：`send(message)`、`receive(message)`；message size **fixed or variable**。
- P、Q 要通訊：**establish a communication link** → exchange via send/receive。

- Implementation 選擇：**direct or indirect**；**synchronous or asynchronous**；**automatic or explicit buffering**。

Direct communication [3.59]：

- **Processes must name each other explicitly**：`send(P, message)`、`receive(Q, message)`。
- Link **established automatically**；**associated with exactly one pair**；**between each pair exactly one link**。
- **Disadvantage：limited modularity** — 改一個 process 嘅 identifier 要檢查所有其他 processes。

Indirect communication [3.60–3.61]：

- **Messages sent to and received from mailboxes (ports)**；每個 mailbox 有 unique id；**share a mailbox 先可以通訊**。
- **A link may be associated with more than two processes**；**each pair may have several links**（each = one mailbox）。
- Operations：create mailbox、send/receive through mailbox、destroy mailbox；`send(A, msg)`、`receive(A, msg)`。
- **Mailbox sharing problem**（P1 send，P2 P3 都 receive，邊個收？）解決：
  1. **Allow a link to be associated with at most two processes**
  2. **Allow only one process at a time to execute** `receive()`
  3. **Allow the system to select the receiver arbitrarily; sender is notified who the receiver was**

Synchronization (blocking vs non-blocking) [3.62–3.63] ⭐：

	Blocking (synchronous)	Non-blocking (asynchronous)
Send	sender **blocked until the message is received** (by receiver or mailbox)	sender sends and **resumes**
Receive	receiver **blocks until a message is available**	receiver gets **a valid message or null**

- **Both send and receive blocking** → **rendezvous**；producer–consumer 變得 trivial（`send(next_produced)` / `receive(next_consumed)`）。

Buffering（message queue） [3.64] ⭐：

1. **Zero capacity** — no messages queued；**sender must wait for receiver (rendezvous)**
2. **Bounded capacity** — finite length *n*；**sender must wait if the link is full**
3. **Unbounded capacity** — infinite；**sender never waits**

---

11. Pipes ⭐⭐⭐ [3.65–3.69]

11.1 設計問題（4 個）

1. **Unidirectional or bidirectional?**
2. If bidirectional：**half-duplex**（一次一個方向）or **full-duplex**（同時雙向）？
3. **Must there be a relationship (parent–child)?**
4. **Can pipes be used over a network, or only on the same machine?**

11.2 Ordinary Pipes vs Named Pipes（必背表）

	Ordinary pipe	Named pipe
方向	**Unidirectional**（one-way）	**Bidirectional**（UNIX FIFO：**half-duplex**；Windows：**full-duplex**）
關係	**Requires parent–child relationship**	**No parent–child relationship required**
存取	**Cannot be accessed from outside the process that creates it**	**Several processes can use it**（multiple writers）
生命週期	**Ceases to exist after processes finish communicating and terminate**	**Continues to exist after processes finish, until explicitly deleted**
機器	Same machine	UNIX：same machine；**Windows：same or different machines**
UNIX API	`pipe(int fd[2])`：**fd[0] = read end，fd[1] = write end**	`mkfifo()`；**appear as typical files in the file system**（FIFOs）
Windows	**anonymous pipes**（`ReadFile()` / `WriteFile()`）	`CreateNamedPipe()`

- **UNIX treats a pipe as a special type of file → standard** `read()` **and** `write()`**；child inherits the pipe from the parent.**
- **Parent calls** `pipe()` **first, then** `fork()` → child inherits both ends。
- **If the parent writes and the child reads, the parent must close the read end, and the child must close the write end.** ⭐
- `ls | grep x`：shell 用 pipe 將 ls 嘅 output 接到 grep 嘅 input（tools 唔使特別設計都可以組合）。

> ⚠️ **MC 陷阱**：S26 MC12 ✗「**ordinary pipes can be accessed by arbitrary processes**」；Sp21 MC5 ✗「**named pipes cease to exist after the communicating processes have finished**」；HW1 MC10 同樣 ✗。F20 MC9：「multiple processes, no parent-child relationship, independent from network」→ **named pipes**（socket/RPC 需要 network）。

11.3 Pipe example（Parent → Child） [3.68]

```
int pfds[2]; char buf[30];
pipe(pfds);                         /* 1. create pipe BEFORE fork */
pid_t pid = fork();                 /* 2. child inherits pfds[0], pfds[1] */
if (pid != 0) {                     /* parent = writer */
    close(pfds[0]);                 /* close unused READ end */
    write(pfds[1], "test", 5);
    wait(NULL);
} else {                            /* child = reader */
    close(pfds[1]);                 /* close unused WRITE end */
    read(pfds[0], buf, 5);
    printf("CHILD: read \"%s\"\n", buf);
}
```

📝 **HW1 Q2.5 Model answer**：*(1) The parent calls `pipe(fd)` **before** `fork()` so that both processes share the same pipe (fd[0] read end, fd[1] write end). (2) Call `fork()`; the child inherits both file descriptors. (3) Parent (writer) closes fd[0] and writes data with `write(fd[1], ...)`, then closes fd[1]. (4) Child (reader) closes fd[1] and reads with `read(fd[0], ...)`, then closes fd[0]. Data flows parent → child (one-way).* **HW1 Q3.3(c)**：如果 parent 同 child **各自喺 fork() 之後 call pipe()** → 會建立**兩條獨立 pipe**，佢哋**唔共享** file descriptors → **cannot communicate**。 **HW1 Q3.3(b)**：`Child: 12` 然後 `Parent: done` — 因為 child 嘅 `read()` **blocks until data is written**，而 parent `wait(NULL)` 等 child 結束先印。

11.4 dup / dup2 & I/O redirection（Lab 3 / PA1） ⭐⭐

- **File descriptor table**：0 = stdin，1 = stdout，2 = stderr。
- `dup(oldfd)` — 複製 oldfd 到 **lowest-numbered unused fd**，返回新 fd。
- `dup2(oldfd, newfd)` — 令 newfd 指向 oldfd 嘅 file（如 newfd 已開會先 close）。
- **Input redirection 經典 pattern**：`close(0); dup(fin);` ⟹ fd 0 變成 file → `scanf` 由 file 讀。等價 `dup2(fin, 0);`
- **Output redirection**：`close(1); dup(fout);` 或 `dup2(fout, 1);`
- **Pipe 接 stdout → stdin**（`ls | wc`）：writer child：`dup2(fd[1], 1); close(fd[0]); close(fd[1]); execlp("ls",...)`；reader child：`dup2(fd[0], 0); close(fd[1]); close(fd[0]); execlp("wc",...)`；**parent 要 close 兩端**（否則 reader 永遠收唔到 EOF）。

S26 Q3.3（填空）：child 處理 1.txt、parent 處理 2.txt，數 even numbers，child 經 pipe 傳結果：

```
int fin = open(file_name[i], O_RDONLY);
close(0);                           // Blank 1
dup(fin);                           // Blank 2  → stdin now = file
scanf("%d", &length);
for (j = 0; j < length; j++) {
    scanf("%d", &number);
    if (number % 2 == 0) cnt++;     // Blank 3
}
if (pid == 0) { close(fd[0]); write(fd[1], &cnt, sizeof(int)); exit(0); }  // 4.1 fd[1], 4.2 &cnt
waitpid(pid, &status, 0); close(fd[1]);
read(fd[0], &child_cnt, sizeof(int));                                      // 5.1 fd[0], 5.2 &child_cnt
```

12. Client–Server Communication：Sockets & RPC ⭐ [3.70–3.72]

- **A socket is defined as an endpoint for communication.** A pair of processes over a network uses **a pair of sockets**。
- **A socket is identified by an IP address and a port number.**
- **Server** always running，**listens to a well-known port**；接受 request → 建立 socket connection。
- **Well-known ports（< 1024）**：**SSH 22、FTP 21、HTTP 80**。
- **Only clients can initiate communication.** Client 揀一個 **port number > 1024**（未用緊）。
- IP：IPv4 32-bit（128.32.244.172）、IPv6 128-bit。
- ⭐ **The 4-tuple (source IP address, source port number, destination IP address, destination port number) uniquely determines a pair of communication.**（F19 Q2.2 答案）
- **RPC (Remote Procedure Call) — procedure-call mechanism between systems with network connections；message-based**；常用於 **distributed file systems**（RPC daemons + clients）。

13. 常見陷阱 & MC 速查

敘述	真/假
Program = active, process = passive	✗（program passive；process active）
Stack holds function params, return addresses, local variables	✓
Heap holds global variables	✗（data section；heap = dynamic allocation）

敘述	真/假
Ready → Waiting transition exists	✗
Long-term scheduler controls degree of multiprogramming	✓
Short-term scheduler invoked infrequently	✗（very frequently, ms）
Medium-term scheduler does swapping	✓
Context switch = user→kernel mode switch	✗
Context-switch time is useful work	✗（pure overhead）
fork() returns 0 to parent	✗（0 to child；child PID to parent）
fork() fails → returns −1 to parent, no child	✓
After fork, child shares parent's variables	✗（copy；COW 只係 implementation 優化）
Parent automatically waits for child after fork	✗（需要 wait()）
exec() returns on success	✗（只有 failure 返回 −1）
Zombie still runs user code	✗
Zombie keeps PID & process-table entry	✓
Orphans are adopted by init/systemd	✓
Ordinary pipe: unidirectional, parent-child	✓
Named pipe ceases after processes end	✗
Named pipe (UNIX FIFO) full-duplex	✗（half-duplex；Windows named pipe 先 full-duplex）
Blocking send/receive = synchronous	✓
Zero-capacity buffer: sender never waits	✗（unbounded 先 never waits）
Bounded-buffer shared-memory solution holds BUFFER_SIZE items	✗（BUFFER_SIZE − 1）
Mode bit 1 = kernel	✗（1 = user，0 = kernel）
Privileged instruction in user mode → trap to OS	✓
DMA interrupts after every byte	✗
Trap = software-generated interrupt	✓
Interrupts all have same priority	✗

14. Past-paper 題目（Ch3）

- **S26**：MC1 interrupts（C）、MC2 DMA（B）、MC10 fork（C：7, X=6, Y=1）、MC11 context switch（B）、MC12 ordinary pipes false（B）、MC13 zombie（B）、MC14 leave running（A）、Q3.1 fork tree（7 / 3 / $2^{n+1}-1$）、Q3.2 fork+exec cases、Q3.3 pipe/dup 填空
- **F19**：Q2.1 state transitions、Q2.2 socket 4-tuple、Q2.4 fork+exec、Q2.6（$2^{n+1}-2$）、Q2.7（23）
- **F20**：MC3 interrupt、MC7 fork、MC8 address space（all of above）、MC9 named pipes、MC10 medium-term、Q3.1（8 outputs）、Q3.2（$2(3^n+1)$）
- **Sp21**：MC2 interrupt、MC degree of multiprogramming、MC own address space etc.（all）、MC stack、MC short-term、MC fork incorrect（B）、MC pipes not true（C）、MC leave CPU（all）、MC7（5）、MC8（27）、Q1.1（5）、Q1.2（488）、Q1.3 outputs
- **HW1**：MC1 DMA、MC2 interrupts、MC7 privileged、MC9 fork、MC10 pipes、Q2.3 states、Q2.5 pipe steps、Q3.1–3.3

15. 自我測驗

1. `for (i=0;i<4;i++) fork();` 有幾多個 processes？（16）
2. `fork(); fork() && fork(); fork();` 有幾多？（2 × 3 × 2 = **12**：第一個 fork → 2；`fork() && fork()` 每個變 3；最後 ×2）
3. Zombie 同 orphan 分別？
4. 點解 bounded buffer 只可以放 BUFFER_SIZE − 1 個 item？
5. Named pipe 喺 UNIX 叫咩？點建立？（FIFO；mkfifo()）
6. 寫出三種 mode transfer。（system call, interrupt, trap/exception）

4 Ch4 Threads 線程與並行

📌 **考試範圍提示**：Ch4 通常 **3–4 題 MC** + 有機會 1 題 short question。必考： **thread 有乜自己私有 / 乜同其他 thread 共享、benefits（4 個）、concurrency vs parallelism、data vs task parallelism、Amdahl's Law 計算、multithreading models（many-to-one / one-to-one / many-to-many）、signals（sync vs async）、thread cancellation（asynchronous vs deferred）、fork()/exec() semantics、Linux clone()、heavyweight vs lightweight**。

1. Motivation & Overview（動機） [4.4–4.6]

1.1 點解要 multithreading？

- 好多 applications 本身係 multithreaded：
  - **Web browser**：一個 thread display images/text，另一個 thread 由 network retrieve data。
  - **Word processor**：display graphics、respond to keystrokes、background spelling & grammar checking。
- **Most OS kernels are multithreaded**（Linux boot 時建立多個 kernel threads：managing devices、memory management、interrupt handling）。
- **Take advantage of processing capabilities on multicore systems**（data mining、graphics、AI）。
- ⭐ **Process creation is heavy-weight while thread creation is light-weight** — threads of the same process **share code, data and other resources**。
- 其他例子 [4.5]：embedded systems（elevators, planes）、OS kernels、**database servers**、**network servers**（file/web server、airline reservation）、parallel programming。

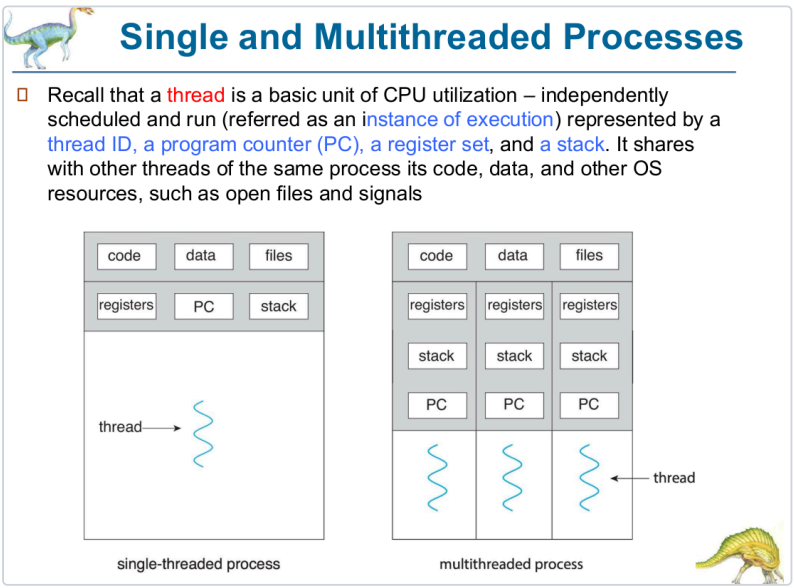
1.2 Thread 定義 ⭐ ⭐ [4.6]

- **A thread is a basic unit of CPU utilization** — independently scheduled and run（**an instance of execution**）。
- **每個 thread 私有（per-thread）：thread ID, program counter (PC), register set, stack**
- **同一 process 嘅 threads 共享（shared）：code (text), data (global variables), heap, open files, signals**（及其他 OS resources）

```
single-threaded process          multithreaded process

┌──────┬──────┬──────┐           ┌──────┬──────┬──────┐
│ code │ data │ files│           │ code │ data │ files│   ← shared
├──────┼──────┼──────┤           ├──────┼──────┼──────┤
│ regs │  PC  │stack │           │ regs │ regs │ regs │   ← private
└──────┴──────┴──────┘           │stack │stack │stack │      (per thread)
                                 │  PC  │  PC  │  PC  │
                                 └──────┴──────┴──────┘
```

📝 **F19 Q2.3**：unique to each thread → **program counter, registers, stack, thread ID (thread state)**；shared → **code, data, files (heap)**。 ⚠️ **F20 MC11**：NOT shared by threads in the same process → **Stack**。



Single- vs multithreaded process [4.6]

2. Benefits（四大好處） ⭐ ⭐ [4.7]

Benefit	解釋
Responsiveness	**may allow continued execution if part of a process (one thread) is blocked** — 對 interactive / UI applications 特別重要
Resource sharing	**threads share the resources of a process by default** — 比 shared memory / message passing 容易（同一 address space）
Economy	**thread creation is much cheaper than process creation**（less time & memory）；**context switching between threads is faster** than between processes
Scalability	process **can take advantage of multicore architectures**（threads run in parallel）

📝 HW2 MC1：app 喺一個 thread blocked 時仍然 respond user → **Responsiveness**。

2.1 Heavyweight vs Lightweight（F20 Q2.3） ⭐

📝 **Model answer**：*A process is called a heavyweight process because creating it requires allocating a new address space and duplicating the program, data and other resources (PCB, memory, open files) — time- and resource-consuming. A thread is lightweight because it only contains an execution unit (thread ID, PC, registers, stack, TCB) and shares the code, data and open files of its process; creating a thread needs no duplication of program and data, and context switching between threads of the same process is cheaper since the address space stays the same.*

---

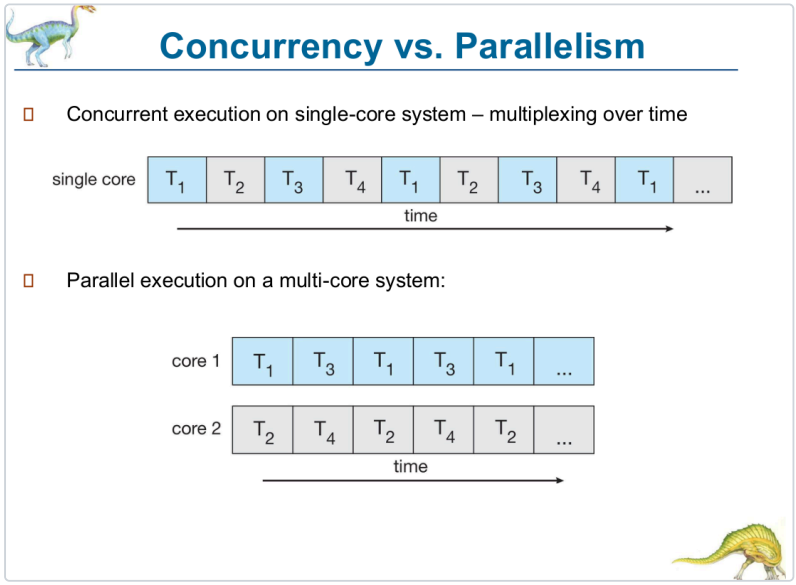
3. Multicore Programming（多核心編程） ⭐ [4.8–4.11]

3.1 五大挑戰 [4.9]

1. **Dividing tasks** — how to divide into separate, concurrent tasks
2. **Balance** — each task performs an "equal" amount of work（value）
3. **Data splitting** — data must be divided to run on separate cores
4. **Data dependency** — **if there is a data dependency, synchronization is required**
5. **Testing and debugging** — **different execution paths make debugging difficult**

3.2 Concurrency vs Parallelism ⭐ ⭐ [4.9–4.10]

- **Parallelism** — **a system can perform more than one task simultaneously**（需要 multicore / multiprocessor）。
- **Concurrency** — **supports more than one task making progress**；**on a single core, the scheduler provides concurrency by multiplexing over time**。
- **Single-core：only concurrency is possible；multicore：parallelism possible.**
- **Concurrency without parallelism 係可以嘅；parallelism 一定 concurrent。**



Concurrency vs parallelism [4.10]

```
Single core (concurrency):   T1 T2 T3 T4 T1 T2 T3 T4 T1 ...   (time →)
Two cores (parallelism):     core 0: T1 T3 T1 T3 ...
                             core 1: T2 T4 T2 T4 ...
```

⚠️ **S26 MC16**：✓「**Concurrency can be supported on a single processor core through time multiplexing by the scheduler**」。✗「Concurrency means performing more than one task at exactly the same time」（嗰個係 parallelism）✗「Parallelism means tasks only make progress over time on a single core」✗「Testing and debugging become easier in multicore systems」 **Sp21 MC**：multiple tasks make progress on a single processor → **concurrency**。

3.3 Data vs Task Parallelism ⭐ [4.11]

- **Data parallelism — distributes subsets of the same data across multiple cores, same operation on each core**（e.g. 分開 sum 一個 array 嘅兩半；distributed ML）。
- **Task parallelism — distributes threads (tasks) across cores, each thread performing a unique operation**。
- **Not mutually exclusive** — an application may use both（hybrid）。

📝 HW2 MC2：「Threads on different cores perform the **same operation on different subsets of an array**」→ **data parallelism**。「One thread handles input, another does spell-check」→ task parallelism。

---

4. Amdahl's Law ⭐⭐⭐（計數題必考）[4.12–4.13]

Amdahl's Law

$$\text{speedup} \leq \frac{1}{S + \dfrac{1-S}{N}}, \qquad \lim_{N\to\infty} \text{speedup} = \frac{1}{S}$$

- **S = serial portion，$1-S$ = parallel portion，N = number of processing cores**。
- **As $N \to \infty$, speedup → $1/S$.**
- **Serial portion has a disproportionate effect on performance gained by adding cores.**
- Amdahl's Law identifies **theoretical speedup in latency at a fixed workload**。

4.1 例子（全部計算過）

題目	S	N	計算	Speedup
Slide：75% parallel	0.25	2	$\frac{1}{0.25+0.75/2} = \frac{1}{0.625}$	**1.6**
Sp21 MC：30% serial	0.3	7	$\frac{1}{0.3+0.7/7} = \frac{1}{0.4}$	**2.5**
HW2 Q3.1(a)：20% serial	0.2	4	$\frac{1}{0.2+0.8/4} = \frac{1}{0.4}$	**2.5**
HW2 Q3.1(b)	0.2	∞	$\frac{1}{S} = \frac{1}{0.2}$	**5**
S26 MC17 A：81% parallel	0.19	3	$\frac{1}{0.19+0.81/3} = \frac{1}{0.46}$	2.17
S26 MC17 B：81% parallel	0.19	5	$\frac{1}{0.19+0.81/5} = \frac{1}{0.352}$	2.84
S26 MC17 C：61% parallel	0.39	3	$\frac{1}{0.39+0.61/3} = \frac{1}{0.5933}$	**1.69 ✓（1.6–1.7）**
S26 MC17 D：61% parallel	0.39	5	$\frac{1}{0.39+0.61/5} = \frac{1}{0.512}$	1.95

4.2 解題 tips

- 留意題目畀 **parallel % 定 serial %**（S = serial！）。
- 反向題：要 speedup = X with N cores → 解 S：$S = \dfrac{1/X - 1/N}{1 - 1/N}$。
- 例：想 4 cores 有 3× speedup → $S = \dfrac{1/3 - 1/4}{1 - 1/4} = \dfrac{1/12}{3/4} = \dfrac{1}{9} \approx$ **11.1% serial**。
- Speedup 上限永遠 $\leq N$ 同 $\leq 1/S$。

---

5. Multithreaded Process & TCB ⭐ [4.14–4.18]

5.1 Multithreaded process [4.14]

- **More than one instance of execution**，每個有自己 PC。
- **Threads share the same address space → can access the same data.**
- **Each thread has its own private set of registers.**
- **Context switch between two threads (T1 → T2) of the same process**：save T1's register state, restore T2's；**address space remains the same → overhead is much smaller。Switching to a thread of a different process → more overhead.**
- **Overlap of I/O with other activities within a single program**（一個 thread 用 CPU，另一個做 I/O）。

5.2 Thread Control Block (TCB) ⭐ [4.15]

- **Thread = single unique execution context = lightweight process**：PC, registers, execution flags, stack。
- Thread 喺 processor 執行 = **resident in the registers**；**PC register holds the address of the executing instruction**。
- **Each thread has a TCB**：
  - **Execution state**：CPU registers, program counter, **pointer to stack**
  - **Scheduling info**：state, priority, CPU time
  - **Accounting info**
  - **Various pointers**（implementing scheduling queues）
  - **Pointer to enclosing process (PCB)**
- ⚠️ Sp21 MC：**NOT in TCB → Process identifier**（PID 喺 PCB；TCB 只有 pointer to enclosing process）。

5.3 Shared vs Private state [4.16] ⭐

- **Threads encapsulate concurrency — "active" component；address space encapsulates protection — "passive" part.**
- **Shared by all threads in a process**：**contents of memory (global variables, heap)**；**I/O state (file descriptors, network connections)**
- **Private to each thread (kept in TCB)**：**CPU registers (including PC)**；**execution stack (parameters, temporary variables, saved PC)**
- **If a process has multiple threads, there is no "process state", only the state of each thread**（summary）。

5.4 Thread lifecycle [4.17–4.18]

- **new → ready → running → waiting → terminated**（同 process 一樣）。
- **"Active" threads are represented by their TCBs; TCBs are organized into queues by state**（ready queue、各 device queues：tape unit、disk unit、ethernet...，每個 queue 有自己 scheduling policy）。

> 📝 **HW2 Q2.1**：同一 process 兩個 threads 可唔可以有唔同 PC / register 值？→ **Yes** — 每個 thread 係獨立 flow of execution，有自己 PC、register set、stack；唔 running 時存喺 **TCB (Thread Control Block)**。

6. User Threads vs Kernel Threads ⭐ ⭐ [4.19–4.20]

- **User threads** — independently executable entities within a program, **created and managed by a user-level thread library**；**visible only to programmers and unknown to the kernel → cannot be scheduled on a CPU directly**。
- **Kernel threads** — **supported and managed by the OS**；**can be scheduled and executed on a CPU**（Windows, Linux, macOS, iOS, Android 都支援）。
- ⭐ **OS only manages and schedules kernel threads.**（F19 Q3.1：OS 排程嘅係 **kernel-level threads**）
- **Three primary thread libraries**：**POSIX Pthreads**、**Windows threads**、**Java threads**（對應 POSIX API、Win32 API、Java API）。
- F20 MC12：provides API for creating and managing threads → **Thread library (e.g. POSIX Pthreads)**。
- If a program defines no user threads, there is no kernel thread to schedule for it（user threads = concurrent & modular entities OS 可利用）。
- **TCB 只同 kernel thread 關聯**；**kernel thread is the only thing the CPU scheduler schedules**（summary）。

7. Multithreading Models ⭐ ⭐ ⭐ [4.21–4.26]

Model	Mapping	優點	缺點	例子
Many-to-One	many user threads → **1 kernel thread**	thread management 喺 user space（efficient）	**one thread blocking (blocking system call) blocks the entire process**；**cannot run in parallel on multicore**（only one kernel thread active at a time）	**Solaris Green Threads, GNU Portable Threads**（few systems now）
One-to-One	each user thread → **its own kernel thread**	**maximum concurrency**（another thread runs when one blocks）；**parallel on multicore**	**creating a user thread requires creating a kernel thread → overhead**；**number of threads per process may be restricted**	**Windows, Linux（most common now）**
Many-to-Many	many user threads → **smaller or equal number** of kernel threads	flexible；OS 可預先建立足夠 kernel threads（**thread pool**）；user threads 數目唔受 kernel threads 限制	**difficult to implement** in practice	Windows ThreadFiber package（otherwise uncommon）
Two-level	many-to-many + **allows a user thread to be bound to a kernel thread**	更靈活	複雜	—

- **Many-to-one 同 many-to-many 要決定邊個 user thread map 去 kernel thread → process-contention scope (PCS)**（Ch5）。

- **Concurrency 比較**：many-to-one < many-to-many < one-to-one。
- **Most OSes now use the one-to-one model**（cores 多咗，限制 kernel threads 數目冇咁重要）[4.26]。
- **LWP (lightweight process)**（summary）：user 同 kernel threads 之間嘅 intermediate data structure；**對 thread library 嚟講 LWP 好似一個 virtual processor**，library 將 user thread schedule 落 LWP；**每個 LWP attached to a kernel thread**；kernel thread block → LWP 都 block。

> ⚠️ **MC 陷阱**：
> - F20 MC13 ✗「**A user thread can be mapped to more than one kernel thread**」（冇 one-to-many）；✓ kernel threads scheduled by OS；✓ users manage user threads；✓ one user thread → at most one kernel thread。
> - Sp21 MC ✗「**The many-to-many model is easy to implement in practice**」。✓ many-to-one = many user → single kernel；✓ one-to-one more concurrency than many-to-one；✓ most OS now one-to-one。
> - **HW2 Q2.2**：many-to-one 喺 4 cores 可唔可以 parallel？→ **No**，只有一個 kernel thread，OS 只 schedule kernel threads，一次只可以喺一個 core 上跑（只有 concurrency 冇 parallelism）；一個 thread blocking system call 仲會 block 成個 process。

---

8. Threading Issues ⭐ ⭐ [4.27–4.33]

8.1 fork() and exec() semantics [4.28]

- **Does `fork()` called by a thread duplicate only the calling thread or all threads?**
- **Some UNIX systems have two versions of fork().**
- **If `exec()` is called immediately after forking → duplicating only the calling thread is appropriate**（exec 會取代成個 process）。
- **`exec()` works as normal — replaces the entire process including all threads.**

8.2 Signal Handling ⭐ ⭐ [4.29–4.31]

- **Signals notify a process that a particular event has occurred.**
- **Synchronous signals — illegal memory access, division by 0 → delivered to the same process that performed the operation causing the signal**。
- **Asynchronous signals — generated by an event external to the running process：terminating with 、timer expires**（`kill`）。
- 處理：有啲 ignore，有啲（illegal memory access）terminate program。
- **Signal handling pattern**：(1) **generated** by an event → (2) **delivered** to a process → (3) **handled** by a **default** or **user-defined** handler。
- **Every signal has a default handler (kernel); a user-defined handler can override the default.**
- **Multithreaded process 要 deliver 去邊？四個選擇**：
  1. **To the thread to which the signal applies**
  2. **To every thread in the process**
  3. **To certain threads in the process**
  4. **Assign a specific thread to receive all signals for the process**
- **Synchronous signals → delivered to the thread causing the signal**（not others）。
- **Some asynchronous signals (e.g. terminate process, Ctrl-C) → sent to all threads.**
- UNIX：`kill(pid_t pid, int signal)`（deliver to a process）；Pthreads：`pthread_kill(pthread_t tid, int signal)`（deliver to a specific thread）。
- ⚠️ 留意：slide 3.41 將 `signal()` 寫成「send a notification to another process」，但實際 **deliver** 用 `kill()`（4.31）；`signal(SIGINT, handler)` 係**安裝 user-defined handler**（PA1）。MC 照題目用字揀。

> ⚠️ **S26 MC18** ✗「**All signals should be delivered to all threads in a process**」。✓ signals can be received sync or async；✓ users can define custom handlers；✓ different signals can have different handlers。 HW2 MC3：**division by zero → synchronous；timer expiration → asynchronous**。 **HW2 Q2.3 (Ctrl+C)**：*Asynchronous — Ctrl+C makes the terminal send SIGINT, generated by an event external to the running process. Yes, the program can handle it differently: register a user-defined handler with `signal(SIGINT, handler)` (or `sigaction`), which overrides the default handler (terminate). (SIGKILL/SIGSTOP cannot be caught.)* — PA1 myshell 就係用 `signal(SIGINT, handle_sigint)`。

8.3 Thread Cancellation ⭐ [4.32–4.33]

- **Terminating a thread before it has finished；the thread to be cancelled = target thread.**
- **Asynchronous cancellation — terminates the target thread immediately**（可能喺 critical update 中途 → OS 未必可以 reclaim 所有 resources）。
- **Deferred cancellation — the target thread periodically checks whether it should terminate, allowing it to terminate itself in an orderly fashion**（**preferred**）。
- **Invoking cancellation only requests it; actual cancellation depends on thread state and type.**
- **Pthreads：if cancellation is disabled, it remains pending until enabled；default type is deferred.**

- Code [4.33]：

```
#include <pthread.h>
pthread_t tid;
pthread_create(&tid, 0, worker, NULL);   /* worker = start routine */
pthread_cancel(tid);                     /* request cancellation */
pthread_join(tid, NULL);                 /* wait for thread to terminate */
```

> 📝 HW2 MC4：deferred cancellation → **target thread periodically checks whether it should terminate**。

8.4 Thread-Local Storage（課本補充，Sp21 MC 考過）

- **Thread-local storage (TLS) is data that is unique to each thread**（類似 static data 但每個 thread 一份；同 local variables 唔同，local 只喺一個 function call 內存在）。

---

9. OS Examples [4.34–4.38]

9.1 Windows Threads

- Windows application = separate process，每個 process 可有多個 threads；**Windows uses the one-to-one mapping**。
- Thread 組成：**thread ID, register set, program counter, separate user and kernel stacks, private data storage area**（run-time libraries / DLLs 用）。
- **Register set + stacks + private storage area = context of the thread.**
- 數據結構：
  - **ETHREAD (executive thread block)** — pointer to process & to KTHREAD（kernel space）
  - **KTHREAD (kernel thread block)** — scheduling & synchronization info, **kernel-mode stack**, pointer to TEB（kernel space）
  - **TEB (thread environment block)** — thread id, **user-mode stack**, **thread-local storage**（**user space**）

9.2 Linux Threads ⭐ [4.37–4.38]

- **Linux does not differentiate between threads and processes — both are called tasks.**
- **A thread in Linux is a process that may share certain resources with other threads**；每個 thread 有自己 `task_struct`。
- 對比 Windows：Windows 一個 PCB（shared resources）指向兩個 TCB；**Linux 就係兩個 normal task_struct，set up to share resources**。
- `clone()` 用 flags 指定 share 咩：

```
clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);
```

- CLONE_VM：share **address space**；CLONE_FS：share file-system info；CLONE_FILES：share **open files**；CLONE_SIGHAND：share **signal handlers**。
- **Instead of copying all data structures like fork(), the new task points to the parent's data structures depending on the flags.**
- **A normal `fork()` can be implemented as `clone(SIGCHLD, 0)`**（七都唔 share → 等於 process）。

---

10. Pthreads API 速查（Lab 4 / Ch6 都用）

```
#include <pthread.h>
int pthread_create(pthread_t *tid, const pthread_attr_t *attr, void *(*start)(void *), void *arg);
int pthread_join(pthread_t tid, void **retval);     /* wait for thread tid */
void pthread_exit(void *retval);
int pthread_cancel(pthread_t tid);
pthread_t pthread_self(void);
/* mutex */
pthread_mutex_t m; pthread_mutex_init(&m, NULL);
pthread_mutex_lock(&m); /* critical section */ pthread_mutex_unlock(&m);
/* semaphore */
#include <semaphore.h>
sem_t s; sem_init(&s, 0 /*shared between threads*/, 1 /*initial value*/);
sem_wait(&s); /* P / wait: decrement or block */ sem_post(&s); /* V / signal: increment */
```

- 編譯：`gcc prog.c -lpthread`（或 `-pthread`）。
- 典型 pattern：

```
void *worker(void *arg) { int id = *(int *)arg; ...; return NULL; }
int main() {
    pthread_t t[4]; int ids[4];
    for (int i = 0; i < 4; i++) { ids[i] = i; pthread_create(&t[i], NULL, worker, &ids[i]); }
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL);
}
```

- ⚠️ 傳 `&i`（loop variable）畀多個 thread 會 race（i 會變）→ 用 `ids[i]` array。

11. 常見陷阱 & MC 速查

敘述	真/假
Each thread has its own address space and file descriptors	✗（share）
Threads of the same process cannot run concurrently on multiple cores	✗（one-to-one 可以 parallel）
Thread creation is usually faster than process creation	✓（S26 MC15）
Multithreaded process can't use multicore	✗
Stack is shared among threads	✗
Heap / global data shared	✓
Concurrency possible on single core	✓
Parallelism possible on single core	✗
Data parallelism = same operation on different data subsets	✓
As $N \to \infty$, Amdahl speedup → $1/S$	✓
Speedup can exceed N	✗
OS schedules user threads directly	✗（kernel threads）
Many-to-one allows parallelism on multicore	✗
Many-to-one: one blocking call blocks whole process	✓
One-to-one: maximum concurrency, but overhead of kernel threads	✓
Many-to-many easy to implement	✗
User thread can map to multiple kernel threads	✗
Most OS use one-to-one now	✓
Division by zero → asynchronous signal	✗（synchronous）
Ctrl-C → asynchronous	✓
All signals delivered to all threads	✗
User-defined handler can override default	✓
Asynchronous cancellation = target checks periodically	✗（嗰個係 deferred）
Pthreads default cancellation type = deferred	✓
Linux distinguishes threads and processes	✗（both tasks）
fork() = clone(SIGCHLD, 0)	✓
PID is in TCB	✗
Windows uses one-to-one	✓

12. Past-paper 題目（Ch4）

- **S26 MC15**：true about multithreaded process → **C. Thread creation is usually faster than process creation**
- **S26 MC16**：concurrency/parallelism → **C**
- **S26 MC17**：Amdahl 1.6–1.7 → **C (61%, 3 cores) = 1.69**
- **S26 MC18**：false about signals → **C. All signals delivered to all threads**
- **F19 Q2.3**：unique vs shared（§1.2）；**Q3.1**：OS schedules kernel threads；user-thread → kernel-thread/LWP scheduling = **PCS (process-contention scope)**，within the same process, based on priority set by programmer；CPU scheduling (SCS) 喺全系統 kernel threads 之間揀
- **F20 MC11** stack；**MC12** thread library；**MC13** user thread → multiple kernel threads（false）；**Q2.3** heavyweight vs lightweight
- **Sp21 MC**：Amdahl 30% serial 7 cores → **2.5**；TCB 唔包括 **process identifier**；single processor 多 task progress → **concurrency**；many-to-many easy to implement（false）；TLS → **unique to each thread**
- **HW2**：MC1 responsiveness、MC2 data parallelism、MC3 sync/async、MC4 deferred、Q2.1 TCB、Q2.2 many-to-one、Q2.3 Ctrl+C、Q3.1 Amdahl（2.5；5）

13. 自我測驗

1. 列出 multithreading 四個 benefits。
2. 50% serial 嘅 program 用 4 cores speedup？（$\frac{1}{0.5 + 0.5/4} = \frac{1}{0.625}$ = **1.6**）；無限 cores？（$1/S$ = **2**）
3. Many-to-one model 兩大缺點？
4. 點解 exec() 緊接 fork() 時只複製 calling thread 就夠？

5. Deferred vs asynchronous cancellation？邊個 preferred？點解？
6. Linux 點樣建立 thread？（clone() with CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND）

5 Ch5 CPU Scheduling CPU 排程

📌 **考試範圍提示：Ch5 係分數最重嘅一章**（S26：27/100；F19：40/100；F20：27+；Sp21：26）。必考：

1. **Gantt chart + average waiting / turnaround / response time**：FCFS、SJF、SRTF、RR、Priority（± RR）、**MLFQ**（每年都有！）、自創 policy（LUF、dynamic priority）
2. **RM / EDF** real-time Gantt + missed deadlines
3. MC / short：preemptive vs non-preemptive、convoy effect、starvation & aging、MLFQ 優點、RR vs FCFS、exponential averaging、Little's formula、PCS vs SCS、processor affinity、load balancing、hard vs soft real-time ⚠️ **Response time 本科定義 = 第一次運行「完結」時間 − arrival time**（slide 5.12），唔係課本「第一次開始」！

---

1. Basic Concepts（基本概念） [5.4–5.7]

1.1 CPU–I/O Burst Cycle

- **Objective of multiprogramming：have a process running at all times → maximize CPU utilization.**
- **Process execution consists of cycles of CPU execution and I/O wait — CPU burst and I/O burst.**
- [5.5] **Histogram of CPU-burst times：many short CPU bursts, few long ones (long-tail distribution)。I/O-bound → many short CPU bursts；CPU-bound → a few long CPU bursts.** 呢個分佈對設計 scheduling algorithm 好重要。
- CPU idle 時，OS 由 ready queue 揀一個 process（除非 ready queue empty）→ 由 **CPU scheduler（process scheduler / short-term scheduler）** 揀。

1.2 When does scheduling happen? ⭐ ⭐ [5.6]

CPU scheduling 可能喺四種情況發生：

1. **Running → waiting**（e.g. I/O request, `wait()`）
2. **Process terminates**
3. **Running → ready**（e.g. **interrupt / timer**）
4. **Waiting → ready**（I/O completion）或者 **new → ready**（新 process 到達，priority 更高）

- **Scheduling under 1 and 2 is non-preemptive (cooperative)** — process **gives up the CPU voluntarily**。
- **Scheduling under 3 and 4 is preemptive.**
- Preemptive 要考慮：**access to shared data (race condition, Ch6)**、**preemption while in kernel mode**、**interrupts during crucial OS activities**。

> 📝 Sp21 MC：CPU scheduling decisions may take place in → **all of the above**（running→waiting、terminates、running→ready）。 📝 HW2 MC5：preemptive 例子 → **time quantum expires, running process moved back to the ready queue**（A terminate、B I/O、D wait() 都係自願 → non-preemptive）。

1.3 Dispatcher [5.7]

- **Dispatcher module gives control of the CPU to the process selected by the CPU scheduler**：
  1. **switching context** from one process to another
  2. **switching to user mode**
  3. **jumping to the proper location in the user program** to restart it
- **Dispatch latency — time for the dispatcher to stop one process and start another（overhead）**。應該越快越好。
- `vmstat`（Linux）可睇 context switches 數目（每秒幾百次）。

---

2. Scheduling Metrics（排程指標） ⭐ ⭐ ⭐ [5.8, 5.12]

Metric	定義	目標
CPU utilization	**fraction of the time that the CPU is busy**	max
Throughput	**number of processes completed per unit time**（favors short processes）	max
Turnaround time (TAT)	**time to execute a particular process = completion time − arrival time**	min
Waiting time (WT)	**total time spent waiting in the ready queue = TAT − CPU burst**	min
Response time (RT)	**time from submission until the first response is produced**	min
Fairness	adequate allocation for different jobs/processes	—

2.1 公式（本科 convention，必背）⭐

Metrics（本科 convention）

$$\text{Turnaround} = \text{Completion} - \text{Arrival}$$
$$\text{Waiting} = \text{Turnaround} - \text{CPU burst}$$
$$\text{Response} = (\text{end of the FIRST run}) - \text{Arrival}$$

- **Response time 嘅「first run」完結** = 第一次上 CPU 嘅嗰段（RR / MLFQ 就係 **end of first quantum**，或者 burst 細過 quantum 就係完成時間）[5.12]。
  - Slide 5.13 例：RR q=4，P1(24), P2(3), P3(3) 全部 t=0 到 → **RT：P1 = 4, P2 = 7, P3 = 10，avg = 7**。
  - S26 MLFQ 官方：P1 response = 4 − 0 = 4（P1 跑 0–4）。
  - **非 preemptive (FCFS/SJF) 嘅 RT = TAT**（第一次 run 就跑完）。
- Waiting time 另一個計法（**gap method**）：將 Gantt chart 上 process 唔喺 CPU 嘅時間（由 arrival 起）加埋。兩個方法結果一樣：
  - S26 P2：gap = (4−2)+(21−8)+(40−29)+(57−45)+(64−58) = 44 ✓；formula = 71 − 2 − 25 = 44 ✓。
- S26 MC19 ✓「**Response time is the amount of time from when a request is submitted until the first response is produced**」。✗ WT = arrival 到完成（嗰個係 TAT）；✗ TAT = 等 ready queue 時間（嗰個係 WT）；✗ throughput = CPU busy fraction（嗰個係 utilization）。

---

3. FCFS — First-Come, First-Served ⭐ [5.9–5.10]

- 按 **arrival order** 排；**non-preemptive**（跑完 current CPU burst 先放）。
- 例：P1 = 24, P2 = 3, P3 = 3（全部 t=0）
  - 次序 P1, P2, P3：`0 P1 24 P2 27 P3 30` → **WT = 0, 24, 27 → avg 17；TAT avg** $= \frac{24+27+30}{3} = 27$
  - 次序 P2, P3, P1：`0 P2 3 P3 6 P1 30` → **WT = 6, 0, 3 → avg 3；TAT avg = 13**
- ⭐ **Convoy effect — short processes stuck behind a long process**，令 average waiting time 好長，完全取決於 arrival order。One CPU-bound + many I/O-bound → **low I/O device utilization**。（例：銀行排隊存張支票，前面有人開戶）

📝 HW2 MC6：convoy effect → **several short processes wait behind a long process that arrived first**。 📝 **F19 Q3.2**：FCFS 問題 = **convoy effect**（short process stuck behind a long process）。

---

4. RR — Round Robin ⭐⭐⭐ [5.11–5.16]

- **FCFS + preemption**：每個 process 得 **time quantum q**（通常 **10–100 ms**）；用完 → **preempted and added to the end (tail) of the ready queue**。
- **n processes → each gets 1/n of CPU time in chunks of at most q；no process waits more than (n − 1)q time units** ⭐（ignoring context switch）。
- **Timer interrupts every quantum**；如果 burst（或 remaining）< q，process 自己完成就放 CPU。
- 例 [5.13]：q = 4，P1 24, P2 3, P3 3 → `0 P1 4 P2 7 P3 10 P1 14 P1 18 P1 22 P1 26 P1 30`
  - **WT：P1 = 6, P2 = 4, P3 = 7 → avg 5.67；TAT avg** $= \frac{30+7+10}{3} = 15.67$ **；RT = 4, 7, 10 → avg 7**
- **RR：average waiting time can be long, but more "fair"；better for short jobs than FCFS；better average response time → good for interactive jobs.**

4.1 Time quantum 大細 [5.14, 5.16] ⭐

- **q large → RR degenerates to FCFS.**（Sp21 MC 答案：**FCFS**）
- **q small → many context switches; q must be large w.r.t. context-switch time (< 10 µs), otherwise overhead too high.**（q 極細 = **processor sharing**：好似 n 個 processor 每個 1/n 速度）
- **Average turnaround time does not necessarily improve as q increases.**
- **TAT improves if most processes finish their CPU burst within a single quantum.**
- **Rule of thumb：80% of CPU bursts should be shorter than q.**

4.2 FCFS vs RR ⭐（F19 Q3.2 考過）[5.15]

- **Is RR always better than FCFS? No!** 例：**10 jobs start at the same time, each needs 100 s, q = 1 s → average turnaround time much worse under RR**（全部差唔多 1000 s 先完；FCFS 平均 550 s），**but average response time much better**。
- 一般而言 **no clear winner between FCFS and RR** in TAT/WT — 取決於 burst times、arrival pattern、quantum size。

📝 **F19 Q3.2 Model answer**：*RR performs worse than FCFS in turnaround time when all processes have the same (equal) CPU burst times that are longer than the time quantum — the jobs are interleaved and all finish near the end, whereas FCFS finishes them one by one.*

4.3 RR 做 Gantt chart 規則（考試用）

1. 用 **queue**（寫低每個時刻 queue 內容！）。
2. **新到達 process 加入 queue tail；quantum 用完嘅 process 都去 tail**。
3. ⚠️ **同一時刻**「新 process 到達」同「running process quantum 用完」→ convention：**新到達嗰個先入 queue，然後先放被 preempt 嗰個**（寫低 assumption）。
4. Process burst 完 → 即刻揀下一個（唔使等 quantum 完）。
5. Queue 空而 running process quantum 用完 → 佢繼續跑（再得一個 quantum）。

---

5. SJF / SRTF — Shortest Job First ⭐ ⭐ ⭐ [5.17–5.24]

5.1 SJF（non-preemptive）

- **Associate each process with the length of its next CPU burst; schedule the shortest.**（更準確叫 **shortest-next-CPU-burst**）
- ⭐ **SJF is optimal — gives the minimum average waiting time for a given set of processes.**
- **Difficulty：knowing the length of the next CPU request.**（所以 practical 難 implement；F20 MC15 答案 **SJF difficult to implement**）
- **Sketch proof：moving a short process before a long one decreases the waiting time of the short process more than it increases the waiting time of the long process → average waiting time decreases.**
- 例 [5.18]：P1 = 6, P2 = 8, P3 = 7, P4 = 3。Slide 個 chart **當全部 t = 0 到**：`0 P4 3 P1 9 P3 16 P2 24` → **avg WT** $= \frac{3+16+9+0}{4} = 7$（slide 答案）。
  - ⚠️ 如果真係跟表入面 arrival（0, 2, 4, 5）：t=0 只有 P1 → `0 P1 6 P4 9 P3 16 P2 24` → WT = 0, 14, 5, 1 → avg 5。考試**一定要睇清楚有冇 arrival time**。

5.2 SRTF — Shortest-Remaining-Time-First（preemptive SJF）⭐

- **New arrival with burst shorter than the remaining time of the running process → preempts it.**
- Scheduling 喺 **every arrival (condition 4)** 都要 invoke。
- 例 [5.23]：P1(0,8), P2(1,4), P3(2,9), P4(3,5) → `0 P1 1 P2 5 P4 10 P1 17 P3 26`
  - **avg WT** $= \frac{(10-1)+(1-1)+(17-2)+(5-3)}{4} = \frac{26}{4} = 6.5$

5.3 比較 [5.24] ⭐

- **SJF/SRTF are the best for minimizing average waiting time (and average turnaround time).**
- **Provably optimal：SJF among non-preemptive, SRTF among preemptive. SRTF is always at least as good as SJF.**
- **SJF/SRTF = FCFS if all processes have the same CPU burst.**
- **Can lead to starvation of long processes if shorter processes keep arriving；fairness cannot be enforced.**
- F20 MC14：best (min) average waiting time → **SJF**（**SRTF** 亦接受）。

5.4 Exponential Averaging（預測下一個 CPU burst）⭐ ⭐ [5.19–5.22]

Exponential averaging

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n, \qquad 0 \le \alpha \le 1$$

- t_n **= actual length of the n-th CPU burst**；τ_{n+1} **= predicted value for the next burst**；通常 $\alpha = 1/2$。
- $\alpha = 0 \Rightarrow \tau_{n+1} = \tau_n$（**recent history does not count**）；$\alpha = 1 \Rightarrow \tau_{n+1} = t_n$（**only the last burst counts**）。
- 展開 → **all past values count but recent ones weigh more；past effect diminishes exponentially**：

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\alpha\, t_{n-1} + \cdots + (1-\alpha)^j \alpha\, t_{n-j} + \cdots + (1-\alpha)^{n+1}\tau_0$$

- 例：$\alpha = 0.5$，10 rounds 後 $(1/2)^{10} = 1/1024$ → 可忽略。
- 預測取決於 **(1) all previous actual bursts with different weights, (2) the initial estimate** τ_0。
- Slide 5.20 例（$\alpha = 1/2$, $\tau_0 = 10$）：

i	0	1	2	3	4	5	6	7
CPU burst t_i	6	4	6	4	13	13	13	…
guess τ_i	10	8	6	6	5	9	11	12

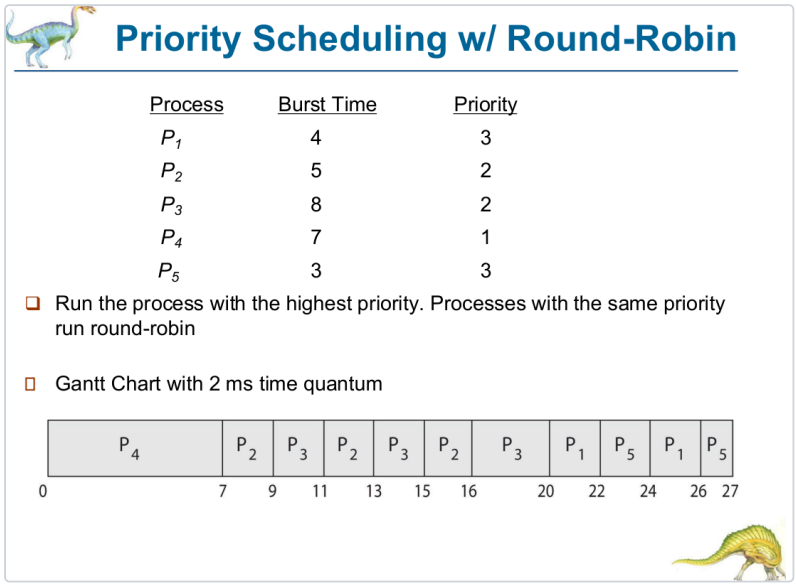
- **HW2 MC9**：$\tau_n = 10,\ t_n = 6,\ \alpha = 0.25 \Rightarrow \tau_{n+1} = 0.25 \times 6 + 0.75 \times 10 = 1.5 + 7.5 =$ **9 ms**。

---

6. Priority Scheduling ⭐ ⭐ [5.25–5.27]

- **Each process has a priority number；CPU allocated to the highest priority（本科：smallest integer = highest priority，除非題目另講）。**

- **Preemptive**（higher-priority arrival preempts）或 **non-preemptive**。
- **Equal priority → FCFS order**（或者 RR）。
- **SJF is a special case of priority scheduling where priority = inverse of predicted next CPU burst.**
- ⭐ **Problem：Starvation (indefinite blocking) — low-priority processes may never execute.**
- ⭐ **Solution：Aging — gradually increase the priority of a process as it waits**（需要每個 process 喺 ready queue 有 timer）。
- Sp21 MC：main problem of priority scheduling → **some processes may never execute (starvation)**；HW2 MC7：aging → **address starvation by gradually increasing the priority of a waiting process**。
- 例 [5.26]（all t=0；P1 10/3, P2 1/1, P3 2/4, P4 1/5, P5 5/2）→ `0 P2 1 P5 6 P1 16 P3 18 P4 19`，**avg WT** $= \frac{6+0+16+18+1}{5} = 8.2$
- **Priority + RR** [5.27]：**run the highest priority；same priority → round-robin**。例 q = 2（burst/priority：P1 4/3, P2 5/2, P3 8/2, P4 7/1, P5 3/3，全部 t=0）→ `0 P4 7 P2 9 P3 11 P2 13 P3 15 P2 16 P3 20 P1 22 P5 24 P1 26 P5 27`
  - P4（priority 1）先跑完；P2、P3（priority 2）RR；P2 喺 16 完咗之後 P3 獨自跑到 20；最後 P1、P5（priority 3）RR。



Priority scheduling with round-robin, q = 2 [5.27]

⚠️ **Priority + RR 有唔同 arrival time（Lab 5：HW / exam 常見）：被高 priority 新 process preempt 嘅 process 要保留佢剩餘嘅 quantum**，唔好 reset（lab 話呢個係 common mistake）。規則同 MLFQ 一樣。

- **Spring 2023 midterm 官方 Gantt（q = 2）**：`0 P1 1 P2 4 P1 5 P3 7 P1 8 P4 10 P1 11 P3 13 P5 15 P1 16 P3 18 P5 20 P3 22 P5 24` — P1 喺 0–1 被 P2 preempt，4–5 只跑剩低嘅 1 unit 就輪到 P3；8–10 P4 preempt 後 P1 10–11 又用返剩低 1 unit。（由 Gantt 推返嘅 table：P1 (0, 5, pr 2)、P2 (1, 3, pr 1)、P3 (2, 8, pr 2)、P4 (8, 2, pr 1)、P5 (9, 6, pr 2)，模擬器重現完全一致 ✓）

7. Multilevel Queue（多級佇列） ⭐ [5.28–5.29]

- **Priority scheduling combined with RR；priority assigned statically；a process remains in the same queue for its whole runtime.**
- **Partition processes into queues based on process type**（例：real-time → system → interactive → batch）。
- **Each queue can have its own scheduling algorithm.**
- **Scheduling among queues**：(1) **fixed-priority preemptive scheduling**（高 queue 空先輪到低 queue → 可能 starvation）或 (2) **time-slice**（each queue gets a certain % of CPU time，e.g. 60%, 20%, 10%, 10%）。
- Sp21 MC：Multilevel queue（固定）有 starvation（free from starvation 嘅只有 **FCFS 同 RR**）。

8. MLFQ — Multilevel Feedback Queue ⭐ ⭐ ⭐（每年必考 Gantt） [5.30–5.35]

- **A process can move between queues；aging can be implemented this way → flexibility.**
- **MLFQ 由以下 parameters 定義：**
  1. **number of queues**
  2. **scheduling algorithm for each queue**
  3. **method to upgrade a process**（to higher queue）
  4. **method to demote a process**（to lower queue）
  5. **method to determine which queue a process enters when it needs service**

8.1 Slide example 規則（考試照用）[5.31] ⭐

- Q0 – RR（slide 5.31: q = 8；例子 5.32–5.34 用 **q = 4**）；Q1 – RR（q = 16 / **8**）；Q2 – **FCFS**
- **A new job always enters Q0 (RR, FCFS order)；it also preempts jobs from Q1 or Q2 currently running.**
- **Gets q_0 in Q0；if not finished → moved to Q1. In Q1 gets q_1 additional；if still not complete → preempted and moved to Q2.**
- ⭐ **If a job from Q1 or Q2 is preempted by a new job from Q0, it joins the HEAD of Q1 or Q2, with the remaining quantum (Q1) or remaining CPU burst (Q2).**（**唔會**攞一個全新 quantum；**唔會**因為被 preempt 而 demote）
- **Q1 job 只會喺 Q0 空先跑；Q2 job 只會喺 Q0 和 Q1 都空先跑。**
- **Q1 job 永遠唔會 preempt Q2 job**（因為有 process 直接 arrive 去 Q1）— summary 重點。
- **A job in Q0 is never preempted**（Q0 內部只有 RR）。
- **Burst $\leq q_0 + q_1$（e.g. 4 + 8 = 12）嘅 job 永遠唔會落 Q2。**
- **This approximates SRTF：CPU-bound jobs drop like a rock；short-running I/O-bound jobs stay near the top.**

8.2 Lecture example [5.32–5.34]（Q0 q=4, Q1 q=8, Q2 FCFS）

Process	Arrival	Burst
P1	0	10
P2	2	15
P3	5	2
P4	12	14
P5	18	6

```
0 P1 4 P2 8 P3 10 P1 12 P4 16 P1 18 P5 22 P1 24 P2 32 P4 40 P5 42 P2 45 P4 47
 Q0   Q0   Q0   Q1   Q0   Q1   Q0   Q1   Q1   Q1   Q1   Q2   Q2
```

- t=4 P1 用完 Q0 quantum → Q1；t=8 P2 → Q1；t=10 P3 完；P1 喺 Q1 跑；t=12 P4 到 → preempt P1（P1 返 Q1 head，剩 6 quantum）；t=16 P4 → Q1 tail；P1 繼續（剩 quantum 6）；t=18 P5 到 → preempt P1；t=22 P5 → Q1；P1 跑到 24 完成（Q1 總共用 2+2+2 = 6 ≤ 8）；P2 24–32 用完 Q1 quantum → Q2；P4 32–40 → Q2；P5 40–42 完成；P2 42–45 完；P4 45–47 完。
- **WT：P1 = 14, P2 = 28, P3 = 3, P4 = 21, P5 = 18 → avg** $= \frac{84}{5} = 16.8$ ✓

8.3 MLFQ advantages ⭐ ⭐（F19 Q3.3、F20 Q2.5 必背）[5.35]

- Used in **BSD Unix, Solaris, Windows NT and later**。

1. **It does not need prior knowledge of the next CPU burst time**（SJF/SRTF 需要）
2. **It handles interactive jobs well — better response time than RR**（short jobs 喺 top queue 完成）
3. **It produces similar performance to SJF/SRTF**
4. **It is "fair" — makes progress on CPU-bound jobs**

- **Starvation** 可以用 **reshuffling jobs periodically**（e.g. 一段時間後將所有 jobs 搬返 top queue）解決。

> 📝 **F20 Q2.5 Model answer**：*(1) MLFQ resembles the performance of SJF/SRTF without requiring prior knowledge of the next CPU burst time; (2) like RR, it gives good response time to interactive (short, I/O-bound) jobs since they finish in the top queues; (3) it is fair — all processes, long or short, make progress (and starvation can be avoided by periodically moving jobs up / aging).*
> 📝 **F19 Q3.3**：*MLFQ approximates SJF because short (interactive, I/O-bound) jobs finish within the top high-priority queue(s), while long CPU-bound jobs are demoted to lower queues — so short jobs get served first like SJF. Its major advantage over SJF is that it does not require knowing the CPU burst times in advance.* ⚠️ S26 MC20 ✓「**Each queue can use a different scheduling algorithm**」；✗ processes never move；✗ requires burst times known；✗ non-preemptive。Sp21 MC ✓「**handles interactive jobs well by delivering similar performance as SJF**」；✗ needs prior knowledge；✗ classification permanent；✗ won't suffer starvation。

8.4 MLFQ 做題 checklist ✍️

1. 每個時間點寫低：**Q0、Q1、Q2 內容（同次序）**、running process、佢喺邊個 queue、**剩餘 quantum**、**剩餘 burst**。
2. 事件：**arrival**（入 Q0 tail；如果 running 係 Q1/Q2 → preempt，被 preempt 嘅返 **head** of own queue，保留剩餘 quantum）；**quantum 用完**（未完成 → 降一級，入 tail）；**完成**。
3. Gantt 上標 **process@Queue**（HW2 要求 label queue）。
4. **同一 process 由 Q0 用完 quantum 立即喺 Q1 繼續跑**（因為 Q0 空）→ Gantt 可合併，但計 response time 要用 Q0 quantum 完結時間（S26 P5：49 − 45 = 4）。
5. **題目規則永遠優先**（S26 第 8 條、F20 aging timer、HW2「retains unused quantum」）。

---

9. Thread Scheduling ⭐ ⭐ [5.36–5.37]

- **On modern OSes, kernel-level threads are scheduled (by the OS); user-level threads are managed by the thread library.**
- **LWP (lightweight process)** — intermediate data structure；**appears as a virtual processor on which user threads are scheduled to run；each LWP attached to a kernel thread**。

- ⭐ **Process-Contention Scope (PCS) — under many-to-one and many-to-many models, the thread library schedules user-level threads onto an LWP；competition among threads belonging to the same process**；typically **by priority set by programmers**（thread library 通常唔會調整 priority）；PCS typically **preempts the running thread in favour of a higher-priority user thread**。
- ⭐ **System-Contention Scope (SCS) — the kernel schedules kernel threads onto an available CPU；competition among all threads in the system** → 就係 CPU scheduling。
- **One-to-one systems (Windows, Linux, Solaris) schedule threads using only SCS.**

> 📝 HW2 MC8：thread library 揀 user thread run on LWP → **PCS, because the competing threads belong to the same process**。 📝 **F19 Q3.1**：*The OS schedules kernel-level threads. Mapping user threads onto kernel threads/LWPs (PCS) is done by the thread library, based on priorities set by the programmer, among the user threads of the same process; CPU scheduling (SCS) decides which kernel thread runs on a CPU among all kernel threads in the system.*

---

10. Multiple-Processor Scheduling ⭐ ⭐ [5.38–5.46]

10.1 AMP vs SMP [5.38–5.39]

- Multiprocessor 而家包括 **multicore CPUs, multithreaded cores, NUMA systems, heterogeneous multiprocessing**。
- **Asymmetric multiprocessing (AMP) — only one processor (the master server) accesses kernel data structures**（唔使 data sharing）；**all scheduling decisions, I/O processing, system activities handled by the master**；other processors execute only user code；⭐ **the master server can become a potential bottleneck**。
- **Symmetric multiprocessing (SMP) — each processor is self-scheduling**，with **one common ready queue** or **a private ready queue per processor**；每個 processor 嘅 scheduler examine ready queue 揀 thread。
  - Common queue：**must ensure two processors don't choose the same thread（race condition）** 同 processes 唔會 lost。
  - **All modern OSes support SMP**（Windows, Linux, macOS, Android, iOS）。

> 📝 **F20 Q2.4 Model answer（common vs per-processor ready queue）**：*A single common ready queue achieves **load balancing** easily — no processor is idle while the queue is non-empty — but it is hard to keep **processor affinity** (a thread may run on a different CPU each time, losing its cache contents) and the shared queue needs locking (possible race condition/contention). Per-processor private ready queues naturally provide processor affinity (and no contention on a shared queue), but need extra mechanisms such as **push/pull migration** to balance the load.*

10.2 Multicore & Multithreaded cores (CMT) ⭐ [5.40–5.44]

- **Multiple cores on the same chip — faster and consume less power, but complicate scheduling.**
- ⭐ **Memory stall：when a processor accesses memory, it spends a significant amount of time waiting for data to become available（modern processors much faster than memory, esp. on cache miss）**。（處理器可能 50% 時間喺等 memory）
- **Chip multithreading (CMT)（Intel = hyperthreading）：multiple hardware threads per core；each hardware thread has its own architectural state (PC, register set) and appears as a logical CPU**。
- **If one thread stalls waiting for memory, the core switches to another hardware thread.**
- **Dual-threaded dual-core → 4 logical processors；quad-core × 2 threads → 8 logical processors；UltraSPARC T3：16 cores × 8 threads = 128 logical processors**。
- **Two levels of scheduling**：
  1. **OS decides which software (kernel) thread runs on each logical CPU（= CPU scheduling）**
  2. **Each core decides which hardware thread runs on the physical core（e.g. RR on UltraSPARC T3）**
- 三層 mapping [5.44]：user thread → LWP/kernel thread（**PCS**, thread library）；kernel (software) thread → logical CPU / hardware thread（**CPU scheduling**, OS）；hardware thread → core（**core 自己，typically RR**）。

> 📝 **F19 Q2.5**：multicore 優點 = **faster + less power**；CMT 原因 = **memory stall** — 一個 hardware thread 等 memory 時 core 可以跑另一個 thread（S26 MC22 ✓「another thread may run when one thread stalls waiting for memory」）。

10.3 Processor Affinity ⭐ ⭐ [5.45]

- **When a thread has run on a processor, that processor's cache holds its memory accesses → the thread has affinity for that processor.**
- **High cost of invalidating and repopulating caches → most SMP systems try to avoid migrating processes between processors.**
- ⭐ **Per-processor ready queues provide processor affinity for free!**
- **Soft affinity — OS attempts to keep a process on the same processor but doesn't guarantee it（可以 migrate during load balancing）**。
- **Hard affinity — a process specifies a subset of processors it may run on.**（Linux：soft affinity + `sched_setaffinity()` for hard affinity）
- **NUMA**：CPU 存取近佢嘅 memory 較快 → affinity to both processor and memory。

📝 **HW2 Q2.4**：hard affinity {CPU0, CPU1}，CPU2 idle → **No**，hard affinity must be respected；load balancing 只可以將 T 由 CPU0 搬去 CPU1。

10.4 Load Balancing ⭐ [5.46]

- **Keep workload evenly distributed**（只喺 per-processor private queues 時需要）。
- **Push migration — a specific task periodically checks loads and pushes tasks from overloaded to idle/less-busy CPUs.**
- **Pull migration — an idle processor pulls a waiting task from a busy processor.**
- **Not mutually exclusive — often both implemented (Linux CFS).**
- ⭐ **Load balancing often counteracts the benefits of processor affinity**（natural tension）。
- S26 MC22 ✗「**The motivation of placing multiple cores on the same chip is to simplify the scheduler**」（其實 complicate scheduling）；✓ AMP master bottleneck；✓ memory stall → run another thread；✓ affinity keeps process on same CPU for cache。

---

11. Real-Time CPU Scheduling ⭐ ⭐ ⭐ [5.47–5.52]

11.1 Hard vs Soft ⭐

- **Real-time scheduling demands performance guarantee — predictability.**
- **Hard real-time — a task must be serviced by its deadline; service after the deadline is the same as no service.**
- **Soft real-time — no guarantee when a critical real-time process will be scheduled; only guarantees it will be given preference over noncritical processes.**
- **Real-time OS scheduler must support a priority-based algorithm with preemption**（但 preemptive priority-based 只保證 **soft** real-time）。
- Sp21 MC ✓「**Soft real-time systems guarantee only that the process will be prioritized over non-critical processes**」；✗ doesn't demand guarantee；✗ hard RT can service after deadline；✗ priority without preemption。

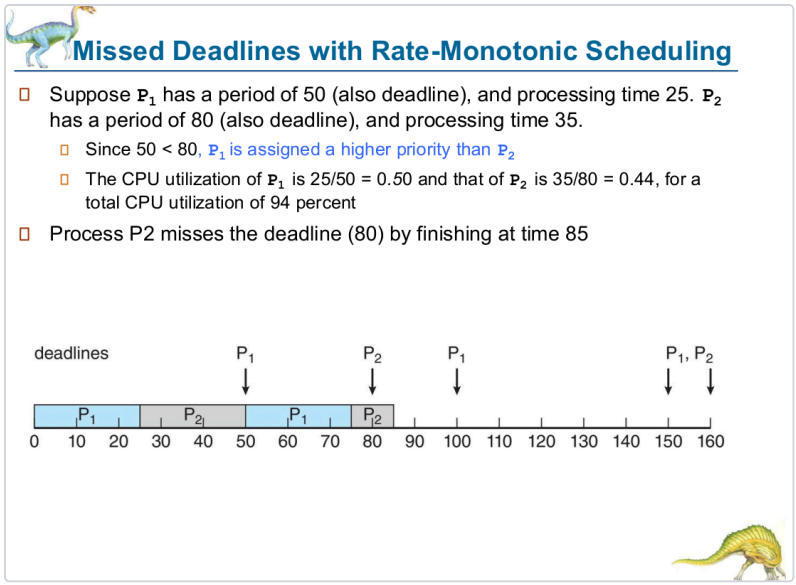
11.2 Periodic tasks [5.48]

- 參數：**processing time** t, **deadline** d, **period** p，$0 \leq t \leq d \leq p$；**rate** = $1/p$。
- **Admission control**：process announces deadline；scheduler **admits only if it can guarantee completion by the deadline**。
- **CPU utilization of task** i = t_i/p_i；total（$U > 1$ → 一定有 miss）：

$$U = \sum_i \frac{t_i}{p_i}$$

11.3 Rate-Monotonic (RM) ⭐ ⭐ [5.49–5.50]

- **Static priority = inverse of period：shorter period → higher priority**（rationale：**tasks requiring CPU more often get higher priority**）。**Preemptive.**
- **RM is optimal among static-priority algorithms：if a set can't be scheduled by RM, no other static-priority algorithm can.**
- **Only handles periodic processes with constant processing time.**
- 例 [5.49]：P1 (p=50, t=20), P2 (p=100, t=35)：U = 0.40 + 0.35 = 75% → OK：`0 P1 20 P2 50 P1 70 P2 75 idle 100`
- 例 [5.50]：P1 (50, 25), P2 (80, 35)：U = 0.50 + 0.44 = **94%** → `0 P1 25 P2 50 P1 75 P2 85` → **P2 misses its deadline (80) by finishing at 85**。

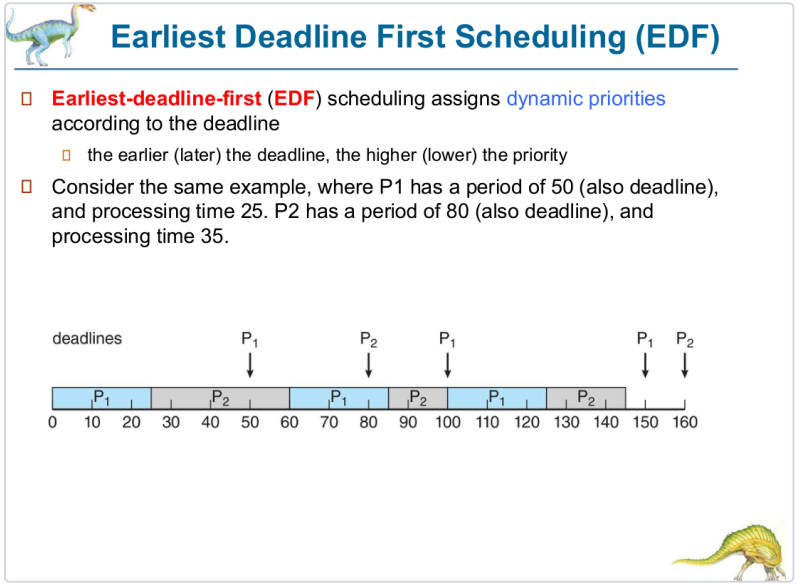


RM: P2 misses its deadline at 80 **[5.50]**

- 課本 bound（slides 冇，但好用）：**RM guaranteed if** $U \leq N\left(2^{1/N}-1\right)$：N=1 → 1；**N=2 → 0.828**；N=3 → 0.780；N=4 → 0.757；$N \to \infty \Rightarrow \ln 2 \approx 0.693$。超過 bound **唔一定** miss（要畫 Gantt 確認）。

11.4 EDF — Earliest Deadline First ⭐ ⭐ [5.51–5.52]

- **Dynamic priorities according to deadline：earlier (absolute) deadline → higher priority.** Preemptive。
- 例 **[5.51]**（P1 50/25, P2 80/35）：`0 P1 25 P2 60 P1 85 P2 100 P1 125 P2 145 idle 150` → **no deadline missed**（t=50 時 P2 deadline 80 < P1 新 deadline 100 → P2 繼續）。



EDF: same tasks, no deadline missed **[5.51]**

- **EDF does not require processes to be periodic, nor a constant CPU time per burst；only requirement：announce its deadline when it becomes runnable.**
- ⭐ **EDF is theoretically optimal — can meet all deadlines with CPU utilization up to 100%**（但實際上 context switching + interrupt handling 令佢做唔到 100%）。
- S26 MC21：assigns priorities according to deadlines → **EDF**。

> 📝 **Sp21 Q2.3(b)**：*EDF is better — unlike RM with static priorities, EDF uses dynamic priorities based on deadlines, which can increase the fraction of met deadlines (schedulable up to 100% utilization).* 📝 **HW2 Q2.5**（A: p=50,t=25；B: p=80,t=35；t=50 時 A 新 instance deadline 100，B 剩 10 deadline 80）→ **No：RM 揀 A**（shorter period，static）→ B 75–85 **miss 80**；**EDF 揀 B**（deadline 80 < 100）→ B 50–60 ✓，A 60–85 ✓。

11.5 RM / EDF 做題方法 ✍️

1. 列出每個 task 嘅 **release times**（0, p, 2p, ...）同 **absolute deadlines**（release + d；冇講 d 就 d = p）。
2. 每個 time unit（或者每個事件：release / completion）揀：
   - **RM**：ready jobs 入面 **period 最短**（固定）嗰個。
   - **EDF**：ready jobs 入面 **absolute deadline 最早**嗰個。

3. **Ties**：題目有講就跟（S26：**smaller process ID**）；冇講：**keep the running job（唔 preempt）**（F19、Sp21 官方答案都係咁），再唔得就 smaller ID。寫低 assumption。
4. **Soft real-time：miss 咗 deadline 嘅 job 會繼續跑完**（Sp21 hint；S26 官方亦係咁），但 count 為 missed。
5. 數 **missed deadlines**：每個 deadline ≤ 時限範圍嘅 job，完成時間 > deadline（或者未完成）→ miss。
6. 先計 $U = \sum_i t_i/p_i$ 做 sanity check。

S26 Q4.2（P1 t=1 d=3 p=4；P2 2/4/5；P3 2/6/7；P4 1/5/8；0–16，ties → smaller ID）：

- **RM**（priority P1 > P2 > P3 > P4）：`0 P1 1 P2 3 P3 4 P1 5 P2 7 P3 8 P1 9 P3 10 P2 12 P1 13 P3 14 P3 15 P2 16`
  - P3 job (d=6) finishes 8 ✗；P4 never runs → misses d=5 ✗ and d=13 ✗；P3 job2 (d=13) finishes 14 ✗ → **RM misses 4 deadlines**（U = 0.25+0.4+0.286+0.125 = 1.06 > 1）
- **EDF**：`0 P1 1 P2 3 P4 4 P3 6 P1 7 P2 9 P1 10 P3 12 P4 13 P2 15 P1 16`
  - P2 job3 (d=14) finishes 15 ✗；P1 job4 (d=15) finishes 16 ✗ → **EDF misses 2 deadlines**

Sp21 Q2.3（P1 1/3/3, P2 2/4/4，0–14）：

- **EDF**：`0 P1 1 P2 3 P1 4 P2 6 P1 7 idle 8 P2 10 P1 11 idle 12 P1 13 P2 14`（t=9 tie deadline 12 → keep P2）
- **RM**：`0 P1 1 P2 3 P1 4 P2 6 P1 7 idle 8 P2 9 P1 10 P2 11 idle 12 P1 13 P2 14`
- 兩者都冇 miss（U = 0.833）。

F19 Q3.4（EDF；P1 p=d=10 t=4；P2 16/2；P3 20/8；到 38）：

- `0 P1 4 P2 6 P3 14 P1 18 P2 20 P1 24 P3 32 P1 36 P2 38`（t=10、t=30 deadline tie → keep P3 running）

---

12. Algorithm Evaluation [5.53–5.60]

- 先定 **criteria**（e.g. maximize CPU utilization **subject to max response time 300 ms**；maximize throughput such that TAT ∝ execution time）。
- **Deterministic modeling — takes a particular predetermined workload and defines the performance of each algorithm for that workload；simple and fast, gives exact numbers, but requires exact input and answers apply only to those cases**。
  - 例 [5.54–5.55]：5 processes at t=0：P1 10, P2 29, P3 3, P4 7, P5 12 → **FCFS avg WT = 28；non-preemptive SJF = 13；RR (q=10) = 23**。
- **Queueing models / analysis** — CPU = server with ready queue；用 **distributions**（exponential, described by mean）；可計 utilization、average queue length、wait time；**classes of algorithms and distributions handled are limited；assumptions often unrealistic**。
  - ⭐ **Little's Formula**：$n = \lambda \times W$（n **= average queue length,** λ **= average arrival rate,** W **= average waiting time in queue**）；**valid for any scheduling algorithm and arrival distribution**（steady state，arrival rate < service rate）。
  - 例：$\lambda = 7/\text{s},\ n = 14 \Rightarrow W = n/\lambda =$ **2 s**；**HW2 MC10**：$\lambda = 3/\text{s},\ n = 12 \Rightarrow W = 12/3 =$ **4 s**。
- **Simulations** — programming a model of the system；**more accurate**；data from **random number generators (distributions)** 或 **trace files**（real system 記錄）。
- **Implementation** — 真系統測試，最準但 **high cost & high risk**；環境會變；可調 parameters；APIs 改 priority（Java, POSIX, Windows）。

---

13. Gantt Chart 萬用流程（考試步驟）✍️⭐⭐⭐

1. **抄表**：Process / Arrival / Burst（/ Priority）。
2. **由 t = 0 開始逐個事件推進**（events：arrival、completion、quantum expiry、check point）。每個事件寫低 **ready queue**（同 MLFQ 各層）。
3. **揀 process 規則**：

Algorithm	揀邊個	Preempt?
FCFS	earliest arrival	No
SJF	shortest burst among **arrived**	No
SRTF	shortest **remaining**	Yes, on arrival
RR	head of queue	Yes, quantum expiry
Priority (NP / P)	highest priority	P: on higher arrival
Priority + RR	highest priority；same → RR	Yes
MLFQ	highest non-empty queue	Yes（Q0 arrival preempts Q1/Q2）
RM	shortest period	Yes
EDF	earliest absolute deadline	Yes

4. **CPU idle**：冇 process ready 就畫 **idle**（Sp21 FCFS `0 P1 2 idle 4 P2…`）。
5. **畫 Gantt**：每段標 **start time / process / end time**；所有 boundary 時間都要寫（partial credit 靠呢個）。

6. **計 table**：Completion → **TAT = C − A** → **WT = TAT − Burst** → **RT = end of first run − A**。
7. **Average**：寫出 $\frac{a+b+\cdots}{n} = x$（S26：$\frac{140}{6} = 23.33$）。
8. **Sanity check**：總 burst 加 idle = 最後完成時間；每個 WT ≥ 0。

13.1 Tie-breaking（寫 assumption！）

- 同時到達、同 burst/priority：**FCFS → lower process ID**。
- **SRTF**：新到達 burst = running remaining → 一般 **keep running**（唔 preempt）；F19 官方 preempt 咗，但兩種 avg WT 一樣 2.5。
- **RR**：新到達 + quantum expiry 同一時刻 → **新到達先入 queue**。
- **EDF**：deadline 相同 → **keep running**，否則 smaller ID（或者跟題目）。
- **Dynamic priority（S26 Q4.3）**：題目講「same priority → larger ID first」— **跟題目**！

---

14. Past-paper 計算題全集（全部用程式驗證 ✓）

14.1 S26 Q4.1 MLFQ（Q0 RR4, Q1 RR8, Q2 FCFS；preempted → head of own queue）

P	Arr	Burst
P1	0	9
P2	2	25
P3	6	7
P4	14	18
P5	45	15
P6	58	6

```
0 P1 4 P2 8 P3 12 P1 14 P4 18 P1 21 P2 29 P3 32 P4 40 P2 45 P5 57 P2 58 P6 64 P2 71 P4 77 P5 80
(Q0)(Q0)(Q0)(Q1)(Q0)(Q1)(Q1)(Q1)(Q1)(Q2)(Q0→Q1 at 49)(Q2)(Q0→Q1 at 62)(Q2)(Q2)(Q2)
```

- WT：P1 12, P2 44, P3 19, P4 45, P5 20, P6 0 → **avg** $= \frac{140}{6} = 23.33$
- RT（end of first run − arrival）：P1 4, P2 6, P3 6, P4 4, P5 4 (49−45), P6 4 (62−58) → **avg** $= \frac{28}{6} = 4.67$

14.2 S26 Q4.3 Dynamic priority（larger = higher；waiting rate α，running rate β；tie → larger ID）

- P1 (0, 7), P2 (3, 5), P3 (6, 6), P4 (9, 4)
- (a) $\alpha = 0,\ \beta = 3$（running process priority 上升，waiting 不變 → 一旦 run 就霸住）：`0 P1 7 P3 13 P4 17 P2 22`
  - t=7：P2, P3 都係 0 → tie → larger ID → **P3**；t=13：P2 (0), P4 (0) → **P4**。
- (b) $\alpha = 1,\ \beta = 0$，只考慮 P1、P2，每 3 個 unit check：`0 P1 3 P2 6 P1 9 P2 11 P1 12`
  - t=3：P1 = 0 (ran 3), P2 = 0 → tie → P2；t=6：P1 = 3, P2 = 0 → P1；t=9：P1 = 3, P2 = 3 → tie → P2；t=11 P2 完 → P1。
- (c) $\alpha = 1,\ \beta = -2$，初始 30，每 3 unit check：`0 P1 3 P2 6 P3 9 P4 12 P1 15 P2 17 P3 20 P4 21 P1 22`
  - t=3：P1 = 30−6 = 24, P2 = 30 → P2；t=6：P1 27, P2 24, P3 30 → P3；t=9：P1 30, P2 27, P3 24, P4 30 → tie → P4；t=12：P1 33 → P1；t=15：P1 27, P2 33, P3 30, P4 27 → P2（17 完）；t=17：P1 29, P3 32, P4 29 → P3（20 完）；t=20：P1 32, P4 32 → tie → P4（21 完）；t=21：P1。

14.3 S26 Q2.3 自訂 policy（揀 f_i 最大）

$$f_i = \frac{1}{a\,c_i + b} \qquad (a, b > 0)$$

- (a) **Yes** — I/O-bound processes have small average CPU bursts c_i → larger f_i → higher priority.
- (b) **Yes** — a CPU-bound process (large c_i, low f_i) can starve if higher-priority I/O-bound processes keep arriving.

14.4 F19 Q3.5 LUF (Least Used First)（揀用 CPU 最少嘅；tie → FCFS；preempt on arrival）

- P1 (0, 25), P2 (15, 25), P3 (25, 5), P4 (40, 5)
- `0 P1 15 P2 25 P3 30 P2 40 P4 45 P1 55 P2 60`

P	TAT	RT	WT
P1	55	15	30
P2	45	10	20
P3	5	5	0
P4	5	5	0

- LUF vs SRTF：相似 — short jobs 較易有高 priority；分別 — **LUF 令跑咗好耐嘅 process 易 starve**；SRTF 令 **long burst 嘅 process 一開始就 starve**。

14.5 F19 Q3.6（P1 0/8, P2 7/13, P3 9/1, P4 11/5, P5 20/7, P6 27/1）

Alg	Gantt	WT (P1..P6)	Avg
FCFS	0 P1 8 P2 21 P3 22 P4 27 P5 34 P6 35	0,1,12,11,7,7	**6.33**
SJF	0 P1 8 P2 21 P3 22 P4 27 P6 28 P5 35	0,1,12,11,8,0	**5.33**
RR q=4	0 P1 8 P2 12 P3 13 P4 17 P2 21 P4 22 P5 26 P2 30 P5 33 P6 34 P2 35	0,15,3,6,6,6	**6.0**（corrected）
SRTF	0 P1 8 P2 9 P3 10 P2 11 P4 16 P2 20 P5 27 P6 28 P2 35（官方，tie 時 preempt）	0,15,0,0,0,0	**2.5**

- SRTF 另一答案（tie 時 keep P2）：`… 16 P2 27 P6 28 P5 35` → WT 0,7,0,0,8,0 → avg 都係 **2.5**。
- 注意 SJF：t=8 只有 P2 到咗 → P2 跑到 21（non-preemptive）。

14.6 F20 Q4.1（P1 0/9, P2 3/3, P3 8/2, P4 10/6, P5 16/8, P6 20/5）

Alg	Gantt	WT	Avg
FCFS	0 P1 9 P2 12 P3 14 P4 20 P5 28 P6 33	0,6,4,4,4,8	**4.33**
SJF	0 P1 9 P3 11 P2 14 P4 20 P6 25 P5 33	0,8,1,4,9,0	**3.67**
SRTF	0 P1 3 P2 6 P1 8 P3 10 P1 14 P4 20 P6 25 P5 33	5,0,0,4,9,0	**3.0**

14.7 F20 Q4.2 Two-level feedback queue with aging（Q0 RR4；Q1 FCFS；Q1 timer 8 ms → 返 Q0；被 preempt 返 Q1 head 並 reset timer）

- P1 0/9, P2 3/3, P3 8/2, P4 10/9, P5 13/4, P6 18/5
- **`0 P1 4 P2 7 P1 8 P3 10 P4 14 P5 18 P1 22 P6 26 P4 30 P6 31 P4 32`**
- 關鍵：t=8 P1（Q1）被 P3 preempt → Q1 head，timer reset 8；t=16 P1 timer 到 8 → 升返 Q0；t=22 P4（14 入 Q1）timer 到 8 → 升 Q0。
- WT = 13, 1, 0, 13, 1, 8 → **avg = 6**

14.8 F20 Q4.3 Priority + RR（smaller = higher；preemptive；same priority RR q=4）

- P1 0/9/2, P2 3/3/1, P3 8/5/4, P4 10/2/4, P5 16/8/5, P6 20/5/3
- **`0 P1 3 P2 6 P1 12 P3 16 P4 18 P3 19 P5 20 P6 25 P5 32`**
- WT = 3, 0, 6, 6, 8, 0 → **avg** $= \frac{23}{6} = 3.83$

14.9 Sp21 Q2.1（P1 0/2, P2 4/12, P3 6/6, P4 9/4, P5 13/5）

Alg	Gantt	WT	Avg
FCFS	0 P1 2 idle 4 P2 16 P3 22 P4 26 P5 31	0,0,10,13,13	**7.2**
SJF	0 P1 2 idle 4 P2 16 P4 20 P5 25 P3 31	0,0,19,7,7	**6.6**
RR q=8	0 P1 2 idle 4 P2 12 P3 18 P4 22 P2 26 P5 31	0,10,6,9,13	**7.6**
SRTF	0 P1 2 idle 4 P2 6 P3 12 P4 16 P5 21 P2 31	0,15,0,3,3	**4.2**

14.10 Sp21 Q2.2 MLFQ（Q0 q=2, Q1 q=4, Q2 FCFS）

- P1 0/10, P2 10/8, P3 8/30, P4 16/2, P5 22/8, P6 6/22
- **`0 P1 2 P1 6 P6 8 P3 10 P2 12 P6 16 P4 18 P3 22 P5 24 P2 28 P5 32 P1 36 P6 52 P3 76 P2 78 P5 80`**
  - (Q0)(Q1)(Q0)(Q0)(Q0)(Q1)(Q0)(Q1)(Q0)(Q1)(Q1)(Q2)(Q2)(Q2)(Q2)(Q2)
- WT：P1 26, P2 60, P3 38, P4 0, P5 50, P6 24 → **avg = 33**

14.11 HW2（P1 0/7, P2 1/3, P3 4/5, P4 10/2）

- **SRTF**：`0 P1 1 P2 4 P3 9 P1 10 P4 12 P1 17` → WT 10,0,0,0 → **avg WT 2.5；avg TAT 6.75**
- **RR q=3**：`0 P1 3 P2 6 P1 9 P3 12 P1 13 P4 15 P3 17` → WT 6,2,8,3 → **avg WT 4.75；avg TAT 9**
- **MLFQ（Q0 4, Q1 8, Q2 FCFS；P1 0/20, P2 1/5, P3 11/2）**：`0 P1(Q0) 4 P2(Q0) 8 P1(Q1) 11 P3(Q0) 13 P1(Q1) 18 P2(Q1) 19 P1(Q2) 27`
  - t=11 P1 被 P3 preempt → **Q1 head（喺 P2 前面），剩 5 ms quantum**（唔係新 8 ms）。

---

15. 常見陷阱 & MC 速查

敘述	真/假
Scheduling on running→waiting / terminate = non-preemptive	✓
Timer interrupt (quantum expiry) = preemptive	✓
FCFS is preemptive	✗（**FCFS must be non-preemptive** — Sp21 MC）
SJF gives minimum average waiting time	✓
SJF easy to implement	✗（need next burst length）
SRTF ≥ SJF in performance	✓

敘述	真/假
RR with very large q → FCFS	✓
RR always better TAT than FCFS	✗
RR: no process waits more than (n−1)q	✓
Small q → high context-switch overhead	✓
Priority scheduling problem = starvation；solution = aging	✓
Multilevel queue: processes move between queues	✗（MLFQ 先可以）
MLFQ requires known burst times	✗
MLFQ: each queue different algorithm	✓
Starvation-free: FCFS and RR	✓（Sp21 MC 答案 A：1, 4）
OS schedules user threads (SCS)	✗（PCS 由 thread library；SCS 係 kernel threads）
One-to-one systems only use SCS	✓
AMP: master can be a bottleneck	✓
Multicore simplifies scheduling	✗
Per-processor queues give affinity for free	✓
Load balancing may counteract affinity	✓
Hard RT: service after deadline acceptable	✗
RM: static priority by period	✓
EDF: dynamic priority by deadline	✓
EDF requires periodic tasks	✗
EDF optimal, up to 100% utilization (theory)	✓
Little's formula $n = \lambda W$ only for FCFS	✗（any algorithm）
Response time (course) = first start − arrival	✗（**end of first run − arrival**）

5

16. 自我測驗

1. P1(0,5), P2(1,3), P3(2,1)：SRTF Gantt？（`0 P1 1 P2 2 P3 3 P2 5 P1 9`；WT = 4,1,0 → 5/3）
2. 10 個 jobs 各 100 s 同時到，RR q=1 vs FCFS 平均 TAT？（RR ≈ 995.5 s；FCFS = 550 s）
3. $\alpha = 0.5,\ \tau_0 = 10$，bursts 6, 4 → τ_2？（$\tau_1 = 8$；$\tau_2 = 6$）
4. 兩個 tasks：A (t=2, p=5)、B (t=4, p=10)：U = 0.8，RM 得唔得？（0.8 ≤ 0.828 → guaranteed）
5. PCS vs SCS？
6. Push vs pull migration？

6 Ch6 Synchronization 同步工具

📌 **考試範圍提示**：Midterm 包到 **Ch6（Synchronization Tools）**。每年都有 **1 大題（8–25 分）+ 3–4 題 MC**。題型：

1. **Race condition — 列出 shared variable 所有可能值**（F19 Q4.3、F20 Q5：每個值 2.5 分！）
2. **Semaphore 次序控制填空**（S26 Q5：wait/signal 填 blank）、**semaphore 輸出序列判斷**（F19 Q4.4）
3. **用 atomic instruction 寫 spinlock**（S26 Q2.4 swap；Sp21 Q3.1 compare_and_reset）
4. **Bounded-waiting test_and_set 算法分析 / 改錯**（S26 MC25、Sp21 Q3.2）
5. 定義：**race condition、critical section 三個條件、spinlock 優缺點、binary semaphore vs mutex**
6. **Pthread mutex + semaphore 程式輸出**（S26 MC23）

---

1. Background（背景） [6.4–6.8]

- **Processes execute concurrently and may be interrupted at any time, partially completing execution.**
- **Concurrent access to shared data may result in data inconsistency.**
- **Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.**

1.1 Producer–Consumer with `counter` [6.5–6.6]

- 用 `counter` 記 buffer 有幾多 item（可以用晒 BUFFER_SIZE 格）：

```
/* Producer */                          /* Consumer */
while (true) {                         while (true) {
  while (counter == BUFFER_SIZE) ;       while (counter == 0) ;
  buffer[in] = next_produced;            next_consumed = buffer[out];
  in = (in + 1) % BUFFER_SIZE;           out = (out + 1) % BUFFER_SIZE;
  counter++;                             counter--;
}                                      }
```

- 問題：`counter++` / `counter--` **唔係 atomic**。

1.2 Race Condition 例子 ⭐⭐ [6.7]

```
counter++  →  register1 = counter;  register1 = register1 + 1;  counter = register1
counter--  →  register2 = counter;  register2 = register2 - 1;  counter = register2
```

counter = 5 initially，interleaving：

Step	Executed	Result
S0	producer: register1 = counter	register1 = 5
S1	producer: register1 = register1 + 1	register1 = 6
S2	consumer: register2 = counter	register2 = 5
S3	consumer: register2 = register2 − 1	register2 = 4
S4	producer: counter = register1	counter = 6
S5	consumer: counter = register2	**counter = 4**（正確應該係 5！）

- 調轉 S4、S5 → counter = 6。**結果取決於執行次序 → race condition。**
- **Kernel race example** [6.8]：P0、P1 同時 `fork()` → race on kernel variable `next_available_pid` → **same pid could be assigned to two different processes** unless mutual exclusion。

---

2. Race Condition & Critical-Section Problem ⭐⭐⭐ [6.9–6.12]

2.1 Race condition 定義（S26 Q2.1 原題，4 分）

- Sp21 MC：「several processes access and manipulate the same data concurrently and the outcome depends on the order」→ **race condition**。
- 根本原因（summary）：**high-level statements (count++) translate into multiple machine instructions, which can be interleaved arbitrarily.**

> 📝 **Model answer**：*A **race condition** is an undesirable situation where several processes (or threads) access and manipulate shared data concurrently, and the final outcome depends on the particular order (timing) in which the accesses take place. Because the execution order is unpredictable (e.g., context switches at untimely points), the result can be incorrect / non-deterministic. **Example**: two threads execute `counter++` on the same shared counter without synchronization; each loads the old value into a register, increments it and stores it back, so one update can be lost and the final value is wrong (e.g., 5 + 1 − 1 gives 4 or 6 instead of 5).*

2.2 Critical Section

- **Critical section：a segment of code in which a process may be changing shared variables, updating a table, writing a file, etc.**
- **When one process is in its critical section, no other may be in its critical section.（mutual exclusion and critical section imply the same thing）**
- **Critical-section problem：design a protocol that processes can use to cooperate.**
- **General structure** [6.10]：

```
do {
    entry section        /* request permission */
        critical section
    exit section
        remainder section
} while (true);
```

2.3 三個 requirements ⭐⭐⭐（F19 Q4.1 原題）[6.11]

1. **Mutual exclusion — if process P_i is executing in its critical section, no other processes can be executing in their critical sections.**
2. **Progress — if no process is executing in its critical section and some processes wish to enter, the selection of the process that will enter next cannot be postponed indefinitely**（只有唔喺 remainder section 嘅 process 先參與決定）。
3. **Bounded waiting — a bound must exist on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.**（i.e. 等待時間有上限，no starvation）

- Assumptions：**each process executes at a nonzero speed；no assumption about the relative speed of the processes.**
- Summary：**bounded waiting also implies progress.**
- Sp21 MC：CS solution **唔需要滿足** → **synchronization**（三個係 mutual exclusion, progress, bounded waiting）；mutual exclusion 定義 → A。

2.4 Critical sections in the kernel [6.12]

- Kernel data structures（**list of open files**, memory allocation, process table, interrupt handling）都有 race condition。
- **Preemptive kernel** — allows preemption of a process running in kernel mode；**not free from race conditions**，SMP 上更難；但 more responsive（適合 real-time）。
- **Non-preemptive kernel** — runs until it exits kernel mode, blocks, or voluntarily yields → **essentially free of race conditions in kernel mode**（single-processor）。

3. Synchronization Hardware ⭐⭐ [6.13–6.21]

3.1 Disabling interrupts [6.13]

- **Uniprocessor：could simply disable interrupts** → current code runs without preemption。
- **Generally inefficient on multiprocessor/SMP systems**（要 disable 所有 CPU 嘅 interrupts，費時；影響 system clock）。
- 層次：**Hardware**（load/store, disable interrupts, **test&set, compare&swap**）→ **High-level APIs**（locks, semaphores）→ programs。

3.2 Atomic instructions [6.14]

- **Atomic = non-interruptible**；保證 instruction 唔會被中途打斷 → no race condition。
- 即使兩個 CPU 同時執行，**they are forced to execute sequentially (serialization)**。
- **Building blocks for more sophisticated synchronization tools.**
- 兩個常用：`test_and_set()`（test a memory word and set a value）、`compare_and_swap()`（swap contents of two words）。
- S26 MC24：commonly used hardware primitive → **test_and_set()**。

3.3 test_and_set ⭐⭐ [6.15–6.16]

```
boolean test_and_set(boolean *target) {   /* executed atomically */
    boolean rv = *target;
    *target = TRUE;
    return rv;                              /* returns the OLD value */
}
/* lock initialized to FALSE */
do {
    while (test_and_set(&lock)) ;   /* spin while old value was TRUE (lock held) */
        /* critical section */
    lock = false;                   /* release */
        /* remainder section */
} while (true);
```

- 第一個 call 嘅 process 攞到 old = FALSE → 入 CS，同時 lock 變 TRUE；其他人攞到 TRUE → spin。
- ✓ **mutual exclusion**；✗ **bounded waiting**（冇保證輪到邊個）。
- Sp21 MC4（lock init 0）：entry section → `while (test_and_set(&lock));`（C）。✗ `while (!test_and_set(&lock))`。

3.4 compare_and_swap (CAS) ⭐ ⭐ [6.17–6.18]

```
int compare_and_swap(int *value, int expected, int new_value) {  /* atomic */
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;                            /* returns the OLD value */
}
/* lock initialized to 0 */
do {
    while (compare_and_swap(&lock, 0, 1) != 0) ;   /* spin until old value was 0 */
        /* critical section */
    lock = 0;
        /* remainder section */
} while (true);
```

- ⚠️ 留意 return 值：**CAS 返回 old value** → 用 `!= 0` 判斷。Sp21 MC4 ✗ `while (compare_and_swap(&lock, 0, 1) == 0)`（相反）、✗ `compare_and_swap(&lock, 1, 0) != 0`。

3.5 Bounded-waiting Mutual Exclusion with test_and_set ⭐ ⭐ ⭐ [6.19–6.20]

```
/* shared: boolean waiting[n] (all false), boolean lock = false */
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
        /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])        /* scan in cyclic order */
        j = (j + 1) % n;
    if (j == i)
        lock = false;          /* no one is waiting, so release the lock */
    else
        waiting[j] = false;    /* unblock process j (hand the lock over directly) */
        /* remainder section */
} while (true);
```

Sketch proof [6.20]：

- **Mutual exclusion**：P_i enters CS only if `waiting[i] == false` or `key == false`. `key` becomes false only via `test_and_set()` — **only the first process to execute test_and_set finds key == false**；others wait. `waiting[i]` becomes false only when another process leaves its CS, and **only one waiting[i] is set to false** → mutual exclusion。
- **Progress**：a process exiting its CS **either sets lock = false or sets waiting[j] = false** — both let a waiting process proceed。
- **Bounded waiting**：the exiting process **scans waiting[] in cyclic order (i+1, i+2, …, n−1, 0, …, i−1)** and designates the first waiting one → **any waiting process enters within n − 1 turns**。

> ⚠️ **S26 MC25**（which is NOT correct）→ **A.「test_and_set(&lock) releases the lock after the critical section」**（錯：釋放係 `lock = false` 或者 `waiting[j] = false`；test_and_set 只會 set lock 為 true）。✓ waiting[i] 表示 P_i 想入 CS；✓ `waiting[j] = false` 表示 P_i 將 lock 交畀 P_j（lock 冇真正釋放）；✓ 執行 `lock = false` 後 P_i 可以再入 CS。

> 📝 **Sp21 Q3.2（改錯題）**：版本用 `j = i + 1; while (j < n && !waiting[j]) j++; if (j < n) waiting[j] = false;`
> **Problems**：(1) **只 scan index 大過 i 嘅 process → index 細過 i 嘅 process 可能 unbounded waiting**（violates bounded waiting）；(2) **冇 process 等嘅時候 lock 永遠唔會 release** → 其他人之後永遠入唔到（violates progress / deadlock）。
> **Correction**：改返 cyclic scan `j = (i + 1) % n; while ((j != i) && !waiting[j]) j = (j + 1) % n;` 同 `if (j == i) lock = false; else waiting[j] = false;`。

3.6 Atomic Variables [6.21]

- **Atomic variable：atomic (uninterruptible) updates on basic data types (integers, booleans)**，用 CAS 實現：

```
void increment(atomic_int *v) {
    int temp;
    do {
        temp = *v;
    } while (temp != compare_and_swap(v, temp, temp + 1));   /* retry if someone changed v */
}
```

3.7 用其他 atomic instruction 寫 spinlock（考試題型）⭐⭐

S26 Q2.4 — `swap(x, v)`：atomically sets x = v and returns the old value of x（`locked` = 1 held / 0 free）：

```
while (swap(locked, 1) == 1) ;   /* acquire: spin while the old value was 1 */
/* critical section */
locked = 0;                       /* release (or swap(locked, 0)) */
```

Sp21 Q3.1 — `compare_and_reset(int *a, int old, int new)`：if (*a == old) { *a = new; return 1; } else return 0;

```
struct spinlock { int value = 0; };
void acquire(struct spinlock *lock) {
    while (compare_and_reset(&lock->value, 0, 1) == 0) ;   /* spin until we changed 0 → 1 */
}
void release(struct spinlock *lock) {
    lock->value = 0;
}
```

萬用套路：搵出「**成功攞到 lock**」時 instruction 嘅 return 值 → `while (instruction(...) 表示失敗) ;` → release 就 set 返 free value。

Instruction	Acquire	Release
`test_and_set(&lock)`（return old）	`while (test_and_set(&lock)) ;`	`lock = false;`
`compare_and_swap(&lock, 0, 1)`（return old）	`while (compare_and_swap(&lock, 0, 1) != 0) ;`	`lock = 0;`
`swap(locked, 1)`（return old）	`while (swap(locked, 1) == 1) ;`	`locked = 0;`
`compare_and_reset(&v, 0, 1)`（return 1 = success）	`while (compare_and_reset(&v, 0, 1) == 0) ;`	`v = 0;`
課本 `swap(&lock, &key)`（exchange）	`key = true; while (key == true) swap(&lock, &key);`	`lock = false;`

4. Mutex Locks ⭐⭐ [6.22–6.23]

- **Simplest software tool most OSes use.**
- **Protect a critical region by first `acquire()` a lock, then `release()` it — both must be atomic**（usually via hardware atomic instructions）。
- Boolean variable `available` 表示 lock 有冇得用：

```
acquire() {
    while (!available) ;   /* busy wait */
    available = false;
}
release() {
    available = true;
}
do { acquire lock;  critical section;  release lock;  remainder section; } while (true);
```

- **This solution requires busy waiting → the lock is called a spinlock.**
- Wait if locked — 等耐咗應該 **sleep / block** 而唔係 spin。

4.1 Spinlock 優缺點 ⭐⭐⭐（S26 Q2.4b 原題）

- 單 CPU 用 spinlock 冇乜意思（spin 緊嘅 thread 佔住 CPU，持有 lock 嘅 thread 跑唔到）。

> 📝 **Model answer**：*Disadvantage: a spinlock **wastes CPU cycles due to busy waiting** (the waiting thread keeps looping). Advantage: **no context switch is required** when a process must wait on a lock, and a context switch may take considerable time. So spinlocks are useful when locks are expected to be **held for short times**, especially on **multiprocessor systems**, where one thread can spin on one processor while another thread executes its critical section on another processor.*

5. Semaphores ⭐⭐⭐ [6.24–6.28]

5.1 定義 [6.24]

- **Semaphore S — a (non-negative) integer variable, a generalized lock；Dijkstra (late 1960s)；main synchronization primitive in original UNIX。**
- **Accessed only through two atomic operations (apart from initialization)：`wait()` and `signal()`**（originally **P()** = *proberen*, to test；**V()** = *verhogen*, to increment）。
- **Atomic → no more than one process can execute wait()/signal() on the same semaphore at the same time (serialization).**

```
wait(S) {                   signal(S) {
    while (S <= 0) ;            S++;
    S--;                    }
}
```

5.2 Counting vs Binary [6.25] ⭐

- **Counting semaphore — value ranges over an unrestricted domain；control access to a resource with a finite number of instances（initialize to the number of available resources）**；wait = 攞一個，signal = 還一個。
- **Binary semaphore — value only 0 or 1；can behave like a mutex lock**，但亦可以用嚟做其他同步。
- **Ordering example**（S1 喺 S2 之前執行）：`synch = 0`

```
P1:  S1;  signal(synch);          P2:  wait(synch);  S2;
```

> 📝 **F19 Q4.2：Binary semaphore = mutex lock? → *No.*** *A binary semaphore initialized to 1 can be used as a mutex lock, but a binary semaphore can also be used for other synchronization purposes, e.g., initialized to 0 to enforce execution order / signaling between processes. (Also a mutex lock is owned — the thread that locks it must unlock it — whereas any process can signal a semaphore.)*

5.3 Implementation with no busy waiting ⭐ ⭐ [6.26–6.27]

- **Each semaphore has a waiting queue**（each entry：value + pointer to next record）。
- **block()** — place the invoking process on the waiting queue（suspend）；**wakeup(P)** — remove a process from the waiting queue and put it in the **ready queue**。

```
typedef struct { int value; struct process *list; } semaphore;
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) { add this process to S->list; block(); }
}
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) { remove a process P from S->list; wakeup(P); }
}
```

- ⭐ **Increment/decrement is done BEFORE checking the value**（同 busy-waiting 版相反）。
- ⭐ **Semaphore value may become negative；if negative, its magnitude = number of processes waiting on it.**（busy-waiting classical 版永遠唔會 negative）

5.4 Deadlock and Starvation ⭐ [6.28]

- **Deadlock — two or more processes waiting indefinitely for an event that can be caused only by one of the waiting processes.**

```
S = Q = 1
P0: wait(S); wait(Q); ... signal(S); signal(Q);
P1: wait(Q); wait(S); ... signal(Q); signal(S);
```

P0 wait(S)、P1 wait(Q) 之後：P0 等 Q（要 P1 signal），P1 等 S（要 P0 signal）→ **deadlock**（extremely difficult to debug）。

- **Starvation (indefinite blocking) — a process may never be removed from the semaphore queue**，例如用 **LIFO order** 或者按 priority 移除 queue 入面嘅 processes。

---

6. 做題方法 ✍️ ⭐ ⭐ ⭐

6.1 Race condition 所有可能值（F19 Q4.3、F20 Q5）

步驟：

1. 將每個 thread 拆成 **atomic reads/writes**（`c = b + a` → read b, read a, write c；**if 條件嘅 read 係另一個 read**！）。
2. 另一個 thread 嘅 writes 可以插喺任何 read 之間（**次序保持**）。
3. 按 **「條件 read 嗰陣見到嘅值」分 branch**，再按「每個 read 見到新值定舊值」分 sub-case。
4. 列出所有 **distinct** 值（唔好漏！每個值獨立計分）。

F19 Q4.3：a = 7, b = 2, c = 5；T1：`if (a < 0) c = b - a; else c = b + a;`；T2：`b = 8; a = -6;`

Case	T1 見到	c
T2 全部喺 T1 之前 → a<0 branch	b = 8, a = −6	8 − (−6) = **14**
else branch；T2 喺 c 計之前完成	b = 8, a = −6	8 + (−6) = **2**
else；T2 喺 T1 之後	b = 2, a = 7	**9**
else；write b 喺 read b 前，write a 喺 read a 後	b = 8, a = 7	**15**
else；read b 喺 write b 前，write a 喺 read a 前	b = 2, a = −6	**−4**

→ **{14, 2, 9, 15, −4}**

F20 Q5：a = 7, b = 2；T1：`if (b < 0) c = b - a; else c = b + a;`；T2：`b = -8; a = 6;`

- b<0 branch（write b 喺條件 read 之前）：a 新 → −8 − 6 = **−14**；a 舊 → −8 − 7 = **−15**
- else branch，第二次 read b 見到新值（−8）：a 新 → −8 + 6 = **−2**；a 舊 → −8 + 7 = **−1**
- else branch，第二次 read b 見到舊值（2）：a 新 → 2 + 6 = **8**；a 舊 → 2 + 7 = **9** → **{−14, −15, −2, −1, 8, 9}**

6.2 Semaphore 次序（dependency）題（S26 Q5）

規則：「X 必須喺 Y 之前完成」→ 用一個 **init 0** 嘅 semaphore s：**X 完成後 `signal(s)`；Y 開始前 `wait(s)`**。每個 dependency edge 一個 semaphore。

S26 Q5：A depends on B and C；B depends on C（即 C → B → A，且 C → A）；s0, s1, s2 全部 init 0。

```
workerA: /*BLANK1*/ wait(&s2);   wait(&s1);  printf("A");          // 等 B (s2) 同 C (s1)
workerB: /*BLANK2*/ wait(&s0);   printf("B");  signal(&s2);        // 等 C (s0)，完成通知 A
workerC: /*BLANK3*/ /* Nothing */ printf("C"); /*BLANK4*/ signal(&s0); signal(&s1);
```

- **BLANK1 = wait(&s2)，BLANK2 = wait(&s0)，BLANK3 = /* Nothing */，BLANK4 = signal(&s0)** → 輸出一定係 **C B A**。

6.3 Semaphore 輸出序列題（F19 Q4.4）

```
semaphore L = 3, R = 0;
P1: while (true) { wait(L); putc('C'); signal(R); }
P2: while (true) { wait(R); putc('A'); putc('B'); signal(R); }
P3: while (true) { wait(R); putc('D'); }
```

方法：數 token。L = 3 → **P1 最多印 3 個 C**（冇人 signal L）。每個 C 產生 1 個 R token；**P2 借 token 印 AB 後還返；P3 食咗 token 唔還**。

- (a) D 數目 = R token 總數 = **3**
- (b) A：**最少 0**（P3 搶晒 token），**最多 infinite**（P2 無限借還）
- (c) `CABABDCDABCABD` → **No**：印到 `C A B A B D C D` 時 C 印咗 2 次、D 食咗 2 個 token → R = 0，下一個 `A`（P2 需要 wait(R)）冇 token，要等第 3 個 C 先得。
- (d) `CABACDBCABDD` → **Yes**：C(R=1) A(P2 借→0) B(還→1) A(借→0) C(→1) D(食→0) B(還→1) C(→2) A(借→1) B(還→2) D(→1) D(→0) ✓
- **技巧**：逐個字元追蹤 semaphore 值同每個 process「做到邊」；任何時刻 token 唔可以 < 0。

6.4 Pthread 程式輸出題（S26 MC23）

```
int counter = 0; pthread_mutex_t mtx; sem_t sem;   /* sem_init(&sem, 0, 0) */
worker1: lock; counter += 2; printf("A %d;", counter); unlock; sem_post(&sem);
worker2: sem_wait(&sem); lock; counter += 3; printf("B %d;", counter); unlock;
```

- sem 初始 0 → worker2 一定等 worker1 post → **輸出 `A 2; B 5;`**（答案 A）。mutex 保證 counter 更新互斥。

---

7. 補充：Peterson（Lab 6）& classic problems（Ch7 預覽）

7.1 Peterson's Solution（two processes, software-only；Lab 6 有教 + 證明，見 Labs notes L6.1）

```
int turn; boolean flag[2];          /* Pi uses j = 1 - i */
do {
    flag[i] = true;  turn = j;
    while (flag[j] && turn == j) ;
        /* critical section */
    flag[i] = false;
        /* remainder section */
} while (true);
```

- 滿足 mutual exclusion、progress、bounded waiting（兩個 process）；但 **modern architectures may reorder instructions → not guaranteed to work** without memory barriers。

7.2 Classic problems with semaphores（Ch7 預覽）

- **Bounded buffer**：`mutex = 1, full = 0, empty = n`

```
Producer: wait(empty); wait(mutex); add item; signal(mutex); signal(full);
Consumer: wait(full);  wait(mutex); remove item; signal(mutex); signal(empty);
```

- ⚠️ 調轉 `wait(mutex)` 同 `wait(empty)` 次序 → 可能 **deadlock**。
- **First readers–writers**：`rw_mutex = 1, mutex = 1, read_count = 0`；第一個 reader `wait(rw_mutex)`，最後一個 reader `signal(rw_mutex)`；writers 可能 **starve**。
- **Condition variables (POSIX)**：`pthread_cond_wait(&cond, &mutex)`（必須持有 mutex；wait 時**釋放 mutex 並 sleep**，醒後**重新 acquire**）；`pthread_cond_signal(&cond)`。

---

8. 常見陷阱 & MC 速查

敘述	真/假
Race condition: outcome depends on order of access	✓
counter++ is a single atomic machine instruction	✗
CS requirements: mutual exclusion, progress, bounded waiting	✓
"Synchronization" is one of the 3 CS requirements	✗
Bounded waiting implies progress	✓（summary）
Disabling interrupts works well on multiprocessors	✗（only uniprocessor）
Atomic = non-interruptible	✓
test_and_set returns the new value	✗（old value）
CAS returns old value	✓
Simple TAS lock satisfies bounded waiting	✗
Bounded-waiting TAS: waiting process enters within n−1 turns	✓
test_and_set releases the lock	✗
Spinlock wastes CPU but avoids context switch	✓
Spinlocks useful on multiprocessors for short critical sections	✓
Semaphore accessed only by wait/signal (except init)	✓
P() = wait, V() = signal	✓
Binary semaphore = mutex lock (equivalent)	✗（can be used as one, but more general）
Counting semaphore initialized to # of resources	✓
Blocking semaphore value can be negative = # waiting	✓
Non-preemptive kernel essentially free of race conditions (uniprocessor)	✓
LIFO semaphore queue may cause starvation	✓
Wrong order of wait() on two semaphores may deadlock	✓

9. Past-paper 題目（Ch6）

- **S26**：MC23（A 2; B 5;）、MC24（test_and_set）、MC25（A 錯）、Q2.1 race condition（4 分）、Q2.4 swap spinlock + 優缺點（4 分）、Q5 semaphore 填空（8 分）
- **F19**：Q4.1 三個條件（4 分）、Q4.2 binary semaphore vs mutex（3 分）、Q4.3 c 可能值（8 分）、Q4.4 semaphore 序列（10 分）
- **F20**：Q5 c 可能值（15 分！）
- **Sp21**：MC mutual exclusion、MC CS 唔需要 synchronization、MC race condition、MC4 TAS entry、Q3.1 compare_and_reset spinlock（4 分）、Q3.2 bounded waiting 改錯（7 分）

10. 自我測驗

1. 寫出 CS 三個 requirements 嘅定義（英文）。
2. 用 `test_and_set` 寫 lock；點解唔滿足 bounded waiting？
3. 一個 semaphore value = −3 代表咩？（3 個 processes 喺 waiting queue）
4. x = 0；T1：`x = x + 1`；T2：`x = x * 2`（每句 read/write 分開）→ 可能值？（**2**: T1→T2；**1**: T2→T1；**1**: T1 read 0, T2 read 0 & write 0, T1 write 1；**0**: T2 read 0, T1 write 1, T2 write 0 → {0, 1, 2}）
5. 三個 threads 要 A → B → C 次序，用最少 semaphores 寫出來。（s1 = s2 = 0；A; signal(s1) | wait(s1); B; signal(s2) | wait(s2); C）

L Labs & PA1/PA2 Lab 1–6 • PA 考點 • lab 例題

📌 **點解要睇**：Midterm email 講明「There may be questions related to **PA** or coding questions using some **APIs we discussed in our lab classes**」；Lab 3 slide 亦講「**PA-related questions in exam** — it is crucial to have a thorough understanding of the project assignments」。S26 Q3.3 直接叫你「refer to the **Lab 3 notes** for open, pipe, read, write」。✓ 本頁所有 fork 數目、semaphore 輸出、race 值、Gantt chart 都已用真 C 程式 / 窮舉 / 模擬器驗證，同 lab 官方答案一致。

L1. Lab 1 — Linux 環境 & Shell 指令

- **Terminal** = 畫面/視窗；**Shell** = 解讀指令嘅程式（bash, tcsh...）→ **shell fetches, interprets, executes** commands（Ch2 [2.8]）。
- 常用指令：

指令	用途	指令	用途
`pwd`	顯示目前目錄	`cd dir` / `cd ..` / `cd ~`	轉目錄
`ls -l -h` / `ls -a`	列出檔案（詳細 / 包括隱藏）	`mkdir` / `rmdir`	建立 / 刪除目錄
`cp a b` / `mv a b` / `rm f`	複製 / 搬或改名 / 刪除	`cat` / `less` / `head` / `tail`	睇檔案
`ps` / `ps aux`	列出 processes（PID）	`kill PID`	送 signal 畀 process
`wc -l`	數行數	`sort` / `grep x`	排序 / 搵字
`diff a b`	比較檔案（PA 對答案）	`chmod u+x f`	加執行權限
`du -ah | sort -rh | less`	睇 disk 用量（CS Lab 2 quota 100MB）	`vim` / `nano`	文字編輯器

L2. Lab 2 — C in Linux、編譯流程、Ch1–3 Review

L2.1 Compile pipeline（同 Ch2 linker/loader 掛鈎）

- 最簡：`gcc prog.c -o prog` → `./prog`（**loader** 將 executable 載入 memory）。
- `gcc -c main.c` → `main.o`（relocatable object）；`gcc -o main main.o -lm` → **linker** 連 math library（`-lm`，用 `<math.h>` 時要加）。
- `-g`：加 debugging info → `gdb prog`；Pthreads：`gcc prog.c -lpthread`（或 `-pthread`）。
- **Makefile**：`target: prerequisites` + 下一行 **TAB**（唔可以用空格）+ command；`make` / `make clean`。

L2.2 I/O Subsystem 補充（Lab 2 review slides）

- **Buffering** — store data temporarily while being transferred；**Caching** — faster device holding a **copy** of data。
  - ⭐ **分別：a buffer may hold the only existing copy of a data item, whereas a cache only holds a copy on faster storage of an item that resides elsewhere.**
- **Spooling** — hold output for a device that **cannot accept interleaved data**（e.g. **printer**）。
- I/O hardware：**port**（connection point）、**bus**（PCI / PCIe、expansion bus for slow devices）、**controller (host adapter)**；device registers：**data-in, data-out, status, control**。
- **Device driver** presents a **uniform device-access interface** to the I/O subsystem（abstraction）。
- Interrupt 補充：**maskable vs nonmaskable**；**interrupt vector = table of interrupt-handler addresses**；**multilevel interrupts**（high vs low priority）；⭐ **a trap (software interrupt) has a relatively LOW interrupt priority compared with device interrupts**；multi-CPU systems can process interrupts concurrently。
- **DMA controller operates the memory bus directly — momentarily prevents the CPU from accessing main memory**（cycle stealing）；完成後 interrupt。

L2.3 Lab 2 fork 題（已驗證 ✓）

```
int main() {
    pid_t pid = fork();  int cnt = 0;
    if (pid == 0) {
        cnt += 10;  pid = fork();
        if (pid != 0) { cnt += 10;  pid = fork(); }
    }
    printf("%d \n", pid);  printf("%d \n", cnt);
    return 0;
}
```

- 4 個 processes 印出：P0（pid≠0, cnt 0）、C1（pid≠0, cnt 20）、C2（pid 0, cnt 10）、C3（pid 0, cnt 20）。
- **non-zero PID 印 2 次；'0' 印 3 次（P0 嘅 cnt + C2、C3 嘅 pid）；'10' 印 1 次。**

填空：保證輸出 process 3 → 2 → 1（每個都印 x = 1）：

```
if (fork()) {            // BLANK1: fork() (> 0 亦可)
    x = x + 1;  wait(0); // BLANK2: wait(0) / wait(NULL)
    printf("...process 1 is %d\n", x);
} else if (fork()) {     // BLANK3: fork()
    x = x + 1;  wait(0); // BLANK4: wait(0)
    printf("...process 2 is %d\n", x);
} else { x = x + 1;  printf("...process 3 is %d\n", x); }
```

L3. Lab 3 — Linux Process & Pipe System Calls（⭐ ⭐ ⭐ PA1 + 考試 API）

L3.1 本 lab 用到嘅 system calls

- **Process**：`getpid`, `getppid`, `sleep`, `fork`, `wait`, `signal`
- **File**：`open`, `read`, `write`, `close`, `pipe`, `dup`, `dup2`
- **Execute**：`execl`, `execlp`, `execle`, `execv`, `execvp`, `execvpe`

Call	重點
`pid_t getpid(void)` / `getppid(void)`	自己 / parent 嘅 PID
`unsigned int sleep(unsigned int s)`	停 s 秒（`usleep` 用 μs；`nanosleep` 更細）
`pid_t fork(void)`	child 0、parent child PID、−1 error（set errno）；child 得到 parent address space（text, data, bss, heap, stack）同 **open file descriptors 嘅獨立 copy**
`pid_t wait(int *status)`	等任何一個 child；`wait(0)` / `wait(NULL)` 唔要 status；返回 child PID；**課程只用 `wait()`**（`waitpid` 唔使）
`signal(SIGINT, handler)`	**signal = software-generated interrupt**；可以用 customized handler 取代 default；**child inherits the customized handler**

L3.2 Zombie / Orphan demo（lab 示範）

- **Orphan**：parent 先完（冇 wait）→ shell prompt 出咗嚟之後 child 先繼續印 → child **被 init (PID 1) adopt & reap**。
- **用 wait() 正確**：parent 印完 `wait(0)` → child 印完先返 shell。
- **Zombie**：child 已完但 parent 仲 `sleep(10)` 未 `wait()` → child 喺嗰段時間係 zombie；parent `wait(0)` 後清走。
- **Exit status**：**voluntary** — normal `exit(0)`（return from main / `exit` / `_exit`）、error `exit(1)`；**involuntary** — fatal error（divide by 0, segmentation fault, core dump）、**killed by another process（`kill()`）**。child 終止時：open files flushed & closed、resources de-allocated、**parent notified via a signal**、exit status via `wait()`。

L3.3 File operations（低階 vs stdio）

```
fd = open("hello.txt", O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR);   /* permission 600 */
write(fd, "hello\n", 6);  close(fd);
fd = open("hello.txt", O_RDONLY);  read(fd, buffer, 6);  close(fd);
```

- `open/read/write/close`（`<fcntl.h>`, `<unistd.h>`）= **low-level system calls**；`fopen/fread/fwrite/fclose`（`<stdio.h>`）= **high-level C library**。
- ⭐ **PA 唔准用 stdio file ops：`FILE*` 係 C library 嘅 struct，cannot be used in pipe operations**。

L3.4 Pipes

- **Key concept：treat a pipe as 2 files**；`int pipe(int pfds[2])` → **pfds[0] read end，pfds[1] write end**。
- Data received **in the order it was sent**；**OS enforces mutual exclusion（only one process at a time）**；**processes sharing the pipe must have the same parent**。
- ⭐ **read/write are blocking**：冇 data 讀 → reader block；pipe 滿 → writer block（pipe size 有限：4 KB page / 64 KB）。
- **Two-way communication → create two pipes**（每個方向一條）。
- Shell pipe `ps aux | grep root | sort | wc -l`：前一個 command 嘅 output 做下一個嘅 input。

L3.5 exec family

- **6 variants**：`execl, execlp, execle, execv, execvp, execvpe` — **replace the current process image；成功唔會 return；error return −1**。
- 字母意思：**l** = arguments as a **list**（數目已知）；**v** = arguments in a **vector/array**（compile time 唔知數目，PA1 用 `execvp`）；**p** = search **PATH**；**e** = pass **environment**。

```
execlp("ls", "ls", "-l", "-h", NULL);            /* same as running "ls -l -h" */
char *args[] = {"ls", "-l", "-h", NULL};  execvp("ls", args);
printf("never printed if exec succeeds\n");
```

L3.6 dup / dup2（redirection 核心）

- `int dup(int oldfd)` — copy 到 **first unused (lowest-numbered) fd**；`int dup2(int oldfd, int newfd)` — copy 到指定 newfd（如已開先 close）；成功返回新 fd，失敗 −1。
- Default：**0 = stdin，1 = stdout，2 = stderr**。
- **Lab 題：以下程式輸出係咩？**

```
fd = open("output.txt", O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR);
close(1);                /* close stdout */
dup(fd);                 /* lowest free fd = 1 → stdout now refers to output.txt */
printf("Hello World!\n"); fflush(stdout);
```

→ **螢幕冇嘢顯示；"Hello World!" 寫入 output.txt**。

- **`ls | wc -l` 實作（lab 版本：parent 做 wc）：**

```
pipe(pfds);
if (fork() == 0) {                  /* child → ls */
    close(1); dup(pfds[1]);         /* stdout = pipe write end */
    close(pfds[0]);
    execlp("ls", "ls", NULL);
} else {                            /* parent → wc -l */
    close(0); dup(pfds[0]);         /* stdin = pipe read end */
    close(pfds[1]);                 /* MUST close write end, else wc never sees EOF */
    wait(0);
    execlp("wc", "wc", "-l", NULL);
}
```

- **dup vs dup2**：兩個都得；**dup2 更 flexible、易讀**；如果連續兩個 `close` 再兩個 duplicate，**應該用 dup2 指定位置**。

L3.7 Lab 3 fork 例題 + 更多歷屆 fork 題（全部驗證 ✓）

題目	Code（重點）	答案
Ex1	`for (i=0;i<3;i++) fork();`	$2^3 = 8$
Ex2	`for(i<2){ fork(); for(j<3) if(!fork()){} }`	8 個連續 fork → $2^8 = 256$
Ex3	`for(i<10){ if(fork()){fork();fork();} else fork(); }`	每輪 ×6（parent 4 + child 2）→ 6^{10}
Fall 2022 Q3	`for(i<n){ if (fork() == fork()) printf(...); }`	processes 4^n；lines $4^0 + 4^1 + \cdots + 4^{n-1} = \frac{4^n - 1}{3}$（n=1,2,3：1, 5, 21 行）
Fall 2024 P3.1	`for(i<M){ pid = fork(); if(!pid) for(j<N) fork(); }`	(a) **child** 執行 line Y（`!pid` true）；(b) M=N=1 → **3**；(c) $(2^N + 1)^M$
Spring 2025 MC12	`if (fork() || fork()) { if (fork() && !fork()) fork(); }`	**9** processes（新建 8 個）

- **Fall 2022 解釋**：每輪每個 process 變 4 個（2 個 fork）→ 4^n；每輪只有「兩個 fork 都返回 0」嗰個 grandchild 印 → 第 k 輪印 4^{k-1} 行。
- **Fall 2024 解釋**：設 $f(k)$ = 剩 k 輪時一個 process 最終變幾多個：parent 自己繼續 → $f(k-1)$；child 做 N 次 fork 變 2^N 個、每個都繼續 → $2^N f(k-1)$；所以 $f(k) = (1 + 2^N) f(k-1)$，$f(M) = (2^N + 1)^M$。
- **Short-circuit 圖解（lab）：**

Pattern	共幾多 processes	入 if-clause	入 else-clause
`fork() && fork()`	3	**1**（兩個都 > 0）	2
`fork() || fork()`	3	**2**	1（兩個都 = 0）

- S25 MC12：外層 `||` → 2 個入 if、1 個入 else；內層 `fork() && !fork()` → 1 個入 inner if（再 fork → 2）、2 個唔入 → 每個變 $4 \to 2 \times 4 + 1 = 9$。
- **Tips（lab）**：見到 loop → **展開成冇 loop 嘅等價 statements**；nested loop → 先展開內層；`if (fork())` → 分別諗 parent 同 child 行咩。

L3.8 PA1 — Simplified Linux Shell（考試可能問）

- 6 種 command pattern：`cmd`、`cmd < in`、`cmd > out`、`cmd < in > out`、`cmd > out < in`、`cmd1 | cmd2 | …`（最多 8 段）。
- **`cd` 唔可以用 exec 做**（要改 shell 自己嘅 working directory）→ **`chdir(path)`**（0 成功 / −1 失敗）。
- **唔准用 `system()` / `popen()`**（會開另一個 shell 代你做）。
- **Redirection** = `open` + `dup/dup2` 取代 stdin/stdout；**multi-level pipe** = 將 2-level pipe 寫成 for-loop。
- `execvp` 嘅 argv array 尾要放 **NULL**（所以 8 個 arguments 要 size 9）。
- **Ctrl-C** → `signal(SIGINT, handler)`，SIGINT = **2**；start/end PID 要一致（冇多餘 processes）。

- Plagiarism：JPlag 檢測，**提供同抄襲雙方都 0 分**。

L4. Lab 4 — Named Pipes & Pthreads

L4.1 Anonymous vs Named pipes（lab 版本）

- Anonymous：unidirectional；cannot be accessed outside the creating process；parent–child；ceases after processes terminate。
- Named (FIFO)：bidirectional；no parent–child needed；several processes（多 writers）；**appear as files**（`mkfifo()`）；**exist until explicitly deleted**。
- ⭐ FIFO 細節：
  - **open / read / write 係 blocking**：**FIFO 要兩端都有 process open 先會 unblock**（冇 reader 時 writer 會 block）；用 `O_NONBLOCK` 可以變 non-blocking。
  - **Data read once then discarded**（唔似 regular file 可以重讀）→ **cannot broadcast to multiple recipients**。
  - 實際上 **一條 FIFO 唔適合雙向**（寫完即刻讀可能讀返自己個 message）→ 雙向用兩條。
  - **Writes ≤ PIPE_BUF bytes are atomic**；更大可能同其他 writer 交錯。
- `popen(cmd, "r"/"w")` / `pclose(fp)`（`<stdio.h>`）：fork 一個 child 執行 cmd，返回 `FILE*` 連住佢嘅 stdout（"r"）或 stdin（"w"）；`pclose` 等佢完並返回 termination status。（**PA1 禁用**）
- FIFO producer：`if (access(FIFO_NAME, F_OK) == -1) mkfifo(FIFO_NAME, 0777);` → `open(FIFO_NAME, O_WRONLY)` → `write` loop → `close`；consumer：`open(..., O_RDONLY)` → `read` until 0 → `close`。
- `access(path, how)`：`F_OK` 存在、`R_OK` / `W_OK` / `X_OK` 權限；0 = OK，−1 = 唔得。

L4.2 Exam example — 兩條 ordinary pipes 雙向通訊（填空）

```
char buffer[10] = {};  int p2c[2], c2p[2];
pipe(p2c);                                  /* BLANK1 */
pipe(c2p);                                  /* BLANK2 */
pid_t pid = fork();
if (pid == 0) {                             /* child */
    const char msg[] = "Child";
    close(c2p[0]);
    write(c2p[1], msg, 6);                  /* BLANK3（5 亦接受） */
    close(p2c[1]);
    read(p2c[0], buffer, 7);                /* BLANK4（6 亦接受） */
    printf("Child received the message: %s\n", buffer);
} else {                                    /* parent */
    const char msg[] = "Parent";
    close(c2p[1]);
    read(c2p[0], buffer, 6);                /* BLANK5（5 亦接受） */
    close(p2c[0]);
    write(p2c[1], msg, 7);                  /* BLANK6（6 亦接受） */
    printf("Parent received the message: %s\n", buffer);
}
```

- 輸出：`Parent received the message: Child` / `Child received the message: Parent`。

L4.3 Pthreads

```
int sum;                                /* shared by the threads */
void *runner(void *param) {
    int upper = atoi(param);  sum = 0;
    for (int i = 1; i <= upper; i++) sum += i;
    pthread_exit(0);
}
int main(int argc, char *argv[]) {
    pthread_t tid;  pthread_attr_t attr;
    pthread_attr_init(&attr);                       /* default attributes */
    pthread_create(&tid, &attr, runner, argv[1]);   /* id, attr, start routine, arg */
    pthread_join(tid, NULL);                        /* wait for the thread */
    printf("sum = %d\n", sum);
}
```

- `pthread_t` = thread id；`pthread_attr_t` = attributes（stack size, scheduling info）；compile `-lpthread`。
- Join 多個 threads：`for (i = 0; i < 10; i++) pthread_join(workers[i], NULL);`
- **Thread trace**：main 印 "main: begin" → create T1、T2 → join → "main: end"；**threads run independently, order depends on scheduling**（T2 可以喺 T1 之前行）。
- **兩個 threads 各印 "begin of X" 同 "end of X"** → 中間 4 行有 $\binom{4}{2} = 6$ 種次序（每個 thread 自己 begin 先過 end），"main: start" 最先、"main: finish" 最尾 → **6 possible outputs**。

L4.4 Exam example — thread vs process 共享 memory（MC）

```
void *child(void *arg) { int *counter = (int *)arg; (*counter)++; return NULL; }
int main() {
    int counter = 0;  pthread_t thread;
    pthread_create(&thread, NULL, child, (void *)&counter);
    pthread_join(thread, NULL);
    printf("%d\n", counter);                /* 1: thread shares memory */
    counter = 0;
    pid_t pid = fork();
    if (pid) wait(NULL);
    else { child((void *)&counter); exit(0); }
    printf("%d\n", counter);                /* 0: child process has its own copy */
}
```

- 選項 A `0 0`、B `1 0`、C `0 1`、D `1 1` → **答案 B（1 / 0）**。**Key：threads share memory space but processes have separate memory spaces.**

L5. Lab 5 — Scheduling Review & PA2（MLFQ）

L5.1 Review 例題（P1 7, P2 4, P3 1, P4 4，全部 t=0，已驗證 ✓）

Algorithm	Gantt	avg WT
FCFS / RR q=8	`0 P1 7 P2 11 P3 12 P4 16`	$\frac{0+7+11+12}{4} = 7.5$
RR q=4	`0 P1 4 P2 8 P3 9 P4 13 P1 16`	$\frac{9+4+8+9}{4} = 7.5$
RR q=2	`0 P1 2 P2 4 P3 5 P4 7 P1 9 P2 11 P4 13 P1 16`	$\frac{9+7+4+9}{4} = 7.25$
SJF	`0 P3 1 P2 5 P4 9 P1 16`（P2、P4 同長 → 先 P2）	$\frac{9+1+0+5}{4} = 3.75$
Priority（4,1,2,3）	`0 P2 4 P3 5 P4 9 P1 16`	$\frac{9+0+4+5}{4} = 4.5$

- **SRTF**（P1 0/7, P2 2/4, P3 4/1, P4 5/4）：`0 P1 2 P2 4 P3 5 P2 7 P4 11 P1 16` → $\frac{9+1+0+2}{4} = 3$。
- 觀察：q 大過最長 burst → RR = FCFS；q 細咗 avg WT 唔一定變差（7.25 < 7.5）。

L5.2 ⭐ Priority + RR 有唔同 arrival time（HW / Exam 熱門）

- ⚠️ **Common mistake：reset the time quantum of a preempted process.** 正確：**被高 priority 新 process preempt 嘅 process 要記住佢剩餘嘅 quantum**（同 MLFQ 規則一樣）。
- **Spring 2023 midterm 官方答案**（q = 2）：`0 P1 1 P2 4 P1 5 P3 7 P1 8 P4 10 P1 11 P3 13 P5 15 P1 16 P3 18 P5 20 P3 22 P5 24`
  - P1 喺 0–1 跑咗 1 unit 被 P2（高 priority）preempt → 4–5 **只跑剩低嘅 1 unit quantum** 然後輪到 P3（同 priority RR）。8–10 同樣：P4 preempt P1，P1 10–11 用返剩低 1 unit。
  - 由 Gantt 推返嘅 table（模擬器重現完全一致 ✓）：P1 (0, 5, pr 2)、P2 (1, 3, pr 1)、P3 (2, 8, pr 2)、P4 (8, 2, pr 1)、P5 (9, 6, pr 2)，細數字 = 高 priority。

L5.3 RM / EDF 例題（deadline < period）

- P1 (t=1, d=2, p=4)、P2 (2, 5, 6)、P3 (3, 7, 9)；$U = \frac{1}{4} + \frac{2}{6} + \frac{3}{9} = 0.917$。
- **RM**（P1 > P2 > P3）：`0 P1 1 P2 3 P3 4 P1 5 P3 6 P2 8 P1 9 P3 …` → **P3 first job finishes at 10 > deadline 7 → misses 1 deadline**。
- **EDF**：`0 P1 1 P2 3 P3 4 P1 5 P3 7 P2 8 P1 9 P2 10 P3 12 …` → **no deadline missed**。

L5.4 PA2 — N-level MLFQ simulator

- **N = 1–4 queues，last queue must be FCFS**（input 用大數 e.g. 1000 代表 FCFS），**≤ 10 processes**。
- Input：`queue_num = 3`、`time_quantum = 2 4 1000`、`process_table_size`、`process_table`（`P1 0 7` …）；`#` 開頭 / 空行忽略。
- Assumptions：**sorted by arrival**；**first arrival = 0**；**no two processes arrive at the same time**；**CPU never idle**；Gantt ≤ 300 steps。
- 用法：`./mlfq < input.txt > output.txt`（**input/output redirection**）；用 `diff --side-by-side your-out.txt sample-out.txt`（`--suppress-common-lines`）對答案。
- Output：parsed values + `Gantt Chart = 0 P1 2 P2 3 P1 4 P3 5 P1 6 P4 8 P1 10 P4 11 P1 12`。
- **Step-by-step 例子（Q0 q=2, Q1 q=4, Q2 FCFS；P1 0/7, P2 1/1, P3 4/1, P4 6/3）**，已用模擬器驗證 ✓：
  - t=1 P2 到 → **no preemption**（P1 都喺 Q0）；t=2 P1 用完 Q0 quantum → Q1；P2 跑。
  - t=3 P2 完 → P1（Q1，quantum 4）；t=4 P3 到 → **P1 preempted**（剩 burst 4，**剩 quantum 3**）。
  - t=5 P3 完 → P1；t=6 P4 到 → P1 preempted（剩 quantum 2）；P4 6–8 用完 Q0 → Q1 tail。

- t=8 Q1 = [P1, P4] → P1 跑 2 → quantum 用完（剩 burst 1）→ Q2；t=10 P4 跑 1 完；t=11 P1（Q2）跑到 12。

L6. Lab 6 — Synchronization Review（Sample Questions）

L6.1 Peterson's solution（software-only，2 processes）

```
/* P_i (j = 1 - i) */
do {
    flag[i] = true;  turn = j;
    while (flag[j] && turn == j) ;   /* busy wait */
        /* critical section */
    flag[i] = false;
        /* remainder section */
} while (true);
```

- Shared：`flag[2]`（想入 CS）、`turn`（輪到邊個）；**assumption：load and store are atomic**。
- **Proof**：ME — 兩個都想入時 `turn` 只可以係 0 或 1 → 只有一個過到 while；Progress — 冇人喺 CS 就即刻入；**Bounded waiting — 最多等對方一次**（對方出嚟會設 `flag[j] = false`，再想入就要設 `turn = i`）。
- 缺點：**複雜（只限兩個 process）、每個 thread code 唔同、busy waiting 浪費 CPU** → OS 提供 mutex / semaphore / condition variable。

L6.2 POSIX primitives 速查

- Mutex：`pthread_mutex_init(&m, NULL)` → `lock` → access → `unlock` → `destroy`；**thread 持 lock 時 crash → 其他人 block**。
- Semaphore（`<semaphore.h>`）：`sem_init(&s, pshared, value)`（**pshared = 0：同一 process 嘅 threads 共用**）、`sem_wait`、`sem_post`、`sem_destroy`。

Initial value	行為	用途
1	一個 permit	**mutual exclusion（當 mutex）**
0	wait() 一定 block 直至有人 signal	**ordering**（等其他 thread 做完先）
N	N 個 permits	**管理 N 個相同 resources**

- Condition variable：`pthread_cond_wait(&cond, &mutex)` **atomically releases the mutex and sleeps；signalled → re-acquires the mutex before returning**；用 `while (!ready)` 包住；`pthread_cond_signal` / `pthread_cond_broadcast`。
- Lab note：**semaphore ≈ integer S + a mutex protecting it + a condition variable whose condition is "S > 0"**。

L6.3 Sample Questions（官方答案，已驗證 ✓）

- **Q0**：binary semaphore = mutex？→ **No** — init 1 可以當 mutex，但 init 0 可以做其他用途（ordering）。
- **Q1**：T1：`wait(S1); CS; signal(S2)`；T2：`wait(S2); CS; signal(S3)`；T3：`wait(S3); CS; signal(S1)` 要 T1, T2, T3, T1... → **S1 = 1, S2 = 0, S3 = 0**。
- **Q2**（P0 → P1 → P2 循環，填 BLANK）：**S0 = 1, S1 = 0, S2 = 0**；P0：`wait(S0)` … **`signal(S1)`**；P1：**`wait(S1)`** … **`signal(S2)`**；P2：**`wait(S2)`** … `signal(S0)`。
- **Q3**（mutex 填空）：`pthread_mutex_t* m = (pthread_mutex_t*) arg;`、`pthread_mutex_lock(m);`、`pthread_mutex_unlock(m);` → **purpose：protect the shared counter；final counter = 10,000**（兩個 threads 各加 5,000）。（`void*` 要 cast 返 `pthread_mutex_t*`，因為 thread function signature 一定係 `void *f(void *)`。）
- **Q4**（X = Y = 0）：A：`wait(X); "b"; signal(Y); wait(Y); "c"; signal(Y);` B：`signal(X); "a"; wait(Y); "d";`
  - **Both complete：abcd, bacd, bcad（3 種）**；如果 B 先攞到 Y → A 永遠 block → **bad, abd**（A 唔會完成）。
- **Q5**（S0 = 1, S1 = S2 = S3 = 0）：P0 loop：`wait(S0); print '1'; release(S1); release(S2); release(S3);`；P1：`wait(S1); release(S0);`；P2：`wait(S2); release(S0); wait(S3); release(S0);` → **'1' 印 4 次**（S0 permits：initial 1 + P1 1 + P2 2 = 4）。
- **Q6**（readers–writers，用兩個 mutex）：

```
/* reader */
pthread_mutex_lock(&mutex);
read_count++;
if (read_count == 1) pthread_mutex_lock(&rw_mutex);    /* first reader locks out writers */
pthread_mutex_unlock(&mutex);
    /* reading */
pthread_mutex_lock(&mutex);
read_count--;
if (read_count == 0) pthread_mutex_unlock(&rw_mutex);  /* last reader lets writers in */
pthread_mutex_unlock(&mutex);
/* writer */
pthread_mutex_lock(&rw_mutex);  /* writing */  pthread_mutex_unlock(&rw_mutex);
```

- **Q7**（race values）：a = 1, b = 3, c = 0；T1：`a = b + 1; c = a - b;`；T2：`a = 13; b = 10; c = 2;` → **c ∈ {−6, 1, 2, 3, 10}**。
  - 2：T2 嘅 `c = 2` 最後寫；1：T1 讀 a=4、b=3 後 T2 全做完，或者 T2 先全做（a = 11, c = 11 − 10）；−6：T1 讀 a=4 後 T2 全做（b=10）；3：T1 寫 a=4 → T2 全做 → T1 讀 a=13, b=10；10：T1 寫 a=4 → T2 寫 a=13 → T1 讀 a=13, b=3 → T2 寫 b, c → T1 寫 c = 10。
  - 方法：**列出每個 read 可能見到嘅值（a ∈ {4, 11, 13}, b ∈ {3, 10}）+ 邊個最後寫 c**，再剔走不可能組合（a=11 表示 b 已經係 10）。

L7. 常見陷阱（Labs / PA）

敘述	真/假
PA 可以用 `fopen` / `FILE*` 做 pipe	✗（FILE* 唔用得喺 pipe operations）
`wait(0)` 同 `wait(NULL)` 一樣	✓
`dup(fd)` 一定返回 1	✗（返回 lowest unused fd；要先 `close(1)`）
`exec*()` 成功後會返回 0	✗（唔會 return；失敗 −1）
`execvp` 嘅 argv 唔使 NULL 結尾	✗
`cd` 可以用 `execvp("cd", …)` 做	✗（要 `chdir`）
FIFO 開 write end 時冇 reader 會 block	✓
FIFO 啲 data 可以畀多個 reader 各讀一次（broadcast）	✗
Child 會繼承 parent 嘅 customized signal handler	✓
Threads share memory，processes 唔 share	✓
Priority + RR 被 preempt 後 quantum 重新計	✗（保留剩餘 quantum）
PA2：最後一層 queue 係 FCFS	✓
Peterson：assume load/store atomic	✓
`sem_init(&s, 0, 0)` 嘅 0（第二個）表示 threads 共用	✓（pshared = 0）
Condition wait 會放開 mutex	✓（醒返會重新攞）

Q Past Papers Solved 歷屆長題全解

Spring 2026 Midterm（lecturer 提供）

S26-Q2.1 Race condition (4 pts) CH6

What is a race condition? Give an example.

Answer.
A race condition occurs when two or more processes/threads access and manipulate shared data concurrently, and the final result depends on the order in which the accesses take place; since the order is unpredictable, the result may be incorrect or inconsistent. (2.5) Example: two threads increment the same `counter` without synchronization — `counter++` is load/add/store, so both may load the same old value and one update is lost. (1.5)

S26-Q2.2 Caching (4 pts) CH1

What is the fundamental idea of caching? Is a high cache hit ratio alone enough to guarantee good performance? Explain by considering hit time and miss penalty.

Answer.
Caching temporarily copies a subset of information from a slower but larger storage into a faster but smaller storage so that frequently used data can be accessed more quickly. (2) No — hit ratio is a major criterion but not sufficient (1); performance is the **average access time = hit ratio × hit time + miss ratio × miss time**, which also depends on the hit time and the miss penalty (1). A slow cache or a huge miss penalty can still give poor performance.

S26-Q2.3 Custom policy $f_i = 1/(a \cdot c_i + b)$ (4 pts) CH5

The OS keeps c_i = average past CPU burst of P_i and picks the process with the highest $f_i = \dfrac{1}{a\,c_i + b}$ $(a, b > 0)$, running it for one quantum or until it gives up the CPU. (a) Does this prioritize I/O-bound over CPU-bound processes? (b) Can it cause starvation?

Answer.
(a) **Yes** — I/O-bound processes have small CPU bursts c_i, hence a larger f_i and higher priority. (b) **Yes** — a CPU-bound process (large c_i, low f_i) can starve if higher-priority I/O-bound processes keep arriving.

S26-Q2.4 swap spinlock (4 pts) CH6

`swap(x, v)` atomically sets x to v and returns the old value of x. `locked` = 1 when held, 0 otherwise. (a) Write the line that acquires the spinlock with a busy-spin loop. (b) Advantage and disadvantage of a spinlock.

Answer.
(a) `while (swap(locked, 1) == 1);` (release: `locked = 0;`) (b) Disadvantage: wastes CPU cycles due to busy waiting. Advantage: no context switch is needed when a process must wait on a lock (a context switch can take considerable time) — good for short critical sections on multiprocessors.

S26-Q3.1 fork tree with break (8 pts) CH3

```
for (int i = 0; i < n; i++) {
    if (fork()) fork();
    else break;
}
```

(a) n = 2: draw the process tree; total processes and number of breaks. (b) Relationship between total processes and n.

Answer.
(a) **7 processes, 3 breaks.** Tree: P0 → P1 (break), P2, P3 (break), P4; P2 → P5 (break), P6. (b) Each active process per iteration creates 2 children: the first breaks, the second keeps looping → active count doubles: $A_k = 2^k$. After n iterations: active $= 2^n$, stopped $= 1 + 2 + \cdots + 2^{n-1} = 2^n - 1$. **Total** $= 2^{n+1} - 1$ (breaks $= 2^n - 1$).

S26-Q3.2 fork + exec scenarios (8 pts) CH3

```
int ret = fork();
if (ret == 0) { printf("Hello1\n"); exec("some_executable"); printf("Hello2\n"); }
else if (ret > 0) { wait(NULL); printf("Hello3\n"); }
else { printf("Hello4\n"); }
```

List all possible scenarios and outputs.

Answer.

- Case I — fork succeeds, exec succeeds: **Hello1, Hello3** (exec replaces the image; Hello2 never runs).
- Case II — fork succeeds, exec fails (returns −1): **Hello1, Hello2, Hello3**.
- Case III — fork fails: **Hello4**. (Hello3 always last because the parent waits.)

S26-Q3.3 pipe + dup fill-in (8 pts) CH3

Child counts even numbers in 1.txt, parent in 2.txt (file: length then elements, read with scanf); child sends its count via pipe; parent prints the total.

Answer.

- Blank 1: `close(0)` · Blank 2: `dup(fin)` (stdin now refers to the file)
- Blank 3: `if (number % 2 == 0) cnt++`
- Blank 4.1: `fd[1]` · 4.2: `&cnt` (child: `write(fd[1], &cnt, sizeof(int))`)
- Blank 5.1: `fd[0]` · 5.2: `&child_cnt` (parent: `read(fd[0], &child_cnt, sizeof(int))`)

S26-Q4.1 MLFQ Q0 RR4 / Q1 RR8 / Q2 FCFS (8 pts) CH5

P1 (0, 9), P2 (2, 25), P3 (6, 7), P4 (14, 18), P5 (45, 15), P6 (58, 6). New jobs → Q0; full quantum → demote; Q0 arrival preempts Q1/Q2 immediately; preempted job → head of its queue; no demotion when preempted.

Answer.

Gantt: `0 P1 4 P2 8 P3 12 P1 14 P4 18 P1 21 P2 29 P3 32 P4 40 P2 45 P5 57 P2 58 P6 64 P2 71 P4 77 P5 80` (P5 45–49 Q0 then 49–57 Q1; P6 58–62 Q0 then 62–64 Q1). Waiting = completion − arrival − burst: P1 21−0−9 = 12; P2 71−2−25 = 44; P3 32−6−7 = 19; P4 77−14−18 = 45; P5 80−45−15 = 20; P6 64−58−6 = 0 → **avg** $= \frac{140}{6} = 23.33$. Response (end of first run − arrival): 4, 6, 6, 4, 4 (49−45), 4 (62−58) → **avg** $= \frac{28}{6} = 4.67$ (6.33 also accepted if P5/P6 merged).

S26-Q4.2 RM / EDF with D < T (10 pts) CH5

(t, d, p): P1 (1, 3, 4), P2 (2, 4, 5), P3 (2, 6, 7), P4 (1, 5, 8); all arrive at 0; draw 0–16; ties → smaller ID; count missed deadlines.

Answer.

- **RM** (P1 > P2 > P3 > P4 by period): `0 P1 1 P2 3 P3 4 P1 5 P2 7 P3 8 P1 9 P3 10 P2 12 P1 13 P3 14 P3 15 P2 16` → missed: P3 (d 6, finished 8), P4 (d 5, never), P3 (d 13, finished 14), P4 (d 13, never) → **4 missed**. (U = 1.06 > 1.)
- **EDF**: `0 P1 1 P2 3 P4 4 P3 6 P1 7 P2 9 P1 10 P3 12 P4 13 P2 15 P1 16` → missed: P2 (d 14, finished 15), P1 (d 15, finished 16) → **2 missed**.

S26-Q4.3 Dynamic priority (9 pts) CH5

Larger value = higher priority; waiting changes priority at rate α, running at rate β; tie → larger ID. P1 (0, 7), P2 (3, 5), P3 (6, 6), P4 (9, 4).

Answer.

- (a) $\alpha = 0,\ \beta = 3$, initial 0: **`0 P1 7 P3 13 P4 17 P2 22`**
- (b) $\alpha = 1,\ \beta = 0$, only P1, P2, check every 3: **`0 P1 3 P2 6 P1 9 P2 11 P1 12`**
- (c) $\alpha = 1,\ \beta = -2$, initial 30, check every 3: **`0 P1 3 P2 6 P3 9 P4 12 P1 15 P2 17 P3 20 P4 21 P1 22`**

S26-Q5 Semaphore ordering (8 pts) CH6

A depends on B and C; B depends on C. s0, s1, s2 initialised to 0. workerA: BLANK1; wait(&s1); print A. workerB: BLANK2; print B; signal(&s2). workerC: BLANK3; print C; BLANK4; signal(&s1).

Answer.

BLANK1 `wait(&s2)` · BLANK2 `wait(&s0)` · BLANK3 `/* Nothing */` · BLANK4 `signal(&s0)` → order always C, B, A.

Q

Fall 2019 Midterm

F19-Q1.1 Linker and loader (2 pts) CH2

What do the linker and loader do, respectively?

Answer.
The linker combines relocatable object files (and libraries) into a single binary executable file; the loader loads the binary executable file into memory so it can run.

F19-Q1.2 Locality (3 pts) CH1

What are temporal locality and spatial locality?

Answer.
Temporal locality: a memory location accessed recently is likely to be accessed again in the near future. Spatial locality: locations near a recently accessed location are likely to be accessed soon.

F19-Q1.3 Why separate API and system calls (2 pts) CH2

Two reasons for separating APIs from the underlying system calls?

Answer.
(1) Program portability — the API stays the same across platforms. (2) Hide the complex details of system calls from programmers.

F19-Q1.4 LKM vs layered (3 pts) CH2

Similarity and difference between the loadable-kernel-module approach and the layered approach?

Answer.
Similarity: functionality is divided into modules, each with a well-defined interface. Difference: LKM is more flexible — any module can call any other module (no higher/lower layer restriction), and modules can be loaded/removed at run time.

F19-Q2.1 State transitions (3 pts) CH3

Fill in the actions for: running→ready, ready→running, running→waiting, waiting→ready.

Answer.
running→ready: **interrupt**; ready→running: **scheduler dispatch (CPU scheduling)**; running→waiting: **I/O or event wait**; waiting→ready: **I/O or event completion**.

F19-Q2.2 Socket (2 pts) CH3

Four pieces of information that uniquely define a socket connection?

Answer.
Source IP address, source port number, destination IP address, destination port number.

F19-Q2.3 Thread unique vs shared (3 pts) CH4

Three elements unique to each thread and three shared.

Answer.
Unique: program counter, registers (state), stack (thread ID). Shared: code, data (globals/heap), open files.

F19-Q2.4 Shell system calls (3 pts) CH3

Two system calls a shell must execute to start a new process on UNIX?

Answer.
`fork()` — clones the calling process (creates a child); `exec()` — overlays/replaces the child's address space with a new program (loads the program to run).

F19-Q2.5 Multicore vs SMP, CMT (4 pts) CH5

Two advantages of multicore vs conventional SMP; reason for chip multithreading (hyper-threading).

Answer.
Multicore is faster (on-chip communication) and consumes less power than separate single-core chips. CMT exploits **memory stall**: processors are much faster than memory, so a core may spend up to ~50% of its time waiting for data; with multiple hardware threads the core switches to another thread while one waits.

Q

F19-Q2.6 Lines printed vs n (5 pts) CH3

```
for (i = 0; i < n; ++i) { if (fork()) printf("Fork!\n"); else printf("Fork!\n"); }
```

Answer.
Each iteration doubles the processes and every process prints one line: iteration i prints 2^i lines → total $2 + 4 + \cdots + 2^n = 2^{n+1} - 2$ ✓

F19-Q2.7 fork()==fork() || fork()!=fork() (5 pts) CH3

```
if (fork() == fork() || fork() != fork()) fork();
printf("Fork!\n");
```

Answer.
23. After `fork()==fork()` there are 4 processes; only the one that got 0 twice is true → it skips the rest of `||` and forks → 2 lines. The other 3 run `fork()!=fork()` → 12 processes; 9 are true and fork again → 18; 3 false → 3. Total 2 + 18 + 3 = 23 ✓

F19-Q3.1 Which threads does the OS schedule (4 pts) CH5

Which threads are scheduled by the OS? How does mapping user threads onto kernel threads/LWPs differ from CPU scheduling?

Answer.
The OS schedules **kernel-level threads**. Scheduling user threads onto LWPs/kernel threads (PCS) is done by the thread library, usually by programmer-set priority, among threads of the **same process**; CPU scheduling (SCS) chooses which kernel thread runs on a CPU among **all** kernel threads in the system.

F19-Q3.2 FCFS problem; RR worse than FCFS (3 pts) CH5

Answer.
FCFS: **convoy effect** — short processes stuck behind a long one. RR has worse turnaround than FCFS when all processes have the **same CPU burst length that is larger than the quantum** (they all finish near the end).

F19-Q3.3 MLFQ approximates SJF (3 pts) CH5

Answer.
Short interactive jobs finish in the top (high-priority) queue, while long CPU-bound jobs sink to lower queues — so short jobs are served first, like SJF. Advantage: **no prior knowledge of CPU burst times is needed**.

F19-Q3.4 EDF to t = 38 (6 pts) CH5

P1 (period = deadline 10, t 4), P2 (16, 2), P3 (20, 8).

Answer.
`0 P1 4 P2 6 P3 14 P1 18 P2 20 P1 24 P3 32 P1 36 P2 38` (deadline ties at t = 10 and 30 → the running P3 continues). No deadline missed.

F19-Q3.5 Least Used First (12 pts) CH5

LUF picks the process that has used the fewest CPU cycles; tie → FCFS; preempted process goes to the end of the queue; scheduler runs on completion and on new arrival. P1 (0, 25), P2 (15, 25), P3 (25, 5), P4 (40, 5).

Answer.
(a) `0 P1 15 P2 25 P3 30 P2 40 P4 45 P1 55 P2 60` (b) TAT / RT / WT: P1 55 / 15 / 30; P2 45 / 10 / 20; P3 5 / 5 / 0; P4 5 / 5 / 0. (c) Similar to SRTF: short jobs tend to get high priority. Different: under LUF a process that has already run a long time is likely to starve; under SRTF a process with a long burst is likely to starve from the beginning.

Q

F19-Q3.6 FCFS / SJF / RR4 / SRTF (12 pts) CH5

P1 (0, 8), P2 (7, 13), P3 (9, 1), P4 (11, 5), P5 (20, 7), P6 (27, 1).

Answer.

- FCFS `0 P1 8 P2 21 P3 22 P4 27 P5 34 P6 35` — WT 0,1,12,11,7,7 → **6.33**
- SJF `0 P1 8 P2 21 P3 22 P4 27 P6 28 P5 35` — WT 0,1,12,11,8,0 → **5.33**
- RR q=4 `0 P1 8 P2 12 P3 13 P4 17 P2 21 P4 22 P5 26 P2 30 P5 33 P6 34 P2 35` — WT 0,15,3,6,6,6 → **6.0** (corrected official)
- SRTF `0 P1 8 P2 9 P3 10 P2 11 P4 16 P2 20 P5 27 P6 28 P2 35` — WT 0,15,0,0,0,0 → **2.5** (official; keeping P2 on the tie at t=20 also gives 2.5)

F19-Q4.1 CS requirements (4 pts) CH6

Answer.

(1) **Mutual exclusion** — if a process is executing in its CS, no other process can be in its CS. (2) **Progress** — if no process is in its CS and some want to enter, the selection of the next one cannot be postponed indefinitely. (3) **Bounded waiting** — there is a bound on the number of times other processes may enter their CS after a process has requested entry and before that request is granted.

F19-Q4.2 Binary semaphore vs mutex (3 pts) CH6

Answer.

No. A binary semaphore initialised to 1 can be used as a mutex lock, but it can also be used for other synchronization purposes (e.g., initialised to 0 for ordering/signalling).

F19-Q4.3 Possible values of c (8 pts) CH6

a = 7, b = 2, c = 5. T1: `if (a < 0) c = b - a; else c = b + a;` T2: `b = 8; a = -6;`

Answer.

14, 2, 9, 15, −4 — (a<0 branch: 8−(−6) = 14); else-branch: T2 done before c computed: 8+(−6) = 2; T2 after: 2+7 = 9; b written before read but a after: 8+7 = 15; b read before write, a written before read: 2+(−6) = −4.

F19-Q4.4 Semaphores L = 3, R = 0 with putc (10 pts) CH6

P1: wait(L); putc('C'); signal(R). P2: wait(R); putc('A'); putc('B'); signal(R). P3: wait(R); putc('D').

Answer.

(a) **3** D's. (b) A's: min **0**, max **infinite**. (c) `CABABDCDABCABD` — **No** (after "CABABDCD", R = 0, so P2 cannot print the next A before a third C). (d) `CABACDBCABDD` — **Yes**.

Fall 2020 Midterm

F20-Q2.1 Microkernel (4 pts) CH2

Answer.

Main idea: remove all nonessential components from the kernel and implement them as system/user-level programs in separate address spaces; communicate by message passing. Advantage: small kernel, easier to extend and port, more secure and reliable. Disadvantage: performance suffers due to increased system-function overhead (user↔kernel message passing).

F20-Q2.2 Linker and loader (4 pts) CH2

Answer.

Linker: combines several relocatable object modules into a single binary executable image (stored on disk). Loader: loads the executable into memory for execution.

F20-Q2.3 Heavyweight vs lightweight (4 pts) CH4

Answer.

A process is heavyweight because it owns an address space and resources; creating one duplicates program and data. A thread is lightweight — only an execution unit (PC, registers, stack) that shares code, data and files with other threads of its process, so creating it needs no duplication.

Q

F20-Q2.4 Common vs per-processor ready queue (4 pts) CH5

Answer.
Common queue: easy load balancing (no CPU idle while the queue is non-empty) but hard to keep processor affinity (and needs locking). Per-processor queues: natural processor affinity but need push/pull migration for load balancing.

F20-Q2.5 MLFQ benefits (4 pts) CH5

Answer.
(1) Resembles SJF/SRTF performance without prior knowledge of the next CPU burst (2 pts); (2) RR-like good response time for interactive jobs (1); (3) fair — all processes make progress (1).

F20-Q3.1 Possible outputs (10 pts) CH3

```
if (fork() == 0) {
    if (fork() > 0) { printf("1"); wait(NULL); printf("2"); }
    else printf("3");
} else printf("4");
```

Answer.
Unbuffered: **8** outputs — "2" after "1" and "3"; "4" anywhere: 1342, 1432, 3142, 4132, 3412, 4312, 1324, 3124. Buffered ("1" and "2" flushed together): 3 outputs — 4312, 3412, 3124.

F20-Q3.2 Count "a" (13 pts) CH3

```
if (!fork()) { for (int i = 0; i < n; i++) if (fork()) fork(); }
fork(); printf("a");
```

Answer.
First fork: the child enters the loop; each iteration every process becomes 3 → 3^n processes from the child branch. Plus the original → $3^n + 1$. The final fork doubles → $2(3^n + 1)$ **"a"s** ✓

F20-Q4.1 FCFS / SJF / SRTF (9 pts) CH5

P1 (0, 9), P2 (3, 3), P3 (8, 2), P4 (10, 6), P5 (16, 8), P6 (20, 5).

Answer.
- FCFS `0 P1 9 P2 12 P3 14 P4 20 P5 28 P6 33` → (0+6+4+4+4+8)/6 = **4.33**
- SJF `0 P1 9 P3 11 P2 14 P4 20 P6 25 P5 33` → (0+8+1+4+9+0)/6 = **3.67**
- SRTF `0 P1 3 P2 6 P1 8 P3 10 P1 14 P4 20 P6 25 P5 33` → (5+0+0+4+9+0)/6 = **3.0**

F20-Q4.2 Two-level feedback queue with aging (10 pts) CH5

Q0 RR 4 ms (high), Q1 FCFS; new → Q0; Q0 arrival preempts Q1; preempted Q1 process → head of Q1 with timer reset; a Q1 process whose timer reaches 8 ms rejoins Q0. P1 (0, 9), P2 (3, 3), P3 (8, 2), P4 (10, 9), P5 (13, 4), P6 (18, 5).

Answer.
`0 P1 4 P2 7 P1 8 P3 10 P4 14 P5 18 P1 22 P6 26 P4 30 P6 31 P4 32` → WT (13+1+0+13+1+8)/6 = **6**. (P1 ages back to Q0 at 16; P4 at 22.)

F20-Q4.3 Priority + RR (8 pts) CH5

Smaller number = higher priority; preemptive; same priority → RR q = 4. P1 (0, 9, 2), P2 (3, 3, 1), P3 (8, 5, 4), P4 (10, 2, 4), P5 (16, 8, 5), P6 (20, 5, 3).

Answer.
`0 P1 3 P2 6 P1 12 P3 16 P4 18 P3 19 P5 20 P6 25 P5 32` → (3+0+6+6+8+0)/6 = **3.83**

F20-Q5 Possible values of c (15 pts) CH6

a = 7, b = 2. T1: `if (b < 0) c = b - a; else c = b + a;` T2: `b = -8; a = 6;`

Answer.
−14, −15, −2, −1, 8, 9 (2.5 marks each).

Q

Spring 2021 Midterm

Sp21-Q1.1 pid1/pid2 (5 pts) CH3

```
pid1 = fork(); pid2 = fork();
if (pid1 > 0) if (pid2 == 0) fork();
```

Answer.
5 — 4 processes after two forks; only the child created by the parent's second fork has pid1 > 0 and pid2 == 0, so one more process ✓

Sp21-Q1.2 Nested loop (5 pts) CH3

```
if (fork() == 0) { for (int i = 0; i < 5; i++) if (fork() == 0) fork(); }
fork();
```

Answer.
$(3^5 + 1) \times 2 = 488$ ✓

Sp21-Q1.3 Output orderings (8 pts) CH3

```
printf("A\n");
if (fork() == 0) printf("B\n");
else { printf("C\n"); ___A___ }
printf("D\n");
```

Answer.
(a) empty: ABDCD, ABCDD, ACBDD, ACDBD. (b) `wait(NULL)` : ABDCD, ABCDD, ACBDD. (c) `printf("E\n")` : ABDCED, ABCDED, ABCEDD, ACBDED, ACBEDD, ACEBDD, ACEDBD.

Sp21-Q2.1 FCFS / SJF / RR8 / SRTF (10 pts) CH5

P1 (0, 2), P2 (4, 12), P3 (6, 6), P4 (9, 4), P5 (13, 5).

Answer.
FCFS 7.2 · SJF 6.6 · RR(8) 7.6 · SRTF 4.2 (Gantt charts in Ch5 §14.9; note the idle gap 2–4).

Sp21-Q2.2 MLFQ q=2, q=4, FCFS (8 pts) CH5

P1 (0, 10), P2 (10, 8), P3 (8, 30), P4 (16, 2), P5 (22, 8), P6 (6, 22).

Answer.
`0 P1 2 P1 6 P6 8 P3 10 P2 12 P6 16 P4 18 P3 22 P5 24 P2 28 P5 32 P1 36 P6 52 P3 76 P2 78 P5 80` → WT 26, 60, 38, 0, 50, 24 → **33**.

Sp21-Q2.3 EDF vs RM (8 pts) CH5

P1 (1, 3, 3), P2 (2, 4, 4), 0–14, soft real-time.

Answer.
EDF `0 P1 1 P2 3 P1 4 P2 6 P1 7 idle 8 P2 10 P1 11 idle 12 P1 13 P2 14`; RM `0 P1 1 P2 3 P1 4 P2 6 P1 7 idle 8 P2 9 P1 10 P2 11 idle 12 P1 13 P2 14`. EDF is better: dynamic deadline-based priorities can meet more deadlines (up to 100% utilization) than RM's static priorities.

Sp21-Q3.1 compare_and_reset spinlock (4 pts) CH6

Answer.
`struct spinlock { int value = 0; };` · acquire: `while (compare_and_reset(&lock->value, 0, 1) == 0) ;` · release: `lock->value = 0;`

Sp21-Q3.2 Fix the bounded-waiting algorithm (7 pts) CH6

Answer.
Problems: (1) only processes with index > i are checked → lower-index processes can wait unboundedly; (2) the lock is never released when nobody is waiting. Fix: cyclic scan `j = (i + 1) % n; while ((j != i) && !waiting[j]) j = (j + 1) % n; if (j == i) lock = 0; else waiting[j] = false;`

Q

Homework 1（Fall 2026）

HW1-Q2.1 AMAT + locality (6 pts) CH1

Hit ratio 90%, hit time 2 ns, miss time 50 ns.

Answer.
$0.9 \times 2 + 0.1 \times 50 =$ **6.8 ns**. Temporal: recently accessed items are likely accessed again (e.g., loop counter, loop body instructions). Spatial: items near recent accesses are likely accessed soon (e.g., sequential array elements a[i], a[i+1]).

HW1-Q2.2 Multiprogramming vs time sharing (6 pts) CH1

Answer.
Multiprogramming keeps several jobs in memory; when one waits for I/O, the CPU switches to another, so the CPU always has work → higher utilization. Time sharing switches among jobs so frequently (timer + CPU scheduling) that users can interact with each program while it runs → short response time, interactive computing.

HW1-Q2.3 State sequence (6 pts) CH3

Answer.
New → Ready → Running → Waiting → Ready → Running → Ready → Running → Terminated. When leaving Running, the PCB saves the process state, program counter, CPU registers, plus updated scheduling, accounting and I/O-status information.

HW1-Q2.4 read(): user → kernel → user (6 pts) CH2

Answer.
(1) read() in libc puts the system-call number and parameters in registers and executes a trap instruction. (2) The trap switches to kernel mode; the kernel indexes the system-call table and runs the read service. (3) The kernel stores the return value and executes return-from-trap, which switches back to user mode and resumes after the call.

HW1-Q2.5 Parent → child through a pipe (6 pts) CH3

Answer.
Call `pipe(fd)` **before** `fork()`; after fork, the parent (writer) closes `fd[0]` and writes to `fd[1]`; the child (reader) closes `fd[1]` and reads from `fd[0]`; close the used ends when done.

HW1-Q3.1 Values of x (15 pts) CH3

Answer.
C1: 3, C2: 5, P: 2 (each process has its own copy of x). Outputs: "C1: 3, C2: 5, P: 2" or "C2: 5, C1: 3, P: 2". Moving the first wait before `x *= 2` guarantees C1, C2, P order; values unchanged.

HW1-Q3.2 exec worker (15 pts) CH3

Answer.
3 processes (parent → child → grandchild; the grandchild execs worker). X is not printed (successful exec never returns). Outputs: A P W C D · A W P C D · A W C P D.

HW1-Q3.3 Pipe parent → child (10 pts) CH3

Answer.
(a) Parent closes fd[0], writes to fd[1]; child closes fd[1], reads fd[0]; parent → child. (b) "Child: 12" then "Parent: done" — read blocks until data arrives, and the parent waits for the child. (c) No — separate pipe() calls after fork create two unrelated pipes, with no shared descriptors.

Homework 2（Fall 2026）

HW2-Q2.1 Different PC/registers (6 pts) CH4

Answer.
Yes — each thread has its own PC, register set and stack; saved in its **TCB** when not running.

HW2-Q2.2 Many-to-one on 4 cores (6 pts) CH4

Answer.
No — all user threads map to one kernel thread, which runs on one core at a time; only concurrency, no parallelism (and one blocking call blocks all).

HW2-Q2.3 Ctrl+C (6 pts) CH4

Answer.
Asynchronous (generated by an event external to the process). Yes — install a user-defined handler with `signal(SIGINT, handler)` to override the default (terminate).

HW2-Q2.4 Hard affinity (6 pts) CH5

Answer.
No — hard affinity restricts T to CPU 0 and CPU 1; load balancing may move it between those two only.

HW2-Q2.5 RM vs EDF at t = 50 (6 pts) CH5

Answer.
No. RM picks A (shorter period, static) → B finishes at 85 and misses 80. EDF picks B (deadline 80 < 100) → B finishes at 60, A at 85; both meet deadlines.

HW2-Q3.1 Amdahl (10 pts) CH4

Answer.
(a) $\dfrac{1}{0.2 + 0.8/4} = \mathbf{2.5}$. (b) $N \to \infty$: $\dfrac{1}{0.2} = \mathbf{5}$.

HW2-Q3.2 SRTF and RR q=3 (20 pts) CH5

P1 (0, 7), P2 (1, 3), P3 (4, 5), P4 (10, 2).

Answer.
SRTF `0 P1 1 P2 4 P3 9 P1 10 P4 12 P1 17` → avg WT 2.5, avg TAT 6.75. RR(3) `0 P1 3 P2 6 P1 9 P3 12 P1 13 P4 15 P3 17` → avg WT 4.75, avg TAT 9.

HW2-Q3.3 MLFQ (10 pts) CH5

Q0 RR4, Q1 RR8, Q2 FCFS; P1 (0, 20), P2 (1, 5), P3 (11, 2); preempted Q1 job keeps the unused part of its quantum.

Answer.
`0 P1(Q0) 4 P2(Q0) 8 P1(Q1) 11 P3(Q0) 13 P1(Q1) 18 P2(Q1) 19 P1(Q2) 27`. At t = 11, P1 goes to the head of Q1 (ahead of P2) with 5 ms of its Q1 quantum left.

Fall 2022 Midterm（lab slides 引用）

F22-Q3 fork() == fork() in a loop (10 pts) CH3

```
int main() {
    int i, n;
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        if (fork() == fork()) {
            printf("fork() == fork()!\n");
            fflush(stdout);
        }
    }
    return 0;
}
```

(a) Number of processes and of printed lines for n = 1, 2, 3. (b) In terms of n.

Answer.
(a) n = 1: 4 processes, 1 line; n = 2: 16, 5; n = 3: 64, 21. (b) Each iteration every process does two forks → ×4, so 4^n **processes**. In each iteration only the grandchild that received 0 from both forks sees equal return values, so the iteration that starts with 4^k processes prints 4^k lines. Total lines $= 4^0 + 4^1 + \cdots + 4^{n-1} = \dfrac{4^n - 1}{3}$. ✓ (verified)

Q

Fall 2024 Midterm（lab slides 引用）

F24-P3.1 fork in nested loops (10 pts) CH3

```
for (i = 0; i < M; i++) {
    pid_t pid = fork();          // line X
    if (!pid) {
        for (j = 0; j < N; j++)  // line Y
            fork();
    }
}
```

(a) Which process executes line Y? (b) M = N = 1: how many processes? (c) In terms of M and N.

Answer.
(a) The **child** — `!pid` is true only where `fork()` returned 0. (b) **3** — the parent forks a child (2), the child forks once in the inner loop (3). (c) Let $f(k)$ be the number of processes one process becomes with k outer iterations left: the parent continues ($f(k-1)$) and the child becomes 2^N processes that all continue ($2^N f(k-1)$), so $f(k) = (2^N + 1)\, f(k-1)$ and the total is $(2^N + 1)^M$. ✓ (verified)

Spring 2025 Midterm（lab slides 引用）

S25-MC12 Nested short-circuit forks CH3

```
if (fork() || fork()) {
    if (fork() && !fork()) {
        fork();
    }
}
```

How many processes are there in total (suppose all fork() succeed)?

Answer.
9. `fork() || fork()` : 2 processes enter the if-clause, 1 the else-clause. Inside, `fork() && !fork()` : 1 process enters the inner if and forks once (→ 2), 2 processes skip it → each outer-if process becomes 4. Total $2 \times 4 + 1 = 9$ (8 newly created). ✓ (verified)

Spring 2023 Midterm（lab slides 引用）

S23-PRR Preemptive priority + RR with different arrivals (q = 2) CH5

Smaller number = higher priority; processes with equal priority share the CPU round-robin with q = 2; a higher-priority arrival preempts. Process table reconstructed from the official answer: P1 (0, 5, 2), P2 (1, 3, 1), P3 (2, 8, 2), P4 (8, 2, 1), P5 (9, 6, 2).

Answer.
`0 P1 1 P2 4 P1 5 P3 7 P1 8 P4 10 P1 11 P3 13 P5 15 P1 16 P3 18 P5 20 P3 22 P5 24` (official Gantt). A preempted process **keeps the remaining part of its quantum** — P1 runs only 1 unit at 4–5 and at 10–11. Resetting the quantum is the common mistake. Avg WT $= \dfrac{11 + 0 + 12 + 0 + 9}{5} = 6.4$.

Lab 2 Exercises

Lab2-Q1 Counting printed values CH3

```
pid_t pid = fork();  int cnt = 0;
if (pid == 0) {
    cnt += 10;  pid = fork();
    if (pid != 0) { cnt += 10;  pid = fork(); }
}
printf("%d \n", pid);  printf("%d \n", cnt);
```

How many non-zero PIDs, how many '0's and how many '10's are printed?

Answer.
4 processes print (pid, cnt): P0 (≠0, 0), C1 (≠0, 20), C2 (0, 10), C3 (0, 20) → **2** non-zero PIDs, **3** zeros, **1** ten. ✓ (verified)

Q

Lab2-Q2 Force the order process 3 → 2 → 1 CH3

```
int x = 0;
if (BLANK1) { x = x + 1; BLANK2; printf("The value x in process 1 is %d\n", x); }
else if (BLANK3) { x = x + 1; BLANK4; printf("The value x in process 2 is %d\n", x); }
else { x = x + 1; printf("The value x in process 3 is %d\n", x); }
```

Fill the blanks so the output is always process 3, then 2, then 1 (each printing x = 1).

Answer.
BLANK1 `fork()` · BLANK2 `wait(0)` · BLANK3 `fork()` · BLANK4 `wait(0)` (`fork() > 0` / `wait(NULL)` also fine). Each process waits for the child it created, so 3 prints before 2, and 2 before 1; every process has its own copy of x, so each prints 1.

Lab 4 Exam Example

Lab4-EX1 Two ordinary pipes for two-way communication CH3

Parent sends "Parent" to the child and the child sends "Child" to the parent using `p2c[2]` and `c2p[2]`. Fill BLANK1–BLANK6 (create pipes; child writes then reads; parent reads then writes).

Answer.
BLANK1 `pipe(p2c)` · BLANK2 `pipe(c2p)` · BLANK3 `write(c2p[1], msg, 6)` · BLANK4 `read(p2c[0], buffer, 7)` · BLANK5 `read(c2p[0], buffer, 6)` · BLANK6 `write(p2c[1], msg, 7)` (5 / 6 bytes also accepted). Output: "Parent received the message: Child" and "Child received the message: Parent".

Lab 6 Sample Questions（官方答案）

Lab6-Q1 Semaphores for strict thread order CH6

T1: `wait(S1); CS; signal(S2);` T2: `wait(S2); CS; signal(S3);` T3: `wait(S3); CS; signal(S1);` — initial values for the order T1, T2, T3, T1, ...?

Answer.
S1 = 1, S2 = 0, S3 = 0 (only T1 can start; each thread passes the turn to the next).

Lab6-Q2 P0 → P1 → P2 ordering blanks CH6

S0 = BLANK1, S1 = BLANK2, S2 = BLANK3; P0: `wait(S0); CS; BLANK4;` P1: `BLANK5; CS; BLANK6;` P2: `BLANK7; CS; signal(S0);`

Answer.
S0 = **1**, S1 = **0**, S2 = **0**; BLANK4 `signal(S1)`; BLANK5 `wait(S1)`; BLANK6 `signal(S2)`; BLANK7 `wait(S2)`.

Lab6-Q3 Mutex blanks CH6

`void *add(void *arg) { pthread_mutex_t* m = BLANK1; for (5000 times) { BLANK2; ++counter; BLANK3; } }` — two threads run `add` with `&mutex`. Missing code? Purpose? Final counter?

Answer.
BLANK1 `(pthread_mutex_t*) arg` · BLANK2 `pthread_mutex_lock(m)` · BLANK3 `pthread_mutex_unlock(m)`. Purpose: protect the shared variable `counter` (critical section). Final value **10,000**.

Lab6-Q4 Possible outputs with semaphores X, Y = 0 CH6

Process A: `wait(X); print "b"; signal(Y); wait(Y); print "c"; signal(Y);` Process B: `signal(X); print "a"; wait(Y); print "d";`

Answer.
If both complete: **abcd, bacd, bcad**. If B takes Y first, A blocks forever at its second `wait(Y)` → **bad, abd** (A never finishes). ✓ (verified by exhaustive search)

Q

Lab6-Q5 Counting semaphore permits CH6

S0 = 1, S1 = S2 = S3 = 0. P0: `while(true){ wait(S0); print '1'; release(S1); release(S2); release(S3); }` P1: `wait(S1); release(S0);` P2: `wait(S2); release(S0); wait(S3); release(S0);` — how many '1's?

Answer.

4 — S0 permits: initial 1 + P1 gives 1 + P2 gives 2. After the first print P0 releases S1, S2, S3, so P1 and P2 can produce all of them; then P0 blocks forever. ✓ (verified)

Lab6-Q6 Readers–writers with two mutexes CH6

Fill the reader's blanks (mutex protects read_count; rw_mutex protects the data).

Answer.

Entry: `pthread_mutex_lock(&mutex); read_count++; if (read_count == 1) pthread_mutex_lock(&rw_mutex); pthread_mutex_unlock(&mutex);` Exit: `pthread_mutex_lock(&mutex); read_count--; if (read_count == 0) pthread_mutex_unlock(&rw_mutex); pthread_mutex_unlock(&mutex);` The first reader locks writers out and the last reader lets them in.

Lab6-Q7 Race condition values CH6

a = 1, b = 3, c = 0. Thread 1: `a = b + 1; c = a - b;` Thread 2: `a = 13; b = 10; c = 2;` All possible final values of c?

Answer.

−6, 1, 2, 3, 10 — 2 if T2's `c = 2` is last; 1 (T1 reads a=4, b=3 then T2 runs; or T2 first then a = 11, c = 11 − 10); −6 (T1 reads a=4, then T2 sets b=10); 3 (T2 runs after T1's `a = 4`, T1 reads a=13, b=10); 10 (T1 reads a=13 and b=3 between T2's writes). ✓ (verified by exhaustive search)

Q

歷屆 MC（S26 / F20 / Sp21 / HW1 / HW2 / Lab 4）

S26-MC1 Interrupts → C

Which of the following statements about interrupts is **false**?

A. Interrupts may be generated by hardware devices **B.** Interrupts may be generated by software exceptions or traps **C. All interrupts are serviced in exactly the same way, with no difference in priority** **D.** Interrupts are handled by the kernel

interrupt 有 **priority levels**，high-priority 可以 preempt low-priority interrupt [3.24]。

S26-MC2 DMA → B

Which of the following statements is **true** about DMA?

A. DMA is used to move data for slow devices, while CPU is used for fast devices **B. DMA allows data transfer between device and memory with less CPU involvement** **C.** DMA eliminates interrupts completely **D.** CPU and DMA can't execute concurrently

DMA 用於 **fast devices / large data**（A 相反）；完成後仍然 **interrupt CPU**（C 錯）；CPU 同 DMA 可以 concurrent（D 錯）[3.25]。

S26-MC3 Storage hierarchy → D

Which of the following statements is true about storage hierarchy?

A. Registers are slower than main memory **B.** Main memory is nonvolatile **C.** Hard-disk drives are smaller than volatile memory **D. Main memory is byte addressable**

each address refers to one byte [1.30]。Registers 最快；main memory volatile；HDD 大過 memory。

S26-MC4 von Neumann → B

Which of the following is a core characteristic of the von Neumann architecture?

A. Instructions and data are stored in separate physical memories **B. Instructions and data are stored in the same memory space, accessed by address** **C.** The CPU does not contain an ALU; all computations are done by an external co-processor **D.** The Program Counter (PC) is stored in external mass storage devices

stored-program architecture [1.26]。A 係 Harvard architecture。

S26-MC5 OS services → C

Which of the following is **false** about services of operating systems?

A. User interface can be CLI or GUI **B.** The logging service of OS should keep track of computer resources that each user uses **C. Errors only occur in memory and I/O devices** **D.** Resource allocation service should allocate resources like CPU cycles, memory, file storage

errors 可以喺 **CPU and memory, I/O devices, user programs** [2.5]。

S26-MC6 API → A

Which of the following is an advantage of using APIs instead of directly invoking system calls?

A. APIs provide a more convenient programming interface **B.** APIs always execute faster than system calls **C.** APIs can bypass kernel protection **D.** APIs eliminate the need for linkers and loaders

API 好處 = **portability + hide complex details（convenient）** [2.17]。

S26-MC7 Microkernel → B（C 亦接受）

Which is **false** about microkernel system structure?

A. It is easier to extend a microkernel operating system **B. It typically has better performance than a monolithic kernel** **C. It runs memory management in user space** **D.** It is easier to port a microkernel operating system to new architectures

microkernel **performance suffers**（message passing overhead）[2.48]；microkernel 保留 minimal memory management 喺 kernel，所以 C 都算錯。

S26-MC8 printf vs write → B

What's the relationship between printf() and write()?

A. Both printf() and write() are system calls provided directly by the kernel **B. printf() invokes write() system call to get service from operating system** **C.** The program enters kernel mode immediately when printf() is called **D.** write() system call invokes printf() to perform the print function

printf() 係 C library function，佢 call write() system call [2.26]。

S26-MC9 Layered design → B

Which of the following statements about a layered operating-system design is correct?

A. Any layer may directly access any other layer **B. The interfaces between adjacent layers are usually well defined** **C.** Layered design eliminates all performance overhead **D.** Pure layered design is the dominant structure in all modern operating systems

每層只用下層服務、interface well defined（information hiding）[2.44–2.45]。A 係 LKM 特色；layered 有 overhead；好少 pure layered。

S26-MC10 fork count → C

```
int main() {
   int i;
   for (i = 0; i < 3; i++) {
      if (fork() == 0) {
         fork();
         printf("X ");
         return 1;
      }
   }
   printf("Y ");
   return 0;
}
```

What is the total number of processes (including the original), the number of "X " printed, and the number of "Y " printed?

A. Total processes = 4, "X " = 3, "Y " = 1 **B.** Total processes = 4, "X " = 1, "Y " = 3 **C. Total processes = 7, "X " = 6, "Y " = 1** **D.** Total processes = 7, "X " = 1, "Y " = 6

只有 original 繼續 loop；每輪生 1 個 child，child 再 fork → 2 個印 X 然後 return。3 輪 × 2 = **6 X**；original 印 **1 Y**；total = 1 + 6 = **7**（已用 C 程式驗證）。

S26-MC11 Context switch → B

Which statement is true about context switch?

A. Context switch means switching from user mode to kernel mode **B. During context switch, OS saves the state of one process/thread and restores another** **C.** Context switch requires the same hardware on all CPU architectures **D.** Context switch is shorter if the OS is more complex

[3.20]。A 係 mode switch；越複雜 OS/PCB → context switch **越長**。

S26-MC12 Ordinary pipes → B

Which statement about ordinary pipes is **false**?

A. They can be accessed like file descriptors **B. They can be accessed by arbitrary processes** **C.** They disappear after the communicating processes terminate **D.** They are typically used between parent and child processes

ordinary pipe cannot be accessed from outside the process that creates it（只限 parent–child）[3.65–3.66]。

S26-MC13 Zombie → B

Which statement about zombie processes is correct?

A. A zombie process is still executing user code normally **B. A zombie process has terminated but its parent has not yet collected its status** **C.** A zombie process is created when exec() fails **D.** A zombie process owns no process identifier

[3.49] terminated 但 parent 未 call wait()；**PID、process-table entry、PCB 仍然存在**。

S26-MC14 Leaving running state → A

Which event typically causes a process to leave the running state?

A. It issues an I/O request **B.** It calls a function **C.** It increases a local variable **D.** It reads from cache

I/O request → running → **waiting** [3.9]。

S26-MC15 Multithreaded process → C

Which of the following is true about the multithreaded process?

A. Each thread has its own address space and file descriptors **B.** Threads can't run concurrently on multiple cores if they belong to the same process **C. Thread creation is usually faster than process creation** **D.** It can't take advantage of processing capabilities on multicore systems

Economy — thread creation cheaper [4.4, 4.7]；threads share address space & files。

S26-MC16 Concurrency vs parallelism → C

Which statement is correct about concurrency and parallelism in multicore systems?

A. Concurrency means a system can perform more than one task at exactly the same time **B.** Parallelism means tasks only make progress over time on a single processor core **C. Concurrency can be supported on a single processor core through time multiplexing by the scheduler** **D.** Testing and debugging become easier in multicore systems because execution paths are fixed

[4.9]。A 係 parallelism 定義；debugging 變**難**。

S26-MC17 Amdahl's Law → C

Which combination gives a speedup between 1.6 and 1.7?

A. parallel 81%, 3 cores **B.** parallel 81%, 5 cores **C. parallel 61%, 3 cores** **D.** parallel 61%, 5 cores

$\text{speedup} = \frac{1}{S + (1-S)/N}$。A：$\frac{1}{0.19 + 0.27} = 2.17$；B：2.84；**C：** $\frac{1}{0.39 + 0.2033} = 1.69$；D：1.95。

S26-MC18 Signals → C

Which statement about signal handling is **false**?

A. Signals can be received synchronously or asynchronously **B.** Users can define custom signal handlers for certain signals **C. All signals should be delivered to all threads in a process** **D.** Different signals can have different handlers in the same process

synchronous signals 只 deliver 畀 **causing thread**；只有部分 asynchronous（e.g. Ctrl-C terminate）先畀所有 threads [4.30–4.31]。

S26-MC19 Scheduling criteria → C

Which statement about scheduling criteria is correct?

A. Waiting time is the amount of time from process arrival until the process finishes execution **B.** Turnaround time is the amount of time a process spends waiting in the ready queue **C. Response time is the amount of time from when a request is submitted until the first response is produced** **D.** Throughput is the fraction of time the CPU is busy

A 係 turnaround；B 係 waiting；D 係 CPU utilization [5.8, 5.12]。

S26-MC20 MLFQ → B

Which statement about MLFQ is correct?

A. Processes never move between queues **B. Each queue can use a different scheduling algorithm** **C.** It requires burst times to be known exactly in advance **D.** It is a non-preemptive algorithm

[5.30–5.31]。MLFQ 可以搬 queue、唔需要知 burst、preemptive。

S26-MC21 EDF → B

Which scheduling algorithm assigns priorities according to deadlines?

A. RM **B. EDF** **C.** FCFS **D.** RR

EDF = dynamic priority by deadline；RM = static by period [5.51]。

S26-MC22 Multiprocessor scheduling → C

Which statement about multiprocessor scheduling is **false**?

A. For asymmetric multiprocessing, the master server can become a potential bottleneck **B.** In a multithreaded system, another thread may run when one thread stalls while waiting for memory **C. The motivation of placing multiple processor cores on the same physical chip is to simplify the scheduler** **D.** Processor affinity tries to keep a process on the same CPU to improve cache performance

multicore 係為咗 **faster + less power**，而且令 scheduling **更複雜** [5.40]。

S26-MC23 Mutex + semaphore output → A

```
int counter = 0; pthread_mutex_t mtx; sem_t sem;   /*
sem_init(&sem, 0, 0) */
void *worker1(void *arg) { pthread_mutex_lock(&mtx); counter
= counter + 2;
    printf("A %d;", counter); pthread_mutex_unlock(&mtx);
sem_post(&sem); return NULL; }
void *worker2(void *arg) { sem_wait(&sem);
pthread_mutex_lock(&mtx); counter = counter + 3;
    printf("B %d;", counter); pthread_mutex_unlock(&mtx);
return NULL; }
/* main creates t1 (worker1) and t2 (worker2), then joins
both */
```

What is the output?

A. A 2; B 5; **B.** B 3; A 5; **C.** A 3; B 5; **D.** B 2; A 4;

sem 初始 0 → worker2 一定等 worker1 `sem_post` → 先 A（0+2=2）後 B（2+3=5）。

S26-MC24 Hardware primitive → B

Which hardware primitive is commonly used to build synchronization tools?

A. exec() **B. test_and_set()** **C.** fork() **D.** open()

atomic hardware instructions：**test_and_set()**、**compare_and_swap()** [6.14]。

S26-MC25 Bounded-waiting TAS → A

In the bounded-waiting mutual exclusion algorithm using test_and_set() (waiting[], key, lock; exit section scans j = (i+1)%n ... and either sets lock = false or waiting[j] = false), which statement is **not correct**?

A. test_and_set(&lock) releases the lock after the critical section

B. waiting[i] is used to indicate that process i wants to enter the critical section **C.** waiting[j] = false indicates that process i releases the lock (hands it to j) **D.** Process i can re-enter the critical section after executing lock = false

test_and_set 只會 **set lock = true（acquire）**；release 係 `lock = false` 或者 `waiting[j] = false` [6.19]。

F20-MC1 Locality → B

Locality is essential to ___

A. Memory **B. Cache** **C.** Hard disks **D.** Non-volatile memory (NVM)

caching works because of **temporal/spatial locality** [1.33]。

F20-MC2 Virtualization → D

___ allows a guest OS to run as an application on a native OS.

A. Operating system **B.** Emulation **C.** User interface **D. Virtualization**

[1.43] "allows an OS to run as an application within another OS"。

F20-MC3 Interrupt → C

Which of the following statements about interrupts is **NOT** true?

A. Interrupts enable hardware to interact with the operating system **B.** Interrupt alerts the CPU that some event requires attention **C. Interrupt allows software to access hardware including the CPU** **D.** Interrupt can be caused by both hardware and software

C 講緊 **system call** 嘅功能。

F20-MC4 API → C

Programmers interface with ___ to obtain services from the OS.

A. Shell **B.** System call interfaces **C. APIs (application programming interfaces)** **D.** System calls

programmers 用 **API**（best answer）[2.15]。

F20-MC5 Parameter passing → D

How can parameters be passed to the OS in system calls?

A. Registers **B.** Stack **C.** Block or table **D. All of the above**

三種方法 [2.20]。

F20-MC6 LKM → B

___ refers to the OS design that maintains a set of core components in the kernel and adds additional services via modules during run time.

A. Layered approach **B. Loadable kernel modules (LKMs)** **C.** Microkernel **D.** Monolithic

[2.49]。

M

F20-MC7 fork → A

Which of the following statements on fork() is **NOT** true?

A. The parent process waits until the completion of the child process **B.** The child process duplicates the entire address space of the parent process **C.** The parent and child processes run the same program **D.** The parent and child processes run concurrently in the system

等 child 係 **wait()** 嘅工作，fork 本身唔會等。

F20-MC8 Address space → D

Which of the following is contained in the address space of a process?

A. Stack **B.** Code **C.** Data **D. All of the above**

text (code)、data、heap、stack [3.5]。

F20-MC9 Named pipes → C

___ enables communication for multiple processes with no parent-child relationship, independent from network.

A. RPCs **B.** Ordinary pipes **C. Named pipes** **D.** Socket

named pipes 唔需要 parent-child、多 processes 用 [3.69]；socket/RPC 要 network。

F20-MC10 Medium-term scheduler → D

___ swaps a process out of memory and temporarily stores it on disk.

A. The long-term scheduler **B.** A device scheduler **C.** The CPU scheduler **D. The medium-term scheduler**

swapping [3.18]。

F20-MC11 Not shared by threads → B

Which item below is **NOT** shared by threads of the same process?

A. Code **B. Stack** **C.** Files **D.** Data

每個 thread 有自己 **stack, registers, PC** [4.6]。

F20-MC12 Thread library → A

___ provides the programmer with an API for creating and managing threads.

A. Thread library (such as POSIX Pthreads) **B.** Signals **C.** System calls **D.** Inter-process communications

[4.20]。

F20-MC13 Multithreading models → C

Which of the following statements is **NOT** true?

A. Kernel threads are scheduled by the OS **B.** Programmers or users manage user threads **C. A user thread can be mapped to more than one kernel thread** **D.** One user thread can be mapped to at most one kernel thread

冇 one-to-many mapping；many-to-many 係 many user → **smaller or equal** kernel threads。

F20-MC14 Minimum average waiting time → C（D 亦接受）

Given a set of processes, which algorithm produces the minimum average waiting time?

A. FCFS **B.** RR **C. SJF** **D. SRTF**

SJF optimal among non-preemptive；SRTF optimal among preemptive [5.17, 5.24]。

F20-MC15 Hard to implement → A

Which of the following scheduling algorithms is difficult to implement in practice?

A. SJF **B.** RR **C.** MLFQ **D.** FCFS

好難知道 **next CPU burst length** [5.17]。

Sp21-MC1 von Neumann → A

In a von Neumann architecture, a processing unit contains a/an ___ and processor registers.

A. arithmetic logic unit (ALU) **B.** instruction register (IR) **C.** program counter (PC) **D.** all of the mentioned

[1.26] "a CPU that contains an ALU and processor registers"；IR 屬 control unit。

Sp21-MC2 Interrupt trigger → D

Which of the following events will directly trigger an interrupt?

A. kernel code execution **B.** I/O job execution **C.** user code execution **D. I/O job completion**

device controller 完成操作 → **interrupt** CPU [3.21]。

Sp21-MC3 Dual mode → C

The two separate modes of operation in an operating system are ___.

A. supervisor mode and system mode **B.** physical mode and logical mode **C. user mode and kernel mode** **D.** kernel mode and privileged mode

[3.27]。

Sp21-MC4 System calls → C

___ provide an interface to the services provided by an operating system.

A. Inter-process communications **B.** Remote procedure calls **C. System calls** **D.** User interfaces

[2.14]。

Sp21-MC5 Original UNIX → C

Which operating system structure is adopted by the original UNIX?

A. layered structure **B.** microkernel **C. monolithic structure** **D.** modular approach

[2.40–2.41]。

Sp21-MC6 Degree of multiprogramming → B

The ___ refers to the number of processes in the memory.

A. program counter **B. degree of multi-programming** **C.** long-term scheduler **D.** CPU scheduler

[3.14]。

Sp21-MC7 Amdahl → B

Using Amdahl's Law, what is the speedup for an application that is 30% serial on 7 processing cores?

A. 2 **B. 2.5** **C.** 3 **D.** 3.5

$\frac{1}{0.3 + 0.7/7} = \frac{1}{0.4} = \mathbf{2.5}$。

Sp21-MC8 Process owns → D

In an operating system, each process has its own ___.

A. address space **B.** local/global variables and open files **C.** pending alarms, signals and signal handlers **D. all of the above**

process = address space + resources。

Sp21-MC9 Stack → D

Which section of a process contains temporary data like function parameters, return addresses and local variables?

A. text section **B.** data section **C.** heap **D. stack**

[3.5]。

Sp21-MC10 Short-term scheduler → A

Which of the following selects from among the processes in the ready queue to execute on the CPU?

A. Short-term scheduler **B.** Medium-term scheduler **C.** Long-term scheduler **D.** Context switcher

= CPU scheduler [3.17]。

Sp21-MC11 fork incorrect → B

During process creation with fork(), which of the following is **incorrect**?

A. The child process can run concurrently with the parent **B. The child process can call wait() in order to finish earlier than the parent** **C.** The child process can call exec() to load a new program into it **D.** The child can be an exact duplicate of the parent

wait() 係等 **child** 完成，唔係令自己早完。

Sp21-MC12 Pipes → C

Which of the following statements about pipes is **not** true?

A. Named pipes allow bi-directional communication **B.** Only the parent and child processes can use ordinary pipes for communication **C. Named pipes cease to exist after the communicating processes have finished** **D.** Reading and writing to ordinary pipes on both UNIX and Windows can be performed like ordinary file I/O

named pipes **continue to exist until explicitly deleted** [3.69]。

Sp21-MC13 Leave the CPU → D

Which of the following events might force a process to leave the CPU?

A. make a fork system call **B.** make an I/O request **C.** interrupt **D. all of the above**

system call / I/O / interrupt 都可以令 process 離開 CPU。

M

Sp21-MC14 fork with || → C

```
if (fork() || fork()) {
   fork();
}
```

How many processes are created in total (including the original)?
A. 3 **B.** 4 **C. 5** **D.** 6
P0（true，跳過第二個 fork）→ fork → 2；C1（false）→ fork2：C1 true → fork → 2；C2 false → 1。**2 + 2 + 1 = 5** ✓

Sp21-MC15 fork in loop → C

```
for (int i = 0; i < 3; ++i) {
   if (fork() == 0) fork();
}
```

How many processes in total?
A. 3 **B.** 8 **C. 27** **D.** 64
每輪每個 process 變 3 個 → $3^3 = 27$ ✓

Sp21-MC16 TCB → D
Which of the following is **not** in the Thread Control Block (TCB)?
A. Execution state **B.** Scheduling information **C.** Pointer to enclosing process **D. Process identifier**
TCB 有 execution state、scheduling info、accounting、pointers、**pointer to enclosing PCB** [4.15]；PID 喺 PCB。

Sp21-MC17 Concurrency → B
Which refers to the capability to allow multiple tasks to make progress on a single processor?
A. parallelism **B. concurrency** **C.** data parallelism **D.** task parallelism
[4.9]。

Sp21-MC18 Multithreading models → C
Which of the following is **not** true about multithreading models?
A. Many-to-one means many user-level threads are mapped to a single kernel thread **B.** One-to-one model provides more concurrency than many-to-one model **C. The many-to-many model is easy to implement in practice** **D.** Most operating systems now use the one-to-one model
many-to-many **difficult to implement** [4.26]。

Sp21-MC19 Thread-local storage → D
Thread-local storage is data that ___.
A. is not associated with any process **B.** has been modified by the thread, but not yet updated to the parent process **C.** is generated by the thread independent of the thread's process **D. is unique to each thread**
TLS = **unique to each thread**（Windows TEB 入面有 TLS [4.36]）。

Sp21-MC20 When scheduling occurs → D
CPU scheduling decisions may take place when ___.
A. a process switches from running to waiting state **B.** a process terminates **C.** a process switches from running to ready state **D. all of the above**
四種情況 [5.6]。

Sp21-MC21 Priority problem → B
The main problem with a priority scheduling algorithm is ___.
A. how to set the priority of a process **B. some processes may never execute** **C.** how to determine the length of the next CPU burst **D.** how to determine the length of the time quantum
starvation；solution = aging [5.25]。

Sp21-MC22 RR large quantum → B
For RR scheduling, when the time quantum q gets too large, it degenerates to ___.
A. SJF **B. FCFS** **C.** Shortest-remaining-time-first **D.** Multilevel queue
[5.14, 5.16]。

Sp21-MC23 Must be non-preemptive → A
Which of the following scheduling algorithms must be non-preemptive?
A. FCFS **B.** SJF **C.** Priority scheduling **D.** Multi-level feedback queue
FCFS 一定 non-preemptive；SJF / priority 有 preemptive 版本；MLFQ preemptive [5.11]。

Sp21-MC24 MLFQ true statement → C
In the multilevel feedback queue algorithm, which statement is true?
A. Prior knowledge on the next CPU burst time is required for scheduling **B.** Classification of the ready queue is permanent **C. It handles interactive jobs well by delivering similar performance as SJF** **D.** It won't suffer from starvation problem
[5.35]。B 係 multilevel queue（冇 feedback）；MLFQ 都可能 starvation（要 reshuffle/aging）。

Sp21-MC25 Real-time → C
Which statement about real-time CPU scheduling is true?
A. Real-time scheduling doesn't demand performance guarantee **B.** In hard real-time systems, a task can be serviced after its deadline **C. Soft real-time systems guarantee only that the process will be prioritized over non-critical processes** **D.** The scheduler for a real-time system must support a priority-based algorithm without preemption
[5.47]。RT scheduler 要 **priority-based with preemption**。

Sp21-MC26 Starvation-free → A
Which algorithms are free from starvation? 1. FCFS 2. Non-preemptive SJF 3. Preemptive SJF 4. Round Robin 5. Multi-level Queue
A. 1 4 **B.** 1 2 4 **C.** 1 3 4 **D.** 1 2 4 5
SJF/SRTF 長 job 可能 starve；multilevel queue（fixed priority）低 queue 可能 starve；**FCFS、RR 唔會**。

Sp21-MC27 Mutual exclusion → A
Mutual exclusion implies that ___.
A. if a process is executing in its critical section, then no other process can be executing in their critical sections **B.** if a process is executing in its critical section, then only one of the other processes can be executing in its critical section **C.** if a process is executing in its critical section, then all the resources of the system must be blocked until it finishes execution **D.** none of the above
[6.11]。

Sp21-MC28 CS requirements → C
A solution to the critical section problem does **not** have to satisfy which requirement?
A. mutual exclusion **B.** progress **C. synchronization** **D.** bounded waiting
三個 requirements = ME + progress + bounded waiting。

Sp21-MC29 Race condition → B
A situation where several processes access and manipulate the same data concurrently and the outcome depends on the particular order of access is called ___.
A. data consistency **B. race condition** **C.** aging **D.** starvation
[6.9]。

Sp21-MC30 Entry section → C
Suppose the binary variable lock is initialized to 0. Which can be an implementation of the entry section to solve the critical-section problem?
A. while (compare_and_swap(&lock, 0, 1) == 0) ; **B.** while (compare_and_swap(&lock, 1, 0) != 0) ; **C. while (test_and_set(&lock)) ;** **D.** while (!test_and_set(&lock)) ;
TAS 返回 old value；lock = 0 時第一個 process 得 0 → 跳出 loop 入 CS [6.16]。A 嘅條件相反（應該 `!= 0`）。

HW1-MC1 DMA → C
Which statement about I/O operations and DMA is **NOT** true?
A. In programmed I/O, the CPU executes special I/O instructions to move data between a device controller and memory **B.** A DMA controller can transfer a block of data between an I/O device and memory without continuous CPU involvement **C. During a DMA transfer, the DMA controller sends an interrupt to the CPU after every byte is transferred** **D.** After a DMA block transfer completes, the DMA controller can interrupt the CPU
DMA **block 完成先 interrupt 一次** [3.25–3.26]。

HW1-MC2 Interrupts and traps → B
Which statement about interrupts and traps is **NOT** true?
A. Hardware interrupts are generally asynchronous to the instruction currently being executed **B. Interrupts can only be generated by I/O devices** **C.** A trap or exception may be caused by an error or by a user request such as a system call **D.** Interrupts may have different priority levels
trap/exception = **software-generated interrupt** [3.24]。

M

HW1-MC3 Resource allocator → C

Several jobs are ready to run on a shared computer. The OS decides which job receives CPU time and how memory is allocated. Which role does this illustrate?

A. The OS provides a user interface **B.** The OS provides a communication service **C. The OS acts as a resource allocator** **D.** The OS acts as an application program

[1.23] ◦

HW1-MC4 NUMA → D

Which statement about multiprocessor, multicore, and NUMA systems is **NOT** true?

A. In SMP, processors have their own registers and local caches but share physical memory **B.** A multicore system places multiple computing cores on a single physical chip **C.** In NUMA, a CPU can access its local memory faster than remote memory connected to another CPU group **D. In NUMA, every CPU accesses every part of memory with exactly the same latency**

NUMA = **non-uniform** latency [1.40] ◦

HW1-MC5 OS structures → B

Which statement about operating-system structures is **true**?

A. A monolithic kernel moves most OS services into user-space processes that communicate by message passing **B. A microkernel keeps only essential functions in the kernel and provides other services outside the kernel through communication mechanisms such as message passing** **C.** In a layered OS, each layer may directly call any other layer without restriction **D.** A loadable kernel module always requires the entire kernel to be recompiled and the machine to be rebooted

[2.46–2.47] ◦ A 描述 microkernel 唔係 monolithic；C 錯；LKM 唔使 recompile/reboot ◦

HW1-MC6 read() path → C

A C program calls read() to obtain data from a file. Which sequence best describes how the request reaches the OS?

A. The application directly executes the device's privileged I/O instructions while remaining in user mode **B.** The read() API completes the entire file operation in user mode without invoking the kernel **C. The API and run-time environment use the system-call interface to invoke the corresponding kernel service, and control later returns to the application** **D.** The OS creates a new process to execute every read() request

[2.18–2.19] ◦

HW1-MC7 Privileged instruction → A

What normally happens if a user program attempts to execute a privileged instruction in user mode?

A. The hardware treats the instruction as illegal and traps to the operating system **B.** The CPU automatically changes the instruction into a non-privileged instruction **C.** The user program changes the mode bit by itself and continues in kernel mode **D.** The instruction is always executed, but its result is discarded

[3.28] ◦

HW1-MC8 Virtualization → B

Which statement about virtualization is **NOT** true?

A. A virtual machine manager creates and manages virtual machines **B. An application always runs faster inside a virtual machine than directly on the physical machine** **C.** Each guest OS runs in its own VM and behaves as if it has access to hardware **D.** Virtualization allows multiple operating systems to run on one physical machine

VM 只會 **minor performance decrease** [1.46] ◦

HW1-MC9 fork → C

Which statement about fork() is **NOT** true?

A. After a successful fork(), the child receives 0 and the parent receives the child's PID **B.** The parent and child both continue execution at the instruction following fork() **C. If the child changes a global variable after fork(), the parent's copy changes automatically** **D.** The execution order of parent and child depends on scheduling unless synchronization is used

各自有 **own copy** ◦

HW1-MC10 Pipes → D

Which statement about pipes is **NOT** true?

A. An ordinary pipe is typically used between processes with a parent-child relationship **B.** A named pipe can be used by processes that do not have a parent-child relationship **C.** An ordinary pipe is typically unidirectional **D. A named pipe ceases to exist as soon as the communicating processes terminate**

named pipe 要 **explicitly deleted** ◦

HW2-MC1 Responsiveness → B

An interactive application continues responding to the user while one of its threads is blocked. Which benefit does this illustrate?

A. Economy **B. Responsiveness** **C.** Resource sharing **D.** Scalability

[4.7] ◦

HW2-MC2 Data parallelism → D

Which arrangement is an example of data parallelism?

A. Threads on different cores each perform a different operation **B.** One thread processes all elements of an array sequentially **C.** One thread handles user input while another thread performs spelling checks **D. Threads on different cores perform the same operation on different subsets of an array**

A、C = task parallelism；B = serial [4.11] ◦

HW2-MC3 Signal classification → A

Which statement correctly classifies signals?

A. Division by zero can generate a synchronous signal, whereas timer expiration can generate an asynchronous signal **B.** Division by zero can generate an asynchronous signal, whereas timer expiration can generate a synchronous signal **C.** Both generate synchronous signals **D.** Both generate asynchronous signals

[4.29] ◦

HW2-MC4 Deferred cancellation → C

Which statement describes deferred thread cancellation?

A. The target thread is terminated immediately when cancellation is requested **B.** Requesting cancellation terminates every thread in its process **C. The target thread periodically checks whether it should terminate, allowing it to terminate in an orderly manner** **D.** The target thread can be canceled only after it has finished its normal execution

A 係 asynchronous cancellation [4.32] ◦

HW2-MC5 Preemptive scheduling → C

Which event is an example of preemptive CPU scheduling?

A. A running process completes its execution and terminates **B.** A running process requests I/O and enters the waiting state **C. A time quantum expires, and the running process is moved back to the ready queue** **D.** A running process calls wait() and enters the waiting state

running → ready（involuntary）= preemptive [5.6] ◦

HW2-MC6 Convoy effect → B

Which situation illustrates the convoy effect in FCFS?

A. A waiting process gradually gains a higher priority **B. Several short processes wait behind a long process that arrived first** **C.** A process moves to a lower-priority queue after using its time quantum **D.** A running process is preempted when its RR quantum expires

[5.10] ◦ A = aging；C = MLFQ demotion ◦

HW2-MC7 Aging → A

What is the purpose of aging in priority scheduling?

A. To address starvation by gradually increasing the priority of a waiting process **B.** To keep each process permanently in the same priority queue **C.** To predict the length of a process's next CPU burst **D.** To give higher priority to periodic tasks with shorter periods

[5.25] ◦ D = RM ◦

HW2-MC8 PCS vs SCS → D

In a many-to-many model, a thread library selects a user thread to run on an available LWP. Which scheduling scope applies?

A. SCS, because several user threads are competing **B.** SCS, because the library schedules kernel threads onto CPUs **C.** PCS, because all kernel threads in the system are competing **D. PCS, because the competing threads belong to the same process**

[5.37] ◦

M

HW2-MC9 Exponential averaging → C

The predicted CPU burst is 10 ms, the actual burst is 6 ms, $\alpha = 0.25$. What is the next prediction?

A. 6 ms **B.** 7 ms **C. 9 ms** **D.** 10 ms

$\tau_{n+1} = 0.25 \times 6 + 0.75 \times 10 = \mathbf{9}$ ◦

HW2-MC10 Little's formula → B

3 processes arrive per second on average; average queue length is 12. Average waiting time?

A. 0.25 s **B. 4 s** **C.** 9 s **D.** 36 s

$W = n/\lambda = 12/3 = \mathbf{4\ s}$ [5.57] ◦

Lab4-MC Threads share memory, processes do not → B

```
void *child(void *arg) { int *counter = (int *) arg;
(*counter)++; return NULL; }
int main() {
    int counter = 0;  pthread_t thread;
    pthread_create(&thread, NULL, child, (void *)&counter);
    pthread_join(thread, NULL);
    printf("%d\n", counter);
    counter = 0;
    pid_t pid = fork();
    if (pid) wait(NULL);
    else { child((void *)&counter); exit(0); }
    printf("%d\n", counter);
}
```

What is the output?

A. 0 then 0 **B. 1 then 0** **C.** 0 then 1 **D.** 1 then 1

thread 同 main 共用 `counter` → 印 1；fork 出嚟嘅 child 改嘅係自己嗰份 copy → parent 印 0 ◦ **Threads share memory space, processes have separate memory spaces.**（Lab 4）

新練習 MC（針對 Fall 2026 slides）

P-ch1-1 Storage managed by → A

According to the storage characteristics table, which level of storage is managed by the **compiler**?

A. Registers **B.** Cache **C.** Main memory **D.** Solid-state disk

registers – compiler；cache – hardware；main memory / SSD / disk – OS [1.34] ◦

P-ch1-2 Bootstrap → C

Where is the bootstrap program (the first program run at power-on) typically stored?

A. DRAM **B.** Cache **C. EEPROM** **D.** Hard disk

[1.30] bootstrap program stored on EEPROM（firmware）◦

P-ch1-3 Multiprocessor speedup → B

Why is the speed-up ratio with N processors less than N?

A. Each processor runs a different instruction set **B. Overhead, e.g., contention for shared resources such as the bus or memory** **C.** Only one processor can execute user processes **D.** Processors must share a single register set

[1.37] ◦

P-ch1-4 Multiprocessor advantages → D

Which is **NOT** an advantage of multiprocessor systems listed in the lecture?

A. Increased throughput **B.** Economy of scale **C.** Increased reliability (graceful degradation) **D. Simpler parallel programming**

multiprocessor 令 parallel programming 更複雜 [1.37, summary] ◦

P-ch1-5 NUMA drawback → B

What is the potential drawback of a NUMA system?

A. All CPUs must share a single slow bus **B. Increased latency when a CPU must access remote memory across the system interconnect** **C.** CPUs cannot share one physical address space **D.** It consumes more power than multicore chips

[1.40] ◦

P-ch1-6 Cloud models → B

A database server provided as a ready-to-use software stack over the Internet is an example of:

A. SaaS **B. PaaS** **C.** IaaS **D.** MaaS

SaaS = applications；**PaaS = software stack (e.g. database server)**；IaaS = servers/storage [1.48] ◦

P-ch1-7 VMM → B

Which statement about virtualization components is correct?

A. The guest OS creates and manages the virtual machine manager **B. The VMM (hypervisor) creates and manages VMs, providing an interface identical to the host** **C.** The host is the virtual copy given to each guest **D.** Programs in a VM usually run several times faster than natively

[1.44] ◦

P-ch1-8 Timesharing → B

Which statement is correct?

A. Multiprogramming is an extension of timesharing **B. Timesharing switches jobs so frequently that users can interact with each job while it runs** **C.** Timesharing requires that only one program be in memory **D.** Multiprogramming is not used in modern systems

timesharing = logical extension of multiprogramming [1.42] ◦

P-ch1-9 Mechanical vs electrical storage → B

Compared with mechanical storage (e.g., HDD), electrical storage (NVM) is generally:

A. Larger and cheaper per byte **B. More costly, smaller, more reliable and faster** **C.** Slower but more reliable **D.** Volatile

[1.31] ◦

P-ch1-10 AMAT → C

A cache has hit ratio 95%, hit time 1 ns and miss time 100 ns. The average access time is:

A. 1.0 ns **B.** 5.0 ns **C. 5.95 ns** **D.** 100 ns

$0.95 \times 1 + 0.05 \times 100 = 0.95 + 5 = \mathbf{5.95\ ns}$ ◦

P-ch2-1 ioctl → C

In UNIX, `ioctl()` belongs to which category of system calls?

A. Process control **B.** File management **C. Device management** **D.** Protection

[2.25] device management：ioctl()、read()、write() ◦

P-ch2-2 chmod → C

`chmod()`, `umask()` and `chown()` are examples of which category of system calls?

A. Information maintenance **B.** Communications **C. Protection** **D.** File management

[2.25] ◦

P-ch2-3 Parameter passing limit → B

Which parameter-passing methods do **not** limit the number or length of parameters?

A. Registers only **B. Block (table) and stack methods** **C.** Registers and stack **D.** None of them

[2.20] ◦

P-ch2-4 Policy vs mechanism → B

Deciding how long the timer should be set for a particular user is an example of:

A. A mechanism **B. A policy** **C.** A system call **D.** A system program

timer 本身 = mechanism；設幾耐 = policy [2.34] ◦

P-ch2-5 Shell → B

What is the primary functionality of a shell?

A. Schedule kernel threads **B. Fetch a command from the user, interpret it and execute it** **C.** Manage virtual memory **D.** Load device drivers

[2.8] ◦

M

P-ch2-6 DLL → B

What is a benefit of dynamically linked libraries (DLLs)?

A. They are linked statically into every executable **B. They are loaded when needed and shared by all programs using the same version** **C.** They remove the need for a loader **D.** They make executables larger

[2.31] ◦

P-ch2-7 MS-DOS → B

Which statement about MS-DOS is correct?

A. It is a multitasking system with dual mode **B. Application programs could access I/O directly, and it had no dual mode or hardware protection** **C.** It is a microkernel design **D.** It creates a new process for every program

[2.36, 2.39] ◦

P-ch2-8 Microkernel services → B

A microkernel typically provides:

A. File systems and device drivers only **B. Minimal process and memory management, plus a communication facility** **C.** The GUI and shells **D.** All OS services in kernel mode

[2.46] ◦

P-ch2-9 Darwin → B

Darwin, the kernel environment of macOS/iOS, consists of:

A. Linux kernel and glibc **B. Mach microkernel and BSD UNIX kernel** **C.** Windows NT kernel and personalities **D.** ART and JNI

[2.54, 2.57] (hybrid，XNU kernel) ◦

P-ch2-10 Android → B

Which statement about Android is **false**?

A. Java applications execute on the Android RunTime (ART) **B. Programs written using JNI are generally portable across hardware** **C.** Android uses the Bionic C library instead of glibc **D.** The HAL abstracts hardware to provide a consistent view

JNI programs **generally not portable** [2.58] ◦

P-ch2-11 Windows structure → B

Windows is described as:

A. Pure microkernel **B. Largely monolithic with microkernel-like subsystems (personalities) and loadable modules** **C.** Pure layered **D.** Simple structure like MS-DOS

[2.52] ◦

P-ch2-12 High-level language → B

Which is a main advantage of implementing an OS in a high-level language?

A. Always faster than assembly **B. Easier to port to other hardware** **C.** Less storage needed **D.** No compiler needed

[2.35]；缺點 = reduced speed、more storage ◦

P-ch3-1 Default file descriptors → A

On UNIX, each process by default has three open file descriptors. Which are they?

A. stdin (0), stdout (1), stderr (2) **B.** stdin (1), stdout (2), stderr (3) **C.** pipe read end, pipe write end, stderr **D.** stdin, stdout, swap

[3.7] ◦

P-ch3-2 Linux PCB → B

In Linux, the PCB is represented by:

A. struct pcb in a hash table **B. struct task_struct elements in a circular doubly linked list (task list)** **C.** TCB in a binary tree **D.** ETHREAD

[3.13] ◦

P-ch3-3 Long-term scheduler → B

Which statement about the long-term scheduler is correct?

A. It is invoked every few milliseconds **B. It controls the degree of multiprogramming and is mainly used in mainframes, not PCs** **C.** It swaps processes out to disk **D.** It selects the next process from the ready queue

[3.17] ◦ C = medium-term；D = short-term ◦

P-ch3-4 Register sets → B

Some processors (e.g., SUN UltraSPARC) provide multiple register sets. What is the effect on context switch?

A. Context switch becomes impossible **B. A context switch may just change the pointer to a different register set** **C.** The PCB is no longer needed **D.** Context switch time increases

[3.20] ◦

P-ch3-5 Copy-on-write → B

In Linux, fork() uses copy-on-write. What does this achieve?

A. Parent and child never share any pages **B. Copying of the address space is delayed until a page is written, speeding up process creation** **C.** The child always runs exec() first **D.** All pages are copied immediately

[3.46] ◦

P-ch3-6 Mode bit → B

The hardware mode bit is ___ in user mode and ___ in kernel mode.

A. 0, 1 **B. 1, 0** **C.** 1, 1 **D.** 0, 0

[3.28] ◦

P-ch3-7 Protection rings → B

On Intel processors with four protection rings:

A. Ring 3 is kernel mode and ring 0 is user mode **B. Ring 0 is kernel mode and ring 3 is user mode** **C.** All four rings are heavily used **D.** Ring 1 is for the VMM only

[3.29]；rings 1、2 rarely used ◦

P-ch3-8 Trap → C

A segmentation fault (protection violation) causes which type of mode transfer?

A. System call **B.** Interrupt **C. Trap or exception** **D.** Context switch only

trap/exception = **internal synchronous event** [3.31] ◦

P-ch3-9 Bounded buffer → B

The shared-memory bounded-buffer solution with `in` and `out` and no counter can hold at most:

A. BUFFER_SIZE items **B. BUFFER_SIZE − 1 items** **C.** BUFFER_SIZE + 1 items **D.** Unlimited items

[3.54] ◦

P-ch3-10 Direct communication → B

Which is a property of direct communication in message passing?

A. Messages go through mailboxes **B. A link is associated with exactly one pair of processes** **C.** A link may be associated with more than two processes **D.** It provides better modularity than indirect communication

[3.59]；缺點 = limited modularity ◦

P-ch3-11 Zero capacity → C

With a zero-capacity message queue:

A. The sender never waits **B.** The sender waits only if the queue is full **C. The sender must wait for the receiver (rendezvous)** **D.** Messages are lost

[3.64] ◦

P-ch3-12 UNIX FIFO → B

Which statement about named pipes (FIFOs) in UNIX is correct?

A. They support full-duplex transmission over networks **B. They are created with mkfifo(), appear as files, support half-duplex, and processes must be on the same machine** **C.** They disappear when the creating process exits **D.** They require a parent-child relationship

[3.69]；Windows named pipes 先係 full-duplex、可跨機。

P-ch3-13 Ports → B

Which statement about client–server sockets is correct?

A. Servers initiate the connection to clients **B. All ports below 1024 are well known; clients pick a port above 1024** **C.** HTTP uses port 22 **D.** A socket is identified by the MAC address only

[3.71]；HTTP 80、SSH 22、FTP 21 ◦

P-ch3-14 Orphan → B

When a parent terminates without calling wait(), its child process:

A. Is immediately deleted by the kernel in all OSes **B. Becomes an orphan and is re-parented to init/systemd, which periodically calls wait()** **C.** Becomes a zombie forever **D.** Takes over the parent's PID

[3.50] ◦

P-ch3-15 exec → C

Which is true about a successful exec() call?

A. It returns the PID of the new program **B.** It returns 0 to the child **C. It does not return; the process image is replaced** **D.** It creates a new process

exec **唔會建立新 process**，只係 replace 現有 process 嘅 image [3.41] ◦

M

P-ch3-16 fork count → C

How many processes are there in total after: `fork(); fork() && fork(); fork();`

A. 8 **B.** 10 **C. 12** **D.** 16

2 × 3 × 2 = **12**（`fork() && fork()` 令每個 process 變 3 個）。

P-ch3-17 Interrupt vector → B

The interrupt vector is:

A. A list of processes waiting for I/O **B. A table of addresses of interrupt handlers, indexed by interrupt number** **C.** The register that holds the next instruction address **D.** A queue of DMA requests

Ch3 summary。

P-ch3-18 Nonmaskable interrupt → B

A nonmaskable interrupt is reserved for events such as:

A. Keyboard input **B. Unrecoverable memory errors and power failure** **C.** Timer expiry for RR **D.** Disk I/O completion

Ch3 summary；maskable 用於 device controllers。

P-ch4-1 Multicore challenges → D

Which is **NOT** one of the multicore programming challenges listed in the lecture?

A. Dividing tasks **B.** Data dependency **C.** Testing and debugging **D. Reducing the number of system calls**

五大：dividing tasks、balance、data splitting、data dependency、testing & debugging [4.9]。

P-ch4-2 Windows TEB → C

In Windows, which thread data structure lives in **user space** and contains the thread id, user-mode stack and thread-local storage?

A. ETHREAD **B.** KTHREAD **C. TEB** **D.** PCB

[4.36]。ETHREAD、KTHREAD 喺 kernel space。

P-ch4-3 clone() → B

Which call creates a Linux task that behaves like a thread?

A. clone(SIGCHLD, 0) **B. clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0)** **C.** fork() followed by exec() **D.** pthread_kill()

share address space、fs info、files、signal handlers [4.38]；A = normal fork。

P-ch4-4 Linux threads → B

How does Linux treat threads?

A. As lightweight processes with separate TCB and PCB structures **B. As tasks — a thread is a process that shares certain resources with other tasks** **C.** Only user-level threads are supported **D.** Through the many-to-one model

[4.37]。

P-ch4-5 Signal delivery → B

A synchronous signal such as division by zero should be delivered to:

A. Every thread in the process **B. The thread that caused the signal** **C.** A random thread **D.** The parent process

[4.31]。

P-ch4-6 Pthread cancellation default → B

In Pthreads, the default cancellation type is:

A. Asynchronous **B. Deferred** **C.** Disabled permanently **D.** Immediate

[4.32]。

P-ch4-7 Amdahl limit → B

An application is 50% parallel. As the number of cores approaches infinity, the maximum speedup is:

A. 1.5 **B. 2** **C.** 4 **D.** Infinite

$1/S = 1/0.5 = \mathbf{2}$。

P-ch4-8 One-to-one drawback → C

What is the main drawback of the one-to-one model?

A. One blocking call blocks the whole process **B.** Threads cannot run in parallel **C. Creating a user thread requires creating a kernel thread, so the number of threads may be restricted** **D.** It is difficult to implement

[4.23]。A、B 係 many-to-one；D 係 many-to-many。

P-ch4-9 Thread context switch → B

Why is a context switch between two threads of the same process cheaper than between processes?

A. Threads have no registers **B. The address space stays the same** **C.** Threads do not have stacks **D.** The OS does not save any state

[4.14]。

P-ch4-10 fork in multithreaded process → B

If exec() is called immediately after fork() in a multithreaded process, which fork() version is appropriate?

A. Duplicate all threads **B. Duplicate only the calling thread** **C.** Duplicate no threads **D.** It does not matter for correctness or efficiency

exec 會 replace 成個 process，複製所有 threads 係浪費 [4.28]。

P-ch5-1 Dispatcher → D

Which is **NOT** a function of the dispatcher?

A. Switching context **B.** Switching to user mode **C.** Jumping to the proper location in the user program **D. Selecting the next process using the scheduling algorithm**

揀 process 係 **CPU scheduler**；dispatcher 只負責交 CPU 畀佢 [5.7]。

P-ch5-2 RR waiting bound → B

With n processes and quantum q under RR (ignoring context switches), no process waits more than:

A. nq **B. (n − 1)q** **C.** q / n **D.** n + q

[5.11]。

P-ch5-3 Quantum rule of thumb → B

A common rule of thumb for choosing the RR quantum is:

A. q should be smaller than the context-switch time **B. 80% of CPU bursts should be shorter than q** **C.** q should equal the longest CPU burst **D.** q should be 1 ms

[5.16]。

P-ch5-4 Exponential averaging α = 0 → B

With exponential averaging, if $\alpha = 0$ then:

A. Only the last CPU burst counts **B. Recent history does not count; the prediction never changes** **C.** All bursts are weighted equally **D.** The prediction is always 0

$\tau_{n+1} = \tau_n$ [5.22]。

P-ch5-5 SJF as priority → B

SJF is a special case of priority scheduling where the priority is:

A. The arrival time **B. The inverse of the predicted next CPU burst** **C.** The process ID **D.** The period of the task

[5.25]。

P-ch5-6 MLFQ preemption rule → C

In the lecture's MLFQ, when a job running from Q1 is preempted by a new arrival in Q0, it:

A. Is demoted to Q2 **B.** Joins the tail of Q1 with a fresh quantum **C. Joins the head of Q1 with its remaining quantum** **D.** Is moved to Q0

[5.31]。

P-ch5-7 Logical CPUs → D

A quad-core processor with 2 hardware threads per core appears to the OS as how many logical processors?

A. 2 **B.** 4 **C.** 6 **D. 8**

4 × 2 = 8 [5.42]。

P-ch5-8 Push migration → B

Push migration means:

A. An idle processor pulls a waiting task from a busy processor **B. A specific task periodically checks loads and moves tasks from overloaded to idle/less-busy CPUs** **C.** A process specifies the CPUs it may run on **D.** A thread is pinned to one core forever

A = pull migration [5.46]；C = hard affinity [5.45]。

P-ch5-9 Affinity system call → A

In Linux, which system call supports hard affinity?

A. sched_setaffinity() **B.** setpriority() **C.** clone() **D.** nice()

[5.45]。

M

P-ch5-10 RM optimality → B

Rate-monotonic scheduling is optimal in the sense that:

A. It always achieves 100% CPU utilization **B. If a task set cannot be scheduled by RM, it cannot be scheduled by any other static-priority algorithm** **C.** It never misses deadlines **D.** It handles aperiodic tasks best

[5.52] ◦

P-ch5-11 EDF requirement → C

What is the only requirement EDF places on a process?

A. It must be periodic **B.** It must have a constant CPU burst **C. It must announce its deadline to the scheduler when it becomes runnable** **D.** Its period must be a power of 2

[5.52] ◦

P-ch5-12 RM utilization → B

Two periodic tasks: P1 (period 50, burst 25), P2 (period 80, burst 35). Under RM:

A. Total utilization is 75% and no deadline is missed **B. Total utilization ≈ 94% and P2 misses its deadline at 80 (finishes at 85)** **C.** P1 misses its deadline **D.** EDF would also miss a deadline

[5.50–5.51]；EDF 可以全部 meet ◦

P-ch5-13 Deterministic modeling → B

Which statement about deterministic modeling is correct?

A. It uses probability distributions of CPU bursts **B. It takes a predetermined workload and gives exact numbers, but results apply only to that workload** **C.** It requires running the real system **D.** It is the most accurate method

[5.54] ◦

P-ch5-14 Response time (course definition) → C

RR with q = 4; P1 (24), P2 (3), P3 (3) all arrive at 0 in that order. Using the course definition, the average response time is:

A. 3.67 **B.** 5.67 **C. 7** **D.** 15.67

end of first run − arrival = 4, 7, 10 → avg **7** [5.13] ◦ (3.67 係課本「first start」定義；5.67 係 avg WT；15.67 係 avg TAT)

P-ch5-15 Non-preemptive kernel / scheduling → B

Scheduling that happens only when a process switches from running to waiting or terminates is called:

A. Preemptive **B. Non-preemptive (cooperative)** **C.** Real-time **D.** Multilevel

[5.6] ◦

P-ch5-16 SMP common queue issue → A

With a single common ready queue in SMP, what must the system guard against?

A. Two processors choosing to schedule the same thread (race condition) **B.** Processor affinity being too strong **C.** Too many push migrations **D.** Memory stalls

[5.39] ◦

P-ch6-1 Disabling interrupts → B

Why is disabling interrupts not a good critical-section solution on multiprocessors?

A. It is not allowed in kernel mode **B. It is inefficient — interrupts must be disabled on all processors** **C.** It causes race conditions on uniprocessors **D.** It requires semaphores

[6.13] ◦

P-ch6-2 CAS return value → C

compare_and_swap(&value, expected, new_value) returns:

A. new_value **B.** expected **C. The original value of *value** **D.** TRUE if the swap happened

[6.17] ◦

P-ch6-3 Negative semaphore → B

In the blocking (no busy waiting) semaphore implementation, a value of −3 means:

A. The semaphore is broken **B. Three processes are waiting on the semaphore** **C.** Three resources are available **D.** Three signals were lost

[6.26] ◦

P-ch6-4 Spinlock usage → B

Spinlocks are most appropriate when:

A. Locks are held for a long time on a uniprocessor **B. Locks are held for short times on multiprocessor systems** **C.** Processes must sleep while waiting **D.** Context switches are free

[6.22] ◦

P-ch6-5 Preemptive kernel → B

Which statement is correct?

A. A preemptive kernel is free from race conditions **B. A non-preemptive kernel is essentially free from race conditions in kernel mode (on a single processor)** **C.** Non-preemptive kernels are harder to make correct on SMP than preemptive ones **D.** Kernel code never has race conditions

[6.12] ◦

P-ch6-6 Kernel race example → B

The lecture's kernel race-condition example involves two processes calling fork() concurrently. Which variable is at risk?

A. counter **B. next_available_pid** **C.** lock **D.** turn

same PID could be assigned to two processes [6.8] ◦

P-ch6-7 P and V → B

In Dijkstra's notation, P() and V() correspond to:

A. signal() and wait() **B. wait() and signal()** **C.** acquire() and test_and_set() **D.** lock() and fork()

P = proberen (test)，V = verhogen (increment) [6.24] ◦

P-ch6-8 Semaphore deadlock → B

P0: wait(S); wait(Q); ... P1: wait(Q); wait(S); ... with S = Q = 1. What can happen?

A. Starvation only **B. Deadlock** **C.** Race condition on S only **D.** Nothing; this is always safe

[6.28] ◦

P-ch6-9 Starvation in semaphore queue → A

Starvation (indefinite blocking) can occur with semaphores when:

A. Processes are removed from the waiting queue in LIFO order **B.** The semaphore is initialized to 1 **C.** signal() is atomic **D.** The waiting queue is FIFO

[6.28] ◦

P-ch6-10 Atomic increment → B

In `do { temp = *v; } while (temp != compare_and_swap(v, temp, temp + 1));`, why might the loop repeat?

A. Because CAS always fails the first time **B. Because another thread changed *v between reading temp and the CAS, so the CAS returned a different value** **C.** Because temp + 1 overflows **D.** It never repeats

[6.21] ◦

P-ch6-11 Counting semaphore initial value → C

A counting semaphore controlling access to 5 identical printers should be initialized to:

A. 0 **B.** 1 **C. 5** **D.** −5

initialized to the number of resources available [6.25] ◦

P-ch6-12 Simple TAS lock → B

The simple lock `while (test_and_set(&lock)); /*CS*/ lock = false;` satisfies:

A. Mutual exclusion and bounded waiting **B. Mutual exclusion but not bounded waiting** **C.** Bounded waiting but not mutual exclusion **D.** None of the requirements

要用 waiting[] 版本 [6.19] 先有 bounded waiting ◦

M

A C / Pthreads API Lab / PA API 速查

> Announcement：「There may be questions related to PA or coding questions using some APIs we discussed in our lab classes」→ 呢頁就係用嚟填空 / 寫 code。

1. Process APIs

Call	Header	Return	重點
`pid_t fork(void)`	`<unistd.h>`	child: **0**；parent: **child PID (>0)**；fail: **−1**	child 由下一句開始；複製 address space（COW）
`int execlp(const char *file, const char *arg0, ..., (char *)NULL)`	`<unistd.h>`	**success: no return**；fail: −1	用 PATH 搵 program；**arg0 = program name**；最後一定要 `NULL`
`int execvp(const char *file, char *const argv[])`	`<unistd.h>`	同上	`argv` 最後一個元素要係 **NULL**（PA1：所以 array size 係 9 而唔係 8）
`int execl(path, arg0, ..., NULL)` / `execv(path, argv)`	`<unistd.h>`	同上	要 full path（冇 p = 唔搵 PATH）
`pid_t wait(int *status)`	`<sys/wait.h>`	terminated child 嘅 PID；冇 child → −1	`wait(NULL)` 唔要 status；等**任何一個** child
`pid_t waitpid(pid_t pid, int *status, int options)`	`<sys/wait.h>`	該 child PID；error −1	`waitpid(pid, &status, 0)` 等指定 child；`WNOHANG` 唔 block
`void exit(int status)`	`<stdlib.h>`	—	結束 process，status 經 wait 交畀 parent（`WEXITSTATUS(status)`）
`pid_t getpid(void)` / `getppid(void)`	`<unistd.h>`	自己 / parent PID	
`int chdir(const char *path)`	`<unistd.h>`	0 / −1	PA1 `cd`（唔可以用 exec 做 cd，因為要改 shell 自己嘅 cwd）
`sighandler_t signal(int signum, sighandler_t handler)`	`<signal.h>`	舊 handler	`signal(SIGINT, handler)`；`SIG_IGN` 忽略；`SIG_DFL` default
`int kill(pid_t pid, int sig)`	`<signal.h>`	0 / −1	send signal to process
`unsigned int sleep(unsigned int s)`	`<unistd.h>`	剩餘秒數	停 s 秒；`usleep` = μs；`nanosleep`（`<time.h>`）更細

exec family（Lab 3）：六個 — `execl, execlp, execle, execv, execvp, execvpe`；全部 **replace the current process image，成功唔會 return，失敗 −1**。

字母	意思	例
l (list)	arguments 逐個列出，數目 compile time 已知	`execlp("ls", "ls", "-l", "-h", NULL);`
v (vector)	arguments 放喺 array（尾要 NULL）；數目唔知時用	`char *a[] = {"ls", "-l", NULL}; execvp("ls", a);`
p (path)	用 **PATH** 搵 program（唔使 full path）	`execlp` / `execvp` / `execvpe`
e (env)	額外傳 environment array	`execle` / `execvpe`

1.1 經典 fork–exec–wait

```
pid_t pid = fork();
if (pid < 0) { perror("fork"); exit(1); }
else if (pid == 0) {                        /* child */
    execlp("ls", "ls", "-l", (char *)NULL);
    perror("exec failed");                   /* only reached if exec fails */
    exit(1);
} else {                                    /* parent */
    int status;
    waitpid(pid, &status, 0);               /* or wait(NULL) */
}
```

1.2 Signal handler（PA1 Feature 2）

```
void handle_sigint(int signum) {
    printf("Myshell (pid=%d) terminates by Ctrl-C with signal: %d\n", getpid(), signum);
    exit(0);
}
int main() { signal(SIGINT, handle_sigint); /* ... */ }
```

- SIGINT = **2**（Ctrl-C）；SIGKILL (9)、SIGSTOP 唔可以 catch。

2. File I/O（POSIX，唔係 stdio）

Call	Return	重點
`int open(const char *path, int flags[, mode_t mode]) <fcntl.h>`	**lowest unused fd** / −1	flags：`O_RDONLY`、`O_WRONLY`、`O_RDWR`、`O_CREAT`、`O_TRUNC`、`O_APPEND`；建立檔案要 mode（e.g. `0644`）
`ssize_t read(int fd, void *buf, size_t count)`	bytes read；**0 = EOF**；−1 error	blocking（pipe 冇 data 會等；所有 write ends close 晒先 return 0）
`ssize_t write(int fd, const void *buf, size_t count)`	bytes written / −1	
`int close(int fd)`	0 / −1	**用完要 close**，尤其 pipe write end
`int dup(int oldfd)`	新 fd = **lowest unused**	`close(0); dup(fin);` → fd 0 指向 file
`int dup2(int oldfd, int newfd)`	newfd	newfd 已開 → 先 close；`dup2(fd, 1)` = stdout 指向 fd

- **fd 0 = stdin，1 = stdout，2 = stderr**（`STDIN_FILENO`, `STDOUT_FILENO`, `STDERR_FILENO`）。
- Lab 3 permission 寫法：`open("hello.txt", O_CREAT | O_WRONLY, S_IRUSR | S_IWUSR)`（**600**：owner read + write）；`O_RDONLY` 開讀。
- ⚠️ **唔好用 stdio 嘅 `fopen/fread/fwrite/fclose`（`FILE*`）做 PA**：`FILE*` 係 C library struct，**cannot be used in pipe operations**。
- **Output redirection**（`cmd > out.txt`）：

```
int fout = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fout, STDOUT_FILENO);  close(fout);   /* then exec */
```

- **Input redirection**（`cmd < in.txt`）：

```
int fin = open("in.txt", O_RDONLY);
dup2(fin, STDIN_FILENO);  close(fin);      /* or: close(0); dup(fin); */
```

3. Pipes

Call	重點
`int pipe(int fd[2]) <unistd.h>`	**fd[0] = read end，fd[1] = write end**；0 / −1；**一定喺 fork() 之前 call**
`int mkfifo(const char *path, mode_t mode) <sys/stat.h>`	named pipe（FIFO），之後用 `open()` / `read()` / `write()`；要 `unlink()` 刪除；0 / −1
`int access(const char *path, int how) <unistd.h>`	`F_OK` 存在、`R_OK` / `W_OK` / `X_OK` 權限；0 = OK，−1 = 唔得（Lab 4：`if (access(FIFO, F_OK) == -1) mkfifo(FIFO, 0777);`）
`FILE *popen(const char *cmd, const char *type)` / `int pclose(FILE *fp) <stdio.h>`	fork 一個 child 跑 cmd；`"r"` 讀佢 stdout、`"w"` 寫佢 stdin；`pclose` 等佢完返回 status（**PA1 禁用**）

- **FIFO 特性（Lab 4）**：`open/read/write` **blocking，要兩端都有 process open 先繼續**（可用 `O_NONBLOCK`）；data **讀咗就冇**（唔可以 broadcast）；**≤ `PIPE_BUF` bytes 嘅 write 係 atomic**；一條 FIFO 實際唔適合雙向（可能讀返自己寫嘅）→ 雙向用兩條。
- **兩條 ordinary pipes 做雙向**：`pipe(p2c); pipe(c2p);` → child：`close(c2p[0]); write(c2p[1], …); close(p2c[1]); read(p2c[0], …);`；parent：`close(c2p[1]); read(c2p[0], …); close(p2c[0]); write(p2c[1], …);`（Lab 4 exam example）。

3.1 Parent → child

```
int fd[2]; pipe(fd);
if (fork() == 0) {                /* child: reader */
    close(fd[1]);
    int n; read(fd[0], &n, sizeof(n));
    close(fd[0]); exit(0);
} else {                          /* parent: writer */
    close(fd[0]);
    int n = 6; write(fd[1], &n, sizeof(n));
    close(fd[1]); wait(NULL);
}
```

A

3.2 Shell pipeline `ls | wc -l`

```
int fd[2]; pipe(fd);
if (fork() == 0) {              /* child 1: ls → pipe */
    dup2(fd[1], STDOUT_FILENO);
    close(fd[0]); close(fd[1]);
    execlp("ls", "ls", (char *)NULL);
}
if (fork() == 0) {              /* child 2: pipe → wc */
    dup2(fd[0], STDIN_FILENO);
    close(fd[0]); close(fd[1]);
    execlp("wc", "wc", "-l", (char *)NULL);
}
close(fd[0]); close(fd[1]);     /* parent MUST close both, else wc never sees EOF */
wait(NULL); wait(NULL);
```

- **Multi-level pipe（PA1 Feature 5）**：n segments → n−1 pipes；segment i：`dup2(fds[i-1][0], 0)`（if i > 0）、`dup2(fds[i][1], 1)`（if i < n−1）；每個 child close 所有 pipe fds；parent close 所有 fds 再 wait 全部。

3.3 常見錯誤

1. `pipe()` 喺 `fork()` 之後 call → 兩條唔相關嘅 pipe（HW1 Q3.3c）。
2. 冇 close 未用嘅 write end → reader `read()` 永遠唔會得到 EOF（hang）。
3. `write(fd[0], …)` / `read(fd[1], …)` 搞錯方向（**0 讀 1 寫**：記「0 = stdin-like，1 = stdout-like」）。
4. `read` / `write` 第二個參數要 **address**（`&cnt`），第三個係 **size**（`sizeof(int)`）。

---

4. Pthreads（`#include <pthread.h>`，compile with `-pthread`）

Call	重點
`int pthread_create(pthread_t *tid, const pthread_attr_t *attr, void *(*start)(void *), void *arg)`	return 0 = success；attr 用 `NULL`；start routine signature `void *f(void *)`
`int pthread_join(pthread_t tid, void **retval)`	等 thread 結束（類似 wait）；`retval` 可 NULL
`void pthread_exit(void *retval)`	thread 自己結束
`int pthread_cancel(pthread_t tid)`	request cancellation（default **deferred**）
`pthread_t pthread_self(void)`	自己 tid
`int pthread_kill(pthread_t tid, int sig)`	signal 指定 thread
`int pthread_attr_init(pthread_attr_t *attr)`	default attributes（stack size, scheduling info）→ 再傳畀 `pthread_create`（Lab 4 sum example）
`pthread_cond_broadcast(&cond)`	叫醒**所有**等緊嘅 threads（`pthread_cond_signal` 只叫醒一個）

```
void *worker(void *arg) { int id = *(int *)arg; printf("thread %d\n", id); return NULL; }
int main() {
    pthread_t t[3]; int id[3];
    for (int i = 0; i < 3; i++) { id[i] = i; pthread_create(&t[i], NULL, worker, &id[i]); }
    for (int i = 0; i < 3; i++) pthread_join(t[i], NULL);
    return 0;
}
```

4.1 Mutex

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;   /* or pthread_mutex_init(&m, NULL) */
pthread_mutex_lock(&m);   /* critical section */   pthread_mutex_unlock(&m);
pthread_mutex_destroy(&m);
```

4.2 POSIX unnamed semaphore（`#include <semaphore.h>`）

```
sem_t s;
sem_init(&s, 0, 1);     /* pshared = 0 → shared between threads of this process; initial value 1 */
sem_wait(&s);           /* P(): if value > 0 decrement, else block */
sem_post(&s);           /* V(): increment, wake a waiter */
sem_destroy(&s);
```

- **Ordering**：`sem_init(&s, 0, 0)`；先做嘅 thread 做完 `sem_post(&s)`，後做嘅 thread 開頭 `sem_wait(&s)`。
- **Mutual exclusion**：init 1；`sem_wait` → CS → `sem_post`。
- **Counting resources**：init = 資源數量 n。

A

4.3 Condition variables

```
pthread_mutex_t m; pthread_cond_t c; int ready = 0;
/* waiter */
pthread_mutex_lock(&m);
while (!ready) pthread_cond_wait(&c, &m);   /* releases m while sleeping, re-acquires on wake */
pthread_mutex_unlock(&m);
/* signaller */
pthread_mutex_lock(&m); ready = 1; pthread_cond_signal(&c); pthread_mutex_unlock(&m);
```

- 一定要持有 mutex 先 call；用 **while**（唔係 if）再 check 條件。

---

5. Compile / Run（Lab 1–2）

```
gcc -pedantic -Wall -Wextra -std=c99 -o myshell myshell.c
gcc prog.c -o prog -lpthread       # pthread programs (or -pthread)
gcc buggy.c -o buggy -g            # debug info, then: gdb buggy
./mlfq < input.txt > output.txt    # PA2: input/output redirection
diff --side-by-side my-out.txt sample-out.txt   # compare outputs (--suppress-common-lines)
gcc main.c -o main -lm             # link math library (linker!)
./prog
```

- `gcc -c main.c` → `main.o`（compiler）；`gcc -o main main.o -lm`（linker）；`./main`（loader）[2.32]。

G Glossary A-Z 術語表

Term	Definition（exam English）	中文	Ref
Accounting information	PCB field: CPU used, clock time elapsed, time limits	記帳資訊	3.10
Address space	The memory a program executes in, distinct from other programs' (text, data, heap, stack)	位址空間	3.4–3.5
Admission control	Real-time scheduler admits a task only if it can guarantee its deadline	准入控制	5.48
Aging	Gradually increasing the priority of a process that waits a long time, to prevent starvation	老化	5.25
Amdahl's Law	$\text{speedup} \le \frac{1}{S + (1-S)/N}$; as $N \to \infty$, speedup $\to 1/S$	阿姆達爾定律	4.12
API	A set of functions available to programmers (parameters + return values) that invoke system calls on their behalf	應用程式介面	2.15
Asymmetric multiprocessing (AMP)	Master processor handles scheduling, I/O and kernel data; others run user code only; master may bottleneck	非對稱多處理	1.37, 5.38
Asynchronous cancellation	Terminates the target thread immediately	非同步取消	4.32
Asynchronous signal	Signal generated by an event external to the running process (Ctrl-C, timer expiry)	非同步信號	4.29
Atomic	Non-interruptible; executes as one uninterruptible unit	原子性	6.14
Atomic variable	Provides atomic updates on basic data types (e.g., increment via CAS)	原子變數	6.21
Average access time	Hit ratio × hit time + miss ratio × miss time	平均存取時間	1.32
Binary semaphore	Semaphore whose value is only 0 or 1; can act like a mutex	二元信號量	6.25
Blocking (synchronous) send/receive	Sender blocks until the message is received; receiver blocks until a message is available	阻塞式	3.62
Bootstrap program	First program run at power-on, stored in EEPROM	開機程式	1.30
Bounded buffer	Fixed-size buffer shared by producer and consumer	有界緩衝區	3.53
Bounded waiting	A bound exists on how many times others may enter their CS after a process requests entry	有限等待	6.11
Busy waiting	Looping continuously while waiting for a condition; wastes CPU	忙碌等待	6.22
Cache	Faster, smaller storage holding a copy of a subset of slower, larger storage	快取	1.32
Cascading termination	When a process terminates, all its children are also terminated by the OS	級聯終止	3.48
Chip multithreading (CMT) / hyperthreading	Multiple hardware threads per core, each appearing as a logical CPU; hides memory stalls	晶片多執行緒	5.40–5.42
clone()	Linux call creating a task that shares selected resources with its parent (flags)	複製	3.46, 4.38
Cloud computing	Delivers computing, storage and apps as a service over a network; extension of virtualization	雲端運算	1.47
Compare-and-swap (CAS)	Atomically: if *value == expected set *value = new; return original value	比較並交換	6.17
Concurrency	More than one task making progress (possibly by time multiplexing on one core)	並行（並發）	4.9
Context switch	Saving the state of the old process/thread and loading the saved state of the new one; pure overhead	上下文切換	3.20
Convoy effect	Short processes stuck waiting behind a long process (FCFS)	護航效應	5.10
Cooperating process	A process that can affect or be affected by other processes	協作進程	3.51
Copy-on-write (COW)	Parent and child share pages until one writes; then the page is copied	寫入時複製	3.46
CPU burst	A period of CPU execution between I/O waits	CPU 爆發	5.4
CPU-bound process	Spends most time computing; few long CPU bursts	CPU 密集	3.14
CPU scheduler (short-term)	Selects a process from the ready queue and allocates a CPU core	短期排程器	3.17, 5.6
CPU utilization	Fraction of time the CPU is busy	CPU 使用率	5.8
Critical section	Code segment in which shared data is accessed/modified	臨界區	6.9
Data parallelism	Same operation on different subsets of data across cores	數據並行	4.11
Deadlock	Two or more processes wait indefinitely for an event only one of the waiting processes can cause	死結	6.28
Deferred cancellation	Target thread periodically checks whether it should terminate (orderly)	延遲取消	4.32
Degree of multiprogramming	Number of processes currently in memory	多道程式度	3.14
Deterministic modeling	Evaluates algorithms on a predetermined workload; exact but specific	確定性建模	5.54
Device driver	Kernel module that controls a device	裝置驅動程式	1 (summary)

Term	Definition（exam English）	中文	Ref
Direct communication	Processes name each other explicitly; one link per pair	直接通訊	3.59
Dispatch latency	Time for the dispatcher to stop one process and start another	分派延遲	5.7
Dispatcher	Gives CPU control to the selected process: context switch, switch to user mode, jump to restart location	分派器	5.7
DLL (dynamically linked library)	Library loaded when needed and shared by all programs using it	動態連結庫	2.31
DMA	Controller transfers blocks directly between device and memory without CPU; interrupts once when done	直接記憶體存取	3.25
Dual-mode operation	User mode vs kernel mode distinguished by a hardware mode bit (1 = user, 0 = kernel)	雙模式	3.27–3.28
dup / dup2	Duplicate a file descriptor (dup → lowest free fd; dup2 → specified fd)	複製描述符	Lab/PA1
EDF	Earliest-deadline-first: dynamic priority, earlier deadline = higher priority; theoretically optimal	最早截止期限優先	5.51
Exec()	Replaces the current process image with a new program; returns only on failure	執行新程式	3.41
Exponential averaging	$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$: predicts the next CPU burst	指數平均	5.19
FCFS	First-come first-served; non-preemptive; convoy effect	先到先得	5.9
fork()	Creates a child duplicating the parent's address space; returns 0 to child, child PID to parent, −1 on failure	分叉	3.35–3.37
Hard affinity	Process specifies the subset of CPUs it may run on	硬親和性	5.45
Hard real-time	Task must be serviced by its deadline; late = no service	硬實時	5.47
Heap	Memory dynamically allocated at run time	堆積	3.5
Hybrid system	OS combining several structures (Linux, Windows, macOS)	混合式	2.52
Indirect communication	Messages sent to/received from mailboxes (ports)	間接通訊	3.60
Interrupt	Signal from hardware/software requiring CPU attention; handled by an ISR	中斷	3.21–3.24
Interrupt vector	Table of interrupt-handler addresses indexed by interrupt number	中斷向量	3 (summary)
I/O-bound process	Spends more time on I/O; many short CPU bursts	I/O 密集	3.14
IPC	Mechanism for cooperating processes to exchange data: shared memory or message passing	進程間通訊	3.51
Kernel	The one program running at all times on the computer	核心	1.24
Kernel thread	Thread supported and scheduled by the OS	核心執行緒	4.19
Layered approach	OS divided into layers (0 = hardware … N = UI); each uses only lower layers	分層結構	2.44
Linker	Combines relocatable object files and libraries into one executable	連結器	2.31
Little's formula	$n = \lambda \times W$ (queue length = arrival rate × waiting time)	利特爾公式	5.57
Load balancing	Keep workload evenly distributed across CPUs (push / pull migration)	負載平衡	5.46
Loadable kernel module (LKM)	Core kernel plus modules linked in at boot or run time; any module can call any other	可載入核心模組	2.49–2.50
Loader	Loads the executable file into memory for execution	載入器	2.31
Locality (temporal / spatial)	Recently accessed items are reused soon / nearby items are accessed soon	局部性	1.33
Long-term scheduler (job scheduler)	Selects which jobs are brought into memory; controls degree of multiprogramming	長期排程器	3.17
LWP (lightweight process)	Virtual processor between user threads and a kernel thread	輕量級進程	5.36
Many-to-one / one-to-one / many-to-many	Mappings from user threads to kernel threads	多對一 / 一對一 / 多對多	4.21–4.24
Medium-term scheduler	Swaps processes out to disk and back to reduce degree of multiprogramming	中期排程器	3.18
Memory stall	Processor waits for data from memory (e.g., cache miss)	記憶體停頓	5.40
Message passing	IPC via explicit send()/receive() without shared memory	訊息傳遞	3.57
Microkernel	Minimal kernel; other services run in user space and communicate via message passing	微核心	2.46–2.48
MLFQ	Multilevel feedback queue: processes move between queues; approximates SRTF without burst knowledge	多級回饋佇列	5.30–5.35
Mode bit	Hardware bit: 1 = user mode, 0 = kernel mode	模式位元	3.28
Monolithic	All kernel functionality in one static binary in a single address space; fast but hard to maintain	單體式	2.41
Multicore	Multiple computing cores on a single chip; faster on-chip communication, less power	多核心	1.39

Term	Definition（exam English）	中文	Ref
Multilevel queue	Processes permanently assigned to queues by type; each queue its own algorithm	多級佇列	5.28–5.29
Multiprogramming	Keeping several jobs in memory so the CPU always has one to execute	多道程式	1.42
Mutex lock	Lock acquired before and released after a critical section (acquire/release atomic)	互斥鎖	6.22
Mutual exclusion	If one process is in its CS, no other process can be in its CS	互斥	6.11
Named pipe (FIFO)	Bidirectional (UNIX half-duplex) pipe with a file name; no parent–child needed; persists until deleted	具名管道	3.69
Non-preemptive scheduling	Process keeps the CPU until it terminates or blocks	非搶佔式	5.6
NUMA	Each CPU has fast local memory; remote memory access slower	非一致記憶體存取	1.40
Ordinary pipe	Unidirectional pipe between parent and child; fd[0] read, fd[1] write; gone after use	普通管道	3.66
Orphan	Child whose parent terminated without wait(); adopted by init/systemd	孤兒進程	3.50
Parallelism	Performing more than one task simultaneously (needs multiple cores)	平行	4.9
PCB	Process control block: state, PC, registers, scheduling, memory, accounting, I/O info	進程控制塊	3.10
PCS (process-contention scope)	Thread library schedules user threads onto LWPs, among threads of the same process	進程競爭範圍	5.37
Pipe	IPC channel treated as a special file; read()/write()	管道	3.65
Policy vs mechanism	Policy = what will be done; mechanism = how to do it	策略與機制	2.34
Preemptive scheduling	CPU can be taken away involuntarily (interrupt, higher-priority arrival)	搶佔式	5.6
Priority scheduling	CPU to highest-priority process (smallest number); starvation → aging	優先權排程	5.25
Privileged instruction	Instruction executable only in kernel mode (I/O, timer, interrupt management)	特權指令	3.28
Process	A program in execution; active entity with PC and resources	進程	3.4–3.5
Processor affinity	Tendency to keep a process on the CPU whose cache holds its data	處理器親和性	5.45
Program	Passive entity — executable file on disk	程式	3.5
Progress	If no one is in the CS, selection of the next entrant cannot be postponed indefinitely	前進	6.11
Pull / push migration	Idle CPU pulls tasks / a task pushes work from overloaded CPUs	拉 / 推遷移	5.46
Race condition	Outcome depends on the order of concurrent access to shared data	競爭條件	6.9
Ready queue	Processes in memory ready and waiting to run	就緒佇列	3.15
Rate-monotonic (RM)	Static priority = shorter period higher priority; optimal among static-priority algorithms	速率單調	5.49
Response time	(Course) end of the first run − arrival; time until the first response is produced	回應時間	5.12
Round robin (RR)	FCFS with a time quantum; preempted process to tail; wait ≤ (n−1)q	輪轉	5.11
RPC	Remote procedure call: message-based procedure call between networked systems	遠端程序呼叫	3.70
SCS (system-contention scope)	Kernel schedules kernel threads onto CPUs, among all threads in the system	系統競爭範圍	5.37
Semaphore	Integer accessed only via atomic wait() (P) and signal() (V)	信號量	6.24
Shared memory	IPC by reading/writing a shared region	共享記憶體	3.51
Signal	UNIX notification to a process that an event occurred	信號	4.29
SJF	Shortest-job-first: optimal minimum average waiting time; hard to know next burst	最短工作優先	5.17
SMP	Symmetric multiprocessing: each processor self-schedules; own registers & cache, shared memory	對稱多處理	1.38, 5.39
Socket	Endpoint for communication identified by IP address + port	套接字	3.70
Soft affinity	OS tries but does not guarantee to keep a process on the same CPU	軟親和性	5.45
Soft real-time	Critical tasks get preference over non-critical ones; no guarantee	軟實時	5.47
Spinlock	Lock that busy-waits; no context switch, wastes CPU; good for short CS on multiprocessors	自旋鎖	6.22
SRTF	Shortest-remaining-time-first: preemptive SJF	最短剩餘時間優先	5.23
Stack	Temporary data: function parameters, return addresses, local variables	堆疊	3.5
Starvation	A process waits indefinitely (low priority / semaphore queue)	飢餓	5.25, 6.28

Term	Definition（exam English）	中文	Ref
System call	Programming interface to OS services	系統呼叫	2.14
System program	Utilities shipped with the OS providing a convenient environment (not part of kernel)	系統程式	2.27
Task parallelism	Different threads performing unique operations on cores	任務並行	4.11
TCB	Thread control block: registers, PC, stack pointer, scheduling info, pointer to PCB	執行緒控制塊	4.15
Test-and-set	Atomically returns the old value of a boolean and sets it to TRUE	測試並設置	6.15
Thread	Basic unit of CPU utilization: thread ID, PC, register set, stack	執行緒	4.6
Thread-local storage	Data unique to each thread	執行緒區域儲存	(Sp21 MC)
Throughput	Number of processes completed per unit time	吞吐量	5 (summary)
Timesharing (multitasking)	CPU switches among jobs so frequently that users can interact with each	分時	1.42
Trap / exception	Software-generated interrupt caused by an error or a system call request	陷阱 / 例外	3.24, 3.31
Turnaround time	Completion time − arrival time	周轉時間	5.8
User thread	Thread managed by a user-level library, invisible to the kernel	使用者執行緒	4.19
Virtualization	Abstracts one computer's hardware into multiple execution environments (VMs)	虛擬化	1.43
VMM / hypervisor	Creates and manages VMs, presenting an interface identical to the host	虛擬機監控器	1.44
von Neumann architecture	Stored-program: instructions and data in the same memory; fetch–decode–execute	馮紐曼架構	1.26
Waiting time	Time spent in the ready queue = turnaround − burst	等待時間	5.8
Zombie	Terminated process whose parent has not yet called wait(); PID & PCB remain	殭屍進程	3.49

I Index 索引

A

Accounting information · 29
Address space · 21, 23, 28, 31, 32, 35, 87
Admission control · 53
Aging · 5, 47, 50, 51, **57**, 88, 89
Amdahl's Law · 4, 39, **41**, 86, 87
AMP · 12, 15, **52**, 53, 58
Android · 9, 20, 23, 24, 42, 52, 91
API · **17**, 24, 42, **44**, **67**, **94**
ART · 23, 24, 91
Asymmetric multiprocessing (AMP) · 12, 52, 86
Asynchronous cancellation · 5, 43, 45, 46, 89
Asynchronous signal · 45, 89
Atomic · 7, 59, **60**, **61**, **62**
Atomic variable · 61
Average access time · 4, **12**, 15, 73, 90

B

Binary semaphore · 5, 59, 63, 65, 71, 77
blocking · 4, 36, 38, 42, 69, 93, 95
Blocking (synchronous) send/receive · 18, 35, 36, 38, 43
bootstrap · 11, 90
Bootstrap program · 11, 90
Bounded buffer · 4, **35**, 38, 64, 91
bounded capacity · 36
Bounded waiting · 5, 8, 60, 61, 64, 65, 93
buffering · 14, 19, **35**, 36, 66
Busy waiting · 62, **63**, 71, 73, 93
byte addressable · 4, 11, 14, 15, 85

C

Cache · 10, 11, 12, 15, 52, 73, 90
Cascading termination · 32
chdir · 68, 72, 94
Chip multithreading (CMT) / hyperthreading · 52
CLI · 16, 17, 19, 85
clone() · **31**, 39, 44, 46, 92
close() · 7, 37, 67, 68, 69, 95, 96
cloud · 9, **14**, 23, 90
Cloud computing · **14**
CMT · **52**, 75
compare_and_reset · 8, 59, 62, 65, 79
compare_and_swap · 8, 60, **61**, 62, 88, 93
Concurrency · **40**, 41, 43, 45, 86, 88
condition variable · 71
consumer · **35**, 36, **59**, 64, 69
Context switch · 5, **30**, 31, 38, 85, 91
control program · 9
Convoy effect · 7, 47, 48, 76, 89
copy-on-write · **31**, 91
Copy-on-write (COW) · **31**, 91
counter++ · 59, 65, 73
counting semaphore · 63, 65, 84, 93
CPU burst · 47, 48, **49**, 51, 76, 88
CPU scheduler (short-term) · 31, 42, 47, 87, 92
CPU utilization · 13, 29, 39, 47, 53, 54, 55
CPU-bound · 30, 47, 48, 51, 56, 73, 76
CPU-bound process · 30, 56
Critical section · 44, **60**, 61, 62, 86, 88
Ctrl-C · 43, 45, 68, 86, 94
cycle stealing · 66

D

Darwin · 21, 22, 23, 24, 91
Data parallelism · 41, 45, 88, 89
deadline · 53, **54**, 55, **70**, 93
Deadlock · 61, **63**, 64, 65, 93
Deferred cancellation · 43, 44, 89
Degree of multiprogramming · 30, 38, 87
Deterministic modeling · 55, 93
Device driver · 10, 14, 26, 66
Direct communication · 36, 91
Dispatch latency · 47
Dispatcher · **47**, 92
DLL · 91
DLL (dynamically linked library) · 91
DMA · 14, **26**, 27, 38, 85, 88
dual mode · 21, **26**, 87, 91
Dual-mode operation · **27**, 28
dup / dup2 · **7**, **37**, **68**, 95
dup() · 7, 37, 68, 72, 74, 95
dup2 · 7, **37**, **68**, 95, 96
dynamic priority · 4, 7, 47, **56**, 58, 74

E

economy · 12, 15, 40, 86, 89, 90
EDF · **54**, 55, 58, **70**, 79, 93
EEPROM · 11, 90
entry section · 60, 61, 88
exception · 26, 27, 38, 88, 91
Exec() · 5, 31, 32, **43**, 91, 92
execl · 67, 94
execle · 67, 94
execlp · 7, 32, 34, 37, 67, 68, 94
execv · 67, 94
execvp · 7, 67, 68, 72, 94
execvpe · 67, 94
exit section · 60, 86
exit() · 18, 27, 32
Exponential averaging · 4, 8, **49**, 90, 92

F

FCFS · **48**, **51**, **56**, **57**, 58
fflush · 68, 81
FIFO · 5, 36, 38, 69, 72, 91, 95
fopen · 67, 72, 95
fork() · **6**, **31**, **32**, 33, **43**, 68
full-duplex · 36, 38, 91

G

Gantt · 4, **49**, **50**, **55**, 57, 70
gcc · 19, 44, 66, 97
getpid · 18, 67, 94
getppid · 67, 94
guest · 13, 14, 15, 86, 89, 90
GUI · 16, 17, 19, 85, 91

H

half-duplex · 5, 36, 38, 91
Hard affinity · 52, 53, 81, 92
Hard real-time · 53, 88
HDD · 10, 11, 15, 85, 90
Heap · 5, 28, 37, 39, 42, 45, 87
heavyweight · 39, **40**, 45, 77
hit ratio · 12, 15, 73, 80, 90
hybrid · 14, 16, 21, **23**, 24, 41
hyperthreading · 52
hypervisor · 13, 15, 90

I

I/O-bound · **14**, **26**, **37**, **47**, 61, 73, 96
I/O-bound process · **14**, **26**, **37**, **47**, 61, 73, 96
IaaS · 14, 90
Indirect communication · 36
Interrupt · **26**, 27, 38, 66, 87, 92
Interrupt vector · 26, 66, 92
IPC · 22, 23, 26, **35**

J

JNI · 23, 91

K

Kernel · 21, **23**, **27**, **42**, **60**
Kernel thread · 42, 43, 44, 51, 52, 76, 87
kill() · 32, 43, 67, 94

L

layered · 4, 21, **22**, 23, 24, 85
Layered approach · **22**, 75, 86
Least Used First · **56**, 76
lightweight · 39, **40**, 42, 43, 45, 77
Linker · 19, 20, 24, **66**, 75, 77
Linux · 21, 31, **44**, **66**, **67**, **68**
Little's formula · 4, 5, 47, 55, 58, 90
LKM · 4, 16, 21, 23, 24, 75, 85
Load balancing · 47, 52, **53**, 58, 78, 81
loadable kernel module · 89
Loadable kernel module (LKM) · 89
Loader · 19, 20, 24, **66**, 75, 77
locality · 9, **12**, 15, 75, 80, 86
Locality (temporal / spatial) · 9, **12**, 15, 75, 80, 86
long-term scheduler · 38, 87, 91
Long-term scheduler (job scheduler) · 38, 87, 91
LUF · 7, 47, **56**, 76
LWP · 43, 45, 51, 52, 89
LWP (lightweight process) · 43, 45, 51, 52, 89

M

Mach · 21, 22, 23, 24, 35, 91
mailbox · 36
main memory · 4, **11**, 14, 15, 26, 85
Makefile · 66
many-to-many · 5, 39, 42, 43, 45, 52, 88
many-to-one · 39, 42, 43, 45, 52, 88, 92
Many-to-one / one-to-one / many-to-many · 39, 42, 43, 45, 52, 88, 92
maskable · 26, 66, 92
mechanism · 5, **20**, 24, 25, 35, 90
Medium-term scheduler · 30, 38, 87
Memory stall · 52, 53, 75
Message passing · 4, 16, 23, 24, **35**, 89
Microkernel · 21, **22**, 23, 24, 85, 91
middleware · 9
mkfifo · 36, 38, 69, 91, 95
MLFQ · 4, **50**, **51**, **56**, **57**, **70**
Mode bit · 5, 18, 27, 38, 89, 91
Monolithic · 4, **21**, 22, 23, 24, 89
MS-DOS · 20, **21**, 91
Multicore · **12**, **40**, **52**, 75, 86
multilevel feedback queue · **50**, 88
Multilevel queue · 5, **50**, 58, 88
multiprocessor · **12**, 13, 15, 40, 86, 90
Multiprogramming · **13**, 15, 30, 38, 80, 90
multitasking · 13, 20, 91
mutex · 5, **62**, 63, 64, 71, 84, **96**
Mutex lock · 11, 63, 65, 77
Mutual exclusion · 59, 60, **61**, 65, 88, 93

N

named pipe · 5, 36, 38, 89, 95
Named pipe (FIFO) · 5, 36, 38, 89, 95
next_available_pid · 59, 93
non-blocking · 36, 69
non-preemptive · 5, 47, **49**, 57, 88, 93
nonmaskable · 26, 66, 92

NUMA · 4, **13**, 15, 52, 89, 90
NVM · 10, 11, 86, 90

O

O_CREAT · 7, 67, 68, 95
O_RDONLY · 7, 37, 67, 69, 95
O_WRONLY · 7, 67, 68, 69, 95
one-to-one · 39, 43, 44, 45, 52, 88, 92
open() · 7, 18, 37, 67, 68, 69, 95
Ordinary pipe · 5, 36, 38, 85, 89
Orphan · 26, **32**, 38, **67**, 91

P

PA1 · **37**, 43, **66**, **67**, **68**, **94**
PA2 · **66**, **70**, 72, 97
PaaS · 14, 90
Parallelism · **40**, **41**, 45, 86, 88
parameter passing · 16, **18**, 24, 86, 90
PCB · **29**, 30, 32, 42, 85, 91
pclose · 69, 95
PCS · 42, 45, 47, 52, 58, 76, 89
PCS (process-contention scope) · 42, 45, 47, 52, 58, 76, 89
Peterson · **64**, **71**, 72
Pipe · **7**, 36, **37**, **67**, 95
policy · 5, **20**, 24, 25, **56**, 90
Policy vs mechanism · 16, **20**, 25, 90
popen · 68, 69, 95
port · 10, 14, 20, 21, 37, 75, 91
possible values · 77, 78
preemptive · 47, **49**, **57**, 88, 93
Preemptive scheduling · 50, 89
printf · 6, 19, 34, 35, 64, 78, 85
Priority scheduling · **49**, 50, 58, 88, 89, 92
Privileged instruction · 24, 27, 38, 89
Process · **27**, **28**, **29**, **31**, **32**, 87
process state · 5, 26, **28**, 29, 42, 80
process tree · 6, 26, 33, 34, 73
Processor affinity · 13, 47, **52**, 53, 78, 86
producer · **35**, 36, **59**, 64, 69
Program · 13, 16, 19, **28**, 31, 90
programmed I/O · **14**, 16, **26**, 29, **37**, **47**
Progress · 5, 41, 51, 60, 61, 65, 88
ps aux · 66, 67
pthread_cancel · 44, 96
pthread_cond_wait · 64, 71, 97
pthread_create · 44, 69, 70, 90, 96
pthread_exit · 44, 69, 96
pthread_join · 44, 69, 70, 90, 96
pthread_mutex_init · 44, 71, 96
pthread_mutex_lock · 44, 71, 83, 84, 86, 96, 97
Pthreads · 42, 43, **44**, **69**, **94**, **96**
Pull / push migration · 52, 53, 58, 78, 92
pull migration · 52, 53, 58, 78, 92
push migration · 53, 92

R

Race condition · **59**, 60, **63**, 65, 73, 88
rate-monotonic · **53**, 93
Rate-monotonic (RM) · **53**, 93
read() · 18, 24, 27, 36, 37, 80, 89
read_count · 64, 71, 84
Ready queue · 30, 42, 47, 48, 50, 52, 55
rendezvous · 36, 91
resource allocator · 9, 15, 89
resource sharing · **16**, 31, 89
Response time · 4, 13, 47, 48, 51, 55, 93
responsiveness · 40, 45, 89
RM · **53**, **54**, 55, **70**, 93
round robin · **48**, 88
Round robin (RR) · **48**, 88
RPC · 36, **37**, 87
RR · **48**, **49**, 50, 51, **57**, **70**
rw_mutex · 64, 71, 84

S

SaaS · 14, 90
scalability · 40, 89
SCS · 45, 47, 52, 58, 76, 89
SCS (system-contention scope) · 45, 47, 52, 58, 76, 89
sem_destroy · 71, 96
sem_init · 44, 64, 71, 72, 86, 96
sem_post · 44, 64, 71, 86, 96
sem_wait · 44, 64, 71, 86, 96
Semaphore · 63, **64**, 65, 71, 93, **96**
Shared memory · 16, **35**, 40
shell · 17, 20, **66**, 67, **68**, **96**
short-circuit · 32, **33**, 68, 82
short-term scheduler · 30, 38, 47, 87
SIGCHLD · 44, 45, 92
SIGINT · 32, 43, 67, 68, 81, 94
Signal · **32**, **43**, 63, 64, 71, **94**
signal handler · 72, **94**
SJF · **49**, 51, 57, 78, 87, 88
SMP · **12**, **52**, 60, 75, 93
Socket · 36, 37, 38, 75, 87, 91
Soft affinity · 52
Soft real-time · 4, 47, 53, 55, 79, 88
spatial locality · 12, 15, 75, 86
speedup · 4, 8, 15, **41**, 45, 86
Spinlock · 7, 59, **62**, 65, 73, 79
spooling · 14, 66
SRTF · **49**, 51, 56, 57, 76, 78
Stack · 18, 24, 28, 39, 42, 87, 90
Starvation · 50, 51, 58, **63**, 88, 93
STDOUT_FILENO · 95, 96
storage hierarchy · 9, **10**, 11, 15, 85
swap() · 8, 62, 73
swapping · 13, 30, 38, 87
synchronous signal · 89, 92
System call · 4, **17**, **18**, 24, 27
System program · 24

T

Task parallelism · 39, **41**, 89
task_struct · 29, 44, 91
TCB · 40, **41**, **42**, 45, 88
temporal locality · 12, 75
test_and_set · 8, **60**, **61**, 65, 86
Thread · **39**, **42**, **43**, **44**, 52
thread cancellation · 39, **43**, 89
thread library · 42, 43, 45, 51, 52, 58, 87
Thread-local storage · **44**, 88, 92
Throughput · 12, 15, 47, 48, 55, 86, 90
time quantum · 47, **48**, 70, 88, 89
timesharing · 9, **13**, 15, 90
Timesharing (multitasking) · 9, **13**, 15, 90
trap · 18, 26, 27, 38, 80, 88, 91
Trap / exception · 18, 26, 27, 38, 80, 88, 91
turnaround · 4, 47, **48**, 49, 76, 86
Turnaround time · 47, 48, 49, 86
two-level · **57**, 78

U

unbounded capacity · 36
UNIX · 17, **21**, 32, 36, 87, 91
User thread · 5, 42, 43, 45, 52, 87, 89
usleep · 67, 94
utilization · 4, 8, 47, 48, 54, 55, 93

V

Virtualization · 9, **13**, 14, 15, 86, 89
VMM · 13, 14, 15, 27, 90, 91
VMM / hypervisor · 13, 14, 15, 27, 90, 91
volatile · 4, 10, 11, 15, 85, 86, 90
von Neumann · 4, 9, **10**, 15, 85, 87
von Neumann architecture · **10**, 87

W

wait queue · 30
wait() · 5, 31, 32, 34, 47, 67, 87
Waiting time · 47, 48, 49, 55, 57, 86, 87
waitpid · 7, 37, 67, 94
wc -l · 66, 67, 68, 96
Windows · 17, 21, 22, 42, **44**, 91
write() · 4, 16, 18, 19, 24, 27, 85

Z

zero capacity · 36, 91
Zombie · 5, **32**, 38, **67**, 85